

CyberLab Reference Guide

Field quick-reference for the kit
capability → tool → command → every option explained

Generated from the repository. Markdown is the source of truth.
github.com/project-cyberlab/cyberlab-reference-guide

Contents

Overview	6
Capability Index	9
Kit Tool List	15
Tool Reference	
62	Acquire & preserve (23)
	affcat 63
	affconvert 65
	affinfo 67
	bdeinfo 68
	dc3dd 70
	dcfldd 72
	dd 74
	dumpcap 76
	ewfacquire 81
	ewfexport 86
	ewfinfo 89
	ewfmount 91
	ewfverify 93
	HashMyFiles-gui 95
	img_stat 97
	md5sum 98
	ntfs-3g 101
	rahash2 102
	sha256sum 105
	sigtool 107
	ssdeep 110
	tshark 112
	vshadowinfo 119
	Build the timeline (5)
	log2timeline.py 121
	mactime 134
	pinfo.py 136
	psort.py 139
	tsk_gettimes 147
	Decode & deobfuscate (7)
	CryptoTester-gui 149
	cyberchef 151
	hashcat 152
	hydra 164
	john 165
	openssl 169
	rax2 170
	Examine the filesystem (20)
	binwalk 173

blks	180
bulk_extractor	183
ffind	189
file	191
fls	197
foremost	201
fsstat	203
icat	205
ils	207
istat	209
mmls	212
photorec	214
rafind2	216
scalpel	217
strings	220
tcpxtract	223
testdisk	225
tsk_recover	227
xxd	229
Malware triage — documents (23)	
mraptor	232
msodde	234
msoffcrypto-tool	237
OffVis-gui	239
olebrowse	241
oledir	242
oledump.py	244
olefile	250
oleid	252
olemap	253
olemeta	254
oleobj	256
oletimes	258
olevba	260
pcodedmp	264
pdf-parser	266
pdf-parser.py	270
pdfid	274
pdfid.py	277
PDFStreamDumper-gui	280
pyxswf	282
rtfobj	285
xlmdeobfuscator	287
Malware triage — static (22)	
7za	291
base64dump.py	293

capa	297
CFF-Explorer-gui	300
clamscan	302
die	307
die-gui	308
diec	312
floss	316
freshclam	318
numbers-to-string.py	321
objdump	324
PE-Detective-gui	330
rabin2	332
radiff2	336
readelf	340
readelf.py	346
Signature-Explorer-gui	349
unzip	351
upx	353
yara	356
yarac	361
Memory forensics (5)	
aeskeyfind	363
rsakeyfind	365
vol	366
volatility3	370
volshell	374
Network analysis (10)	
arp-scan	377
capinfos	378
editcap	380
inetsim	384
mergecap	386
ngrep	388
nmap	392
nping	397
reordercap	403
tcpflow	405
Report & support (1)	
ezhexviewer	409
Reverse engineering (15)	
dnSpy-gui	410
frida	412
frida-discover	418
frida-kill	421
frida-ls-devices	423
frida-ps	424

frida-trace	427
idr-gui	434
r2	436
rasm2	440
VB-Decompiler-gui	443
vbdec-gui	445
vdbbin	446
vivbin	448
xortool	451

Windows artifacts (24)

AccessEnum-gui	455
AmcacheParser	457
AppCompatCacheParser	460
chainsaw	463
esedbexport	465
esedbinfo	467
EvtxECmd	469
evtexport	473
evtinfo	476
hayabusa	478
hivexsh	479
MFTECmd	480
PECmd	484
RECmd	487
regfexport	491
regfinfo	493
regfmount	495
regipy-diff	497
regipy-dump	498
regipy-parse-header	501
regipy-plugins-run	502
regripper	504
rip.pl	505
SrumECmd	506
	508

Alphabetical Tool Index

CyberLab Reference Guide

A field quick-reference for the kit. You open it mid-mission or mid-exercise when you need a **capability** and do not remember which tool provides it — it routes you to the tool, the exact command, and **every option, explained**.

Start here

You are...	Go to
Facing a problem — " <i>I need to carve deleted files</i> "	Capability Index — 46 capabilities
Asking what the kit even has	Kit Tool List — 1,006 tools, VM → category → tool
Wanting a finished page	fls · vol · yara
Carrying it into the field	PDF — 372 pages
Wanting to know how pages are built and why	The Format

Where it stands

Kit inventory	1,006 tools, 5 platforms, 54 categories
Captured from a real binary	945 commands
Capabilities	46 , across 10 investigation phases
Tool pages	135 , every one with a complete options table
Flags with usage guidance	153 across 20 reviewed tools
Mined invocations	360 real commands from cyberlab
Citations checked	255 URLs — 245 live
Linter	0 errors , 269 warnings (tracked debt)

What makes this different

Every DFIR quick-reference in the field either organises by tool (so you must already know the tool) or skips flag documentation. The most complete one in public circulation states outright that it "*assumes operator familiarity*."

This guide inverts that:

- **Capability-first.** You search by what you need to *do*, not what the binary is called.
- **Options completely listed out** — every flag, with *what it does* and *when you'd use it*.
- **Tools attached to commands**, each tagged with the VM that carries it, so you know whether you can run it on the box in front of you.

Scope is binding

Only tools in `catalog/kit-tools.json` may be documented. It is generated from the kit VMs' own upstream manifests:

Platform	Tools	Source of truth
Kali Linux	403	<code>kali-meta</code> <code>debian/control</code> (<code>kali-tools-*</code>)
REMnux	268	<code>REMnux/docs</code> <code>discover-the-tools</code>
SIFT Workstation	162	<code>teamdfir/sift-saltstack</code>
FLARE-VM	137	<code>mandiant/flare-vm</code> <code>config.xml</code>
Security Onion	36	kit tracker CSV

Every flag is real

Options are **captured from the running binary** (`--help` / `man`, inside the kit container), stored raw under `capture/`, and enforced by `scripts/lint.py`: a documented flag absent from a capture is a **build error**.

This answers a measured failure in the predecessor project, where fabricated CLI flags reached ~44 of 61 modules. Rationale in `docs/FORMAT.md`.

Citations are liveness-checked too — see `capture/SOURCES.md`.

Rebuild everything

```
python scripts/build_kit_list.py      # kit inventory from upstream manifests
python scripts/candidates.py          # candidate command names
# scripts/probe_container.sh runs inside the kit container on rick
python scripts/merge_coverage.py      # merge probe results -> coverage report
python scripts/mine_cyberlab.py       # candidate invocations (read-only)
python scripts/generate_pages.py      # tool pages (never overwrites hand-written)
python scripts/build_index.py         # capability index
python scripts/lint.py                # gate: 0 errors required
python scripts/validate_sources.py    # citation liveness
python scripts/build_pdf.py           # print render
```

Relationship to cyberlab

Separate repository, deliberately. The `cyberlab training lab` is **on hold** and receives no changes from this work; it is read-only input for mining.

Layout

catalog/	kit tool list – the binding scope (generated)
reference/	the guide: INDEX.md + one page per tool, grouped by capability
capture/	raw --help/man output, committed as evidence
docs/	FORMAT.md (how pages are built), PLAN.md (roadmap)
scripts/	build, probe, mine, generate, lint, validate, render

Capability Index

Start here when you have a problem. Find what you need to *do*; it points you at the tool and its page.

Phases follow [NIST SP 800-86](#) (collect → examine → analyse → report), ordered the way an investigation actually runs.

Legend: **bold** = captured from a real binary, page can be written · plain = in the kit but not captured yet (needs a booted VM or has no CLI).

Phases

- [Acquire & preserve](#)
- [Examine the filesystem](#)
- [Build the timeline](#)
- [Windows artifacts](#)
- [Memory forensics](#)
- [Network analysis](#)
- [Malware triage — static](#)
- [Malware triage — documents](#)
- [Reverse engineering](#)
- [Decode & deobfuscate](#)
- [Report & support](#)

Acquire & preserve

Image a disk, volume or device

[dc3dd](#), [dcfldd](#), [dd](#), [ewfacquire](#), [affconvert](#), [guymager](#)

Verify evidence integrity with hashes

[rahash2](#), [ssdeep](#), [sha256sum](#), [md5sum](#), [sigtool](#), [hashdeep](#), [md5deep](#), [sha256deep](#)

Inspect or mount a forensic image container

[ewfinfo](#), [ewfmount](#), [ewfverify](#), [ewfexport](#), [affinfo](#), [affcat](#), [img_stat](#), [ntfs-3g](#), [vshadowinfo](#), [bdeinfo](#)

Capture live network traffic

[dumpcap](#), [tshark](#), [tcpdump](#)

Examine the filesystem

See the partition and volume layout

[mmls](#), [fsstat](#), [img_stat](#), [testdisk](#)

List files and directories, including deleted ones

[fls](#), [ffind](#), [ils](#), [tsk_recover](#)

Recover deleted or lost files

[tsk_recover](#), [icat](#), [photorec](#), [testdisk](#), [blkls](#), [ext4magic](#), [extundelete](#), [ext3grep](#)

Carve files out of unstructured data

[foremost](#), [scalpel](#), [binwalk](#), [bulk_extractor](#), [tcpxtract](#), [magicrescue](#)

Inspect metadata for one file or inode

[istat](#), [ils](#), [file](#), [stat](#), [exiftool](#), [trid](#), [magika](#)

Search raw data for a pattern

[rafind2](#), [strings](#), [grep](#), [xxd](#), [bulk_extractor](#), [lightgrep](#)

Build the timeline

Build a super-timeline from many artifact sources

[log2timeline.py](#), [psort.py](#), [pinfo.py](#), [psteal.py](#)

Build a filesystem MAC-time timeline

[fls](#), [mactime](#), [tsk_gettimes](#)

Windows artifacts

Parse registry hives

[rip.pl](#), [regripper](#), [hivexsh](#), [regfexport](#), [regfinfo](#), [regfmount](#), [regipy-dump](#), [regipy-parse-header](#), [regipy-plugins-run](#), [regipy-diff](#), [RECcmd](#), [hivexget](#), [hivexml](#)

Parse Windows event logs

[evtexport](#), [evtinfo](#), [EvtxECmd](#), [chainsaw](#), [hayabusa](#), [evt_dump](#)

Parse ESE / SRUM / Amcache databases

[esedbexport](#), [esedbinfo](#), [SrumECmd](#), [AmcacheParser](#)

Parse execution and persistence artifacts

[PECmd](#), [AppCompatCacheParser](#), [MFTECmd](#), [AmcacheParser](#), [analyzeMFT](#)

Parse mail stores

[pffexport](#), [pffinfo](#), [readpst](#)

Memory forensics

Analyse a memory image

[vol](#), [volatility3](#), [volshell](#), [vol.py](#), [rekall](#)

Recover encryption keys from memory

[aeskeyfind](#), [rsakeyfind](#), [bulk_extractor](#)

Network analysis

Read and filter packet captures

[tshark](#), [capinfos](#), [ngrep](#), [tcpflow](#)

Split, merge or repair capture files

[editcap](#), [mergcap](#), [reordercap](#)

Extract files and payloads from traffic

[tcpextract](#), [tcpflow](#), [foremost](#)

Detect intrusions in traffic

[suricata](#), [suricatasc](#), [zeek](#), [zeek-cut](#), [rita](#)

Probe or scan hosts and services

[nmap](#), [nping](#), [arp-scan](#), [ncat](#), [netdiscover](#)

Simulate network services for detonation

[inetsim](#), [fakedns](#), [fakenet](#)

Malware triage — static

Identify what a file actually is

[file](#), [die](#), [diec](#), [trid](#), [magika](#), [exiftool](#)

Scan with signatures for known-bad

[yara](#), [yarak](#), [clamscan](#), [freshclam](#), [sigtool](#), [clamscan](#)

Identify capabilities in a binary

[capa](#), [floss](#)

Extract strings, including obfuscated ones

[floss](#), [strings](#), [base64dump.py](#), [numbers-to-string.py](#)

Inspect PE / ELF structure

[rabin2](#), [readelf](#), [readelf.py](#), [objdump](#), [pescan](#), [pecheck.py](#), [peframe](#)

Detect and reverse packing

[upx](#), [die](#), [diec](#), [binwalk](#), [7za](#), [unzip](#)

Compare or cluster samples

[ssdeep](#), [radiff2](#), [bytehist](#)

Malware triage — documents

Analyse Office documents and macros

[olevba](#), [oleid](#), [oledump.py](#), [olemeta](#), [oletimes](#), [olemap](#), [oledir](#), [olebrowse](#), [olefile](#), [oleobj](#), [mraptor](#), [msodde](#), [pcodedmp](#), [pyxswf](#), [xlmdeobfuscator](#), [runxlrd2.py](#), [vipermonkey](#)

Analyse RTF documents and embedded objects

[rtfobj](#), [oleobj](#), [rtfdump.py](#)

Analyse PDFs

[pdfid](#), [pdfid.py](#), [pdf-parser](#), [pdf-parser.py](#), [peepdf](#), [qpdf](#)

Analyse other container formats

[7za](#), [unzip](#), [msoffcrypto-tool](#), [onenoteanalyzer](#)

Reverse engineering

Disassemble and explore a binary

[r2](#), [rabin2](#), [rasm2](#), [objdump](#), [vivbin](#), [vdbbin](#), [rizin](#), [ghidra](#), [cutter](#)

Diff two binaries

[radiff2](#), [bindiff](#)

Emulate or instrument execution

[frida](#), [frida-trace](#), [frida-ps](#), [frida-discover](#), [frida-kill](#), [frida-ls-devices](#), [speakeasy](#), [qiling](#), [sctdbg](#), [unicorn](#)

Analyse shellcode

[rasm2](#), [xortool](#), [sctdbg](#), [shellcode2exe](#)

Decode & deobfuscate

Decode, decrypt or transform encoded data

[cyberchef](#), [base64dump.py](#), [rax2](#), [xxd](#), [openssl](#), [numbers-to-string.py](#)

Break simple obfuscation

[xortool](#), [floss](#), [xlmdeobfuscator](#), [de4dot](#)

Crack passwords and hashes

[hashcat](#), [john](#), [hydra](#), [unshadow](#), [zip2john](#), [rar2john](#), [fcrackzip](#), [hashid](#)

Find hidden data

[binwalk](#), [ssdeep](#), [steghide](#), [stegosuite](#)

Report & support

Fetch and verify external references

[curl](#), [wget](#)

Inspect files by hand

[xxd](#), [ezhexviewer](#), [less](#), [hexdump](#)

Coverage of this index

- **46 capabilities** across 11 phases
- **147 captured tools** are reachable through a capability
- **5 named kit tools map to no capability** (below)
- **2 capabilities have no captured tool** behind them yet

Capabilities with nothing captured yet

These are real needs the kit may cover with GUI or Windows-only tools that a Linux container cannot capture.

- Windows artifacts / Parse mail stores
- Network analysis / Detect intrusions in traffic

Named kit tools not yet mapped to a capability

These are named in the kit manifests, captured from a real binary, and reach no capability in this index. Each is a taxonomy gap to close or an explicit out-of-scope decision. Listed rather than silently dropped, because a missing capability is otherwise invisible.

`aircrack-ng`, `cabextract`, `ccrypt`, `cryptsetup`, `dislocker`

Container-provided commands (not named in the kit manifests)

Discovered by walking the container's `PATH`. They ship in a kit image but no kit manifest names them, so the great majority is OS plumbing rather than investigative tooling. Kept for completeness only.

► 807 container-provided commands

Kit Tool List

The binding scope for the quick-reference guide. A tool absent from this list is **not in the kit** and must not be documented in the guide.

Every entry is derived from an upstream machine-readable manifest; none is written from memory. Sources are listed at the end.

Contents

- [REMnux](#) — 268 tools, 31 categories
- [Kali Linux](#) — 403 tools, 12 categories
- [FLARE-VM](#) — 137 tools, 1 categories
- [SIFT Workstation](#) — 162 tools, 1 categories
- [Security Onion](#) — 36 tools, 9 categories

Total: 1006 tools across 5 platforms.

REMnux

Analyze Documents / Email Messages

Tool	Command(s)	Purpose
emldump.py	—	Parse and analyze EML files.
mail-parser	—	Parse raw SMTP and .MSG email messages and generate a parsed object from them.
msg-extractor	<code>extract_msg</code>	Extract emails and attachments from MSG files.
msgconvert	—	Convert MSG files to MBOX files.

Analyze Documents / General

Tool	Command(s)	Purpose
base64dump.py	—	Locate and decode strings encoded in Base64 and other common encodings.
Tesseract OCR	<code>tesseract</code>	Examine images to identify and extract text using optical character recognition (OCR).

Analyze Documents / Microsoft Office

Tool	Command(s)	Purpose
EvilClippy	—	Modify aspects of Microsoft Office documents.

Tool	Command(s)	Purpose
Hachoir	<code>hachoir-grep</code> , <code>hachoir-metadata</code> , <code>hachoir-strip</code> , <code>hachoir-wx</code>	View, edit, and carve contents of various binary file types.
libolecf	<code>olecfexport</code> , <code>olecfinfo</code> , <code>olecfmount</code> , etc	Microsoft Office OLE2 compound documents.
msoffcrypto-crack.py	—	Recover the password of an encrypted Microsoft Office document.
msoffcrypto-tool	—	Decrypt a Microsoft Office file with password, intermediate key, or private key which generated its escrow key.
msoffice-crypt	—	Encrypt and decrypt OOXML Microsoft Office documents.
oledump.py	—	Analyze OLE2 Structured Storage files.
olefile	—	Python package to parse, read and write MS OLE2 files.
oletools	<code>mraptor</code> , <code>msodde</code> , <code>olebrowse</code> , <code>oledir</code> , <code>oleid</code> , <code>olemap</code> , <code>olemeta</code> , <code>oleobj</code> , <code>oletimes</code> , <code>olevba</code> , <code>pyxswf</code> , <code>rtfobj</code> , <code>ezhexviewer</code>	Microsoft Office OLE2 compound documents.
onedump.py	—	Extract and analyze embedded files from OneNote documents.
pcode2code	—	Decompile VBA macro p-code from Microsoft Office documents.
pcodedmp	—	Disassemble VBA p-code.
rtfdump.py	—	Analyze a suspicious RTF file.
SSView	<code>ssview</code>	Analyze OLE2 Structured Storage files.
XLMMacroDeobfuscator	<code>xlmddeobfuscator</code> , <code>runxlrd2.py</code>	Deobfuscate XLM macros (also known as Excel 4.0 macros) from Microsoft Office files.
xmldump.py	—	Extract contents of XML files, in particular OOXML-formatted Microsoft Office documents.
zipdump.py	—	Analyze zip-compressed files.

Analyze Documents / Pdf

Tool	Command(s)	Purpose
Origamind	<code>pdfcop</code> , <code>pdfdecompress</code> , <code>pdfdecrypt</code> , <code>pdfextract</code> , etc	Parse, modify, generate PDF files.
pdf-parser.py	—	Examine elements of the PDF file.
pdfid.py	—	Identify suspicious elements of the PDF file.

Tool	Command(s)	Purpose
pdfresurrect	—	Extract previous versions of content from PDF files.
pdftk-java	pdftk	Edit, create, and examine PDF files.
pdftool.py	—	Analyze PDF files to identify incremental updates to the document.
peepdf-3	—	Examine elements of the PDF file.
qpdf	—	Manipulate (merge, convert, transform) PDF files.

Dynamically Reverse-Engineer Code / Elf Files

Tool	Command(s)	Purpose
edb	—	An AArch32/x86/x86-64 debugger, well suited for debugging ELF files.
GNU Project Debugger	—	Multi-language debugger.
ltrace	—	Trace library calls and signals.
strace	—	Trace process' system calls and signals.

Dynamically Reverse-Engineer Code / General

Tool	Command(s)	Purpose
Frida	frida , frida-ps , frida-trace , frida-discover , frida-ls-devices , frida-kill	Trace the execution of a process to analyze its behavior.
r2pipe	—	Examine binary files, including disassembling and debugging.
radare2	r2 , rasm2 , rabin2 , rahash2 , rafind2 , r2ai , decai , pdg	Examine binary files, including disassembling and debugging. Includes r2ai and decai plugins for LLM-powered analysis (A
Wine	wine	Run Windows applications.
x64dbg Automate MCP (OpenCode skills)	—	Drive x64dbg on a remote Windows VM from OpenCode on REMnux, with eight AI commands for tracing, unpacking, state snapsh

Dynamically Reverse-Engineer Code / Scripts

Tool	Command(s)	Purpose
box-js	box-js , box-export	Analyze suspicious JavaScript scripts.
JavaScript Deobfuscator	—	Deobfuscate JavaScript by removing common obfuscation techniques such as string arrays and proxy functions.

Tool	Command(s)	Purpose
js_unshroud	—	Monitor and deobfuscate JavaScript behavior in a headless browser to analyze malicious web pages.
JStillery	jstillery	Deobfuscate JavaScript scripts using AST and Partial Evaluation techniques.
objects.js	—	Emulate common browser and PDF viewer objects, methods, and properties when deobfuscating JavaScript.
PowerShell Core	pwsh	Run PowerShell scripts and commands.
Rhino Debugger	rhino-debugger	GUI JavaScript debugger.
SpiderMonkey	js	Execute and deobfuscate JavaScript using Mozilla's standalone JavaScript engine.
SpiderMonkey (Patched)	js-ascii , js-file	Execute and deobfuscate JavaScript using a patched version of Mozilla's standalone JavaScript engine.
STPyV8	—	Python3 and JavaScript interop engine, fork of the original PyV8 project.
Webcrack	—	Deobfuscate, unminify, and unpack bundled JavaScript, including scripts protected with obfuscator.io.

Dynamically Reverse-Engineer Code / Shellcode

Tool	Command(s)	Purpose
libemu	—	A library for x86 code emulation and shellcode detection.
Qiling	—	Emulate code execution of PE files, shellcode, etc. for a variety of OS and hardware platforms.
runsc	—	Run shellcode to trace and analyze its execution.
scdbg	—	Analyze shellcode by emulating its execution.
shcode2exe	—	Convert 32 and 64-bit shellcode to a Windows executable file.
shellcode2exe.bat	—	Convert 32 and 64-bit shellcode to a Windows executable file.
Speakeasy	—	Emulate code execution, including shellcode, Windows drivers, and Windows PE files.
XORSearch	xorsearch	Locate and decode strings obfuscated using common techniques.

Examine Static Properties / .Net

Tool	Command(s)	Purpose
dnfile	—	Analyze static properties of .NET files.
dotnetfile	—	Analyze static properties of .NET files.
monodis	—	Disassemble and extract resources from .NET assemblies.

Examine Static Properties / Deobfuscation

Tool	Command(s)	Purpose
1768.py	—	Analyze Cobalt Strike beacons.
Balbuzard	<code>balbuzard</code> , <code>bbcrack</code> , <code>bbharvest</code> , <code>bbtrans</code>	Extract and deobfuscate patterns from suspicious files.
base64dump.py	—	Locate and decode strings encoded in Base64 and other common encodings.
brxor.py	—	Bruteforce XOR'ed strings to find those that are English words.
Chepy	<code>chepy</code>	Decode and otherwise analyze data using this command-line tool and Python library.
Cobalt Strike Configuration Extractor (CSCE) and Parser	<code>csce</code> , <code>list-cs-settings</code>	Analyze Cobalt Strike beacons.
cs-analyze-processdump.py	—	Analyze Cobalt Strike beacon process dumps to detect sleep mask encoding.
cs-decrypt-metadata.py	—	Decrypt Cobalt Strike metadata.
cs-extract-key.py	—	Extract AES and HMAC keys from Cobalt Strike beacon process memory.
cut-bytes.py	—	Cut out a part of a data stream.
CyberChef	<code>cyberchef</code>	Decode and otherwise analyze data using this browser app.
DC3-MWCP	<code>mwcp</code>	Parsing configuration information from malware.
ex_pe_xor.py	—	Search an XOR'ed file for indications of executable binaries.
FLOSS	<code>floss</code>	Extract and deobfuscate strings from PE executables.
format-bytes.py	—	Decompose structured binary data with format strings.
hex-to-bin.py	—	Convert hexadecimal text dumps to binary data.
Malchive	—	Perform static analysis of various aspects of malicious code.

Tool	Command(s)	Purpose
NoMoreXOR.py	—	Help guess a file's 256-byte XOR by using frequency analysis.
numbers-to-string.py	—	Translate number sequences into ASCII characters.
re-search.py	—	Search files using regular expressions from a built-in library or custom patterns.
sets.py	—	Perform set operations on lines or bytes in text files.
strdeob.pl	—	Locate and decode stack strings in executable files.
translate.py	—	Translate bytes according to a Python expression.
unicode	—	Display Unicode character properties.
unXOR	—	Deobfuscate XOR'ed files.
xor-kpa.py	—	Implement a XOR known plaintext attack.
xorBruteForcer.py	—	Bruteforce an XOR-encoded file.
XORSearch	<code>xorsearch</code>	Locate and decode strings obfuscated using common techniques.
xorsearch.py	—	Search for XOR, ROL, ROT, and SHIFT encoded strings with YARA and regex support.
XORStrings	—	Search for XOR encoded strings in a file.
xortool	—	Analyze XOR-encoded data.

Examine Static Properties / Elf Files

Tool	Command(s)	Purpose
pyelftools	<code>readelf.py</code>	Python library for parsing and analyzing ELF files and DWARF debugging information.

Examine Static Properties / General

Tool	Command(s)	Purpose
7-Zip	<code>7za</code>	Compress and decompress files using a variety of algorithms.
binwalk	—	Extract and analyze firmware images.
bulk_extractor	—	Extract interesting strings from binary files.
ClamAV	<code>clamscan</code> , <code>freshclam</code>	Scan files for malware signatures.
Detect-It-Easy	—	Determine types of files and examine file properties.

Tool	Command(s)	Purpose
disitool	<code>disitool.py</code>	Manipulate embedded digital signatures.
DroidLysis	<code>droidlysis</code>	Perform static analysis of Android applications.
ExifTool	<code>exiftool</code>	Tool to read from, write to, and edit EXIF metadata of various file types.
file	—	Identify file type using "magic" numbers.
file-magic.py	—	Identify file types using the Python magic module.
Hachoir	<code>hachoir-grep</code> , <code>hachoir-metadata</code> , <code>hachoir-strip</code> , <code>hachoir-wx</code>	View, edit, and carve contents of various binary file types.
Hash ID	<code>hash-id.py</code>	Identify different types of hashes.
LIEF	—	Parse and analyze PE, ELF, MachO, DEX, OAT, VDEX, ART, and DWARF executable formats.
Magika	—	Identify file type using signatures.
Malcat Lite	—	Analyze binary files using a hex editor, disassembler, and file dissector.
msitools	—	Create, inspect and extract Windows Installer (.msi) files.
Name-That-Hash	<code>nth</code>	Identify different types of hashes.
numbers-to-string.py	—	Convert decimal numbers to strings.
re-search.py	—	Search the file for built-in regular expressions of common suspicious artifacts.
signsrch	—	Find patterns of common encryption, compression, or encoding algorithms.
Sleuth Kit	—	Analyze disk images and recover files from them.
ssdeep	—	Compute Context Triggered Piecewise Hashes (CTPH), also known as fuzzy hashes.
strings.py	—	Extract ASCII and Unicode strings from binary files with length sorting and filtering.
thefuzz	—	Fuzzy String Matching in Python.
TrID	<code>trid</code> , <code>tridupdate</code>	Identify file type using signatures.
wxHexEditor	—	Hex editor.
Yara Rules	—	Scan a file with YARA rules to identify capabilities and behaviors (packer detection, anti-debug, networking).

Tool	Command(s)	Purpose
YARA-Forge Rules	—	Scan files with curated YARA rules from 45+ sources for malware family identification.

Examine Static Properties / Go

Tool	Command(s)	Purpose
GoReSym	GoReSym	Extract metadata and symbols from Go binaries, including stripped ones.
Redress	redress	Analyze stripped Go binaries to recover symbols, types, source structure, and integrate with Radare2.

Examine Static Properties / Pe Files

Tool	Command(s)	Purpose
bearparser	bearcommander	Parse PE file contents.
debloat	—	Remove junk contents from bloated Windows executables.
disitool.py	—	Extract, delete, copy, and inject digital signatures in PE files.
dllcharacteristics.py	—	Read and set DLL characteristics of a PE file.
Manalyze	—	Perform static analysis of suspicious PE files.
PE Tree	pe-tree	Examine contents and structure of PE files.
pecheck.py	—	Analyze static properties of PE files.
pedump	—	Statically analyze PE files and extract their components (e.g., resources).
pefile	—	Python library for analyzing static properties of PE files.
PEframe	peframe	Statically analyze PE and Microsoft Office files.
pev	pestr , readpe , pedis , pehash , pescan , peldd , peres	Analyze PE files and extract strings from them.
PortEx	portex	Statically analyze PE files.
readpe (formerly pev)	readpe , pestr , pedis , pehash , pescan , pesec , peldd , pepack , peres , ofs2rva , rva2ofs	Analyze PE files and extract strings from them.

Explore Network Interactions / Connecting

Tool	Command(s)	Purpose
Anomy	<code>anomy</code>	A wrapper around wget, ssh, sftp, ftp, and telnet to route these connections through Tor to anonymize your traffic.
cURL	<code>curl</code>	Interact with servers via supported protocols, including HTTP, HTTPS, FTP, IMAP, etc. using this command-line tool.
EPIC IRC Client	<code>epic5</code>	Examine IRC activities with this IRC client.
GNU Wget	<code>wget</code>	Interact with servers via HTTP, HTTPS, FTP, and FTPS using this command-line tool.
netcat	<code>nc</code>	Read and write data across network connections.
thug	—	Examine suspicious website using this low-interaction honeyclient.
tor	—	Obfuscate your origins by routing traffic through a network of anonymizing nodes.
Unfurl	—	Deconstruct and decode data from a URL.
zbarimg	<code>zbarimg</code>	Decode QR codes and barcodes from image files.

Explore Network Interactions / Monitoring

Tool	Command(s)	Purpose
Burp Suite Community Edition	<code>burpsuite</code>	Investigate website interactions using this web proxy.
cs-parse-traffic.py	—	Decrypt and parse Cobalt Strike beacon network traffic.
mitmproxy	<code>mitmproxy</code> , <code>mitmdump</code> , <code>mitmweb</code>	Investigate website interactions using this web proxy.
monitor-network	—	Monitor traffic on the first active network interface using tshark, printing a live summary to the screen or saving it t
Network Miner Free Edition	<code>networkminer</code>	Examine network traffic and carve PCAP capture files.
ngrep	—	Look for patterns in network traffic.
PolarProxy	<code>polarproxy</code>	Intercept and decrypt TLS traffic.
tcpdump	—	Capture and analyze network traffic with this command-line sniffer.
tcpflow	—	Analyze the flow of network traffic.
tcpick	—	Capture and analyze network traffic with this command-line sniffer.
tcpxtract	—	Extract files from network traffic.

Tool	Command(s)	Purpose
tshark	—	Capture and analyze network traffic with this console-based sniffer.
wireshark	—	Capture and analyze network traffic with this sniffer.

Explore Network Interactions / Services

Tool	Command(s)	Purpose
accept-all-ips	—	Accept connections to all IPv4 and IPv6 addresses and redirect it to the corresponding local port.
dnsresolver.py	—	DNS resolver tool for dynamic analysis with wildcard and tracking support.
fakedns	—	Respond to DNS queries with the specified IP address.
fakemail	—	Intercept and examine SMTP email activity with this fake SMTP server.
FakeNet-NG	—	Emulate common network services and interact with malware.
INetSim	<code>inetsim</code>	Emulate common network services and interact with malware.
inspircd 3	—	Examine IRC activity with this IRC server.
netcat	<code>nc</code>	Read and write data across network connections.
Nginx	—	Web server.

Gather And Analyze Data / General

Tool	Command(s)	Purpose
DeXRAY	<code>dexray</code>	Extract and decode data from antivirus quarantine files.
dissect	—	Perform a variety of forensics and incident response tasks using this DFIR framework and toolset.
dnslib	—	Python library to encode/decode DNS wire-format packets.
ioc_parser	—	Extract IOCs from security report PDFs.
ipwhois	—	Retrieve and parse whois data for IP addresses.
malwoverview	<code>malwoverview</code>	Query public repositories of malware data (e.g., VirusTotal, HybridAnalysis).

Tool	Command(s)	Purpose
nsrlookup	—	Look up MD5 file hashes in the NIST National Software Reference Library (NSRL).
pdnstool	—	Query passive DNS databases for DNS data.
Scalpel	—	Carve contents out of binary files, such as partitions.
time-decode	—	Decode and encode date and timestamps.
virustotal-search	<code>virustotal-search.py</code>	Search VirusTotal for file hashes.
virustotal-submit	<code>virustotal-submit.py</code>	Submit files to VirusTotal.
Yara	<code>yara</code>	Identify and classify malware samples using Yara rules.
YARA-X	—	Scan files using YARA rules, the next generation of YARA written in Rust.

General Utilities / General

Tool	Command(s)	Purpose
7-Zip	<code>7za</code>	Compress and decompress files using a variety of algorithms.
cabextract	—	Extract Microsoft cabinet (cab) files.
cURL	<code>curl</code>	Interact with servers via supported protocols, including HTTP, HTTPS, FTP, IMAP, etc. using this command-line tool.
Docker	—	Run and manage containers.
Firefox	<code>firefox</code>	Web browser.
GNOME Calculator	<code>calculator</code>	Calculator.
IBus	<code>ibus-setup</code>	Adjust input methods for the GUI.
Info-ZIP	<code>zip</code> , <code>unzip</code>	Compress and decompress files using the zip algorithm.
myip	—	Determine the IP address of the default network interface.
myjson-filter.py	—	Filter data formatted using the JSON format used by Didier Stevens' tools.
nasm	—	An x86-64 assembler.
Nautilus	—	Graphical file manager.
OpenSSH	<code>sftp</code> , <code>ssh</code> , etc	Initiate and receive SSH and SFTP connections.
PowerShell Core	<code>pwsh</code>	Run PowerShell scripts and commands.
RAR	<code>rar</code>	Compress and decompress files using a variety of algorithms.

Tool	Command(s)	Purpose
REMnux Installer	—	Install and update the REMnux distro.
restrict-egress	—	Restrict outbound network access to an allowlist of domains and CIDRs using an nftables default-deny egress policy. It i
sortcanon.py	—	Sort text files using canonicalization functions built into this tool.
SQLite	<code>sqlite3</code>	Manage and interact with SQL database files.
sshpas	<code>sshpas</code>	Supply a password to SSH non-interactively for automated logins.
texteditor.py	—	Edit text files from the command line using search-and-replace commands.
unrar-free	<code>unrar</code>	Decompress files using a variety of algorithms.
Wine	<code>wine</code>	Run Windows applications.

Investigate System Interactions / General

Tool	Command(s)	Purpose
ProcDOT	<code>procdot</code>	Visualize and examine the output of Process Monitor.
ProcmonMCP	—	MCP server that lets AI assistants analyze Process Monitor (Procmon) XML captures.
sandfly-processdecloak	—	Find hidden processes on the local Linux system.
Unhide	—	Find hidden processes or connections on the local Linux system.

Perform Memory Forensics / General

Tool	Command(s)	Purpose
AESKeyFinder	<code>aeskeyfind</code>	Find 128-bit and 256-bit AES keys in a memory image.
<code>bulk_extractor</code>	—	Extract interesting strings from binary files.
RSKeyFinder	<code>rsakeyfind</code>	Find BER-encoded RSA private keys in a memory image.
Volatility Framework	—	Memory forensics tool and framework.

Statically Analyze Code / .Net

Tool	Command(s)	Purpose
de4dot	—	Deobfuscate and unpack .NET programs.
ILSpy	—	Examine and decompile .NET programs.

Statically Analyze Code / Android

Tool	Command(s)	Purpose
androguard	—	Examine Android files.
AndroidProjectCreator	—	Convert an Android APK application file into an Android Studio project for easier analysis.
APKiD	<code>apkid</code>	Identify compilers, packers, and obfuscators used to protect Android APK and DEX files.
apktool	—	Reverse-engineer Android APK files.
baksmali	—	Disassembler for the dex format used by Dalvik, Android's Java VM implementation.
dex2jar	<code>dex-tools</code> , <code>d2j-dex2jar</code>	Examine Dalvik Executable (dex) files.
DroidLysis	<code>droidlysis</code>	Perform static analysis of Android applications.
JADX	<code>jadx</code> , <code>jadx-gui</code>	Generate Java source code from Dalvik Executable (dex) and Android APK files.

Statically Analyze Code / General

Tool	Command(s)	Purpose
Cutter	—	Reverse engineering platform powered by Rizin.
Detect-It-Easy	—	Determine types of files and examine file properties.
Ghidra	—	Software reverse engineering tool suite.
<code>objdump</code>	—	Disassemble binary files.
Qiling	—	Emulate code execution of PE files, shellcode, etc. for a variety of OS and hardware platforms.
radare2	<code>r2</code> , <code>rasm2</code> , <code>rabin2</code> , <code>rahash2</code> , <code>rafind2</code> , <code>r2ai</code> , <code>decai</code> , <code>pdg</code>	Examine binary files, including disassembling and debugging. Includes r2ai and decai plugins for LLM-powered analysis (A
Vivisect	<code>vivbin</code> , <code>vdbbin</code>	Statically examine and emulate binary files.

Statically Analyze Code / Java

Tool	Command(s)	Purpose
cfr	—	Java decompiler.
Java IDX Parser	<code>idx_parser.py</code>	Analyze Java IDX files.
Javassist	—	Java bytecode engineering toolkit/library.
JD-GUI Java Decompiler	<code>jd-gui</code>	Java decompiler with GUI.

Tool	Command(s)	Purpose
Procyon	<code>procyon</code>	Java decompiler.

Statically Analyze Code / Pe-Files

Tool	Command(s)	Purpose
binee (Binary Emulation Environment)	—	Analyze I/O operations of a suspicious PE file by emulating its execution.
capa	—	Detect suspicious capabilities in PE files.
Malchive	—	Perform static analysis of various aspects of malicious code.
mbcscan	<code>mbcscan.py</code>	Scan a PE file to list the associated Malware Behavior Catalog (MBC) details.
Speakeasy	—	Emulate code execution, including shellcode, Windows drivers, and Windows PE files.

Statically Analyze Code / Python

Tool	Command(s)	Purpose
Decompyle++	<code>pycdas</code> , <code>pycdc</code>	Python bytecode disassembler and decompiler.
PyInstaller Extractor	<code>pyinstxtractor.py</code>	Extract contents of a PyInstaller-generated PE files.
pyinstxtractor-ng	<code>pyinstxtractor-ng</code>	Extract contents of PyInstaller-generated executables without requiring a matching Python version.
uncompyle6	<code>uncompyle6</code>	Python cross-version bytecode decompiler for Python 1.0 through 3.8.

Statically Analyze Code / Scripts

Tool	Command(s)	Purpose
AutoIt-Ripper	<code>autoit-ripper</code>	Extract AutoIt scripts embedded in PE binaries.
decode-vbe.py	—	Decode encoded VBS scripts (VBE).
GootLoaderAutoJsDecode.py	—	Statically deobfuscate GootLoader (GOOTLOADER) malicious JScript to recover the payload and extract C2 domains.
JS Beautifier	<code>js-beautify</code>	Reformat JavaScript scripts for easier analysis.

Statically Analyze Code / Unpacking

Tool	Command(s)	Purpose
binwalk	—	Extract and analyze firmware images.
Bytehist	<code>bytehist</code>	Generate byte-usage-histograms for all types of files with a focus on PE files.
ClamAV	<code>clamscan</code> , <code>freshclam</code>	Scan files for malware signatures.
TrID	<code>trid</code> , <code>tridupdate</code>	Identify file type using signatures.
UPX	<code>upx</code>	Pack and unpack PE files.

Use Artificial Intelligence / General

Tool	Command(s)	Purpose
GhidraAssistMCP	—	MCP server for AI-assisted reverse engineering in Ghidra.
OpenCode	—	Open-source AI coding agent for the terminal.
ProcmonMCP	—	MCP server that lets AI assistants analyze Process Monitor (Procmon) XML captures.
radare2	<code>r2</code> , <code>rasm2</code> , <code>rabin2</code> , <code>rahash2</code> , <code>rafind2</code> , <code>r2ai</code> , <code>decai</code> , <code>pdg</code>	Examine binary files, including disassembling and debugging. Includes r2ai and decai plugins for LLM-powered analysis (A)
REMnux MCP Server	<code>remnux-mcp-server</code>	MCP server for using the REMnux malware analysis toolkit via AI assistants.
x64dbg Automate MCP (OpenCode skills)	—	Drive x64dbg on a remote Windows VM from OpenCode on REMnux, with eight AI commands for tracing, unpacking, state snapsh

View Or Edit Files / General

Tool	Command(s)	Purpose
dos2unix	<code>dos2unix</code> , <code>mac2unix</code> , <code>unix2dos</code> , <code>unix2mac</code>	Convert text files with Windows or macOS line breaks to Unix line breaks and vice versa.
Evince	<code>evince</code>	View documents in a variety of formats, including PDF.
feh	—	View images.
ImageMagick	<code>display</code> , <code>convert</code> , <code>mogrify</code> , <code>etc</code>	View and manipulate image and related files.
Mermaid Viewer	—	View Mermaid diagrams, such as AI-generated code-analysis workflow diagrams, in a local browser.
SciTE	—	Edit text files.
VBinDiff	<code>vbindiff</code>	Compare binary files.

Tool	Command(s)	Purpose
Visual Studio Code	<code>code</code>	Powerful source code editor.
wxHexEditor	—	Hex editor.

Kali Linux

Crypto Stego

Tool	Command(s)	Purpose
aesfix	<code>aesfix</code>	Tool for correcting bit errors in an AES key schedule.
aeskeyfind	<code>aeskeyfind</code>	Tool for locating AES keys in a captured memory image.
ccrypt	<code>ccrypt</code>	Secure encryption and decryption of files and streams.
steghide	<code>steghide</code>	Steganography hiding tool.
stegosuite	<code>stegosuite</code>	Steganography tool to hide information in image files.
stegsnow	<code>stegsnow</code>	Steganography using ASCII files.

Database

Tool	Command(s)	Purpose
jsql-injection	<code>jsql-injection</code>	Java tool for automatic database injection.
mdbtools	<code>mdbtools</code>	JET / MS Access database (MDB) tools.
oscanner	<code>oscanner</code>	Oracle assessment framework.
sidguesser	<code>sidguesser</code>	Guesses sids against an Oracle database.
sqldict	<code>sqldict</code>	Dictionary attack tool for SQL Server.
sqlitebrowser	<code>sqlitebrowser</code>	GUI editor for SQLite databases.
sqlmap	<code>sqlmap</code>	Automatic SQL injection tool.
sqlninja	<code>sqlninja</code>	SQL server injection and takeover tool.
sqlsus	<code>sqlsus</code>	MySQL injection tool.
tnscmd10g	<code>tnscmd10g</code>	Tool to prod the oracle tnslsnr process.

Exploitation

Tool	Command(s)	Purpose
armitage	<code>armitage</code>	Cyber attack management for Metasploit.
beef-xss	<code>beef-xss</code>	Browser Exploitation Framework (BeEF).
exploitdb	<code>exploitdb</code>	Searchable Exploit Database archive.

Tool	Command(s)	Purpose
metasploit-framework	<code>metasploit-framework</code>	Framework for exploit development and vulnerability research.
msfpayload	<code>msfpayload</code>	MSFvenom Payload Creator (MSFPayload).
set	<code>set</code>	Social-Engineer Toolkit.
shellnoob	<code>shellnoob</code>	Shellcode writing toolkit.
sqlmap	<code>sqlmap</code>	Automatic SQL injection tool.
termineter	<code>termineter</code>	Smart meter testing framework.

Forensics

Tool	Command(s)	Purpose
7zip	<code>7zip</code>	7-Zip file archiver with a high compression ratio.
afflib-tools	<code>afflib-tools</code>	Advanced Forensics Format Library (utilities).
apktool	<code>apktool</code>	Tool for reverse engineering Android apk files.
autopsy	<code>autopsy</code>	Graphical interface to SleuthKit.
binwalk	<code>binwalk</code>	Tool library for analyzing binary blobs and executable code.
binwalk3	<code>binwalk3</code>	Tool library for analyzing binary blobs and executable code.
bulk-extractor	<code>bulk-extractor</code>	
bytecode-viewer	<code>bytecode-viewer</code>	Java 8+ Jar & Android APK Reverse Engineering Suite.
cabextract	<code>cabextract</code>	Microsoft Cabinet file unpacker.
chkrootkit	<code>chkrootkit</code>	Rootkit detector.
creddump7	<code>creddump7</code>	Python tool to extract credentials and secrets from Windows registry hives.
dc3dd	<code>dc3dd</code>	Patched version of GNU dd with forensic features.
dcfldd	<code>dcfldd</code>	Enhanced version of dd for forensics and security.
ddrescue	<code>ddrescue</code>	Data recovery and protection tool.
dumpzilla	<code>dumpzilla</code>	Mozilla browser forensic tool.
edb-debugger	<code>edb-debugger</code>	Cross platform x86/x86-64 debugger.
ewf-tools	<code>ewf-tools</code>	Collection of tools for reading and writing EWF files.
exifprobe	<code>exifprobe</code>	Read metadata from digital pictures.

Tool	Command(s)	Purpose
exiv2	<code>exiv2</code>	EXIF/IPTC/XMP metadata manipulation tool.
ext3grep	<code>ext3grep</code>	Tool to help recover deleted files on ext3 filesystems.
ext4magic	<code>ext4magic</code>	Recover deleted files from ext3 or ext4 partitions.
extundelete	<code>extundelete</code>	Utility to recover deleted files from ext3/ext4 partition.
fcrackzip	<code>fcrackzip</code>	Password cracker for zip archives.
firmware-mod-kit	<code>firmware-mod-kit</code>	Deconstruct and reconstruct firmware images.
foremost	<code>foremost</code>	Forensic program to recover lost files.
forensic-artifacts	<code>forensic-artifacts</code>	Knowledge base of forensic artifacts (data files).
forensics-colorize	<code>forensics-colorize</code>	Show differences between files using color graphics.
galleta	<code>galleta</code>	Internet Explorer cookie forensic analysis tool.
gdb	<code>gdb</code>	GNU Debugger.
gpart	<code>gpart</code>	Guess PC disk partition table, find lost partitions.
gparted	<code>gparted</code>	GNOME partition editor.
grokevt	<code>grokevt</code>	Scripts for reading Microsoft Windows event log files.
guymager	<code>guymager</code>	Forensic imaging tool based on Qt.
hashdeep	<code>hashdeep</code>	Recursively compute hashsums or piecewise hashings.
inetsim	<code>inetsim</code>	Software suite for simulating common internet services.
jadx	<code>jadx</code>	Dex to Java decompiler.
javasnoop	<code>javasnoop</code>	Intercept Java applications locally.
libhivex-bin	<code>libhivex-bin</code>	Utilities for reading and writing Windows Registry hives.
libsmali-java	<code>libsmali-java</code>	Assembler/disassembler for Android's dex format.
lvm2	<code>lvm2</code>	Linux Logical Volume Manager.
lynis	<code>lynis</code>	Security auditing tool for Unix based systems.
mac-robber	<code>mac-robber</code>	Collects data about allocated files in mounted filesystems.

Tool	Command(s)	Purpose
magicrescue	<code>magicrescue</code>	Recover files by looking for magic bytes.
md5deep	<code>md5deep</code>	
mdbtools	<code>mdbtools</code>	JET / MS Access database (MDB) tools.
memdump	<code>memdump</code>	Utility to dump memory contents to standard output.
metacam	<code>metacam</code>	Extract EXIF information from digital camera files.
missidentify	<code>missidentify</code>	Find win32 applications.
myrescue	<code>myrescue</code>	Rescue data from damaged disks.
nasm	<code>nasm</code>	General-purpose x86 assembler.
nasty	<code>nasty</code>	Tool which helps you to recover your GPG passphrase.
ollydbg	<code>ollydbg</code>	32-bit assembler level analysing debugger.
parted	<code>parted</code>	Disk partition manipulator.
pasco	<code>pasco</code>	Internet Explorer cache forensic analysis tool.
pdf-parser	<code>pdf-parser</code>	Parses PDF files to identify fundamental elements.
pdfid	<code>pdfid</code>	Scans PDF files for certain PDF keywords.
plaso	<code>plaso</code>	Super timeline all the things -- metapackage.
polenum	<code>polenum</code>	Extracts the password policy from a Windows system.
pst-utils	<code>pst-utils</code>	Tools for reading Microsoft Outlook PST files.
python3-capstone	<code>python3-capstone</code>	Lightweight multi-architecture disassembly framework - Python bindings.
python3-dfdatetime	<code>python3-dfdatetime</code>	Digital Forensics date and time library for Python 3.
python3-dfvfs	<code>python3-dfvfs</code>	Digital Forensics Virtual File System.
python3-dfwinreg	<code>python3-dfwinreg</code>	Digital Forensics Windows Registry library for Python 3.
python3-distorm3	<code>python3-distorm3</code>	Powerful disassembler library for x86/AMD64 binary streams (Python3 bindings).
radare2	<code>radare2</code>	Free and advanced command line hexadecimal editor.
readpe	<code>readpe</code>	Command-line tools to manipulate Windows PE files.

Tool	Command(s)	Purpose
recoverdm	<code>recoverdm</code>	Recover files on disks with damaged sectors.
recoverjpeg	<code>recoverjpeg</code>	Recover JFIF (JPEG) pictures and MOV movies.
recstudio	<code>recstudio</code>	
reglookup	<code>reglookup</code>	Utility to analysis for Windows NT-based registry.
regripper	<code>regripper</code>	Perform forensic analysis of registry hives.
rephrase	<code>rephrase</code>	Specialized passphrase recovery tool for GnuPG.
rifiuti	<code>rifiuti</code>	MS Windows recycle bin analysis tool.
rifiuti2	<code>rifiuti2</code>	Replacement for rifiuti, a MS Windows recycle bin analysis tool.
rizin-cutter	<code>rizin-cutter</code>	Reverse engineering platform powered by rizin.
rkhunter	<code>rkhunter</code>	Rootkit, backdoor, sniffer and exploit scanner.
rsakeyfind	<code>rsakeyfind</code>	Locates BER-encoded RSA private keys in memory images.
rz-ghidra	<code>rz-ghidra</code>	Ghidra decompiler and sleigh disassembler for rizin.
safecopy	<code>safecopy</code>	Data recovery tool for problematic or damaged media.
samdump2	<code>samdump2</code>	Dump Windows 2k/NT/XP password hashes.
scalpel	<code>scalpel</code>	Fast filesystem-independent file recovery.
scrounge-ntfs	<code>scrounge-ntfs</code>	Data recovery program for NTFS filesystems.
sleuthkit	<code>sleuthkit</code>	Tools for forensics analysis on volume and filesystem data.
sqlitebrowser	<code>sqlitebrowser</code>	GUI editor for SQLite databases.
ssdeep	<code>ssdeep</code>	Recursive piecewise hashing tool.
tcpdump	<code>tcpdump</code>	Command-line network traffic analyzer.
tcpflow	<code>tcpflow</code>	TCP flow recorder.
tcpick	<code>tcpick</code>	TCP stream sniffer and connection tracker.
tcpreplay	<code>tcpreplay</code>	Tool to replay saved tcpdump files at arbitrary speeds.
truecrack	<code>truecrack</code>	Bruteforce password cracker for TrueCrypt volumes.

Tool	Command(s)	Purpose
undbx	<code>undbx</code>	Tool to extract, recover and undelete e-mail messages from .dbx files.
unhide	<code>unhide</code>	Forensic tool to find hidden processes and ports.
unrar	<code>unrar</code>	
upx-ucl	<code>upx-ucl</code>	Efficient live-compressor for executables.
vinetto	<code>vinetto</code>	Forensics tool to examine Thumbs.db files.
wce	<code>wce</code>	
winregfs	<code>winregfs</code>	Windows registry FUSE filesystem.
wireshark	<code>wireshark</code>	Network traffic analyzer - graphical interface.
xmount	<code>xmount</code>	Tool for crossmounting between disk image formats.
xplico	<code>xplico</code>	Network Forensic Analysis Tool (NFAT).
yara	<code>yara</code>	Pattern matching swiss knife for malware researchers.

Fuzzing

Tool	Command(s)	Purpose
afl++	<code>afl++</code>	Instrumentation-driven fuzzer for binary formats.
sfuzz	<code>sfuzz</code>	Black Box testing utilities.
spike	<code>spike</code>	Network protocol fuzzer.
wfuzz	<code>wfuzz</code>	Web application bruteforcer.

Information Gathering

Tool	Command(s)	Purpose
Otrace	<code>Otrace</code>	Traceroute tool that can run within an existing TCP connection.
arping	<code>arping</code>	Sends IP and/or ARP pings (to the MAC address).
braa	<code>braa</code>	Mass SNMP scanner.
dmitry	<code>dmitry</code>	Deepmagic Information Gathering Tool.
dnsenum	<code>dnsenum</code>	Tool to enumerate domain DNS information.
dnsmap	<code>dnsmap</code>	DNS domain name brute forcing tool.
dnsrecon	<code>dnsrecon</code>	Powerful DNS enumeration script.
dnstracer	<code>dnstracer</code>	Trace DNS queries to the source.

Tool	Command(s)	Purpose
dnswalk	dnswalk	Checks dns zone information using nameserver lookups.
enum4linux	enum4linux	Enumerates info from Windows and Samba systems.
fierce	fierce	Domain DNS scanner.
firewalk	firewalk	Active reconnaissance network security tool.
fping	fping	Sends ICMP ECHO_REQUEST packets to network hosts.
fragrouter	fragrouter	IDS evasion toolkit.
ftester	ftester	Tool for testing firewalls and Intrusion Detection System (IDS).
hping3	hping3	Active Network Smashing Tool.
ike-scan	ike-scan	Discover and fingerprint IKE hosts (IPsec VPN Servers).
intrace	intrace	Traceroute-like application piggybacking on existing TCP connections.
irpas	irpas	
lbd	lbd	Load balancer detector.
legion	legion	Semi-automated network penetration testing tool.
maltego	maltego	
masscan	masscan	TCP port scanner.
metagoofil	metagoofil	Tool designed for extracting metadata of public documents.
nbtscan	nbtscan	Scan networks searching for NetBIOS information.
ncat	ncat	NMAP netcat reimplemention.
netdiscover	netdiscover	Active/passive network address scanner using ARP requests.
netmask	netmask	Helps determine network masks.
nmap	nmap	The Network Mapper.
onesixtyone	onesixtyone	Fast and simple SNMP scanner.
p0f	p0f	Passive OS fingerprinting tool.
qsslcaudit	qsslcaudit	Test SSL/TLS clients how secure they are.
recon-ng	recon-ng	Web Reconnaissance framework written in Python.
smbmap	smbmap	Handy SMB enumeration tool.

Tool	Command(s)	Purpose
smtp-user-enum	smtp-user-enum	Username guessing tool for the SMTP service.
snmpcheck	snmpcheck	SNMP service enumeration tool.
ssldump	ssldump	SSLv3/TLS network protocol analyzer.
sslh	sslh	Applicative protocol multiplexer.
ssllscan	ssllscan	Tests SSL/TLS enabled services to discover supported cipher suites.
sslyze	sslyze	Fast and full-featured SSL scanner.
swaks	swaks	SMTP command-line test tool.
thc-ipv6	thc-ipv6	The Hacker Choice's IPv6 Attack Toolkit.
theharvester	theharvester	Tool for gathering e-mail accounts and subdomain names from public sources.
tlssled	tlssled	Evaluates the security of a target SSL/TLS (HTTPS) server.
twofi	twofi	Twitter words of interest.
unicornscan	unicornscan	Userland distributed TCP/IP stack.
urlcrazy	urlcrazy	
wafw00f	wafw00f	Identify and fingerprint Web Application Firewall products.
zenmap	zenmap	

Passwords

Tool	Command(s)	Purpose
cewl	cewl	Custom word list generator.
chntpw	chntpw	NT SAM password recovery utility.
cisco-auditing-tool	cisco-auditing-tool	Scans Cisco routers for vulnerabilities.
cmospwd	cmospwd	Decrypt BIOS passwords from CMOS.
crackle	crackle	Crack and decrypt BLE encryption.
creddump7	creddump7	Python tool to extract credentials and secrets from Windows registry hives.
crunch	crunch	Tool for creating wordlist.
fcrackzip	fcrackzip	Password cracker for zip archives.
freerdp3-x11	freerdp3-x11	RDP client for Windows Terminal Services (X11 client transitional package).
gpp-decrypt	gpp-decrypt	Group Policy Preferences decrypter.
hash-identifier	hash-identifier	Tool to identify hash types.
hashcat	hashcat	World's fastest and most advanced password recovery utility.

Tool	Command(s)	Purpose
hashcat-utils	hashcat-utils	Set of small utilities for advanced password cracking.
hashid	hashid	Identify the different types of hashes used to encrypt data.
hydra	hydra	Very fast network logon cracker.
hydra-gtk	hydra-gtk	Very fast network logon cracker - GTK+ based GUI.
john	john	Active password cracking tool.
johnny	johnny	GUI for John the Ripper.
maskprocessor	maskprocessor	High-performance word generator with a per-position configurable charset.
medusa	medusa	Fast, parallel, modular, login brute-forcer for network services.
mimikatz	mimikatz	Uses admin rights on Windows to display passwords in plaintext.
ncrack	ncrack	High-speed network authentication cracking tool.
onesixtyone	onesixtyone	Fast and simple SNMP scanner.
ophcrack	ophcrack	Microsoft Windows password cracker using rainbow tables (gui).
ophcrack-cli	ophcrack-cli	Microsoft Windows password cracker using rainbow tables (cmdline).
pack	pack	Password analysis and cracking kit.
pack2	pack2	Password analysis and cracking kit 2.
passing-the-hash	passing-the-hash	Patched tools to use password hashes as authentication input.
patator	patator	Multi-purpose brute-forcer.
pdfcrack	pdfcrack	PDF files password cracker.
pipal	pipal	Statistical analysis on password dumps.
polenum	polenum	Extracts the password policy from a Windows system.
rainbowcrack	rainbowcrack	Rainbow table password cracker.
rarcrack	rarcrack	Password cracker for rar archives.
rcracki-mt	rcracki-mt	Version of rcrack that supports hybrid and indexed tables.
rsmangler	rsmangler	Wordlist mangling tool.
samdump2	samdump2	Dump Windows 2k/NT/XP password hashes.

Tool	Command(s)	Purpose
seclists	<code>seclists</code>	Collection of multiple types of security lists.
sipcrack	<code>sipcrack</code>	SIP login dumper/cracker.
sipvicious	<code>sipvicious</code>	Tools to audit SIP based VoIP systems.
smbmap	<code>smbmap</code>	Handy SMB enumeration tool.
sqldict	<code>sqldict</code>	Dictionary attack tool for SQL Server.
statsprocessor	<code>statsprocessor</code>	Word generator based on per-position Markov chains.
sucrack	<code>sucrack</code>	Multithreaded su bruteforcer.
thc-pptp-bruter	<code>thc-pptp-bruter</code>	THC PPTP Brute Force.
truecrack	<code>truecrack</code>	Bruteforce password cracker for TrueCrypt volumes.
twofi	<code>twofi</code>	Twitter words of interest.
wordlists	<code>wordlists</code>	Contains the rockyou wordlist.

Post Exploitation

Tool	Command(s)	Purpose
cymothoa	<code>cymothoa</code>	Stealth backdooring tool.
dbd	<code>dbd</code>	Netcat clone with encryption.
dns2tcp	<code>dns2tcp</code>	TCP-over-DNS tunnel server and client.
exe2hexbat	<code>exe2hexbat</code>	Convert EXE to bat.
iodine	<code>iodine</code>	Tool for tunneling IPv4 data through a DNS server.
laudanum	<code>laudanum</code>	Collection of injectable web files.
mimikatz	<code>mimikatz</code>	Uses admin rights on Windows to display passwords in plaintext.
miredo	<code>miredo</code>	Teredo IPv6 tunneling through NATs.
nishang	<code>nishang</code>	Collection of PowerShell scripts and payloads.
powersploit	<code>powersploit</code>	PowerShell Post-Exploitation Framework.
proxychains4	<code>proxychains4</code>	Redirect connections through socks/http proxies (proxychains-ng).
proxytunnel	<code>proxytunnel</code>	Help SSH and other protocols through HTTP(S) proxies.
ptunnel	<code>ptunnel</code>	Tunnel TCP connections over ICMP packets.
pwnat	<code>pwnat</code>	NAT to NAT client-server communication.
sbd	<code>sbd</code>	Secure backdoor for Linux and Windows.

Tool	Command(s)	Purpose
shellter	shellter	
ssllh	ssllh	Applicative protocol multiplexer.
stunnel4	stunnel4	Universal SSL tunnel for network daemons - compatibility package.
udptunnel	udptunnel	Tunnel UDP packets over a TCP connection.
veil	veil	Generates payloads to bypass anti-virus solutions.
webacoo	webacoo	Web backdoor cookie script kit.
weevely	weevely	Stealth tiny web shell.

Reverse Engineering

Tool	Command(s)	Purpose
apktool	apktool	Tool for reverse engineering Android apk files.
bytecode-viewer	bytecode-viewer	Java 8+ Jar & Android APK Reverse Engineering Suite.
clang	clang	C, C++ and Objective-C compiler (LLVM based), clang binary.
dex2jar	dex2jar	Tools to work with android .dex and java .class files.
edb-debugger	edb-debugger	Cross platform x86/x86-64 debugger.
jadx	jadx	Dex to Java decompiler.
javasnoop	javasnoop	Intercept Java applications locally.
jd-gui	jd-gui	GUI Java .class decompiler.
metasploit-framework	metasploit-framework	Framework for exploit development and vulnerability research.
ollydbg	ollydbg	32-bit assembler level analysing debugger.
radare2	radare2	Free and advanced command line hexadecimal editor.
recstudio	recstudio	
rizin	rizin	Reverse engineering framework and command-line toolset.
rizin-cutter	rizin-cutter	Reverse engineering platform powered by rizin.
rz-ghidra	rz-ghidra	Ghidra decompiler and sleigh disassembler for rizin.

Sniffing Spoofing

Tool	Command(s)	Purpose
above	<code>above</code>	Network security sniffer for finding vulnerabilities in the network.
bettercap	<code>bettercap</code>	Complete, modular, portable and easily extensible MITM framework.
darkstat	<code>darkstat</code>	Network traffic analyzer.
dnscraf	<code>dnscraf</code>	DNS proxy for penetration testers.
driftnet	<code>driftnet</code>	Picks out and displays images from network traffic.
dsniff	<code>dsniff</code>	Various tools to sniff network traffic for cleartext insecurities.
ettercap-graphical	<code>ettercap-graphical</code>	Ettercap GUI-enabled executable.
ferret-sidejack	<code>ferret-sidejack</code>	Monitors data and extracts interesting data.
fiked	<code>fiked</code>	Cisco VPN attack tool.
hamster-sidejack	<code>hamster-sidejack</code>	Sidejacking tool.
hexinject	<code>hexinject</code>	Versatile packet injector and sniffer.
isr-evilgrade	<code>isr-evilgrade</code>	Evilgrade framework.
macchanger	<code>macchanger</code>	Utility for manipulating the MAC address of network interfaces.
mitmproxy	<code>mitmproxy</code>	SSL-capable man-in-the-middle HTTP proxy.
netsniff-ng	<code>netsniff-ng</code>	Linux network packet sniffer toolkit.
rebind	<code>rebind</code>	DNS rebinding tool.
responder	<code>responder</code>	LLMNR/NBT-NS/mDNS Poisoner.
sniffjoke	<code>sniffjoke</code>	Transparent TCP connection scrambler.
sslsniff	<code>sslsniff</code>	SSL/TLS man-in-the-middle attack tool.
sslsplit	<code>sslsplit</code>	Transparent and scalable SSL/TLS interception.
tcpflow	<code>tcpflow</code>	TCP flow recorder.
tcpreplay	<code>tcpreplay</code>	Tool to replay saved tcpdump files at arbitrary speeds.
wifi-honey	<code>wifi-honey</code>	Wi-Fi honeypot.
wireshark	<code>wireshark</code>	Network traffic analyzer - graphical interface.

Vulnerability

Tool	Command(s)	Purpose
afl++	<code>afl++</code>	Instrumentation-driven fuzzer for binary formats.

Tool	Command(s)	Purpose
bed	<code>bed</code>	A network protocol fuzzer.
cisco-auditing-tool	<code>cisco-auditing-tool</code>	Scans Cisco routers for vulnerabilities.
cisco-global-exploiter	<code>cisco-global-exploiter</code>	Simple and fast Cisco exploitation tool.
cisco-ocs	<code>cisco-ocs</code>	Mass Cisco scanner.
cisco-torch	<code>cisco-torch</code>	Cisco device scanner.
copy-router-config	<code>copy-router-config</code>	Copies Cisco configs via SNMP.
dhcpig	<code>dhcpig</code>	DHCP exhaustion script using scapy network library.
enumiax	<code>enumiax</code>	IAX protocol username enumerator.
gvm	<code>gvm</code>	Remote network security auditor - metapackage and useful scripts.
iaxflood	<code>iaxflood</code>	VoIP flooder tool.
inviteflood	<code>inviteflood</code>	SIP/SDP INVITE message flooding over UDP/IP.
legion	<code>legion</code>	Semi-automated network penetration testing tool.
lynis	<code>lynis</code>	Security auditing tool for Unix based systems.
nikto	<code>nikto</code>	
nmap	<code>nmap</code>	The Network Mapper.
ohrwurm	<code>ohrwurm</code>	RTP fuzzer.
peass	<code>peass</code>	Privilege Escalation Awesome Scripts SUITE.
protos-sip	<code>protos-sip</code>	SIP test suite.
rtpbreak	<code>rtpbreak</code>	Detects, reconstructs, and analyzes RTP sessions.
rtpflood	<code>rtpflood</code>	Tool to flood any RTP device.
rtpinsertsound	<code>rtpinsertsound</code>	Inserts audio into a specified stream.
rtpmixsound	<code>rtpmixsound</code>	Mixes pre-recorded audio in real-time.
sctpscan	<code>sctpscan</code>	SCTP network scanner for discovery and security.
sfuzz	<code>sfuzz</code>	Black Box testing utilities.
siege	<code>siege</code>	HTTP regression testing and benchmarking utility.
siparmyknife	<code>siparmyknife</code>	SIP fuzzing tool.
sipp	<code>sipp</code>	Traffic generator for the SIP protocol.
sipsak	<code>sipsak</code>	SIP Swiss army knife.
sipvicious	<code>sipvicious</code>	Tools to audit SIP based VoIP systems.

Tool	Command(s)	Purpose
slowhttptest	<code>slowhttptest</code>	Application layer Denial of Service attacks simulation tool.
spike	<code>spike</code>	Network protocol fuzzer.
t50	<code>t50</code>	Multi-protocol packet injector tool.
thc-ssl-dos	<code>thc-ssl-dos</code>	Stress tester for the SSL handshake.
unix-privesc-check	<code>unix-privesc-check</code>	Script to check for simple privilege escalation vectors.
voiphopper	<code>voiphopper</code>	Runs a VLAN hop security test.
yersinia	<code>yersinia</code>	Network vulnerabilities check software.

Web

Tool	Command(s)	Purpose
apache-users	<code>apache-users</code>	Enumerate usernames on systems with Apache UserDir module.
apache2	<code>apache2</code>	Apache HTTP Server.
beef-xss	<code>beef-xss</code>	Browser Exploitation Framework (BeEF).
burpsuite	<code>burpsuite</code>	Platform for security testing of web applications.
cadaver	<code>cadaver</code>	Command-line WebDAV client.
commix	<code>commix</code>	Automated All-in-One OS Command Injection and Exploitation Tool.
cutycapt	<code>cutycapt</code>	Utility to capture WebKit's rendering of a web page.
davtest	<code>davtest</code>	Testing tool for WebDAV servers.
default-mysql-server	<code>default-mysql-server</code>	MySQL database server binaries and system database setup (metapackage).
dirb	<code>dirb</code>	URL bruteforcing tool.
dirbuster	<code>dirbuster</code>	Web server directory brute-forcer.
dotdotpwn	<code>dotdotpwn</code>	Directory Traversal Fuzzer.
eyewitness	<code>eyewitness</code>	Rapid web application triage tool.
ferret-sidejack	<code>ferret-sidejack</code>	Monitors data and extracts interesting data.
ftester	<code>ftester</code>	Tool for testing firewalls and Intrusion Detection System (IDS).
hakrawler	<code>hakrawler</code>	Web crawler designed for easy, quick discovery of endpoints and assets.
hamster-sidejack	<code>hamster-sidejack</code>	Sidejacking tool.
heartleech	<code>heartleech</code>	Scanner detecting systems vulnerable to the heartbleed OpenSSL bug.

Tool	Command(s)	Purpose
httpprint	httpprint	
httrack	httrack	Copy websites to your computer (Offline browser).
hydra	hydra	Very fast network logon cracker.
hydra-gtk	hydra-gtk	Very fast network logon cracker - GTK+ based GUI.
jboss-autopwn	jboss-autopwn	JBoss script for obtaining remote shell access.
joomscan	joomscan	OWASP Joomla Vulnerability Scanner Project.
jsql-injection	jsql-injection	Java tool for automatic database injection.
laudanum	laudanum	Collection of injectable web files.
lbd	lbd	Load balancer detector.
maltego	maltego	
medusa	medusa	Fast, parallel, modular, login brute-forcer for network services.
mitmproxy	mitmproxy	SSL-capable man-in-the-middle HTTP proxy.
ncrack	ncrack	High-speed network authentication cracking tool.
nikto	nikto	
nishang	nishang	Collection of PowerShell scripts and payloads.
nmap	nmap	The Network Mapper.
osscanner	osscanner	Oracle assessment framework.
owasp-mantra-ff	owasp-mantra-ff	
padbuster	padbuster	Script for performing Padding Oracle attacks.
paros	paros	Web application proxy.
patator	patator	Multi-purpose brute-forcer.
php	php	Server-side, HTML-embedded scripting language (default).
php-mysql	php-mysql	MySQL module for PHP [default].
proxychains4	proxychains4	Redirect connections through socks/http proxies (proxychains-ng).
proxytunnel	proxytunnel	Help SSH and other protocols through HTTP(S) proxies.
qsslcaudit	qsslcaudit	Test SSL/TLS clients how secure they are.

Tool	Command(s)	Purpose
redsocks	<code>redsocks</code>	Arbitrary TCP connection redirector to a SOCKS or HTTPS proxy server.
sidguesser	<code>sidguesser</code>	Guesses sids against an Oracle database.
siege	<code>siege</code>	HTTP regression testing and benchmarking utility.
skipfish	<code>skipfish</code>	Fully automated, active web application security reconnaissance tool.
slowhttptest	<code>slowhttptest</code>	Application layer Denial of Service attacks simulation tool.
sqldict	<code>sqldict</code>	Dictionary attack tool for SQL Server.
sqlitebrowser	<code>sqlitebrowser</code>	GUI editor for SQLite databases.
sqlmap	<code>sqlmap</code>	Automatic SQL injection tool.
sqlninja	<code>sqlninja</code>	SQL server injection and takeover tool.
sqlsus	<code>sqlsus</code>	MySQL injection tool.
ssldump	<code>ssldump</code>	SSLv3/TLS network protocol analyzer.
sslh	<code>sslh</code>	Applicative protocol multiplexer.
sslscan	<code>sslscan</code>	Tests SSL/TLS enabled services to discover supported cipher suites.
sslsniff	<code>sslsniff</code>	SSL/TLS man-in-the-middle attack tool.
sslsplit	<code>sslsplit</code>	Transparent and scalable SSL/TLS interception.
sslyze	<code>sslyze</code>	Fast and full-featured SSL scanner.
stunnel4	<code>stunnel4</code>	Universal SSL tunnel for network daemons - compatibility package.
thc-ssl-dos	<code>thc-ssl-dos</code>	Stress tester for the SSL handshake.
tlssled	<code>tlssled</code>	Evaluates the security of a target SSL/TLS (HTTPS) server.
tnscmd10g	<code>tnscmd10g</code>	Tool to prod the oracle tnslsnr process.
uniscan	<code>uniscan</code>	LFI, RFI, and RCE vulnerability scanner.
wafw00f	<code>wafw00f</code>	Identify and fingerprint Web Application Firewall products.
wapiti	<code>wapiti</code>	Web application vulnerability scanner.
watobo	<code>watobo</code>	Semi-automated web application scanner.
webacoo	<code>webacoo</code>	Web backdoor cookie script kit.
webscarab	<code>webscarab</code>	Web application review tool.
webshells	<code>webshells</code>	Collection of webshells.
weevely	<code>weevely</code>	Stealth tiny web shell.
wfuzz	<code>wfuzz</code>	Web application bruteforcer.

Tool	Command(s)	Purpose
whatweb	<code>whatweb</code>	Next generation web scanner.
wireshark	<code>wireshark</code>	Network traffic analyzer - graphical interface.
wpscan	<code>wpscan</code>	
xsser	<code>xsser</code>	XSS testing framework.
zaproxy	<code>zaproxy</code>	Testing tool for finding vulnerabilities in web applications.

FLARE-VM

All Packages (choco)

Tool	Command(s)	Purpose
010editor	<code>010editor</code>	Professional text and hex editor with Binary Templates technology.
7zip	<code>7zip</code>	7-Zip file archiver with a high compression ratio.
advanced-installer	<code>advanced-installer</code>	Advanced Installer is a Windows installer authoring tool that can be used to analyze MSI files.
angr	<code>angr</code>	Angr is a multi-architecture binary analysis toolkit providing features like disassembly, IR lifting, program instr...
apimonitor	<code>apimonitor</code>	API Monitor lets you monitor and control API calls made by applications and services.
apktool	<code>apktool</code>	Tool for reverse engineering Android apk files.
asar	<code>asar</code>	Asar decompresses .asar archives.
autoit-ripper	<code>autoit-ripper</code>	Extracts compiled Autolt scripts from PE executables.
binaryninja	<code>binaryninja</code>	Binary Ninja is an interactive decompiler, disassembler, debugger, and binary analysis platform built by reverse en...
bindiff	<code>bindiff</code>	A comparison tool for binary files that assists in quickly finding differences and similarities in disassembled code.
blobrunner	<code>blobrunner</code>	BlobRunner is a simple tool to quickly debug shellcode extracted during malware analysis.

Tool	Command(s)	Purpose
blobrunner64	<code>blobrunner64</code>	BlobRunner is a simple tool to quickly debug shellcode extracted during malware analysis.
bytecodeviewer	<code>bytecodeviewer</code>	A lightweight user-friendly Java/Android Bytecode Viewer, Decompiler and more.
capa	<code>capa</code>	Capa detects capabilities in executable files. You run it against a PE file or shellcode and it tells you what it t...
capa-explorer-web	<code>capa-explorer-web</code>	Web interface for exploring and understanding capa results.
chrome.extensions	<code>chrome.extensions</code>	A package for multiple useful chrome extensions from the Chrome webstore.
cmdr	<code>cmdr</code>	Metapackage for cmdr.
codetrack	<code>codetrack</code>	A free .NET Performance Profile and Execution Analyzer.
cryptotester	<code>cryptotester</code>	Utility tool for performing cryptanalysis with a focus on ransomware cryptography.
cutter	<code>cutter</code>	Disconnect routed IP connections.
cyberchef	<code>cyberchef</code>	Cyber Swiss Army Knife.
cygwin	<code>cygwin</code>	Wrapper for cygwin and useful cygwin packages.
de4dot-cex	<code>de4dot-cex</code>	A de4dot fork with full support for vanilla ConfuserEx.
dependencywalker	<code>dependencywalker</code>	Scans PE files and builds a hierarchical tree diagram of all dependent modules.
dex2jar	<code>dex2jar</code>	Tools to work with android .dex and java .class files.
didier-stevens-beta	<code>didier-stevens-beta</code>	Beta versions of Didier Stevens's software.
didier-stevens-suite	<code>didier-stevens-suite</code>	Tools collection by Didier Stevens.
die	<code>die</code>	Detect It Easy, or abbreviated "DIE" is a program for determining types of files.
dll-to-exe	<code>dll-to-exe</code>	Converts a DLL into a ready-to-use EXE.
dnlib	<code>dnlib</code>	.NET module/assembly reader/writer library.
dnspyex	<code>dnspyex</code>	DnSpyEx is a unofficial continuation of the dnSpy project which is a debugger and .NET assembly editor. You can use...
dotdumper	<code>dotdumper</code>	An automatic unpacker and logger for DotNet Framework targeting files.
dotnet3.5	<code>dotnet3.5</code>	

Tool	Command(s)	Purpose
exeinfope	exeinfope	Displays metadata for a variety of file types and identifies many executable packers.
explorersuite	explorersuite	A suite of tools including CFF Explorer and a process viewer.
extreme_dumper	extreme_dumper	.NET Assembly Dumper from memory of processes.
ezviewer	ezviewer	Standalone, zero dependency viewer for .doc, .docx, .xls, .xlsx, .txt, .log, .rtf, .otd, .htm, .html, .mht, .csv, a...
fakenet-ng	fakenet-ng	FakeNet-NG is a next generation dynamic network analysis tool for malware analysts and penetration testers.
fiddler	fiddler	Intercepts, decrypts, and analyzes HTTPS traffic.
file	file	Recognize the type of data in a file using "magic" numbers.
floss	floss	FLOSS uses advanced static analysis techniques to automatically deobfuscate strings from malware binaries. You can...
garbageman	garbageman	A set of tools designed for .NET heap analysis.
ghidra	ghidra	Software Reverse Engineering Framework.
goresym	goresym	Go symbol recovery tool.
gostringungarbler	gostringungarbler	GoStringUngarbler deobfuscates strings in Go binaries obfuscated by garble.
hashmyfiles	hashmyfiles	HashMyFiles is small utility that allows you to calculate the MD5 and SHA1 hashes of one or more files in your syst...
hollowshunter	hollowshunter	Scans all running processes. Recognizes and dumps a variety of potentially malicious implants (replaced/implanted P...
hxd	hxd	Freeware hex editor.
ida.plugin.capa	ida.plugin.capa	Capa explorer is an IDAPython plugin that integrates capa with IDA Pro.
ida.plugin.comida	ida.plugin.comida	IDA Plugin that help analyzing modules using COM.
ida.plugin.delphihelper	ida.plugin.delphihelper	DelphiHelper.
ida.plugin.dereferencing	ida.plugin.dereferencing	IDA Pro plugin that implements new registers and stack views.
ida.plugin.diaphora	ida.plugin.diaphora	Diaphora is a program diffing IDA plugin.
ida.plugin.flare	ida.plugin.flare	IDA Pro plugins used by the FLARE team.

Tool	Command(s)	Purpose
ida.plugin.flare-emu	<code>ida.plugin.flare-emu</code>	A user friendly scriptable emulation framework that supports multiple binary analysis tools.
ida.plugin.hashdb	<code>ida.plugin.hashdb</code>	Malware string hash lookup plugin for IDA Pro.
ida.plugin.hrtng	<code>ida.plugin.hrtng</code>	IDA Pro plugin with features such as decryption, automation, deobfuscation, patching, lib code recognition and pseu...
ida.plugin.ifl	<code>ida.plugin.ifl</code>	Interactive Functions List IDA Pro plugin.
ida.plugin.xray	<code>ida.plugin.xray</code>	Hexrays decompiler plugin that colorizes and filters the decompiler's output based on regular expressions.
ida.plugin.xrefer	<code>ida.plugin.xrefer</code>	Custom navigation interface within IDA.
idafree	<code>idafree</code>	Free version of IDA, a powerful Interactive DisAssembler and debugger.
idr	<code>idr</code>	Interactive Delphi Reconstructor.
ifpstools	<code>ifpstools</code>	IFPSTools.NET: tools for working with RemObject PascalScript compiled bytecode files.
ilspy	<code>ilspy</code>	ILSpy is a .NET assembly browser and decompiler.
innoextract	<code>innoextract</code>	Tool for extracting data from an Inno Setup installer.
innounp	<code>innounp</code>	Unpacker for Inno Setup installers.
internet_detector	<code>internet_detector</code>	Tool that changes the background and a taskbar icon if it detects internet connectivity.
ipython	<code>ipython</code>	A powerful interactive Python shell.
isd	<code>isd</code>	Inno Setup Decompiler.
js-beautify	<code>js-beautify</code>	JavaScript beautifier and deobfuscator.
js-deobfuscator	<code>js-deobfuscator</code>	Deobfuscator to remove common JS obfuscation techniques.
keystone	<code>keystone</code>	OpenStack identity service.
libraries.python3	<code>libraries.python3</code>	Metapackage to install common Python 3.9 libraries.
magika	<code>magika</code>	Magika is an AI powered file type detection tool that uses deep learning to provide accurate detection.
malware-jail	<code>malware-jail</code>	Sandbox for semi-automatic Javascript malware analysis, deobfuscation and payload extraction.

Tool	Command(s)	Purpose
map	<code>map</code>	Handful of small utility type applications useful for analyzing malicious code.
microsoft-office	<code>microsoft-office</code>	Microsoft Office ProPlusRetail.
nasm	<code>nasm</code>	General-purpose x86 assembler.
net-reactor-slayer	<code>net-reactor-slayer</code>	NETReactorSlayer is an open source (GPLv3) deobfuscator and unpacker for Eziriz .NET Reactor.
nmap	<code>nmap</code>	The Network Mapper.
notepadplusplus	<code>notepadplusplus</code>	Wrapper for Notepad++.
notepadpp.plugin.compare	<code>notepadpp.plugin.compare</code>	ComparePlus plugin for Notepad++.
notepadpp.plugin.jstool	<code>notepadpp.plugin.jstool</code>	A JavaScript (JSON) tool for Notepad++ (formerly JSMinNpp).
notepadpp.plugin.xmltools	<code>notepadpp.plugin.xmltools</code>	XML Tools plugin for Notepad++.
obfuscator-io-deobfuscator	<code>obfuscator-io-deobfuscator</code>	A deobfuscator for scripts obfuscated by Obfuscator.io.
offvis	<code>offvis</code>	The Microsoft Office Visualization Tool (OffVis) is a tool from Microsoft that helps understanding the Microsoft Of...
onenoteanalyzer	<code>onenoteanalyzer</code>	A C# based tool for analyzing malicious OneNote documents.
pdbsym	<code>pdbsym</code>	Download PDBs.
pdfstreamdumper	<code>pdfstreamdumper</code>	PDFStreamDumper is a free, open source tool to analyze malicious PDF documents.
pe_unmapper	<code>pe_unmapper</code>	Small tool to convert between the PE alignments (raw and virtual).
pebear	<code>pebear</code>	Delivers fast and flexible "first view" for malware analysts.
peid	<code>peid</code>	PEiD detects most common packers, cryptors and compilers for PE files.
pesieve	<code>pesieve</code>	Pe-sieve recognizes and dumps variety of implants within the scanned process.
pestudio	<code>pestudio</code>	The goal of pestudio is to spot artifacts of executable files in order to ease and accelerate Malware Initial Asses...
pkg-unpacker	<code>pkg-unpacker</code>	Unpacker for pkg applications.
pma-labs	<code>pma-labs</code>	Binaries for the book Practical Malware Analysis.
procdot	<code>procdot</code>	Creates visual graphs from procmon output.

Tool	Command(s)	Purpose
processdump	<code>processdump</code>	Process Dump is a Windows reverse-engineering command-line tool to dump malware memory components back to disk for...
psnotify	<code>psnotify</code>	A POC tool to fight .NET anti-dumping tricks.
pycdas	<code>pycdas</code>	Python byte-code disassembler.
pycdc	<code>pycdc</code>	Python decompiler.
pylingual	<code>pylingual</code>	Python decompiler for modern Python versions.
rat-king-parser	<code>rat-king-parser</code>	Multi-family RAT config parser/extractor.
recaf	<code>recaf</code>	Java bytecode editor.
reg_export	<code>reg_export</code>	A CLI that exports the raw content of a registry value to a file.
regcool	<code>regcool</code>	In addition to all the features that you can find in RegEdit and RegEdt32, RegCool adds many powerful features that...
regshot	<code>regshot</code>	Regshot is a small, free and open-source registry compare utility that allows you to quickly take a snapshot of you...
resourcehacker	<code>resourcehacker</code>	Resource Hacker is a resource editor for 32bit and 64bit Windows applications.
rundotnetdll	<code>rundotnetdll</code>	A simple utility to list all methods of a given .NET Assembly and to invoke them.
scdbg	<code>scdbg</code>	Scdbg is an emulation based shellcode API logger and debugger.
sclauncher	<code>sclauncher</code>	A small program to load 32-bit shellcode and allow for execution or debugging. Can also output PE files from shellc...
sclauncher64	<code>sclauncher64</code>	A small program to load 64-bit shellcode and allow for execution or debugging. Can also output PE files from shellc...
sfextract	<code>sfextract</code>	Command-line utility to extract files from single file bundles in .NET.
shellcode_launcher	<code>shellcode_launcher</code>	Shellcode launcher utility.
sysinternals	<code>sysinternals</code>	Sysinternals suite.
systeminformer	<code>systeminformer</code>	A free, powerful, multi-purpose tool that helps you monitor system resources, debug software and detect malware.
ttd	<code>ttd</code>	Time travel debugging command line utility.
uncompyle6	<code>uncompyle6</code>	A decompiler for Python 1.0-3.8.

Tool	Command(s)	Purpose
uniextract2	<code>uniextract2</code>	Universal Extractor 2 is an unofficial updated and extended version of the original UniExtract by Jared Breland.
unpyc3	<code>unpyc3</code>	A decompiler for Python 3.7+.
upx	<code>upx</code>	UPX is a free, secure, portable, extendable, high-performance executable packer for several executable formats.
vb-decompiler-lite	<code>vb-decompiler-lite</code>	VB Decompiler is a decompiler for Visual Basic, VB.NET and C# applications.
vbdec	<code>vbdec</code>	VBDec works as a VB6 disassembler, PCode debugger, structure viewer for all vb6 executables, and can generate IDA s...
vcbuildtools	<code>vcbuildtools</code>	Metapackage that requires the dependencies below: - visualstudio2017buildtools - visualstudio2017-workload-vctools.
vcredist-all	<code>vcredist-all</code>	
vscode	<code>vscode</code>	VSCode is a modern, open-source code editor.
vscode.extension.jupyter	<code>vscode.extension.jupyter</code>	Jupyter notebook support, interactive programming and computing that supports Intellisense, debugging and more.
vscode.extension.python	<code>vscode.extension.python</code>	Python language support with extension access points for IntelliSense (Pylance), Debugging (Python Debugger), linti...
windbg	<code>windbg</code>	WinDbg is a debugger that can be used to analyze crash dumps, debug live user-mode and kernel-mode code, and examin...
windows-terminal	<code>windows-terminal</code>	Windows Terminal is a new, modern, feature-rich, productive terminal application for command-line users.
wireshark	<code>wireshark</code>	Network traffic analyzer - graphical interface.
x64dbg	<code>x64dbg</code>	An open-source x64/x32 debugger for Windows.
x64dbg.plugin.dbgchild	<code>x64dbg.plugin.dbgchild</code>	DbgChild is an x64dbg plugin to automatically attach to spawned child processes.
x64dbg.plugin.ollydumpex	<code>x64dbg.plugin.ollydumpex</code>	This plugin is process memory dumper for OllyDbg and Immunity Debugger. OllyDumpEx = OllyDump + PE Dumper - obsolet...

Tool	Command(s)	Purpose
x64dbg.plugin.scyllahide	x64dbg.plugin.scyllahide	ScyllaHide is an advanced open-source x64/x86 user mode Anti-Anti-Debug library.
x64dbg.plugin.x64dbgpy	x64dbg.plugin.x64dbgpy	Automating x64dbg using Python.
yara	yara	Pattern matching swiss knife for malware researchers.

SIFT Workstation

Forensic Packages

Tool	Command(s)	Purpose
absent	absent	
aeskeyfind	aeskeyfind	Tool for locating AES keys in a captured memory image.
afflib-tools	afflib-tools	Advanced Forensics Format Library (utilities).
aircrack-ng	aircrack-ng	Wireless WEP/WPA cracking utilities.
android-sdk-platform-tools	android-sdk-platform-tools	Tools for interacting with an Android platform.
arp-scan	arp-scan	Arp scanning and fingerprinting tool.
autopsy	autopsy	Graphical interface to SleuthKit.
avfs	avfs	Virtual filesystem to access archives, disk images, remote locations.
aws-cli	aws-cli	
bless	bless	A full featured hexadecimal editor.
blt	blt	Graphics extension library for Tcl/Tk - runtime.
bulk-extractor	bulk-extractor	
cabextract	cabextract	Microsoft Cabinet file unpacker.
ccrypt	ccrypt	Secure encryption and decryption of files and streams.
chromium-browser	chromium-browser	Transitional package - chromium-browser -> chromium snap.
clamav	clamav	Anti-virus utility for Unix - command-line interface.
claude-code	claude-code	
cmospwd	cmospwd	Decrypt BIOS passwords from CMOS.
cryptcat	cryptcat	Lightweight version netcat extended with twofish encryption.

Tool	Command(s)	Purpose
cryptsetup	cryptsetup	Disk encryption support - startup scripts.
dc3dd	dc3dd	Patched version of GNU dd with forensic features.
dcfldd	dcfldd	Enhanced version of dd for forensics and security.
default-jre	default-jre	Standard Java or Java compatible Runtime.
disktype	disktype	Detection of content format of a disk or disk image.
dislocker	dislocker	Read/write encrypted BitLocker volumes.
docker	docker	Transitional package.
dos2unix	dos2unix	Convert text file line endings between CRLF and LF.
dotnet	dotnet	
driftnet	driftnet	Picks out and displays images from network traffic.
dsniff	dsniff	Various tools to sniff network traffic for cleartext insecurities.
e2fsprogs	e2fsprogs	Ext2/ext3/ext4 file system utilities.
ent	ent	Pseudorandom number sequence test program.
epic5	epic5	Epic irc client, version 5.
etherape	etherape	Graphical network monitor.
ettercap-graphical	ettercap-graphical	Ettercap GUI-enabled executable.
ewf-tools	ewf-tools	Collection of tools for reading and writing EWF files.
exfat-extras	exfat-extras	
exfat-fuse	exfat-fuse	Read and write exFAT driver for FUSE.
exif	exif	Command-line utility to show EXIF information in JPEG files.
extundelete	extundelete	Utility to recover deleted files from ext3/ext4 partition.
fdupes	fdupes	Identifies duplicate files within given directories.
feh	feh	Imlib2 based image viewer.
file	file	Recognize the type of data in a file using "magic" numbers.
flex	flex	Fast lexical analyzer generator.
foremost	foremost	Forensic program to recover lost files.

Tool	Command(s)	Purpose
gawk	<code>gawk</code>	GNU awk, a pattern scanning and processing language.
gdb	<code>gdb</code>	GNU Debugger.
gddrescue	<code>gddrescue</code>	GNU data recovery tool.
ghex	<code>ghex</code>	GNOME Hex editor for files.
graphviz	<code>graphviz</code>	Rich set of graph drawing tools.
grepcidr	<code>grepcidr</code>	Filter IP addresses matching IPv4/IPv6 CIDR/network specification.
gthumb	<code>gthumb</code>	Image viewer and browser.
gzrt	<code>gzrt</code>	Gzip recovery toolkit.
hashdeep	<code>hashdeep</code>	Recursively compute hashsums or piecewise hashings.
hexedit	<code>hexedit</code>	Viewer and editor in hexadecimal or ASCII for files or devices.
hydra	<code>hydra</code>	Very fast network logon cracker.
hydra-gtk	<code>hydra-gtk</code>	Very fast network logon cracker - GTK+ based GUI.
init	<code>init</code>	Metapackage ensuring an init system is installed.
ipython3	<code>ipython3</code>	Enhanced interactive Python 3 shell.
jq	<code>jq</code>	Lightweight and flexible command-line JSON processor.
kdiff3	<code>kdiff3</code>	Compares and merges 2 or 3 files or directories.
kpartx	<code>kpartx</code>	Create device mappings for partitions.
lft	<code>lft</code>	Layer-four traceroute.
lvm2	<code>lvm2</code>	Linux Logical Volume Manager.
magnus	<code>magnus</code>	Very simple screen magnifier.
mdadm	<code>mdadm</code>	Tool for managing Linux MD devices (software RAID).
mtd-utils	<code>mtd-utils</code>	Memory Technology Device Utilities.
nbd-client	<code>nbd-client</code>	Network Block Device protocol - client.
nbtscan	<code>nbtscan</code>	Scan networks searching for NetBIOS information.
netcat	<code>netcat</code>	TCP/IP swiss army knife -- transitional package.
netpbm	<code>netpbm</code>	Graphics conversion tools between image formats.
netsted	<code>netsted</code>	Network packet-altering stream editor.

Tool	Command(s)	Purpose
netwox	<code>netwox</code>	Networking utilities.
nfdump	<code>nfdump</code>	Netflow capture daemon.
ngrep	<code>ngrep</code>	Grep for network traffic.
nikto	<code>nikto</code>	
ntfs-3g	<code>ntfs-3g</code>	Read/write NTFS driver for FUSE.
okular	<code>okular</code>	Universal document viewer.
onboard	<code>onboard</code>	Simple On-screen Keyboard.
open-iscsi	<code>open-iscsi</code>	ISCSI initiator tools.
openjdk	<code>openjdk</code>	Metapackage for OpenJDK to ensure all packages use the same OpenJDK version.
ophcrack	<code>ophcrack</code>	Microsoft Windows password cracker using rainbow tables (gui).
ophcrack-cli	<code>ophcrack-cli</code>	Microsoft Windows password cracker using rainbow tables (cmdline).
orca	<code>orca</code>	Scriptable screen reader.
outguess	<code>outguess</code>	Universal steganographic tool.
p0f	<code>p0f</code>	Passive OS fingerprinting tool.
p7zip-full	<code>p7zip-full</code>	7z and 7za file archivers with high compression ratio.
patch	<code>patch</code>	Apply a diff file to an original.
pdftk-java	<code>pdftk-java</code>	Pdftk port to java - a tool for manipulating PDF documents.
perl	<code>perl</code>	Larry Wall's Practical Extraction and Report Language.
pev	<code>pev</code>	Text-based tool to analyze PE files.
pff-tools	<code>pff-tools</code>	Utilities for MS Outlook PAB, PST and OST files.
phonon	<code>phonon</code>	
pkg-config	<code>pkg-config</code>	Manage compile and link flags for libraries (transitional package).
plaso-tools	<code>plaso-tools</code>	
powershell	<code>powershell</code>	PowerShell is an automation and configuration management platform.
pst-utils	<code>pst-utils</code>	Tools for reading Microsoft Outlook PST files.
pv	<code>pv</code>	Shell pipeline element to meter data passing through.
python-flowgrep	<code>python-flowgrep</code>	

Tool	Command(s)	Purpose
python3-debian	python3-debian	Python 3 modules to work with Debian-related data formats.
python3-dfvfs	python3-dfvfs	Digital Forensics Virtual File System.
python3-fuse	python3-fuse	Python bindings for FUSE (Filesystems in Userspace) (Python 3 package).
python3-keyrings-alt	python3-keyrings-alt	
python3-m2crypto	python3-m2crypto	Python wrapper for the OpenSSL library (Python 3 modules).
python3-magic	python3-magic	Python3 interface to the libmagic file type identification library.
python3-pefile	python3-pefile	Portable Executable (PE) parsing module for Python.
python3-plaso	python3-plaso	Super timeline all the things -- Python 3.
python3-pypff	python3-pypff	Python 3 bindings for libpff.
python3-pyqt5	python3-pyqt5	Python 3 bindings for Qt5.
python3-pytsk3	python3-pytsk3	
python3-redis	python3-redis	Python bindings for Redis, a persistent key-value database.
python3-setuptools-rust	python3-setuptools-rust	Setuptools Rust extension plugin.
python3-tk	python3-tk	Tkinter - Writing Tk applications with Python 3.x.
python3-tsk	python3-tsk	Python Bindings for The Sleuth Kit.
python3-virtualenv	python3-virtualenv	Python virtual environment creator.
python3-wxgtk4	python3-wxgtk4	
python3-xlswriter	python3-xlswriter	Python 3 module for creating Excel XLSX files.
python3-yara	python3-yara	Python 3 bindings for YARA.
qemu	qemu	Fast processor emulator, dummy package.
qemu-utils	qemu-utils	QEMU utilities.
radare2	radare2	Free and advanced command line hexadecimal editor.
rar	rar	
rsakeyfind	rsakeyfind	Locates BER-encoded RSA private keys in memory images.
safecopy	safecopy	Data recovery tool for problematic or damaged media.
samdump2	samdump2	Dump Windows 2k/NT/XP password hashes.
scalpel	scalpel	Fast filesystem-independent file recovery.

Tool	Command(s)	Purpose
silversearcher-ag	<code>silversearcher-ag</code>	Very fast grep-like program, alternative to ack.
sleuthkit	<code>sleuthkit</code>	Tools for forensics analysis on volume and filesystem data.
socat	<code>socat</code>	Multipurpose relay for bidirectional data transfer.
squashfs-tools	<code>squashfs-tools</code>	Tool to create and append to squashfs filesystems.
ssdeep	<code>ssdeep</code>	Recursive piecewise hashing tool.
ssldump	<code>ssldump</code>	SSLv3/TLS network protocol analyzer.
sslsniff	<code>sslsniff</code>	SSL/TLS man-in-the-middle attack tool.
stunnel4	<code>stunnel4</code>	Universal SSL tunnel for network daemons - compatibility package.
swig	<code>swig</code>	Generate scripting interfaces to C/C++ code.
tcl	<code>tcl</code>	Tool Command Language (default version) - shell.
tcpflow	<code>tcpflow</code>	TCP flow recorder.
tcpick	<code>tcpick</code>	TCP stream sniffer and connection tracker.
tcpreplay	<code>tcpreplay</code>	Tool to replay saved tcpdump files at arbitrary speeds.
tcpslice	<code>tcpslice</code>	Extract pieces of and/or glue together tcpdump files.
tcpstat	<code>tcpstat</code>	Network interface statistics reporting tool.
tcptrace	<code>tcptrace</code>	Tool for analyzing tcpdump output.
tcptrack	<code>tcptrack</code>	TCP connection tracker, with states and speeds.
tcpxtract	<code>tcpxtract</code>	Extract files from network traffic based on file signatures.
testdisk	<code>testdisk</code>	Partition scanner and disk recovery tool, and PhotoRec file recovery tool.
tofromdos	<code>tofromdos</code>	Converts DOS <-> Unix text files, alias tofromdos.
transmission	<code>transmission</code>	Lightweight BitTorrent client.
ugrep	<code>ugrep</code>	Faster grep with an interactive query UI.
unity-control-center	<code>unity-control-center</code>	Utilities to configure the GNOME desktop.
unrar	<code>unrar</code>	
upx-ucl	<code>upx-ucl</code>	Efficient live-compressor for executables.

Tool	Command(s)	Purpose
vbindiff	<code>vbindiff</code>	Visual binary diff, visually compare binary files.
virtuoso-minimal	<code>virtuoso-minimal</code>	High-performance database - core dependency package.
vmfs-tools	<code>vmfs-tools</code>	Tools to access VMFS filesystems.
winbind	<code>winbind</code>	Service to resolve user and group information from Windows NT servers.
wine	<code>wine</code>	Windows API implementation - standard suite.
wireshark	<code>wireshark</code>	Network traffic analyzer - graphical interface.
xdot	<code>xdot</code>	Interactive viewer for Graphviz dot files.
xfsprogs	<code>xfsprogs</code>	Utilities for managing the XFS filesystem.
xmount	<code>xmount</code>	Tool for crossmounting between disk image formats.
zenity	<code>zenity</code>	Display graphical dialog boxes from shell scripts.
zlib1g-dev	<code>zlib1g-dev</code>	Compression library - development.

Security Onion

DB/Search

Tool	Command(s)	Purpose
Elasticsearch	—	NoSQL index for logs/metadata/alerts
ILM	—	Index Lifecycle Management (aging/rolling/deletion)
Kibana	—	Search analytics & dashboards

Detection

Tool	Command(s)	Purpose
ATT&CK Navigator	—	Visual defensive-coverage matrix
ElastAlert 2	—	Continuous alert-indicator scanner
Playbook	—	Custom detections mapped to MITRE ATT&CK
Sigma / Sigma-CLI	—	Translate YAML detection rules to Elastic queries
so-idstools	—	Manage/update Suricata ET rules

Diagnostics

Tool	Command(s)	Purpose
Grafana	—	Infra/node-health dashboards
InfluxDB	—	Time-series store for hardware metrics
Telegraf	—	Hardware health/CPU/mem/disk agent

File Analysis

Tool	Command(s)	Purpose
DomainStats	—	Domain-age checks to flag malicious sites
FreqServer	—	Entropy analysis to detect DGAs
Strelka	—	Real-time file extraction & YARA/file carving

Host/EDR

Tool	Command(s)	Purpose
Elastic Agent	—	Host log/metric/audit collector
Elastic Fleet	—	Central agent config & deployment
OpenCanary	—	Network honeypot / decoy services
Osquery	—	Live endpoint queries via SQL
Wazuh	—	HIDS - File Integrity Monitoring & rootkit checks

IR/AI

Tool	Command(s)	Purpose
CyberChef	—	Visual data decode/parse/analysis
Onion AI Assistant	—	AI triage assistance for analysts
Security Onion Console (SOC)	—	Web UI for threat hunting & grid management
TheHive	—	Collaborative IR & case management

Identity/Infra

Tool	Command(s)	Purpose
ManagerHype	—	Hypervisor integration for VM mapping
Nginx (so-nginx)	—	Reverse proxy for internal web traffic
Ory Kratos & Dex	—	IAM - SSO/auth/sessions
so-apt-cacher-ng	—	Local package caching proxy
so-firewall	—	iptables/nftables wrapper for inter-node comms

Log Routing

Tool	Command(s)	Purpose
Filebeat	—	Lightweight log shipper
Logstash	—	Data pipeline - ECS normalize & enrich
Redis	—	Log ingestion queue/broker
Rsyslog / Syslog-ng	—	Ingest traditional syslog from appliances

Network Visibility

Tool	Command(s)	Purpose
Sensoroni	—	Backend API agent retrieving/extracting PCAPs
Stenographer	—	High-speed raw full packet capture daemon
Suricata	—	Signature-based IDS/IPS & packet capture
Zeek	—	Network traffic analyzer & protocol metadata logging

Sources

- `remnux_docs` — <https://raw.githubusercontent.com/REMnux/docs/master/discover-the-tools>
- `kali_control` — <https://gitlab.com/kalilinux/packages/kali-meta/-/raw/kali/master/debian/control>
- `flare_config` — <https://raw.githubusercontent.com/mandiant/flare-vm/main/config.xml>

Tool Reference

One page per tool. Every option below was read off the real binary and is checked by the linter.

affcat

Kit	Kali Linux · SIFT Workstation
Capability	Inspect or mount a forensic image container
Version	affcat version 3.7.20
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Stream the raw contents of an AFF container to stdout, so tools that cannot read AFF can be fed the image through a pipe rather than a full conversion to raw.

When you'd reach for this

An analyst reaches for affcat when examining AFF files to extract or verify specific segments, pages, or sectors of a disk image, often after acquiring or recovering data, as it allows precise control over output with options like `-s`, `-p`, `-S`, and `-b`, making it preferable for targeted forensic analysis over broader tools like `affverify` or `affstats`.

Sources: <https://www.kali.org/tools/afflib/>

Synopsis

```
affcat [options] infile [... more infiles]
```

Options

All 7 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
<code>-q</code>	—	--- quiet; don't print to STDERR if a page is skipped	
<code>-n</code>	—	--- noisy; tell when pages are skipped.	
<code>-l</code>	—	--- List all of the segment names	
<code>-L</code>	—	--- List segment names, lengths, and args	

Flag	Argument	What it does	When you would use it
<code>-d</code>	—	--- debug. Print the page numbers to stderr as data goes to stdout	
<code>-b</code>	—	--- Output BADFALG for bad blocks (default is NULLs)	
<code>-v</code>	—	--- Just print the version number and exit.	

See also

`ewfinfo`, `ewfmount`, `ewfverify`, `ewfexport`, `affinfo`, `img_stat`, `ntfs-3g`, `vshadowinfo`

affconvert

Kit	Kali Linux · SIFT Workstation
Capability	Image a disk, volume or device
Version	affconvert version 3.7.20
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

← [Capability index](#) · [Kit tool list](#)

Purpose

Convert between AFF and raw images in either direction. The usual reason is a tool that only reads one of them; keep the original, because converting to raw discards the metadata and hashes AFF was carrying.

When you'd reach for this

When an analyst needs to convert files between RAW and AFF formats, they use affconvert, often after acquiring raw data or before processing with other AFF tools, as it directly handles format conversion unlike affcopy which focuses on reordering and recompression.

Sources: <https://www.kali.org/tools/afflib/>

Synopsis

```
affconvert [options] file1 [... files]
```

Common invocations

```
# Convert between raw disk images and AFF files
affconvert file1.raw
# Convert between AFF and raw formats for data processing
affconvert file1.raw file2.raw file3.raw
```

Options

No option definitions could be parsed from this tool's help output. It may be subcommand-driven or have no flags; needs manual review.

See also

`dc3dd`, `dcfldd`, `dd`, `ewfacquire`

affinfo

Kit	Kali Linux · SIFT Workstation
Capability	Inspect or mount a forensic image container
Version	affinfo version 3.7.20
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Print the metadata of an AFF forensic container: acquisition details, hashes and segment layout. The AFF equivalent of `ewfinfo`, and the fastest way to see what an .aff file claims about its own provenance.

When you'd reach for this

An analyst reaches for affinfo when examining an AFF file to validate its integrity or extract metadata, often after acquiring the file or before decrypting it with a passphrase; they choose it for its specific capabilities to verify hashes, list segments, and identify file structures, which are critical for forensic analysis.

Sources: <https://www.kali.org/tools/afflib/>

Synopsis

```
affinfo [options] infile
-a = print ALL segments (normally data segments are suppressed)
-b = print how many bad blocks in each segment (implies -a)
-i = identify the files, don't do info on them.
-w = wide output; print more than 1 line if necessary.
-s segment = Just print information about 'segment'.
```

Options

No option definitions could be parsed from this tool's help output. It may be subcommand-driven or have no flags; needs manual review.

See also

`ewfinfo`, `ewfmount`, `ewfverify`, `ewfexport`, `affcat`, `img_stat`, `ntfs-3g`, `vshadowinfo`

bdeinfo

Kit	SIFT Workstation (libyal)
Capability	Inspect or mount a forensic image container
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Use bdeinfo to determine information about a BitLocker Drive

When you'd reach for this

An analyst reaches for bdeinfo after confirming a partition is BitLocker encrypted using hex dumps and tools like fls, which cannot recognize BitLocker; they run it to extract volume details like the recovery key and encryption algorithm, preferring it over SleuthKit because SleuthKit does not support BitLocker.

Sources: <https://bebinary4n6.blogspot.com/2020/01/how-to-handle-bitlocker-encrypted.html> · <https://www.aldeid.com/wiki/Category:Encryption/Bitlocker>

Synopsis

```
bdeinfo [ -k keys ] [ -o offset ] [ -p password ]  
[ -r password ] [ -s filename ] [ -hvV ] source
```

Common invocations

```
# Retrieve BitLocker encryption details from a drive  
bdeinfo -p Password /dev/sda1  
# Extract BitLocker encryption details from disk image  
bdeinfo -o $((512*128)) image.dd
```

Options

All 8 options parsed from the captured help text; 5 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	shows this help	

Flag	Argument	What it does	When you would use it
<code>-k</code>	—	the full volume encryption key and tweak key formatted in base16 and separated by a : character e.g. FVEK:TWEAK	
<code>-o</code>	—	specify the volume offset in bytes	An analyst would use the -o flag with bdeinfo when specifying the correct byte offset for a BitLocker-encrypted volume in a disk image, after confirming the volume's presence through hexdump analysis or partition layout checks.
<code>-p</code>	—	specify the password/passphrase	An analyst would use the -p flag when providing a password to access a BitLocker-encrypted volume during forensic examination.
<code>-r</code>	—	specify the recovery password	An analyst would use the -r flag when providing a recovery password to access a BitLocker Drive Encrypted volume.
<code>-s</code>	—	specify the file containing the startup key. typically this file has the extension .BEK	An analyst would use the -s flag when providing a file containing a startup key to unlock a BitLocker-encrypted volume.
<code>-v</code>	—	verbose output to stderr	An analyst would use the -v flag when they need detailed error, verbose, or debug output during the analysis of a BitLocker Drive Encrypted volume.
<code>-V</code>	—	print version	

See also

`ewfinfo`, `ewfmount`, `ewfverify`, `ewfexport`, `affinfo`, `affcat`, `img_stat`, `ntfs-3g`

dc3dd

Kit	Kali Linux · SIFT Workstation
Capability	Image a disk, volume or device
Version	dc3dd (dc3dd) 7.2.646
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

dc3dd [OPTION 1] [OPTION 2] ... [OPTION N]

When you'd reach for this

An analyst reaches for dc3dd when imaging a disk with inline hash verification required, such as during forensic acquisition of a test device or system they own, ensuring the image matches the source through SHA-256 hashing logged in the acquisition file. They may run it after confirming authorization and before handing the image to another analyst, preferring it over similar tools for its detailed logging of hash, byte count, and completion status, which is critical for chain of custody and integrity verification.

Sources: <https://github.com/plaintext-security/plaintext-labs/blob/main/forensics/02-acquisition-imaging/lab.md> · <https://www.kali.org/tools/dc3dd/>

Synopsis

Common invocations

```
# Verify data integrity during forensic imaging
dc3dd if=/var/log/messages of=/tmp/dc3dd hash=sha512
```

Options

All 3 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
<code>--help</code>	—	display this help and exit	

Flag	Argument	What it does	When you would use it
<code>--version</code>	—	output version information and exit	
<code>--flags</code>	—	display compile-time flags and exit	

See also

`dcfldd`, `dd`, `ewfacquire`, `affconvert`

dcfldd

Kit	Kali Linux · SIFT Workstation
Capability	Image a disk, volume or device
Version	dcfldd (dcfldd) 1.9
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

← [Capability index](#) · [Kit tool list](#)

Purpose

Enhanced version of dd for forensics and security.

When you'd reach for this

An analyst reaches for dcfldd when imaging a drive to create a forensic copy, ensuring the source device's permissions are restricted with chmod before use to prevent accidental writes; they run it with hash=md5,sha1 and hashlog to verify data integrity, preferring it over dd due to its safety features like multiple output paths and explicit write-blocking warnings.

Sources: <https://dfir.blog/imaging-using-dcfldd/> · <https://www.mankier.com/1/dcfldd>

Synopsis

```
dcfldd [OPTION]...
```

Common invocations

```
# Overwrite disk with pattern to erase data securely
dcfldd pattern="00FFAACC" of=/dev/sda
# Create zero-filled test file for analysis
dcfldd if=/dev/zero of=test bs=50M count=2
```

Options

All 2 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
<code>--help</code>	—	display this help and exit	

Flag	Argument	What it does	When you would use it
<code>--version</code>	—	output version information and exit	

See also

`dc3dd`, `dd`, `ewfacquire`, `affconvert`

dd

Kit	Base OS — present on every Linux image
Capability	Image a disk, volume or device
Version	dd (coreutils) 9.1
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

← [Capability index](#) · [Kit tool list](#)

Purpose

Copy data block by block, without interpreting it. In forensics this is the plain-raw imaging tool: it will read a whole device including unallocated space, but it has no hashing, no error recovery and no metadata. Prefer `dc3dd`, `dcfldd` or `ewfacquire` for evidence; reach for `dd` when you need a byte range and nothing else.

When you'd reach for this

An analyst reaches for `dd` when copying data between drives or sanitizing a drive, ensuring the syntax is correct before execution and using a write blocker to prevent accidental data loss; they may run it after verifying the source and target devices to avoid overwriting evidence, preferring `dd` for its direct, low-level data handling and reliability in forensic imaging tasks.

Sources: <https://www.forensicfocus.com/articles/linux-dd-basics/>

Synopsis

```
dd [OPERAND]...  
or: dd OPTION
```

Common invocations

```
# Clone hard disk from source to target  
dd if=/dev/sda of=/dev/sdb  
# Wipe hard disk by overwriting with zeros  
dd if=/dev/zero of=/dev/hda bs=4K conv=noerror,sync
```

Options

All 2 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
<code>--help</code>	—	display this help and exit	
<code>--version</code>	—	output version information and exit	

See also

`dc3dd`, `dcfldd`, `ewfacquire`, `affconvert`

dumpcap

Kit	REMnux · Kali Linux · FLARE-VM · SIFT Workstation
Capability	Capture live network traffic
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.wireshark.org

← [Capability index](#) · [Kit tool list](#)

Purpose

Capture packets to a file. It does nothing else — which is the point.

When you'd reach for this

An analyst reaches for dumpcap when capturing live network traffic with specific conditions like duration, file size, or packet count, often using options like `-a` or `-b` to automate stopping or file rotation; they may run it alongside tshark for analysis or use capinfos afterward to inspect capture files, preferring it over similar tools for its dedicated capture capabilities and precise control over capture parameters.

Sources: <https://docsislab.wordpress.com/packet-capture/wireshark-command-line/> · <https://www.wireshark.org/docs/man-pages/dumpcap.html>

Synopsis

```
dumpcap [options] ...
```

Options

All 48 options parsed from the captured help text; 20 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-i</code>	interface	name or idx of interface (def: first non-loopback), or for remote capturing, use one of these formats: <code>rpcap:/// TCP@:</code>	Interface to capture from. <code>-D</code> lists what is available.
<code>--interface</code>	interface	name or idx of interface (def: first non-loopback), or for remote capturing, use one of these formats: <code>rpcap:/// TCP@:</code>	Interface to capture from. <code>-D</code> lists what is available.

Flag	Argument	What it does	When you would use it
<code>--ifname</code>	name	name to use in the capture file for a pipe from which we're capturing	
<code>--ifdescr</code>	description	description to use in the capture file for a pipe from which we're capturing	
<code>-f</code>	capture filter	packet filter in libpcap filter syntax	Capture filter in BPF syntax. Applied before writing, so anything it excludes is gone permanently.
<code>-s</code>	snaplen	packet snapshot length (def: appropriate maximum)	Snapshot length: truncate each packet. Headers only, when payload must not be recorded.
<code>--snapshot-length</code>	snaplen	packet snapshot length (def: appropriate maximum)	Snapshot length: truncate each packet. Headers only, when payload must not be recorded.
<code>-p</code>	—	don't capture in promiscuous mode	Do not enter promiscuous mode — only traffic for this host.
<code>--no-promiscuous-mode</code>	—	don't capture in promiscuous mode	Do not enter promiscuous mode — only traffic for this host.
<code>-I</code>	—	capture in monitor mode, if available	Monitor mode, for 802.11 management and control frames.
<code>--monitor-mode</code>	—	capture in monitor mode, if available	Monitor mode, for 802.11 management and control frames.
<code>-B</code>	buffer size	size of kernel buffer in MiB (def: 2MiB)	Kernel buffer size in MiB. Raise it first when a fast link reports drops; the default is small for modern traffic.
<code>--buffer-size</code>	buffer size	size of kernel buffer in MiB (def: 2MiB)	Kernel buffer size in MiB. Raise it first when a fast link reports drops; the default is small for modern traffic.
<code>-y</code>	link type	link layer type (def: first appropriate)	Force the link-layer type.
<code>--linktype</code>	link type	link layer type (def: first appropriate)	Force the link-layer type.
<code>-D</code>	—	print list of interfaces and exit	List interfaces and exit.
<code>--list-interfaces</code>	—	print list of interfaces and exit	List interfaces and exit.
<code>-L</code>	—	print list of link-layer types of iface and exit	List link-layer types for the chosen interface.

Flag	Argument	What it does	When you would use it
<code>--list-data-link-types</code>	—	print list of link-layer types of iface and exit	List link-layer types for the chosen interface.
<code>--list-time-stamp-types</code>	—	print list of timestamp types for iface and exit	
<code>-d</code>	—	print generated BPF code for capture filter	Print the compiled BPF for a filter, to check it means what you think before capturing hours of the wrong thing.
<code>-S</code>	—	print statistics for each interface once per second	Print per-interface packet statistics once a second, for confirming traffic is arriving before committing to a long capture.
<code>-M</code>	—	for -D, -L, and -S, produce machine-readable output	Machine-readable output for <code>-D</code> , <code>-L</code> and <code>-S</code> .
<code>-c</code>	packet count	stop after n packets (def: infinite)	Stop after N packets.
<code>-a</code>	autostop cond	duration:NUM - stop after NUM seconds filesize:NUM - stop this file after NUM kB files:NUM - stop after NUM files packets:NUM - stop after NUM packets	Autostop condition — duration, filesize or files.
<code>--autostop</code>	autostop cond	duration:NUM - stop after NUM seconds filesize:NUM - stop this file after NUM kB files:NUM - stop after NUM files packets:NUM - stop after NUM packets	An analyst would use the <code>--autostop</code> flag when they need to automatically halt packet capture after a specified duration, upon reaching a certain number of files, or when a capture file reaches a defined size limit.
<code>-w</code>	filename	name of file to save (def: tempfile)	Output file. Without it, dumpcap writes to a temporary file and tells you where, which is rarely what you meant.
<code>-g</code>	—	enable group read access on the output file(s)	
<code>-b</code>	ringbuffer opt	duration:NUM - switch to next file after NUM secs filesize:NUM - switch to next file after NUM kB files:NUM - ringbuffer: replace after NUM files packets:NUM - ringbuffer: replace after NUM packets in	Ring buffer: roll to a new file on duration, filesize or count. The difference between a capture that runs overnight and one that fills the disk at 3am.

Flag	Argument	What it does	When you would use it
<code>--ring-buffer</code>	ringbuffer opt	duration:NUM - switch to next file after NUM secs filesize:NUM - switch to next file after NUM kB files:NUM - ringbuffer: replace after NUM files packets:NUM - ringbuffer: replace after NUM packets in	Ring buffer: roll to a new file on duration, filesize or count. The difference between a capture that runs overnight and one that fills the disk at 3am.
<code>-n</code>	—	use pcapng format instead of pcap (default)	pcapng output (the default).
<code>-P</code>	—	use libpcap format instead of pcapng	Legacy pcap output, for a tool that cannot read pcapng.
<code>--capture-comment</code>	comment	add a capture comment to the output file (only for pcapng)	
<code>--temp-dir</code>	directory	write temporary files to this directory (default: /tmp)	
<code>--log-level</code>	level	sets the active log level ("critical", "warning", etc.)	
<code>--log-fatal</code>	level	sets level to abort the program ("critical" or "warning")	
<code>--log-domains</code>	[!]list	comma separated list of the active log domains	
<code>--log-debug</code>	[!]list	comma separated list of domains with "debug" level	
<code>--log-noisy</code>	[!]list	comma separated list of domains with "noisy" level	
<code>--log-file</code>	path	file to output messages to (in addition to stderr)	
<code>-N</code>	packet_limit	maximum number of packets buffered within dumpcap	
<code>-C</code>	byte_limit	maximum number of bytes used for buffering packets within dumpcap	
<code>-t</code>	—	use a separate thread per interface	
<code>-q</code>	—	don't report packet capture counts	Quiet — no packet-count updates.
<code>-v</code>	—	print version information and exit	
<code>--version</code>	—	print version information and exit	
<code>-h</code>	—	display this help and exit	
<code>--help</code>	—	display this help and exit	

Gotchas

- This is deliberately minimal: it captures and it does not dissect. That is why it is the right thing to run as the privileged process and why `tshark` shells out to it — the analysis code never needs the capture privilege.
- Filtering here is destructive. A capture filter that was slightly wrong cannot be widened afterwards; capture broadly and filter at analysis time whenever the disk allows it.
- Drops are reported at the end, not during. Check the count before treating a capture as complete — a busy link with the default buffer loses packets silently.

See also

`tshark`

ewfacquire

Kit	Kali Linux · SIFT Workstation
Capability	Image a disk, volume or device
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Acquire a disk, volume or device into an EWF/E01 evidence container, with the case metadata and hashes stored inside the image itself.

Synopsis

```
ewfacquire [ -A codepage ] [ -b number_of_sectors ]  
[ -B number_of_bytes ] [ -c compression_values ]  
[ -C case_number ] [ -d digest_type ] [ -D description ]  
[ -e examiner_name ] [ -E evidence_number ] [ -f format ]  
[ -g number_of_sectors ] [ -l log_filename ]  
[ -m media_type ] [ -M media_flags ] [ -N notes ]
```

Common invocations

```
# Create EWF image from device or file  
ewfacquire /dev/sda  
# Convert split optical disc RAW to EWF image  
ewfacquire -T optical.cue optical.iso  
# Convert RAW image to EWF format  
ewfacquire -c best -m fixed -t myfile -S 1T -u [-q] myfile.raw
```

Options

All 32 options parsed from the captured help text; 30 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-A</code>	—	codepage of header section, options: ascii (default), windows-874, windows-932, windows-936, windows-949, windows-950, windows-1250, windows-1251, windows-1252, windows-1253, windows-1254, windows-125	Header codepage, for non-ASCII metadata.
<code>-b</code>	—	specify the number of sectors to read at once (per chunk), options: 16, 32, 64 (default), 128, 256, 512, 1024, 2048, 4096, 8192, 16384 or 32768	Sectors per chunk. Larger chunks read faster but lose more data to each unreadable sector.
<code>-B</code>	—	specify the number of bytes to acquire (default is all bytes)	Acquire a fixed number of bytes rather than the whole device — with <code>-o</code> , the way to image one region.
<code>-c</code>	—	specify the compression values as: level or method:level compression method options: deflate (default), bzip2 (bzip2 is only supported by EWF2 formats) compression level options: none (default), empty	Trade size against time. best on a slow USB source is often faster overall than none , because the bottleneck is the read, not the CPU.
<code>-C</code>	—	specify the case number (default is case_number).	Case number, stored in the image header.
<code>-d</code>	—	calculate additional digest (hash) types besides md5, options: sha1, sha256	Add sha1 or sha256 alongside the default md5. Worth doing once, at acquisition: md5 alone is increasingly challenged, and rehashing later means reading the whole image again.
<code>-D</code>	—	specify the description (default is description).	Description, stored in the image header.
<code>-e</code>	—	specify the examiner name (default is examiner_name).	Examiner name, stored in the image header.
<code>-E</code>	—	specify the evidence number (default is evidence_number).	Evidence number, stored in the image header.
<code>-f</code>	—	specify the EWF file format to write to, options: ewf, smart, ftk, encase2, encase3, encase4, encase5, encase6 (default), encase7, encase7-v2, linen5, linen6, linen7, ewfx	Choose the EWF variant. encase6 is the safe default for interoperability; pick the format the tool that will read it expects, not the newest one available.

Flag	Argument	What it does	When you would use it
<code>-g</code>	—	specify the number of sectors to be used as error granularity	Error granularity — how much data around a bad sector is discarded. Lower values preserve more of a failing disk at the cost of speed.
<code>-h</code>	—	shows this help	
<code>-l</code>	—	logs acquiry errors and the digest (hash) to the log_filename	Write the acquisition log, including errors and hashes, to a file. This is the record you cite later; do not skip it.
<code>-m</code>	—	specify the media type, options: fixed (default), removable, optical, memory	Record the media type (fixed, removable, optical, memory) in the header.
<code>-M</code>	—	specify the media flags, options: logical, physical (default)	Record whether this is a physical device or a logical volume.
<code>-N</code>	—	specify the notes (default is notes).	Free-text notes, stored in the image header. These five metadata flags are the reason to choose E01 over a raw <code>dd</code> image: the container describes its own provenance.
<code>-o</code>	—	specify the offset to start to acquire (default is 0)	Start at an offset rather than sector 0.
<code>-p</code>	—	specify the process buffer size (default is the chunk size)	Process buffer size — a throughput tuning knob.
<code>-P</code>	—	specify the number of bytes per sector (default is 512) (use this to override the automatic bytes per sector detection)	Override bytes-per-sector. Needed on 4Kn drives where the 512-byte assumption is wrong.
<code>-q</code>	—	quiet shows minimal status information	Minimal status output, for logs.
<code>-r</code>	—	specify the number of retries when a read error occurs (default is 2)	Read retries before giving up on a sector. Raise it for a dying drive; lower it when retries are heating a drive that may not survive the acquisition.
<code>-R</code>	—	resume acquiry at a safe point	Resume an interrupted acquisition at a safe point, instead of restarting a multi-hour read.
<code>-s</code>	—	swap byte pairs of the media data (from AB to BA) (use this for big to little endian conversion and vice versa)	Swap byte pairs for a big-endian source. Rare, and wrong unless you know the source endianness differs.

Flag	Argument	What it does	When you would use it
<code>-S</code>	—	specify the segment file size in bytes (default is 1.4 GiB) (minimum is 1.0 MiB, maximum is 7.9 EiB for encase6 and encase7 format and 1.9 GiB for other formats)	Segment size. The 1.4 GiB default suits FAT32 and optical media; raise it on a modern filesystem to avoid hundreds of fragments.
<code>-t</code>	—	specify the target file (without extension) to write to	Set the output name, without extension — <code>ewfacquire</code> appends <code>.E01</code> , <code>.E02</code> ... itself. The one flag you always pass.
<code>-T</code>	—	specify the file containing the table of contents (TOC) of an optical disc. The TOC file must be in the CUE format.	Supply a CUE file when imaging optical media, so the track layout is preserved.
<code>-u</code>	—	unattended mode (disables user interaction)	Unattended — suppresses the interactive prompts. Required for any scripted or headless acquisition.
<code>-v</code>	—	verbose output to stderr	Verbose diagnostics to stderr — worth capturing alongside <code>-l</code> when a source is throwing read errors.
<code>-V</code>	—	print version	
<code>-w</code>	—	zero sectors on read error (mimic EnCase like behavior)	Zero unreadable sectors instead of aborting, the way EnCase does. Keeps offsets aligned so the filesystem still parses.
<code>-x</code>	—	use the chunk data instead of the buffered read and write functions.	Bypass the buffered read/write path.
<code>-2</code>	—	specify the secondary target file (without extension) to write to	Write a second copy in the same pass. Two independent copies for the cost of one read, which matters when the source is failing and may not survive a second acquisition.

Gotchas

- The case metadata flags default to literal placeholder strings — `case_number`, `examiner_name`, `evidence_number`. Omit them and the image ships with those placeholders recorded as fact, which is worse than an empty field because it looks filled in.
- `-t` takes the name **without** an extension. Passing `image.E01` produces `image.E01.E01`.

- Acquisition is not verification. `ewfacquire` records hashes; proving the image still matches them later is `ewfverify`.

See also

`dc3dd`, `dcfldd`, `dd`, `affconvert`

ewfexport

Kit	Kali Linux · SIFT Workstation
Capability	Inspect or mount a forensic image container
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Convert an EWF/E01 image to raw, or to another EWF format, including extracting a subset of it.

When you'd reach for this

An analyst reaches for ewfexport when they need to extract specific data from an EWF image, such as a partition or converting an E01 to another format, often after acquiring the image with ewfacquire; they may use it before further analysis to isolate relevant data, preferring it over other tools due to its flexibility in specifying byte ranges and output formats.

Sources: <https://bromiley.medium.com/tooling-thursday-libewf-ec27b4564c2a> · <https://forensics.wiki/libewf/>

Synopsis

```
ewfexport [ -A codepage ] [ -b number_of_sectors ]  
[ -B number_of_bytes ] [ -c compression_values ]  
[ -d digest_type ] [ -f format ] [ -l log_filename ]  
[ -o offset ] [ -p process_buffer_size ]  
[ -S segment_file_size ] [ -t target ] [ -hqsuvVwx ] ewf_files
```

Common invocations

```
# Convert EWF image to RAW or another EWF format  
ewfexport image.E01
```

Options

All 19 options parsed from the captured help text; 17 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-A</code>	—	codepage of header section, options: ascii (default), windows-874, windows-932, windows-936, windows-949, windows-950, windows-1250, windows-1251, windows-1252, windows-1253, windows-1254, windows-125	Header codepage.
<code>-b</code>	—	specify the number of sectors to read at once (per chunk), options: 16, 32, 64 (default), 128, 256, 512, 1024, 2048, 4096, 8192, 16384 or 32768 (not used for raw and files formats)	Sectors per chunk.
<code>-B</code>	—	specify the number of bytes to export (default is all bytes)	Number of bytes to export — with <code>-o</code> , extracts one partition.
<code>-c</code>	—	specify the compression values as: level or method:level compression method options: deflate (default), bzip2 (bzip2 is only supported by EWF2 formats) compression level options: none (default), empty	Compression for an EWF target.
<code>-d</code>	—	calculate additional digest (hash) types besides md5, options: sha1, sha256 (not used for raw and files format)	Calculate an additional digest over the exported data.
<code>-f</code>	—	specify the output format to write to, options: raw (default), files (restricted to logical volume files), ewf, smart, encase1, encase2, encase3, encase4, encase5, encase6, encase7, encase7-v2, linen5	Output format — <code>raw</code> for tools that cannot read EWF, or another EWF variant for compatibility.
<code>-h</code>	—	shows this help	
<code>-l</code>	—	logs export errors and the digest (hash) to the log_filename	Log the export.
<code>-o</code>	—	specify the offset to start the export (default is 0)	Start offset, to export a region rather than the whole image.
<code>-p</code>	—	specify the process buffer size (default is the chunk size)	Process buffer size.
<code>-q</code>	—	quiet shows minimal status information	Minimal output.

Flag	Argument	What it does	When you would use it
<code>-s</code>	—	swap byte pairs of the media data (from AB to BA) (use this for big to little endian conversion and vice versa)	Swap byte pairs.
<code>-S</code>	—	specify the segment file size in bytes (default is 1.4 GiB) (minimum is 1.0 MiB, maximum is 7.9 EiB for raw, encase6 and encase7 format and 1.9 GiB for other formats) (not used for files format)	Segment size for the output.
<code>-t</code>	—	specify the target file to export to, use - for stdout (default is export) stdout is only supported for the raw format	Target name. <code>-</code> writes to stdout, which lets the image be piped straight into another tool without staging a full raw copy on disk.
<code>-u</code>	—	unattended mode (disables user interaction)	Unattended, for scripted conversion.
<code>-v</code>	—	verbose output to stderr	Verbose diagnostics to stderr.
<code>-V</code>	—	print version	
<code>-w</code>	—	zero sectors on checksum error (mimic EnCase like behavior)	Zero sectors that cannot be read.
<code>-x</code>	—	use the chunk data instead of the buffered read and write functions.	Bypass the buffered read/write path.

Gotchas

- Exporting to raw discards the metadata and hashes that justified using E01. Keep the original: the raw copy is a working artefact, not the evidence.
- A raw export needs the full uncompressed size in free space. E01 compression routinely hides a 2 TB image inside 700 GB.

See also

`ewfinfo`, `ewfmount`, `ewfverify`, `affinfo`, `affcat`, `img_stat`, `ntfs-3g`, `vshadowinfo`

ewfinfo

Kit	Kali Linux · SIFT Workstation
Capability	Inspect or mount a forensic image container
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Show the metadata, hashes and acquisition details recorded inside an EWF/E01 image.

When you'd reach for this

When an analyst is working with an E01 file, they run ewfinfo first to extract and save metadata such as imaging date and tool used, which is crucial for documentation and evidence reference. They may use ewfmount before accessing the raw image, and prefer ewfinfo over other tools because it specifically captures the metadata stored within the EWF wrapper.

Sources: <https://bromiley.medium.com/tooling-thursday-libewf-ec27b4564c2a> · <https://dfir.science/2017/11/EWF-Tools-working-with-Expert-Witness-Files-in-Linux.html>

Synopsis

```
ewfinfo [ -A codepage ] [ -d date_format ] [ -f format ]  
[ -ehimvVx ] ewf_files
```

Common invocations

```
# Check EWF image details post-acquisition  
ewfinfo /Cases/001/001_2017_USB_Gold.E01
```

Options

All 9 options parsed from the captured help text; 7 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-A</code>	—	codepage of header section, options: ascii (default), windows-874, windows-932, windows-936, windows-949, windows-950, windows-1250, windows-1251, windows-1252, windows-1253, windows-1254, windows-125	Header codepage, for images with non-ASCII metadata.
<code>-d</code>	—	specify the date format, options: ctime (default), dm (day/month), md (month/day), iso8601	Date format. <code>iso8601</code> is the unambiguous choice for a report.
<code>-e</code>	—	only show EWF read error information	Read errors recorded at acquisition. Check this before trusting a clean-looking image: unreadable sectors are noted here, not in the filesystem.
<code>-f</code>	—	specify the output format, options: text (default), dfxml	Emit DFXML instead of text, when the output feeds a tool rather than a person.
<code>-h</code>	—	shows this help	
<code>-i</code>	—	only show EWF acquiry information	Acquisition details only — who imaged it, when, with what.
<code>-m</code>	—	only show EWF media information	Media details only — geometry, sector size, media type.
<code>-v</code>	—	verbose output to stderr	Verbose diagnostics to stderr.
<code>-V</code>	—	print version	

Gotchas

- This reads the header only. It reports the hash that was recorded at acquisition; it does not recompute it, so it cannot tell you the image is still intact. Use `ewfverify` for that.

See also

`ewfmount`, `ewfverify`, `ewfexport`, `affinfo`, `affcat`, `img_stat`, `ntfs-3g`, `vshadowinfo`

ewfmount

Kit	Kali Linux · SIFT Workstation
Capability	Inspect or mount a forensic image container
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Use ewfmount to mount an Expert Witness Compression Format (EWF) image file

Synopsis

```
ewfmount [ -f format ] [ -X extended_options ] [ -hvV ] image mount_point
```

Common invocations

```
# Mount EWF image to a folder for forensic analysis
ewfmount image.E01 <folder>
# Mount logical image to access files
ewfmount -f files image.L01 mount_point
```

Options

All 5 options parsed from the captured help text; 1 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-f</code>	—	specify the input format, options: raw (default), files (restricted to logical volume files)	An analyst would use the -f flag when mounting a logical evidence file (such as a L01 or Lx01 file) to access metadata about loose files or directories rather than a disk image.
<code>-h</code>	—	shows this help	
<code>-v</code>	—	verbose output to stderr, while ewfmount will remain running in the foreground	
<code>-V</code>	—	print version	

Flag	Argument	What it does	When you would use it
<code>-X</code>	—	extended options to pass to sub system	

See also

`ewfinfo`, `ewfverify`, `ewfexport`, `affinfo`, `affcat`, `img_stat`, `ntfs-3g`, `vshadowinfo`

ewfverify

Kit	Kali Linux · SIFT Workstation
Capability	Inspect or mount a forensic image container
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Recompute an EWF/E01 image's hashes and check them against the values stored at acquisition.

Synopsis

```
ewfverify [ -A codepage ] [ -d digest_type ] [ -f format ]  
[ -l log_filename ] [ -p process_buffer_size ]  
[ -hqVWx ] ewf_files
```

Common invocations

```
# Verify EWF image integrity and validity  
ewfverify image.E01  
  
# Verify data integrity of E01 file against stored hash  
ewfverify -f files logical.E01
```

Options

All 11 options parsed from the captured help text; 9 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-A</code>	—	codepage of header section, options: ascii (default), windows-874, windows-932, windows-936, windows-949, windows-950, windows-1250, windows-1251, windows-1252, windows-1253, windows-1254, windows-125	Header codepage.

Flag	Argument	What it does	When you would use it
<code>-d</code>	—	calculate additional digest (hash) types besides md5, options: sha1, sha256	Also verify an additional digest such as sha256, when one was recorded at acquisition.
<code>-f</code>	—	specify the input format, options: raw (default), files (restricted to logical volume files)	Output format.
<code>-h</code>	—	shows this help	
<code>-l</code>	—	logs verification errors and the digest (hash) to the log_filename	Write the verification result to a log file — the artefact worth keeping with the case, not just terminal output.
<code>-p</code>	—	specify the process buffer size (default is the chunk size)	Process buffer size.
<code>-q</code>	—	quiet shows minimal status information	Minimal output, for scripted checks.
<code>-v</code>	—	verbose output to stderr	Verbose diagnostics to stderr.
<code>-V</code>	—	print version	
<code>-w</code>	—	zero sectors on checksum error (mimic EnCase like behavior)	Wipe sectors that could not be read.
<code>-x</code>	—	use the chunk data instead of the buffered read and write functions.	Bypass the buffered read/write path.

Gotchas

- This reads every byte, so it takes as long as the acquisition did. Budget for that rather than discovering it mid-deadline.
- A pass proves the image matches what was recorded **at acquisition**. It says nothing about whether the acquisition captured the device correctly — a disk failing mid-image produces a verifiable image of incomplete data.

See also

`ewfinfo`, `ewfmount`, `ewfexport`, `affinfo`, `affcat`, `img_stat`, `ntfs-3g`, `vshadowinfo`

HashMyFiles (GUI)

Capability	acquire preserve
Window title	HashMyFiles
Captured from	C:\Tools\HashMyFiles\HashMyFiles.exe on 2026-08-02 — control tree in capture/gui/HashMyFiles/HashMyFiles.tree.txt

[← Capability index](#) · [Kit tool list](#)

Purpose

Hash a set of files — MD5, SHA-1, SHA-256 and CRC32 — and show them in one sortable list. Sorting by hash collapses duplicates across directories instantly, which is what makes it a triage tool rather than a hashing utility.

When you'd reach for this

Reach for HashMyFiles when you have a folder of files and the question is which of them are the same. It hashes a set at once and shows MD5, SHA-1, SHA-256 and CRC32 in one sortable list.

Sorting by hash is what makes it a triage tool rather than a hashing utility: identical files collapse together immediately, so the same payload dropped under six names across four directories is obvious at a glance. Command-line hashing gives you the same numbers and leaves you to spot the duplicates yourself.

For verifying a single acquisition against its recorded hash, `sha256sum` is the simpler answer.

Sources: https://www.nirsoft.net/utils/hash_my_files.html

Controls

The parts of this window you will actually touch, read from the application's own accessibility tree rather than from a screenshot. The full node list is in [capture/gui/HashMyFiles/HashMyFiles.tree.txt](#).

The window exposes 1 further named controls: **0 file(s)**. The full tree, with every automation id, is in [the capture](#).

Using it

1. Drag files, or a folder, onto the window.
2. Read the count in the status pane to confirm everything you expected was taken — it is the only feedback that files were missed.
3. Sort by hash to group identical files; duplicates across directories collapse immediately.
4. Copy the hashes out for the report, or save them as the record of what was collected.

Gotchas

- Hashing here is for triage and deduplication. It is not acquisition — an image's hash of record comes from the acquisition tool that made it.
- It follows what you dropped and nothing more. A folder dropped without recursion silently hashes only the top level.

img_stat

Kit	REMnux · Kali Linux · SIFT Workstation
Capability	Inspect or mount a forensic image container; See the partition and volume layout
Version	The Sleuth Kit ver 4.11.1
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.sleuthkit.org/sleuthkit

[← Capability index](#) · [Kit tool list](#)

Purpose

Analyze disk images and recover files from them.

Synopsis

```
img_stat [-tvV] [-i imgtype] [-b dev_sector_size] image
```

Options

All 5 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
<code>-t</code>	—	display type only	
<code>-i</code>	imgtype	The format of the image file (use '-i list' for list of supported types)	
<code>-b</code>	dev_sector_size	The size (in bytes) of the device sectors	
<code>-v</code>	—	verbose output to stderr	
<code>-V</code>	—	Print version	

See also

[ewfinfo](#), [ewfmount](#), [ewfverify](#), [ewfexport](#), [affinfo](#), [affcat](#), [ntfs-3g](#), [vshadowinfo](#)

md5sum

Kit	Base OS — present on every Linux image
Capability	Verify evidence integrity with hashes
Version	md5sum (GNU coreutils) 9.1
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Compute or verify MD5 checksums. Still everywhere in DFIR for matching files against hash sets, but MD5 is broken for collisions — use it to say two files are the same, never to prove a file is what it claims.

Synopsis

```
md5sum [OPTION]... [FILE]...
```

Common invocations

```
# Generate MD5 checksum to verify file integrity
md5sum ravi.pdf

# Verify files match stored checksums
md5sum -c files.md5

# Verify multiple files' integrity with stored hashes
md5sum --check hashes
```

Options

All 17 options parsed from the captured help text; 8 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-b</code>	—	read in binary mode	
<code>--binary</code>	—	read in binary mode	

Flag	Argument	What it does	When you would use it
<code>-c</code>	—	read checksums from the FILES and check them	An analyst would use the <code>--check</code> flag when verifying if files have changed by comparing their current state to stored hash values, such as after modifying a file or during automated integrity checks.
<code>--check</code>	—	read checksums from the FILES and check them	An analyst would use the <code>--check</code> flag when verifying if files have changed by comparing their current state to stored hash values, such as after modifying a file or during automated integrity checks.
<code>--tag</code>	—	create a BSD-style checksum	An analyst would use the <code>--tag</code> flag when they need to display the MD5 hash in BSD-style format, as demonstrated in the examples where it formats the output as "MD5 (filename) = hashvalue".
<code>-t</code>	—	read in text mode (default)	
<code>--text</code>	—	read in text mode (default)	
<code>-z</code>	—	end each output line with NUL, not newline, and disable file name escaping	
<code>--zero</code>	—	end each output line with NUL, not newline, and disable file name escaping	
<code>--ignore-missing</code>	—	don't fail or report status for missing files	An analyst would use the <code>--ignore-missing</code> flag when verifying checksums of files that may be intentionally absent, to avoid warnings about missing files and focus on verification failures.
<code>--quiet</code>	—	don't print OK for each successfully verified file	An analyst would use the <code>--quiet</code> flag when checking multiple files to display only the modified files, filtering out unchanged ones during verification.

Flag	Argument	What it does	When you would use it
<code>--status</code>	—	don't output anything, status code shows success	An analyst would use the <code>--status</code> flag when running <code>md5sum</code> in a script to check file integrity and need the command to return a status code (0 for no changes, 1 for mismatches) without producing any output.
<code>--strict</code>	—	exit non-zero for improperly formatted checksum lines	An analyst would use the <code>--strict</code> flag when verifying checksum files to ensure they are properly formatted and to have the tool exit with a non-zero status if any hash lines are invalid.
<code>-w</code>	—	warn about improperly formatted checksum lines	An analyst would use the <code>-w</code> flag when verifying a hash file to identify which line contains an improperly formatted MD5 checksum.
<code>--warn</code>	—	warn about improperly formatted checksum lines	An analyst would use the <code>--warn</code> flag when verifying hash values in a checksum file to detect and alert on improperly formatted or incorrect hash entries during validation.
<code>--help</code>	—	display this help and exit	
<code>--version</code>	—	output version information and exit	

See also

`rahash2`, `ssdeep`, `sha256sum`, `sigtool`

ntfs-3g

Kit	SIFT Workstation
Capability	Inspect or mount a forensic image container
Version	ntfs-3g 2022.10.3
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Configuration type 7, XATTRS are on, POSIX ACLS are on

Synopsis

```
ntfs-3g [-o option[,...]] <device|image_file> <mount_point>
```

Options

No option definitions could be parsed from this tool's help output. It may be subcommand-driven or have no flags; needs manual review.

See also

`ewfinfo`, `ewfmount`, `ewfverify`, `ewfexport`, `affinfo`, `affcat`, `img_stat`, `vshadowinfo`

rahash2

Kit	REMnux
Capability	Verify evidence integrity with hashes
Version	rahash2 6.1.9
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.radare.org/n/radare2.html

[← Capability index](#) · [Kit tool list](#)

Purpose

Examine binary files, including disassembling and debugging. Includes r2ai and decai plugins for LLM-powered analysis (API key or local Ollama required), plus the r2ghidra plugin for Ghidra decompilation via the pdg command.

When you'd reach for this

An analyst reaches for rahash2 when they need to compute hash values for files or text strings, often using the `-s` option for strings or `-a all` to apply multiple algorithms simultaneously; they may run it after acquiring evidence to verify integrity or before submitting files for analysis, preferring it over similar tools for its ability to handle multiple algorithms in one command and its integration with radare for further forensic processing.

Sources: https://book.rada.re/tools/rahash2/rahash_tool.html

Synopsis

```
rahash2 [-BehjkLqRrvX] [-b S] [-a A] [-c H] [-E A] [-s S] [-f O] [-t O] [file] ...
```

Common invocations

```
# Encrypt string using rotation cipher with seed
rahash2 -S 12333 -E xor -s hello
# Verify file integrity by comparing CRC32 hash to expected value
rahash2 -qqa crc32 /bin/ls 63212007
# Verify file integrity and encode data with plugins
rahash2 -L
# List cryptographic plugins loaded
rahash2 -L | grep ^c
# Generate multiple hash values for a file's contents
rahash2 -a all /bin/ls
# Compute hash of file to verify integrity
rahash2 -qqa md5 /bin/ls
```

Options

All 23 options parsed from the captured help text; 5 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
-a	algo	comma separated list of algorithms (default is 'sha256')	An analyst would use the -a flag with the value 'all' when they need to compute multiple hash values for a file or string using all available algorithms known to rahash2.
-b	bsize	specify the size of the block (instead of full file)	
-B	—	show per-block hash	
-c	hash	compare with this hash	An analyst would use the -c flag when verifying if a file's computed hash matches a known hash to confirm its integrity or detect modifications.
-e	—	swap endian (use little endian)	
-E	algo	encrypt. Use -S to set key and -l to set IV	
-D	algo	decrypt. Use -S to set key and -l to set IV	
-f	from	start hashing at given address	
-i	num	repeat hash N iterations (f.ex: 3DES)	

Flag	Argument	What it does	When you would use it
<code>-I</code>	iv	use give initialization vector (IV) (hexa or s:string)	An analyst would use the -I flag when specifying a custom initialization vector (IV) for cryptographic operations such as encryption or decryption in algorithms that require it, like AES-CBC.
<code>-j</code>	—	output in json	
<code>-J</code>	—	new simplified json output (same as -jj)	
<code>-S</code>	seed	use given seed (hexa or s:string) use ^ to prefix (key for -E) (- will slurp the key from stdin, the @ prefix points to a file	An analyst would use the -S flag when encrypting or decrypting data with a specific key or seed value, such as during symmetric encryption operations with plugins like AES-ECB or Blowfish.
<code>-k</code>	—	show hash using the openssh's randomkey algorithm	
<code>-q</code>	—	run in quiet mode (-qq to show only the hash)	
<code>-L</code>	—	list muta plugins (combines with -q, used by -a, -E and -D)	
<code>-r</code>	—	output radare commands	
<code>-R</code>	—	output radare2 sdb commands (k file.=...)	
<code>-s</code>	string	hash this string instead of files	
<code>-t</code>	—	stop hashing at given address	An analyst would use the -t flag when they need to stop hashing at a specific memory address to limit the hash calculation to a particular section of a file or data.
<code>-x</code>	hexstr	hash this hexpair string instead of files	
<code>-X</code>	—	output in hexpairs instead of binary/plain	
<code>-v</code>	—	show version information	

See also

`ssdeep`, `sha256sum`, `md5sum`, `sigtool`

sha256sum

Kit	Base OS — present on every Linux image
Capability	Verify evidence integrity with hashes
Version	sha256sum (GNU coreutils) 9.1
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Print or check SHA256 (256-bit) checksums.

When you'd reach for this

An analyst uses sha256sum to verify file integrity after detecting content changes, running it after appending data to confirm checksum mismatches, and preferring it over MD5/SHA-1 for stronger tamper protection and over sha512sum for a balance between security and efficiency.

Sources: <https://penguin-gym-linux.com/en/articles/tutorials/checksum-md5-sha256>

Synopsis

```
sha256sum [OPTION]... [FILE]...
```

Common invocations

```
# Compute SHA256 hashes for multiple image files
sha256sum *.jpg

# Verify file integrity using SHA256 checksums
sha256sum -c checksums.sha256

# Create checksum file for integrity verification
sha256sum file.zip > checksums.sha256

# Verify files match checksums
sha256sum -c checksums.sha256 --quiet

# Verify files match checksums for integrity
sha256sum -c checksums.sha256 --status
```

Options

All 17 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
<code>-b</code>	—	read in binary mode	
<code>--binary</code>	—	read in binary mode	
<code>-c</code>	—	read checksums from the FILEs and check them	
<code>--check</code>	—	read checksums from the FILEs and check them	
<code>--tag</code>	—	create a BSD-style checksum	
<code>-t</code>	—	read in text mode (default)	
<code>--text</code>	—	read in text mode (default)	
<code>-z</code>	—	end each output line with NUL, not newline, and disable file name escaping	
<code>--zero</code>	—	end each output line with NUL, not newline, and disable file name escaping	
<code>--ignore-missing</code>	—	don't fail or report status for missing files	
<code>--quiet</code>	—	don't print OK for each successfully verified file	
<code>--status</code>	—	don't output anything, status code shows success	
<code>--strict</code>	—	exit non-zero for improperly formatted checksum lines	
<code>-w</code>	—	warn about improperly formatted checksum lines	
<code>--warn</code>	—	warn about improperly formatted checksum lines	
<code>--help</code>	—	display this help and exit	
<code>--version</code>	—	output version information and exit	

See also

[rahash2](#), [ssdeep](#), [md5sum](#), [sigtool](#)

sigtool

Kit	REMnux · SIFT Workstation
Capability	Verify evidence integrity with hashes; Scan with signatures for known-bad
Version	ClamAV 1.4.3
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.clamav.net

[← Capability index](#) · [Kit tool list](#)

Purpose

Inspect and build ClamAV signature databases: unpack a .cvd, list its signatures, or generate a new one from a sample. The bridge between 'ClamAV detects this' and 'here is exactly which signature fired and why'.

When you'd reach for this

An analyst reaches for sigtool when investigating false positives detected by ClamAV, running it after obtaining a signature name from scan reports or logs to unpack the database and search for the specific signature; they use it over other methods because it directly facilitates identifying and analyzing signatures linked to false positives.

Sources: <https://docs.clamav.net/manual/Usage/SignatureManagement.html>

Options

All 35 options parsed from the captured help text; 3 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>--help</code>	—	Show this help	
<code>-h</code>	—	Show this help	
<code>--version</code>	—	Print version number and exit	
<code>-V</code>	—	Print version number and exit	
<code>--quiet</code>	—	Be quiet, output only error messages	
<code>--debug</code>	—	Enable debug messages	
<code>--stdout</code>	—	Write to stdout instead of stderr. Does not affect 'debug' messages.	

Flag	Argument	What it does	When you would use it
<code>--hex-dump</code>	—	Convert data from stdin to a hex string and print it on stdout	An analyst would use the <code>--hex-dump</code> flag when needing to generate a hexadecimal representation of a file's contents for detailed forensic examination or signature creation.
<code>--md5</code>	FILES	Generate MD5 checksum from stdin or MD5 sigs for FILES	
<code>--sha1</code>	FILES	Generate SHA1 checksum from stdin or SHA1 sigs for FILES	
<code>--sha256</code>	FILES	Generate SHA256 checksum from stdin or SHA256 sigs for FILES	
<code>--mdb</code>	FILES	Generate .mdb (section hash) sigs	
<code>--imp</code>	FILES	Generate .imp (import table hash) sigs	
<code>--html-normalise</code>	FILE	Create normalised parts of HTML file	
<code>--ascii-normalise</code>	FILE	Create normalised text file from ascii source	
<code>--utf16-decode</code>	FILE	Decode UTF16 encoded files	
<code>--info</code>	FILE	-i FILE Print database information	
<code>--max-bad-sigs</code>	NUMBER	Maximum number of mismatched signatures When building a CVD. Default: 3000	
<code>--flevel</code>	FLEVEL	Specify a custom flevel. Default: 213	
<code>--cvd-version</code>	NUMBER	Specify the version number to use for the build. Default is to use the value+1 from the current CVD in --datadir. If no datafile is found the default behaviour is to prompt for a version number, this	
<code>--no-cdiff</code>	—	Don't generate .cdiff file	
<code>--unsigned</code>	—	Create unsigned database file (.cud)	
<code>--hybrid</code>	—	Create a hybrid (standard and bytecode) database file	
<code>--print-certs</code>	FILE	Print Authenticode details from a PE	

Flag	Argument	What it does	When you would use it
<code>--server</code>	ADDR	ClamAV Signing Service address	
<code>--datadir</code>	DIR	Use DIR as default database directory	An analyst would use <code>--datadir</code> when they need to specify a non-default directory as the default database location for all sigtool operations.
<code>--unpack</code>	FILE	<code>-u FILE</code> Unpack a CVD/CLD file	An analyst would use the <code>--unpack</code> flag when investigating a potential false positive by unpacking the virus signature database to locate and examine the specific signature causing the detection.
<code>--unpack-current</code>	SHORTNAME	Unpack local CVD/CLD into cwd	
<code>--list-sigs</code>	FILE (optional)	<code>-l[FILE]</code> List signature names	
<code>--find-sigs</code>	REGEX	<code>-fREGEX</code> Find signatures matching REGEX	
<code>--decode-sigs</code>	—	Decode signatures from stdin	
<code>--vba</code>	FILE	Extract VBA/Word6 macro code	
<code>--vba-hex</code>	FILE	Extract Word6 macro code with hex values	
<code>--run-cdiff</code>	FILE	<code>-r FILE</code> Execute update script FILE in cwd	
<code>--tempdir</code>	DIRECTORY	Create temporary files in DIRECTORY	

See also

[rahash2](#), [ssdeep](#), [sha256sum](#), [md5sum](#), [yara](#), [yarak](#), [clamscan](#), [freshclam](#)

ssdeep

Kit	REMnux · Kali Linux · SIFT Workstation
Capability	Verify evidence integrity with hashes; Compare or cluster samples; Find hidden data
Version	2.14.1
Captured from	<code>cyberlab-aio</code> via <code>-h</code> on 2026-08-11 — raw help output
Documentation	https://ssdeep-project.github.io/ssdeep/index.html

← [Capability index](#) · [Kit tool list](#)

Purpose

Compute Context Triggered Piecewise Hashes (CTPH), also known as fuzzy hashes.

When you'd reach for this

An analyst reaches for ssdeep when comparing files for similarity rather than exact matches, running commands like `-r` to generate fuzzy hashes and `-x` or `-k` to compare signatures, as it is a mainstream tool used by NIST and can detect partial overlaps between files.

Sources: [https://dfir.science/2017/07/How-To-Fuzzy-Hashing-with-SSDEEP-\(similarity-matching\).html](https://dfir.science/2017/07/How-To-Fuzzy-Hashing-with-SSDEEP-(similarity-matching).html) · <https://ssdeep-project.github.io/ssdeep/usage.html>

Synopsis

```
ssdeep [-m file] [-k file] [-dpgvrsblcxa] [-t val] [-h|-V] [FILES]
```

Common invocations

```
# Generate file hashes recursively for directory tree
ssdeep -r *

# Compare signature files to detect overlapping entries
ssdeep -r /etc >list1.txt

# Detect source code reuse by comparing file similarities
ssdeep -b foo.txt >hashes.txt

# Detect file similarities using fuzzy hash matching
ssdeep -b -m hashes.txt bar.txt
```

Options

No option definitions could be parsed from this tool's help output. It may be subcommand-driven or have no flags; needs manual review.

See also

[rahash2](#), [sha256sum](#), [md5sum](#), [sigtool](#), [radiff2](#), [binwalk](#)

tshark

Kit	REMnux · Kali Linux · FLARE-VM · SIFT Workstation
Capability	Capture live network traffic; Read and filter packet captures
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.wireshark.org

← [Capability index](#) · [Kit tool list](#)

Purpose

Wireshark's command line: capture, filter, dissect and export packet data, including fields for a timeline.

When you'd reach for this

An analyst reaches for tshark when capturing or analyzing network traffic from the command line, often after setting up capture filters (e.g., `tshark -f !arp`) or before generating statistics (e.g., `tshark -z io,stat`). They may prefer it over GUI tools for scripting, automation, or when working with large captures that require efficient, non-interactive processing.

Sources: <https://docsislab.wordpress.com/packet-capture/wireshark-command-line/> · https://www.wireshark.org/docs/wsug_html_chunked/AppTools.html

Synopsis

```
tshark [options] ...
```

Common invocations

```
# Exclude ARP traffic during capture
tshark -f !arp

# Generate protocol hierarchy statistics for network traffic analysis
tshark -z io,phs

# Capture 100 packets excluding ARP traffic for analysis
tshark -c 100 -f !arp

# Generate TCP conversation statistics
tshark -q -z conv,tcp

# Capture 100 packets for network analysis
tshark -c 100 -n -w 100pkts.pcap
```


Options

All 75 options parsed from the captured help text; 20 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-i</code>	interface	name or idx of interface (def: first non-loopback)	Capture live from an interface instead. Needs capture rights, and on a busy link a live dissect will drop packets.
<code>--interface</code>	interface	name or idx of interface (def: first non-loopback)	Capture live from an interface instead. Needs capture rights, and on a busy link a live dissect will drop packets.
<code>-f</code>	capture filter	packet filter in libpcap filter syntax	Capture filter, in BPF syntax. Applied before packets are written, so what it drops is gone forever.
<code>-s</code>	snaplen	packet snapshot length (def: appropriate maximum)	Snapshot length; truncates each packet as it is captured.
<code>--snapshot-length</code>	snaplen	packet snapshot length (def: appropriate maximum)	Snapshot length; truncates each packet as it is captured.
<code>-p</code>	—	don't capture in promiscuous mode	Do not enter promiscuous mode.
<code>--no-promiscuous-mode</code>	—	don't capture in promiscuous mode	Do not enter promiscuous mode.
<code>-I</code>	—	capture in monitor mode, if available	Monitor mode, for capturing 802.11 management frames.
<code>--monitor-mode</code>	—	capture in monitor mode, if available	Monitor mode, for capturing 802.11 management frames.
<code>-B</code>	buffer size	size of kernel buffer (def: 2MB)	Kernel buffer size. Raise it when a fast link is dropping packets at capture time.
<code>--buffer-size</code>	buffer size	size of kernel buffer (def: 2MB)	Kernel buffer size. Raise it when a fast link is dropping packets at capture time.
<code>-y</code>	link type	link layer type (def: first appropriate)	Force the link-layer type.
<code>--linktype</code>	link type	link layer type (def: first appropriate)	Force the link-layer type.
<code>-D</code>	—	print list of interfaces and exit	List interfaces and exit — how you find the right <code>-i</code> value.
<code>--list-interfaces</code>	—	print list of interfaces and exit	List interfaces and exit — how you find the right <code>-i</code> value.
<code>-L</code>	—	print list of link-layer types of iface and exit	List the link-layer types an interface supports.

Flag	Argument	What it does	When you would use it
<code>--list-data-link-types</code>	—	print list of link-layer types of iface and exit	List the link-layer types an interface supports.
<code>--list-time-stamp-types</code>	—	print list of timestamp types for iface and exit	
<code>-c</code>	packet count	stop after n packets (def: infinite)	Stop after N packets — the fast way to sample a huge file before committing to a full pass.
<code>-a</code>	autostop cond	duration:NUM - stop after NUM seconds filesize:NUM - stop this file after NUM KB files:NUM - stop after NUM files packets:NUM - stop after NUM packets	Autostop condition for a live capture: duration, filesize or file count.
<code>--autostop</code>	autostop cond	duration:NUM - stop after NUM seconds filesize:NUM - stop this file after NUM KB files:NUM - stop after NUM files packets:NUM - stop after NUM packets	Autostop condition for a live capture: duration, filesize or file count.
<code>-b</code>	ringbuffer opt	duration:NUM - switch to next file after NUM secs filesize:NUM - switch to next file after NUM KB files:NUM - ringbuffer: replace after NUM files packets:NUM - switch to next file after NUM packets in	Ring buffer: roll to a new file on time or size, so a long capture cannot fill the disk.
<code>--ring-buffer</code>	ringbuffer opt	duration:NUM - switch to next file after NUM secs filesize:NUM - switch to next file after NUM KB files:NUM - ringbuffer: replace after NUM files packets:NUM - switch to next file after NUM packets in	Ring buffer: roll to a new file on time or size, so a long capture cannot fill the disk.
<code>-r</code>	infile	set the filename to read from (or '-' for stdin)	Read a capture file. The safe default — analysis needs no privileges and cannot disturb the wire.
<code>--read-file</code>	infile	set the filename to read from (or '-' for stdin)	Read a capture file. The safe default — analysis needs no privileges and cannot disturb the wire.
<code>-2</code>	—	perform a two-pass analysis	Two-pass analysis, so fields that depend on later packets — reassembly, response times, stream indexes — are populated.

Flag	Argument	What it does	When you would use it
<code>-M</code>	packet count	perform session auto reset	Reset dissector state every N packets, to bound memory on a very long capture.
<code>-R</code>	read filter	packet Read filter in Wireshark display filter syntax (requires -2)	Read filter, which needs <code>-2</code> . Prefer <code>-Y</code> unless you know why you want this one.
<code>--read-filter</code>	read filter	packet Read filter in Wireshark display filter syntax (requires -2)	Read filter, which needs <code>-2</code> . Prefer <code>-Y</code> unless you know why you want this one.
<code>-Y</code>	display filter	packet display filter in Wireshark display filter syntax	Display filter, in Wireshark syntax. Applied after capture, so nothing is lost and it can be changed later. Confusing these two is the classic tshark mistake.
<code>--display-filter</code>	display filter	packet display filter in Wireshark display filter syntax	Display filter, in Wireshark syntax. Applied after capture, so nothing is lost and it can be changed later. Confusing these two is the classic tshark mistake.
<code>-n</code>	—	disable all name resolutions (def: "mNd" enabled, or as set in preferences)	
<code>-N</code>	name resolve flags	enable specific name resolution(s): "mnNtdv"	
<code>-H</code>	hosts file	read a list of entries from a hosts file, which will then be written to a capture file. (Implies -W n)	
<code>--enable-protocol</code>	proto_name	enable dissection of proto_name	
<code>--disable-protocol</code>	proto_name	disable dissection of proto_name	
<code>--enable-heuristic</code>	short_name	enable dissection of heuristic protocol	
<code>--disable-heuristic</code>	short_name	disable dissection of heuristic protocol	
<code>-w</code>	outfile -	write packets to a pcapng-format file named "outfile" (or '-' for stdout)	Write packets out rather than dissecting them, which is much faster when you only want a filtered subset.
<code>--capture-comment</code>	comment	add a capture file comment, if supported	

Flag	Argument	What it does	When you would use it
<code>-C</code>	config profile	start with specified configuration profile	
<code>-F</code>	output file type	set the output file type, default is pcapng an empty "-F" option will list the file types	
<code>-V</code>	—	add output of packet tree (Packet Details)	
<code>-O</code>	protocols	Only show packet details of these protocols, comma separated	
<code>-P</code>	—	print packet summary even when writing to a file	
<code>--print</code>	—	print packet summary even when writing to a file	
<code>-S</code>	separator	the line separator to print between packets	
<code>-x</code>	—	add output of hex and ASCII dump (Packet Bytes)	
<code>--hexdump</code>	hexoption	add hexdump, set options for data source and ASCII dump all dump all data sources (-x default) frames dump only frame data source ascii include ASCII dump text (-x default) delimit delimit ASCII dump	
<code>-j</code>	protocolfilter	protocols layers filter if -T ek pdml json selected (e.g. "ip ip.flags text", filter does not expand child nodes, unless child is specified also in the filter)	
<code>-J</code>	protocolfilter	top level protocol filter if -T ek pdml json selected (e.g. "http tcp", filter which expands all child nodes)	
<code>-e</code>	field	field to print if -Tfields selected (e.g. tcp.port, _ws.col.Info) this option can be repeated to print multiple fields	Which field to print, repeatable. Only meaningful with <code>-T fields</code> , and the ordering is the column ordering.
<code>-l</code>	—	flush standard output after each packet	

Flag	Argument	What it does	When you would use it
<code>-q</code>	—	be more quiet on stdout (e.g. when using statistics)	An analyst would use the <code>-q</code> flag when running tshark with the <code>-z</code> option to generate specific statistics, as shown in examples like "tshark -q -z io,stat,5,ip.addr==255.255.255.255" and "tshark -q -z conv.
<code>-Q</code>	—	only log true errors to stderr (quieter than <code>-q</code>)	
<code>-g</code>	—	enable group read access on the output file(s)	
<code>-W</code>	n	Save extra information in the file, if supported. n = write network address resolution information	
<code>-U</code>	tap_name	PDU export mode, see the man page for details	
<code>-z</code>	statistics	various statistics, see the man page for details	
<code>--export-tls-session-keys</code>	keyfile	export TLS Session Keys to a file named "keyfile"	
<code>--color</code>	—	color output text similarly to the Wireshark GUI, requires a terminal with 24-bit color support Also supplies color attributes to pdml and psml formats (Note that attributes are nonstandard)	
<code>--no-duplicate-keys</code>	—	If <code>-T json</code> is specified, merge duplicate keys in an object into a single key with as value a json array containing all values	
<code>--temp-dir</code>	directory	write temporary files to this directory (default: /tmp)	
<code>--log-level</code>	level	sets the active log level ("critical", "warning", etc.)	
<code>--log-fatal</code>	level	sets level to abort the program ("critical" or "warning")	
<code>--log-domains</code>	[!]list	comma separated list of the active log domains	
<code>--log-debug</code>	[!]list	comma separated list of domains with "debug" level	
<code>--log-noisy</code>	[!]list	comma separated list of domains with "noisy" level	

Flag	Argument	What it does	When you would use it
<code>--log-file</code>	path	file to output messages to (in addition to stderr)	
<code>-h</code>	—	display this help and exit	
<code>--help</code>	—	display this help and exit	
<code>-v</code>	—	display version info and exit	
<code>--version</code>	—	display version info and exit	
<code>-K</code>	keytab	keytab file to use for kerberos decryption	
<code>-G</code>	report	dump one of several available reports and exit default report="fields" use "-G help" for more help	

Gotchas

- Capture filters (`-f`) and display filters (`-Y`) use **different syntaxes** and apply at different times. `-f` discards packets permanently; `-Y` only hides them. Reaching for the wrong one is the most common way to destroy evidence with this tool.
- Dissecting live on a busy link drops packets silently. Capture to a file first, analyse afterwards, whenever completeness matters.
- `-T fields` prints nothing useful without `-e`. It is not an error, just empty output, which reads like the filter matched nothing.

See also

`dumpcap`, `capinfos`, `ngrep`, `tcpflow`

vshadowinfo

Kit	SIFT Workstation (libyal)
Capability	Inspect or mount a forensic image container
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Use vshadowinfo to determine information about a Windows NT Volume Shadow

Synopsis

```
vshadowinfo [ -o offset ] [ -ahvV ] source
```

Common invocations

```
# Display Volume Shadow Snapshot volume information
vshadowinfo /dev/sda1
# Extract shadow copy metadata from disk image
vshadowinfo -o <partition_byte_offset> disk.raw
```

Options

All 5 options parsed from the captured help text; 2 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-a</code>	—	shows allocation information	An analyst would use the -a flag when examining allocation information related to a Volume Shadow Snapshot (VSS) volume.
<code>-h</code>	—	shows this help	

Flag	Argument	What it does	When you would use it
<code>-o</code>	—	specify the volume offset in bytes	An analyst would use the <code>-o</code> flag when needing to specify a non-default volume offset in bytes to access a particular section of a VSS volume that isn't starting at the beginning of the source file or device.
<code>-v</code>	—	verbose output to stderr	
<code>-V</code>	—	print version	

See also

`ewfinfo`, `ewfmount`, `ewfverify`, `ewfexport`, `affinfo`, `affcat`, `img_stat`, `ntfs-3g`

log2timeline.py

Kit	Kali Linux · SIFT Workstation
Capability	Build a super-timeline from many artifact sources
Version	plaso - log2timeline version 20260512
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Extract timestamped events from evidence into a Plaso storage file, the first half of a super-timeline.

When you'd reach for this

An analyst reaches for log2timeline.py when creating a forensic timeline from disk images or directories, as it extracts timestamps into a Plaso storage file, often preceding psort.py for filtering and sorting. They may use it after acquiring evidence and before analysis, preferring it for its ability to detect partitions and VSS, and for supporting targeted extraction via filter files.

Sources: <https://plaso.readthedocs.io/en/latest/sources/user/Using-log2timeline.html> · <https://www.cyberforensicacademy.com/blog/log2timeline-guide-creating-forensic-timelines>

Synopsis

```
log2timeline.py [-h] [--troubles] [-V] [--artifact_definitions PATH]
[--custom_artifact_definitions PATH] [--data PATH]
[--archives TYPES]
[--artifact_filters ARTIFACT_FILTERS]
[--artifact_filters_file PATH]
[--extract_winreg_binary] [--preferred_year YEAR]
```

Common invocations

```
# List available plugins and parser presets
log2timeline.py --info
# Generate timeline from source data
log2timeline.py --storage-file OUTPUT INPUT
# Generate Plaso storage file from raw disk image data
log2timeline.py case_analysis.plaso /mnt/evidence/image.dd
# Extract targeted file events from storage image
log2timeline.py -f filter --storage-file timeline.plaso test.vhd
# Generate timeline from VHD using Plaso storage
log2timeline.py --logfile test.log --storage-file timeline.plaso test.vhd
```

Options

All 100 options parsed from the captured help text; 15 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	Show this help message and exit.	
<code>--help</code>	—	Show this help message and exit.	
<code>--troubles</code>	—	Show troubleshooting information.	
<code>-V</code>	—	Show the version information.	
<code>--version</code>	—	Show the version information.	
<code>-- artifact_definitions</code>	PATH	Path to a directory or file containing artifact definitions, which are .yaml files. Artifact definitions can be used to describe and quickly collect data of interest, such as specific files or Windows	
<code>--artifact- definitions</code>	PATH	Path to a directory or file containing artifact definitions, which are .yaml files. Artifact definitions can be used to describe and quickly collect data of interest, such as specific files or Windows	
<code>-- custom_artifact_definit ions</code>	PATH	Path to a directory or file containing custom artifact definitions, which are .yaml files. Artifact definitions can be used to describe and quickly collect data of interest, such as specific files or	

Flag	Argument	What it does	When you would use it
<code>--custom-artifact-definitions</code>	PATH	Path to a directory or file containing custom artifact definitions, which are .yaml files. Artifact definitions can be used to describe and quickly collect data of interest, such as specific files or	
<code>--data</code>	PATH	Path to a directory containing the data files.	
<code>--archives</code>	TYPES	Define a list of archive and storage media image types for which to process embedded file entries, such as TAR (archive.tar) or ZIP (archive.zip). This is a comma separated list where each entry is th	
<code>--artifact_filters</code>	ARTIFACT_FILTERS	Names of forensic artifact definitions, provided on the command command line (comma separated). Forensic artifacts are stored in .yaml files that are directly pulled from the artifact definitions proj	
<code>--artifact-filters</code>	ARTIFACT_FILTERS	Names of forensic artifact definitions, provided on the command command line (comma separated). Forensic artifacts are stored in .yaml files that are directly pulled from the artifact definitions proj	
<code>--artifact_filters_file</code>	PATH	Names of forensic artifact definitions, provided in a file with one artifact name per line. Forensic artifacts are stored in .yaml files that are directly pulled from the artifact definitions project.	
<code>--artifact-filters_file</code>	PATH	Names of forensic artifact definitions, provided in a file with one artifact name per line. Forensic artifacts are stored in .yaml files that are directly pulled from the artifact definitions project.	

Flag	Argument	What it does	When you would use it
<code>--extract_winreg_binary</code>	—	Extract binary Windows Registry values. WARNING: This can make processing significantly slower.	
<code>--extract-winreg-binary</code>	—	Extract binary Windows Registry values. WARNING: This can make processing significantly slower.	
<code>--preferred_year</code>	YEAR	When a format's timestamp does not include a year, e.g. syslog, use this as the initial year instead of attempting auto-detection.	
<code>--preferred-year</code>	YEAR	When a format's timestamp does not include a year, e.g. syslog, use this as the initial year instead of attempting auto-detection.	
<code>--skip_compressed_streams</code>	—	Skip processing file content within compressed streams, such as syslog.gz and syslog.bz2.	
<code>--skip-compressed-streams</code>	—	Skip processing file content within compressed streams, such as syslog.gz and syslog.bz2.	
<code>-f</code>	FILE_FILTER	List of files to include for targeted collection of files to parse, one line per file path, setup is /path file - where each element can contain either a variable set in the preprocessing stage or a	Use a file filter to limit what gets processed.
<code>--filter-file</code>	FILE_FILTER	List of files to include for targeted collection of files to parse, one line per file path, setup is /path file - where each element can contain either a variable set in the preprocessing stage or a	Use a file filter to limit what gets processed.
<code>--filter_file</code>	FILE_FILTER	List of files to include for targeted collection of files to parse, one line per file path, setup is /path file - where each element can contain either a variable set in the preprocessing stage or a	Use a file filter to limit what gets processed.

Flag	Argument	What it does	When you would use it
<code>--file-filter</code>	FILE_FILTER	List of files to include for targeted collection of files to parse, one line per file path, setup is /path file - where each element can contain either a variable set in the preprocessing stage or a	An analyst would use the --file-filter flag when processing a full disk image directly to specify individual files or paths for analysis, avoiding the need to create a separate triage collection.
<code>--file_filter</code>	FILE_FILTER	List of files to include for targeted collection of files to parse, one line per file path, setup is /path file - where each element can contain either a variable set in the preprocessing stage or a	Use a file filter to limit what gets processed.
<code>--hasher_file_size_limit</code>	SIZE	Define the maximum file size in bytes that hashers should process. Any larger file will be skipped. A size of 0 represents no limit.	
<code>--hasher-file-size-limit</code>	SIZE	Define the maximum file size in bytes that hashers should process. Any larger file will be skipped. A size of 0 represents no limit.	
<code>--hashers</code>	HASHER_LIST	Define a list of hashers to use by the tool. This is a comma separated list where each entry is the name of a hasher, such as "md5,sha256". "all" indicates that all hashers should be enabled. "none" d	Compute hashes during extraction, saving a second pass.
<code>--parsers</code>	PARSER_FILTER_EXPRESSION	Define which presets, parsers and/or plugins to use, or show possible values. The expression is a comma separated string where each element is a preset, parser or plugin name. Each element can be prep	Restrict to specific parsers. A targeted run is minutes instead of hours.
<code>--yara_rules</code>	PATH	Path to a file containing Yara rules definitions.	
<code>--yara-rules</code>	PATH	Path to a file containing Yara rules definitions.	

Flag	Argument	What it does	When you would use it
<code>--partitions</code>	PARTITIONS	Define partitions to be processed. A range of partitions can be defined as: "3..5". Multiple partitions can be defined as: "1,3,5" (a list of comma separated values). Ranges and lists can also be combined as:	An analyst would use the --partitions flag when processing a disk image with multiple partitions and needing to specify a particular partition number to avoid interactive prompts during the analysis.
<code>--partition</code>	PARTITIONS	Define partitions to be processed. A range of partitions can be defined as: "3..5". Multiple partitions can be defined as: "1,3,5" (a list of comma separated values). Ranges and lists can also be combined as:	An analyst would use the --partitions flag when processing a disk image with multiple partitions and needing to specify a particular partition number to avoid interactive prompts during the analysis.
<code>--volumes</code>	VOLUMES	Define volumes to be processed. A range of volumes can be defined as: "3..5". Multiple volumes can be defined as: "1,3,5" (a list of comma separated values). Ranges and lists can also be combined as:	
<code>--volume</code>	VOLUMES	Define volumes to be processed. A range of volumes can be defined as: "3..5". Multiple volumes can be defined as: "1,3,5" (a list of comma separated values). Ranges and lists can also be combined as:	
<code>--codepage</code>	CODEPAGE	The preferred codepage, which is used for decoding single-byte or multi-byte character extracted strings.	
<code>--language</code>	LANGUAGE_TAG	The preferred language, which is used for extracting and formatting Windows EventLog message strings. Use " --language list" to see a list of supported language tags. The en-US (LCID 0x0409) language	

Flag	Argument	What it does	When you would use it
<code>--no_extract_winevt_resources</code>	—	Do not extract Windows EventLog resources such as event message template strings. By default Windows EventLog resources will be extracted when a Windows EventLog parser is enabled.	
<code>--no-extract-winevt-resources</code>	—	Do not extract Windows EventLog resources such as event message template strings. By default Windows EventLog resources will be extracted when a Windows EventLog parser is enabled.	
<code>-z</code>	TIME_ZONE	preferred time zone of extracted date and time values that are stored without a time zone indicator. The time zone is determined based on the source data where possible otherwise it will default to UT	Time zone of the source machine.
<code>--zone</code>	TIME_ZONE	preferred time zone of extracted date and time values that are stored without a time zone indicator. The time zone is determined based on the source data where possible otherwise it will default to UT	Time zone of the source machine.
<code>--timezone</code>	TIME_ZONE	preferred time zone of extracted date and time values that are stored without a time zone indicator. The time zone is determined based on the source data where possible otherwise it will default to UT	When analyzing loose files, a triage collection, or when the system's time zone cannot be auto-detected, an analyst would use the <code>--timezone</code> flag to explicitly specify the source system's time zone.
<code>--no_vss</code>	—	Do not scan for Volume Shadow Snapshots (VSS). This means that Volume Shadow Snapshots (VSS) are not processed. WARNING: this option is deprecated use <code>--vss_stores=none</code> instead.	
<code>--no-vss</code>	—	Do not scan for Volume Shadow Snapshots (VSS). This means that Volume Shadow Snapshots (VSS) are not processed. WARNING: this option is deprecated use <code>--vss_stores=none</code> instead.	

Flag	Argument	What it does	When you would use it
<code>--vss_only</code>	—	Do not process the current volume if Volume Shadow Snapshots (VSS) have been selected.	
<code>--vss-only</code>	—	Do not process the current volume if Volume Shadow Snapshots (VSS) have been selected.	
<code>--vss_stores</code>	VSS_STORES	Define Volume Shadow Snapshots (VSS) (or stores) that need to be processed. A range of snapshots can be defined as: "3..5". Multiple snapshots can be defined as: "1,3,5" (a list of comma separated val	Also process Volume Shadow Copies — often where the pre-attack state survives.
<code>--vss-stores</code>	VSS_STORES	Define Volume Shadow Snapshots (VSS) (or stores) that need to be processed. A range of snapshots can be defined as: "3..5". Multiple snapshots can be defined as: "1,3,5" (a list of comma separated val	Also process Volume Shadow Copies — often where the pre-attack state survives.
<code>-d</code>	—	Enable debug output.	An analyst would use the -d flag when coupled with --logfile to obtain more detailed debug information during the processing of a storage media image.
<code>--debug</code>	—	Enable debug output.	An analyst would use the -d flag when coupled with --logfile to obtain more detailed debug information during the processing of a storage media image.
<code>-q</code>	—	Disable informational output.	
<code>--quiet</code>	—	Disable informational output.	
<code>-u</code>	—	Enable unattended mode and do not ask the user for additional input when needed, but terminate with an error instead.	
<code>--unattended</code>	—	Enable unattended mode and do not ask the user for additional input when needed, but terminate with an error instead.	

Flag	Argument	What it does	When you would use it
<code>--info</code>	—	Print out information about supported plugins and parsers.	An analyst would use the <code>--info</code> flag when they need to check the list of supported plugins, parsers, and output modules available in <code>log2timeline.py</code> .
<code>--use_markdown</code>	—	Output lists in Markdown format use in combination with " <code>--hashers list</code> ", " <code>--parsers list</code> " or " <code>--timezone list</code> "	
<code>--use-markdown</code>	—	Output lists in Markdown format use in combination with " <code>--hashers list</code> ", " <code>--parsers list</code> " or " <code>--timezone list</code> "	
<code>--no_dependencies_check</code>	—	Disable the dependencies check.	
<code>--no-dependencies-check</code>	—	Disable the dependencies check.	
<code>--logfile</code>	FILENAME	Path of the file in which to store log messages, by default this file will be named: " <code>log2timeline-YYYYMMDDThhmmss.log.gz</code> ". Note that the file will be gzip compressed if the extension is ".gz".	An analyst would use the <code>--logfile</code> flag when they need to redirect all log messages from <code>log2timeline.py</code> to a file for detailed debugging or record-keeping during processing.
<code>--log_file</code>	FILENAME	Path of the file in which to store log messages, by default this file will be named: " <code>log2timeline-YYYYMMDDThhmmss.log.gz</code> ". Note that the file will be gzip compressed if the extension is ".gz".	An analyst would use the <code>--logfile</code> flag when they need to redirect all log messages from <code>log2timeline.py</code> to a file for detailed debugging or record-keeping during processing.
<code>--log-file</code>	FILENAME	Path of the file in which to store log messages, by default this file will be named: " <code>log2timeline-YYYYMMDDThhmmss.log.gz</code> ". Note that the file will be gzip compressed if the extension is ".gz".	An analyst would use the <code>--logfile</code> flag when they need to redirect all log messages from <code>log2timeline.py</code> to a file for detailed debugging or record-keeping during processing.
<code>--status_view</code>	TYPE	The processing status view mode: "file", "linear", "none" or "window".	Change progress display; <code>none</code> for clean logs.
<code>--status-view</code>	TYPE	The processing status view mode: "file", "linear", "none" or "window".	Change progress display; <code>none</code> for clean logs.

Flag	Argument	What it does	When you would use it
<code>--status_view_file</code>	PATH	The name of the status view file.	
<code>--status-view-file</code>	PATH	The name of the status view file.	
<code>--status_view_interval</code>	SECONDS	Number of seconds to update the status view.	
<code>--status-view-interval</code>	SECONDS	Number of seconds to update the status view.	
<code>--buffer_size</code>	BUFFER_SIZE	The buffer size for the output (defaults to 196MiB).	
<code>--buffer-size</code>	BUFFER_SIZE	The buffer size for the output (defaults to 196MiB).	
<code>--bs</code>	BUFFER_SIZE	The buffer size for the output (defaults to 196MiB).	
<code>--queue_size</code>	QUEUE_SIZE	The maximum number of queued items per worker (defaults to 125000)	
<code>--queue-size</code>	QUEUE_SIZE	The maximum number of queued items per worker (defaults to 125000)	
<code>--single_process</code>	—	Indicate that the tool should run in a single process.	
<code>--single-process</code>	—	Indicate that the tool should run in a single process.	
<code>--process_memory_limit</code>	SIZE	Maximum amount of memory (data segment) a process is allowed to allocate in bytes, where 0 represents no limit. The default limit is 4294967296 (4 GiB). This applies to both the main (foreman) process	
<code>--process-memory-limit</code>	SIZE	Maximum amount of memory (data segment) a process is allowed to allocate in bytes, where 0 represents no limit. The default limit is 4294967296 (4 GiB). This applies to both the main (foreman) process	
<code>--temporary_directory</code>	DIRECTORY	Path to the directory that should be used to store temporary files created during processing.	

Flag	Argument	What it does	When you would use it
<code>--temporary-directory</code>	DIRECTORY	Path to the directory that should be used to store temporary files created during processing.	
<code>--vfs_back_end</code>	TYPE	The preferred dfVFS back-end: "auto", "fsext", "fsfat", "fshfs", "fsntfs", "tsk" or "vsgpt".	
<code>--vfs-back-end</code>	TYPE	The preferred dfVFS back-end: "auto", "fsext", "fsfat", "fshfs", "fsntfs", "tsk" or "vsgpt".	
<code>--worker_memory_limit</code>	SIZE	Maximum amount of memory (data segment and shared memory) a worker process is allowed to consume in bytes, where 0 represents no limit. The default limit is 2147483648 (2 GiB). If a worker process exc	
<code>--worker-memory-limit</code>	SIZE	Maximum amount of memory (data segment and shared memory) a worker process is allowed to consume in bytes, where 0 represents no limit. The default limit is 2147483648 (2 GiB). If a worker process exc	
<code>--worker_timeout</code>	MINUTES	Number of minutes before a worker process that is not providing status updates is considered inactive. The default timeout is 15.0 minutes. If a worker process exceeds this timeout it is killed by the	
<code>--worker-timeout</code>	MINUTES	Number of minutes before a worker process that is not providing status updates is considered inactive. The default timeout is 15.0 minutes. If a worker process exceeds this timeout it is killed by the	
<code>--workers</code>	WORKERS	Number of worker processes. The default is the number of available system CPUs minus one, for the main (foreman) process.	Number of worker processes; tune to the host's cores.

Flag	Argument	What it does	When you would use it
<code>--sigsegv_handler</code>	—	Enables the SIGSEGV handler. WARNING this functionality is experimental and will a deadlock worker process if a real segfault is caught, but not signal SIGSEGV. This functionality is therefore primari	
<code>--sigsegv-handler</code>	—	Enables the SIGSEGV handler. WARNING this functionality is experimental and will a deadlock worker process if a real segfault is caught, but not signal SIGSEGV. This functionality is therefore primari	
<code>--profilers</code>	PROFILERS_LIST	List of profilers to use by the tool. This is a comma separated list where each entry is the name of a profiler. Use "- -profilers list" to list the available profilers.	
<code>--profiling_directory</code>	DIRECTORY	Path to the directory that should be used to store the profiling sample files. By default the sample files are stored in the current working directory.	
<code>--profiling-directory</code>	DIRECTORY	Path to the directory that should be used to store the profiling sample files. By default the sample files are stored in the current working directory.	
<code>-- profiling_sample_rate</code>	SAMPLE_RATE	Profiling sample rate (defaults to a sample every 1000 files).	
<code>--profiling-sample- rate</code>	SAMPLE_RATE	Profiling sample rate (defaults to a sample every 1000 files).	
<code>--storage_file</code>	PATH	The path of the storage file. If not specified, one will be made in the form -.plaso	Where the .plaso output goes.
<code>--storage-file</code>	PATH	The path of the storage file. If not specified, one will be made in the form -.plaso	An analyst would use the -- storage-file flag when processing a storage media image to specify the output file where the extracted timeline events will be stored.

Flag	Argument	What it does	When you would use it
<code>--storage_format</code>	FORMAT	Format of the storage file, the default is: sqlite. Supported options: sqlite	
<code>--storage-format</code>	FORMAT	Format of the storage file, the default is: sqlite. Supported options: sqlite	
<code>--task_storage_format</code>	FORMAT	Format for task storage, the default is: sqlite. Supported options: redis, sqlite	
<code>--task-storage-format</code>	FORMAT	Format for task storage, the default is: sqlite. Supported options: redis, sqlite	

Gotchas

- This produces a .plaso database, **not** a timeline you can read. `psort.py` is the second half; running only this looks like nothing happened.
- A full run on a disk image can take hours and tens of GB. Scope with `--parsers` unless you genuinely need everything.

See also

`psort.py` , `pinfo.py`

mactime

Kit	REMnux · Kali Linux · SIFT Workstation
Capability	Build a filesystem MAC-time timeline
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.sleuthkit.org/sleuthkit

← [Capability index](#) · [Kit tool list](#)

Purpose

Turn a TSK body file into a human-readable chronological timeline.

When you'd reach for this

An analyst reaches for mactime after gathering temporal data from file systems, logs, and other sources into a body file using tools like fls, to sort and merge the data into a single timeline. They would run it after collecting and consolidating all temporal data, as it is specifically designed to handle the body file format and create a chronological view, which is critical for event reconstruction.

Sources: <https://github.com/sleuthkit/sleuthkit/wiki/Timelines>

Common invocations

```
# Generate timeline of file events from MAC time data
mactime -b body.txt -d > timeline.csv
# Generate timeline with file paths and timestamps
mactime -b "$OUTPUT/body.txt" -d -z UTC > "$OUTPUT/timeline.csv"
```

Options

All 9 options parsed from the captured help text; 8 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-b</code>	—	Specifies the body file location, else STDIN is used	Read the body file produced by <code>fls -m</code> — the normal input.
<code>-d</code>	—	Output in comma delimited format	Emit CSV rather than the default text, for spreadsheets or further tooling.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	Display a header with session information	Produce HTML output for a report.
<code>-y</code>	—	Dates are displayed in ISO 8601 format	Print dates ISO-style (year first), which sorts correctly.
<code>-m</code>	—	Dates have month as number instead of word (does not work with <code>-y</code>)	Print month numerically instead of by name.
<code>-z</code>	—	Specify the timezone the data came from (in the local system format) (does not work with <code>-y</code>)	Time zone of the evidence machine. Getting this wrong shifts the whole timeline.
<code>-g</code>	—	Specifies the group file location, else GIDs are used	Map group IDs to names using a supplied group file.
<code>-p</code>	—	Specifies the password file location, else UIDs are used	Map user IDs to names using a supplied passwd file.
<code>-V</code>	—	Prints the version to STDOUT	

Gotchas

- `mactime` reports in the time zone you give it, not the one embedded in the evidence. An unstated `-z` silently produces a plausible, wrong timeline — state it explicitly every run.
- A date range is passed as a trailing argument (`mactime -b body.txt 2026-01-01..2026-02-01`), not as a flag.

See also

`fls`, `tsk_gettimes`

pinfo.py

Kit	Kali Linux · SIFT Workstation
Capability	Build a super-timeline from many artifact sources
Version	plaso - pinfo version 20260512
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Shows information about a Plaso storage file, for example how it was collected, what information was extracted from a source, etc.

Synopsis

```
pininfo.py [-h] [--troubles] [-V] [--logfile FILENAME]
[--process_memory_limit SIZE] [--compare STORAGE_FILE]
[--output_format FORMAT] [--hash TYPE] [--report TYPE]
[--sections SECTIONS_LIST] [-v] [-w OUTPUTFILE]
[PATH]
```

Options

All 20 options parsed from the captured help text; 2 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	Show this help message and exit.	
<code>--help</code>	—	Show this help message and exit.	
<code>--troubles</code>	—	Show troubleshooting information.	
<code>-V</code>	—	Show the version information.	
<code>--version</code>	—	Show the version information.	

Flag	Argument	What it does	When you would use it
<code>--logfile</code>	FILENAME	Path of the file in which to store log messages, by default this file will be named: "pinfo-YYYYMMDDThhmmss.log.gz". Note that the file will be gzip compressed if the extension is ".gz".	
<code>--log_file</code>	FILENAME	Path of the file in which to store log messages, by default this file will be named: "pinfo-YYYYMMDDThhmmss.log.gz". Note that the file will be gzip compressed if the extension is ".gz".	
<code>--log-file</code>	FILENAME	Path of the file in which to store log messages, by default this file will be named: "pinfo-YYYYMMDDThhmmss.log.gz". Note that the file will be gzip compressed if the extension is ".gz".	
<code>--process-memory-limit</code>	SIZE	Maximum amount of memory (data segment) a process is allowed to allocate in bytes, where 0 represents no limit. The default limit is 4294967296 (4 GiB). This applies to both the main (foreman) process	
<code>--process-memory-limit</code>	SIZE	Maximum amount of memory (data segment) a process is allowed to allocate in bytes, where 0 represents no limit. The default limit is 4294967296 (4 GiB). This applies to both the main (foreman) process	
<code>--compare</code>	STORAGE_FILE	The path of the storage file to compare against.	
<code>--output_format</code>	FORMAT	Format of the output, the default is: text. Supported options: json, markdown, text.	
<code>--output-format</code>	FORMAT	Format of the output, the default is: text. Supported options: json, markdown, text.	
<code>--hash</code>	TYPE	Type of hash to output in file_hashes report. Supported options: md5, sha1, sha256	

Flag	Argument	What it does	When you would use it
<code>--report</code>	TYPE	Report on specific information. Supported options: browser_search, chrome_extension, environment_variables, file_hashes, list, none, windows_services, winevt_providers	
<code>--sections</code>	SECTIONS_LIST	List of sections to output. This is a comma separated list where each entry is the name of a section. Use " --sections list" to list the available sections and " --sections all" to show all available	
<code>-v</code>	—	Print verbose output.	An analyst would use the -v flag when troubleshooting processing issues, validating the integrity of a storage file, or verifying the completeness of artifact extraction after creating a Plaso storage file.
<code>--verbose</code>	—	Print verbose output.	An analyst would use the --verbose flag when troubleshooting or performing thorough validation of the collection process, or immediately after storage file creation to verify successful artifact extraction and document provenance.
<code>-w</code>	OUTPUTFILE	Output filename.	
<code>--write</code>	OUTPUTFILE	Output filename.	

See also

`log2timeline.py`, `psort.py`

psort.py

Kit	Kali Linux · SIFT Workstation
Capability	Build a super-timeline from many artifact sources
Version	plaso - psort version 20260512
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Filter, sort and output the events in a Plaso storage file.

When you'd reach for this

An analyst reaches for psort.py after generating a plaso.dump file with log2timeline.py to create a timeline, filter events by tags, or extract time slices; they may run pininfo.py first to inspect the dump's contents, and choose psort over similar tools for its ability to handle complex time-based filtering and tag-based queries directly from the plaso data.

Sources: <https://digitalinvestigator.blogspot.com/2026/07/super-timeline-analysis-with.html> · <https://github.com/log2timeline/plaso/blob/main/docs/sources/user/Using-psort.md> · <https://nullsec.us/supertimeline-quick-reference/>

Synopsis

```
psort.py [-h] [--troubles] [-V] [--analysis PLUGIN_LIST]
[--process_memory_limit SIZE]
[--temporary_directory DIRECTORY] [--worker_memory_limit SIZE]
[--worker_timeout MINUTES] [--logfile FILENAME] [-d] [-q] [-u]
[--status_view TYPE] [--status_view_file PATH]
[--status_view_interval SECONDS] [--slice DATE_TIME]
```

Common invocations

```
# List available output modules for psort
psort.py -o list
# List available analysis plugins for psort
psort.py --analysis list
# List supported time zones for output formatting
psort.py --output-time-zone list
# Check parameters for VirusTotal analysis plugin
psort.py --analysis virustotal -h
# Filter timeline events using specified criteria
psort.py -q timeline.plaso FILTER
# Extract and sort event data from Plaso storage
psort.py -w test.log timeline.plaso
```

Options

All 59 options parsed from the captured help text; 9 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	Show this help message and exit.	
<code>--help</code>	—	Show this help message and exit.	
<code>--troubles</code>	—	Show troubleshooting information.	
<code>-V</code>	—	Show the version information.	
<code>--version</code>	—	Show the version information.	
<code>--analysis</code>	PLUGIN_LIST	A comma separated list of analysis plugin names to be loaded or "--analysis list" to see a list of available plugins.	Run analysis plugins (tagging, sessionizing) over events.
<code>--process-memory-limit</code>	SIZE	Maximum amount of memory (data segment) a process is allowed to allocate in bytes, where 0 represents no limit. The default limit is 4294967296 (4 GiB). This applies to both the main (foreman) process	
<code>--process-memory-limit</code>	SIZE	Maximum amount of memory (data segment) a process is allowed to allocate in bytes, where 0 represents no limit. The default limit is 4294967296 (4 GiB). This applies to both the main (foreman) process	

Flag	Argument	What it does	When you would use it
<code>--temporary_directory</code>	DIRECTORY	Path to the directory that should be used to store temporary files created during processing.	
<code>--temporary-directory</code>	DIRECTORY	Path to the directory that should be used to store temporary files created during processing.	
<code>--worker_memory_limit</code>	SIZE	Maximum amount of memory (data segment and shared memory) a worker process is allowed to consume in bytes, where 0 represents no limit. The default limit is 2147483648 (2 GiB). If a worker process exc	
<code>--worker-memory-limit</code>	SIZE	Maximum amount of memory (data segment and shared memory) a worker process is allowed to consume in bytes, where 0 represents no limit. The default limit is 2147483648 (2 GiB). If a worker process exc	
<code>--worker_timeout</code>	MINUTES	Number of minutes before a worker process that is not providing status updates is considered inactive. The default timeout is 15.0 minutes. If a worker process exceeds this timeout it is killed by the	
<code>--worker-timeout</code>	MINUTES	Number of minutes before a worker process that is not providing status updates is considered inactive. The default timeout is 15.0 minutes. If a worker process exceeds this timeout it is killed by the	
<code>--logfile</code>	FILENAME	Path of the file in which to store log messages, by default this file will be named: "psort-YYYYMMDDThhmmss.log.gz". Note that the file will be gzip compressed if the extension is ".gz".	

Flag	Argument	What it does	When you would use it
<code>--log_file</code>	FILENAME	Path of the file in which to store log messages, by default this file will be named: "psort-YYYYMMDDThhmmss.log.gz". Note that the file will be gzip compressed if the extension is ".gz".	
<code>--log-file</code>	FILENAME	Path of the file in which to store log messages, by default this file will be named: "psort-YYYYMMDDThhmmss.log.gz". Note that the file will be gzip compressed if the extension is ".gz".	
<code>-d</code>	—	Enable debug output.	An analyst would use the -d flag when psort encounters an unexpected exception during runtime to debug the issue by printing the traceback and dropping into pdb.
<code>--debug</code>	—	Enable debug output.	An analyst would use the -d flag when psort encounters an unexpected exception during runtime to debug the issue by printing the traceback and dropping into pdb.
<code>-q</code>	—	Disable informational output.	Quiet, for scripted runs.
<code>--quiet</code>	—	Disable informational output.	Quiet, for scripted runs.
<code>-u</code>	—	Enable unattended mode and do not ask the user for additional input when needed, but terminate with an error instead.	
<code>--unattended</code>	—	Enable unattended mode and do not ask the user for additional input when needed, but terminate with an error instead.	
<code>--status_view</code>	TYPE	The processing status view mode: "file", "linear", "none" or "window".	
<code>--status-view</code>	TYPE	The processing status view mode: "file", "linear", "none" or "window".	
<code>--status_view_file</code>	PATH	The name of the status view file.	

Flag	Argument	What it does	When you would use it
<code>--status-view-file</code>	PATH	The name of the status view file.	
<code>--status-view-interval</code>	SECONDS	Number of seconds to update the status view.	
<code>--status-view-interval</code>	SECONDS	Number of seconds to update the status view.	
<code>--slice</code>	DATE_TIME	Date and time to create a time slice around. This parameter, if defined, will display all events that happened X minutes before and after the defined date, where X is controlled by the <code>--slice_size</code> op	
<code>--slice_size</code>	SLICE_SIZE	Defines the slice size. In the case of a regular time slice it defines the number of minutes the slice size should be. In the case of the <code>--slicer</code> it determines the number of events before and after a	
<code>--slice-size</code>	SLICE_SIZE	Defines the slice size. In the case of a regular time slice it defines the number of minutes the slice size should be. In the case of the <code>--slicer</code> it determines the number of events before and after a	
<code>--slicer</code>	—	Create a time slice around every filter match. This parameter, if defined will save all X events before and after a filter match has been made. X is defined by the <code>--slice_size</code> parameter.	An analyst would use the <code>--slicer</code> flag when filtering specific events but also needing to include surrounding context to better understand the timeline or related activities.
<code>--data</code>	PATH	Path to a directory containing the data files.	An analyst would use the <code>--data</code> flag when they need to specify a custom location for filter files or databases, such as when using a non-default <code>winevt-rc.db</code> file.
<code>-a</code>	—	By default the <code>psort</code> removes duplicate entries from the output. This parameter changes that behavior so all events are included.	An analyst would use the <code>-a</code> flag when processing a storage media file or VSS store where duplicate entries need to be preserved for comprehensive analysis.

Flag	Argument	What it does	When you would use it
<code>--include_all</code>	—	By default the psort removes duplicate entries from the output. This parameter changes that behavior so all events are included.	An analyst would use the -a flag when processing a storage media file or VSS store where duplicate entries need to be preserved for comprehensive analysis.
<code>--include-all</code>	—	By default the psort removes duplicate entries from the output. This parameter changes that behavior so all events are included.	An analyst would use the -a flag when processing a storage media file or VSS store where duplicate entries need to be preserved for comprehensive analysis.
<code>--language</code>	LANGUAGE_TAG	The preferred language, which is used for extracting and formatting Windows EventLog message strings. Use " --language list" to see a list of supported language tags. The en-US (LCID 0x0409) language	
<code>--additional_fields</code>	ADDITIONAL_FIELDS	Defines additional fields to be included in the output besides the default fields. Multiple additional field names can be defined as a list of comma separated values. Output formats that support addit	
<code>--additional-fields</code>	ADDITIONAL_FIELDS	Defines additional fields to be included in the output besides the default fields. Multiple additional field names can be defined as a list of comma separated values. Output formats that support addit	
<code>--custom_fields</code>	CUSTOM_FIELDS	Defines custom fields to be included in the output besides the default fields. A custom field is defined as "name:value". Multiple custom field names can be defined as list of comma separated values.	
<code>--custom-fields</code>	CUSTOM_FIELDS	Defines custom fields to be included in the output besides the default fields. A custom field is defined as "name:value". Multiple custom field names can be defined as list of comma separated values.	

Flag	Argument	What it does	When you would use it
<code>--custom_formatter_definitions</code>	PATH	Path to a file containing custom event formatter definitions, which is a .yaml file. Custom event formatter definitions can be used to customize event messages and override the built-in event formatter	
<code>--custom-formatter-definitions</code>	PATH	Path to a file containing custom event formatter definitions, which is a .yaml file. Custom event formatter definitions can be used to customize event messages and override the built-in event formatter	
<code>--dynamic_time</code>	—	Indicate that the output should use dynamic time. Output formats that support dynamic time are: dynamic	
<code>--dynamic-time</code>	—	Indicate that the output should use dynamic time. Output formats that support dynamic time are: dynamic	
<code>--output_time_zone</code>	TIME_ZONE	time zone of date and time values written to the output, if supported by the output format. Use "list" to see a list of available time zones. Output formats that support an output time zone are: dynam	An analyst would use the --output-time-zone flag when they need to adjust the time zone of the date and time values in the output events to match a specific location or requirement, rather than the default UTC.
<code>--output-time-zone</code>	TIME_ZONE	time zone of date and time values written to the output, if supported by the output format. Use "list" to see a list of available time zones. Output formats that support an output time zone are: dynam	An analyst would use the --output-time-zone flag when they need to adjust the time zone of the date and time values in the output events to match a specific location or requirement, rather than the default UTC.
<code>-o</code>	FORMAT	The output format. Use "-o list" to see a list of available output formats.	Output format — <code>l2tcsv</code> , <code>dynamic</code> , <code>json</code> , or a timeline tool.
<code>--output_format</code>	FORMAT	The output format. Use "-o list" to see a list of available output formats.	Output format — <code>l2tcsv</code> , <code>dynamic</code> , <code>json</code> , or a timeline tool.

Flag	Argument	What it does	When you would use it
<code>--output-format</code>	FORMAT	The output format. Use "-o list" to see a list of available output formats.	Output format — <code>l2tcsv</code> , <code>dynamic</code> , <code>json</code> , or a timeline tool.
<code>-w</code>	OUTPUT_FILE	Output filename.	Write output to a file rather than stdout.
<code>--write</code>	OUTPUT_FILE	Output filename.	Write output to a file rather than stdout.
<code>--fields</code>	FIELDS	Defines which fields should be included in the output.	
<code>--profilers</code>	PROFILERS_LIST	List of profilers to use by the tool. This is a comma separated list where each entry is the name of a profiler. Use "-profilers list" to list the available profilers.	
<code>--profiling_directory</code>	DIRECTORY	Path to the directory that should be used to store the profiling sample files. By default the sample files are stored in the current working directory.	
<code>--profiling-directory</code>	DIRECTORY	Path to the directory that should be used to store the profiling sample files. By default the sample files are stored in the current working directory.	
<code>--profiling_sample_rate</code>	SAMPLE_RATE	Profiling sample rate (defaults to a sample every 1000 files).	
<code>--profiling-sample-rate</code>	SAMPLE_RATE	Profiling sample rate (defaults to a sample every 1000 files).	

Gotchas

- The date filter is a positional argument, not a flag. Filtering to the incident window is what makes a multi-million-event timeline usable.
- Output ordering follows the storage file, so always sort or filter explicitly rather than assuming chronology.

See also

`log2timeline.py`, `pinfo.py`

tsk_gettimes

Kit	REMnux · Kali Linux · SIFT Workstation
Capability	Build a filesystem MAC-time timeline
Version	The Sleuth Kit ver 4.11.1
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.sleuthkit.org/sleuthkit

← [Capability index](#) · [Kit tool list](#)

Purpose

Analyze disk images and recover files from them.

Synopsis

```
tsk_gettimes [-vVm] [-i imgtype] [-b dev_sector_size] [-z zone] [-s seconds] image [image]
```

Common invocations

```
# Extract file timestamps from disk image
tsk_gettimes ./image.dd > body.txt
```

Options

All 7 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
<code>-i</code>	imgtype	The format of the image file (use '-i list' for supported types)	
<code>-b</code>	dev_sector_size	The size (in bytes) of the device sectors	
<code>-m</code>	—	Calculate MD5 hash in output (slow)	
<code>-v</code>	—	verbose output to stderr	
<code>-V</code>	—	Print version	

Flag	Argument	What it does	When you would use it
<code>-z</code>	—	Time zone of original machine (i.e. EST5EDT or GMT) (only useful with <code>-l</code>)	
<code>-s</code>	seconds	Time skew of original machine (in seconds) (only useful with <code>-l</code> & <code>-m</code>)	

See also

`fls`, `mactime`

CryptoTester (GUI)

Capability	decode deobfuscate
Window title	CryptoTester
Captured from	C:\Tools\CryptoTester\CryptoTester.exe on 2026-08-02 — control tree in capture/gui/CryptoTester/CryptoTester.tree.txt

[← Capability index](#) · [Kit tool list](#)

Purpose

Try cryptographic and encoding operations against a sample interactively: XOR, block ciphers, hashes and conversions, with entropy and pattern views to judge whether a result is plausible plaintext. Built for the guess-and-check work of recovering a malware configuration blob.

When you'd reach for this

Reach for CryptoTester when you have a blob you believe is encoded or encrypted - a configuration block, a C2 address, part of a dropper - and you are working out how. It applies XOR, block ciphers, hashes and conversions interactively and shows entropy alongside, so you can judge whether a candidate result is plausible plaintext rather than guessing.

It is built for guess-and-check, which is the honest shape of this work: you rarely know the scheme in advance. Once you do, scripting the transform is faster and repeatable - this is the tool for the part before that.

Sources: <https://www.nexttron-systems.com/cryptotester/>

Controls

The parts of this window you will actually touch, read from the application's own accessibility tree rather than from a screenshot. The full node list is in [capture/gui/CryptoTester/CryptoTester.tree.txt](#).

The window exposes 105 further named controls: **Idle, File, Open File, Input, Text, Base64, Zero Bytes, Sequential Bytes, Integer, Chrysanthemum.jpg, Desert.jpg, Save Output, Tools, Blob Analyzer, RSA Calculator, Key Finder, Keystream Finder, RNG Tester, Base Encoder, String Encoder, Unit Tester, ECC Validator, Chunk Viewer, Operations, XOR Files, AND Files, Generate Keystream, Visual Difference, Bruteforce Algorithm, Bruteforce Keys, Attempt Blind Decryption, Advanced, Little Endian, Custom, Rounds, Constant, Position, Matrix, IV, Get IV From Input**, and 65 more. The full tree, with every automation id, is in [the capture](#).

Using it

1. Paste or load the ciphertext.
2. Try the obvious first: single-byte XOR and the common block ciphers cover most malware configuration blobs.

3. Use the entropy and pattern views to judge whether the result is plausible plaintext before believing a key.
4. Record the key and mode that worked; a decryption nobody can reproduce is not a finding.

Gotchas

- It is an experiment bench, not an oracle. It tells you a transformation produced output, not that the output is correct.
- Plausible-looking plaintext from a short sample is often coincidence. Confirm against a second sample before concluding.

Kit	REMnux · FLARE-VM · Security Onion
Capability	Decode, decrypt or transform encoded data
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://github.com/gchq/CyberChef/

← [Capability index](#) · [Kit tool list](#)

Purpose

Decode and otherwise analyze data using this browser app.

When you'd reach for this

When an analyst needs to decrypt, decode, or transform data, they reach for CyberChef due to its versatile interface and client-side processing, which ensures no data is transmitted to third-party servers, making it preferable over tools that may not handle sensitive information as securely.

Sources: <https://github.com/martinmathurine/Cryptography-Decryption-CyberChef-Lab> · <https://www.it-connect.tech/cyberchef-a-web-application-for-decrypting-decoding-and-transforming-data/>

Options

No option definitions could be parsed from this tool's help output. It may be subcommand-driven or have no flags; needs manual review.

See also

[base64dump.py](#), [rax2](#), [xxd](#), [openssl](#), [numbers-to-string.py](#)

hashcat

Kit	Kali Linux
Capability	Crack passwords and hashes
Version	v6.2.6
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

← [Capability index](#) · [Kit tool list](#)

Purpose

Recover passwords from hashes using GPU-accelerated guessing — dictionary, rule-mutated, mask and brute-force attacks across several hundred hash types. In DFIR it is usually pointed at credentials recovered from a host to establish what an attacker could have reused elsewhere.

When you'd reach for this

An analyst reaches for hashcat when dealing with hashes like MD5, using wordlists such as rockyou.txt for brute-force or combination attacks, and runs it after identifying the hash type and preparing input files; they choose it over similar tools due to its GPU-accelerated cracking capabilities and support for advanced attack modes like mask attacks, as demonstrated in the examples.

Sources: <https://github.com/IPIRATExAPTAIN/htb-academy/blob/main/CrackingPasswordsWithHashcat.md> · https://hashcat.net/wiki/doku.php?id=frequently_asked_questions

Synopsis

```
hashcat [options]... hash|hashfile|hccapxfile [dictionary|mask|directory]...
```

Common invocations


```
# Restore a paused hash cracking session using saved state
hashcat --restore --session=my_session
# Coordinate distributed cracking to avoid redundant attempts
hashcat --brain-server
# Crack SHA1 hashes using a wordlist to recover passwords
hashcat -m 100 hashes.txt wordlist.txt
# Display backend info for performance tuning
hashcat --backend-info
# Display available OpenCL devices for selection
hashcat -I
# Test hardware performance for cracking speed
hashcat -b
```

Options

All 143 options parsed from the captured help text; 12 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-m</code>	—	Num Hash-type, references below (otherwise autodetect) -m 1000	An analyst would use the -m flag when specifying the hash type (e.g., MD5, SHA-256) to ensure Hashcat correctly interprets the hash format during cracking attempts.
<code>--hash-type</code>	—	Num Hash-type, references below (otherwise autodetect) -m 1000	An analyst would use the -m flag when specifying the hash type (e.g., MD5, SHA-256) to ensure Hashcat correctly interprets the hash format during cracking attempts.
<code>-a</code>	—	Num Attack-mode, see references below -a 3	An analyst would use the -a flag when performing a combination attack to generate password combinations from two separate wordlists.
<code>--attack-mode</code>	—	Num Attack-mode, see references below -a 3	An analyst would use the -a flag when performing a combination attack to generate password combinations from two separate wordlists.
<code>-V</code>	—	Print version	
<code>--version</code>	—	Print version	
<code>-h</code>	—	Print help	
<code>--help</code>	—	Print help	
<code>--quiet</code>	—	Suppress output	

Flag	Argument	What it does	When you would use it
<code>--hex-charset</code>	—	Assume charset is given in hex	
<code>--hex-salt</code>	—	Assume salt is given in hex	
<code>--hex-wordlist</code>	—	Assume words in wordlist are given in hex	
<code>--force</code>	—	Ignore warnings	
<code>--status</code>	—	Enable automatic update of the status screen	
<code>--status-json</code>	—	Enable JSON format for status output	
<code>--status-timer</code>	—	Num Sets seconds between status screen updates to X <code>--status-timer=1</code>	
<code>--stdin-timeout-abort</code>	—	Num Abort if there is no input from stdin for X seconds <code>--stdin-timeout-abort=300</code>	
<code>--machine-readable</code>	—	Display the status view in a machine-readable format	
<code>--keep-guessing</code>	—	Keep guessing the hash after it has been cracked	
<code>--self-test-disable</code>	—	Disable self-test functionality on startup	
<code>--loopback</code>	—	Add new plains to induct directory	
<code>--markov-hcstat2</code>	—	File Specify hcstat2 file to use <code>--markov-hcstat2=my.hcstat2</code>	
<code>--markov-disable</code>	—	Disables markov-chains, emulates classic brute-force	
<code>--markov-classic</code>	—	Enables classic markov-chains, no per-position	
<code>--markov-inverse</code>	—	Enables inverse markov-chains, no per-position	
<code>-t</code>	—	Num Threshold X when to stop accepting new markov-chains <code>-t 50</code>	
<code>--markov-threshold</code>	—	Num Threshold X when to stop accepting new markov-chains <code>-t 50</code>	
<code>--runtime</code>	—	Num Abort session after X seconds of runtime <code>--runtime=10</code>	

Flag	Argument	What it does	When you would use it
<code>--session</code>	—	Str Define specific session name <code>--session=mysession</code>	An analyst would use the <code>--session</code> flag when resuming an interrupted hashcat session to continue cracking from the last checkpointed position.
<code>--restore</code>	—	Restore session from <code>--session</code>	
<code>--restore-disable</code>	—	Do not write restore file	
<code>--restore-file-path</code>	—	File Specific path to restore file <code>--restore-file-path=x.restore</code>	
<code>-o</code>	—	File Define outfile for recovered hash <code>-o outfile.txt</code>	An analyst would use the <code>-o</code> flag when they need to specify the output file path for storing cracked hashes, such as in the example where 'cracked.txt' is used.
<code>--outfile</code>	—	File Define outfile for recovered hash <code>-o outfile.txt</code>	An analyst would use the <code>-o</code> flag when they need to specify the output file path for storing cracked hashes, such as in the example where 'cracked.txt' is used.
<code>--outfile-format</code>	—	Str Outfile format to use, separated with commas <code>--outfile-format=1,3</code>	
<code>--outfile-autohex-disable</code>	—	Disable the use of \$HEX[] in output plains	
<code>--outfile-check-timer</code>	—	Num Sets seconds between outfile checks to X <code>--outfile-check-timer=30</code>	
<code>-p</code>	—	Char Separator char for hashlists and outfile <code>-p :</code>	
<code>--separator</code>	—	Char Separator char for hashlists and outfile <code>-p :</code>	
<code>--stdout</code>	—	Do not crack a hash, instead print candidates only	An analyst would use the <code>--stdout</code> flag when generating custom wordlists by specifying mask patterns to create targeted combinations of characters for cracking hashes.

Flag	Argument	What it does	When you would use it
<code>--show</code>	—	Compare hashlist with potfile; show cracked hashes	An analyst would use the --show flag to display previously cracked hashes stored in the potfile when verifying results or avoiding redundant cracking efforts.
<code>--left</code>	—	Compare hashlist with potfile; show uncracked hashes	
<code>--username</code>	—	Enable ignoring of usernames in hashfile	
<code>--remove</code>	—	Enable removal of hashes once they are cracked	
<code>--remove-timer</code>	—	Num Update input hash file each X seconds --remove-timer=30	
<code>--potfile-disable</code>	—	Do not write potfile	
<code>--potfile-path</code>	—	File Specific path to potfile --potfile-path=my.pot	
<code>--encoding-from</code>	—	Code Force internal wordlist encoding from X --encoding-from=iso-8859-15	
<code>--encoding-to</code>	—	Code Force internal wordlist encoding to X --encoding-to=utf-32le	
<code>--debug-mode</code>	—	Num Defines the debug mode (hybrid only by using rules) --debug-mode=4	
<code>--debug-file</code>	—	File Output file for debugging rules --debug-file=good.log	
<code>--induction-dir</code>	—	Dir Specify the induction directory to use for loopback --induction=inducts	
<code>--outfile-check-dir</code>	—	Dir Specify the outfile directory to monitor for plains --outfile-check-dir=x	
<code>--logfile-disable</code>	—	Disable the logfile	
<code>--hccapx-message-pair</code>	—	Num Load only message pairs from hccapx matching X --hccapx-message-pair=2	
<code>--nonce-error-corrections</code>	—	Num The BF size range to replace AP's nonce last bytes --nonce-error-corrections=16	

Flag	Argument	What it does	When you would use it
<code>--keyboard-layout-mapping</code>	—	File Keyboard layout mapping table for special hash-modes <code>--keyb=german.hckmap</code>	
<code>--truecrypt-keyfiles</code>	—	File Keyfiles to use, separated with commas <code>--truecrypt-keyf=x.png</code>	
<code>--veracrypt-keyfiles</code>	—	File Keyfiles to use, separated with commas <code>--veracrypt-keyf=x.txt</code>	
<code>--veracrypt-pim-start</code>	—	Num VeraCrypt personal iterations multiplier start <code>--veracrypt-pim-start=450</code>	
<code>--veracrypt-pim-stop</code>	—	Num VeraCrypt personal iterations multiplier stop <code>--veracrypt-pim-stop=500</code>	
<code>-b</code>	—	Run benchmark of selected hash-modes	
<code>--benchmark</code>	—	Run benchmark of selected hash-modes	
<code>--benchmark-all</code>	—	Run benchmark of all hash-modes (requires -b)	
<code>--speed-only</code>	—	Return expected speed of the attack, then quit	
<code>--progress-only</code>	—	Return ideal progress step size and time to process	
<code>-c</code>	—	Num Sets size in MB to cache from the wordfile to X <code>-c 32</code>	
<code>--segment-size</code>	—	Num Sets size in MB to cache from the wordfile to X <code>-c 32</code>	
<code>--bitmap-min</code>	—	Num Sets minimum bits allowed for bitmaps to X <code>--bitmap-min=24</code>	
<code>--bitmap-max</code>	—	Num Sets maximum bits allowed for bitmaps to X <code>--bitmap-max=24</code>	
<code>--cpu-affinity</code>	—	Str Locks to CPU devices, separated with commas <code>--cpu-affinity=1,2,3</code>	
<code>--hook-threads</code>	—	Num Sets number of threads for a hook (per compute unit) <code>--hook-threads=8</code>	

Flag	Argument	What it does	When you would use it
<code>--hash-info</code>	—	Show information for each hash-mode	
<code>--example-hashes</code>	—	Alias of --hash-info	
<code>--backend-ignore-cuda</code>	—	Do not try to open CUDA interface on startup	
<code>--backend-ignore-hip</code>	—	Do not try to open HIP interface on startup	
<code>--backend-ignore-metal</code>	—	Do not try to open Metal interface on startup	
<code>--backend-ignore-opengl</code>	—	Do not try to open OpenGL interface on startup	
<code>-I</code>	—	Show system/environment/backend API info -I or -II	
<code>--backend-info</code>	—	Show system/environment/backend API info -I or -II	
<code>-d</code>	—	Str Backend devices to use, separated with commas -d 1	An analyst would use the -d flag when specifying a particular GPU device in a multi-GPU setup where hashcat encounters mapping errors due to identical or similarly identified devices, requiring manual selection to bypass temperature or fan control issues.
<code>--backend-devices</code>	—	Str Backend devices to use, separated with commas -d 1	An analyst would use the -d flag when specifying a particular GPU device in a multi-GPU setup where hashcat encounters mapping errors due to identical or similarly identified devices, requiring manual selection to bypass temperature or fan control issues.
<code>-D</code>	—	Str OpenCL device-types to use, separated with commas -D 1	An analyst would use the -D flag when they need to specify whether to use the CPU, GPU, or both for hash cracking based on available hardware resources.

Flag	Argument	What it does	When you would use it
<code>--opencl-device-types</code>	—	Str OpenCL device-types to use, separated with commas -D 1	An analyst would use the -D flag when they need to specify whether to use the CPU, GPU, or both for hash cracking based on available hardware resources.
<code>-O</code>	—	Enable optimized kernels (limits password length)	
<code>--optimized-kernel-enable</code>	—	Enable optimized kernels (limits password length)	
<code>-M</code>	—	Disable multiply kernel-accel with processor count	
<code>--multiply-accel-disable</code>	—	Disable multiply kernel-accel with processor count	
<code>-w</code>	—	Num Enable a specific workload profile, see pool below -w 3	An analyst would use the -w flag when optimizing Hashcat performance on a dedicated cracking rig with a GPU not driving a display, specifically setting -w 4 for maximum workload intensity.
<code>--workload-profile</code>	—	Num Enable a specific workload profile, see pool below -w 3	An analyst would use the -w flag when optimizing Hashcat performance on a dedicated cracking rig with a GPU not driving a display, specifically setting -w 4 for maximum workload intensity.
<code>-n</code>	—	Num Manual workload tuning, set outerloop step size to X -n 64	
<code>--kernel-accel</code>	—	Num Manual workload tuning, set outerloop step size to X -n 64	
<code>-u</code>	—	Num Manual workload tuning, set innerloop step size to X -u 256	
<code>--kernel-loops</code>	—	Num Manual workload tuning, set innerloop step size to X -u 256	
<code>-T</code>	—	Num Manual workload tuning, set thread count to X -T 64	
<code>--kernel-threads</code>	—	Num Manual workload tuning, set thread count to X -T 64	

Flag	Argument	What it does	When you would use it
<code>--backend-vector-width</code>	—	Num Manually override backend vector-width to X <code>--backend-vector=4</code>	
<code>--spin-damp</code>	—	Num Use CPU for device synchronization, in percent <code>--spin-damp=10</code>	
<code>--hwmon-disable</code>	—	Disable temperature and fanspeed reads and triggers	
<code>--hwmon-temp-abort</code>	—	Num Abort if temperature reaches X degrees Celsius <code>--hwmon-temp-abort=100</code>	
<code>--script-tmto</code>	—	Num Manually override TMTO value for script to X <code>--script-tmto=3</code>	
<code>-s</code>	—	Num Skip X words from the start <code>-s 1000000</code>	
<code>--skip</code>	—	Num Skip X words from the start <code>-s 1000000</code>	
<code>-l</code>	—	Num Limit X words from the start + skipped words <code>-l 1000000</code>	
<code>--limit</code>	—	Num Limit X words from the start + skipped words <code>-l 1000000</code>	
<code>--keyspace</code>	—	Show keyspace base:mod values and quit	
<code>-j</code>	—	Rule Single rule applied to each word from left wordlist <code>-j 'c'</code>	
<code>--rule-left</code>	—	Rule Single rule applied to each word from left wordlist <code>-j 'c'</code>	
<code>-k</code>	—	Rule Single rule applied to each word from right wordlist <code>-k '^.'</code>	
<code>--rule-right</code>	—	Rule Single rule applied to each word from right wordlist <code>-k '^.'</code>	

Flag	Argument	What it does	When you would use it
<code>-r</code>	—	File Multiple rules applied to each word from wordlists -r rules/best64.rule	An analyst would use the -r flag when applying custom or built-in rule sets to a wordlist to generate password variations during cracking attacks, as demonstrated in the examples involving rules/best64.rule and modifying rules to append specific strings like years to passwords.
<code>--rules-file</code>	—	File Multiple rules applied to each word from wordlists -r rules/best64.rule	An analyst would use the -r flag when applying custom or built-in rule sets to a wordlist to generate password variations during cracking attacks, as demonstrated in the examples involving rules/best64.rule and modifying rules to append specific strings like years to passwords.
<code>-g</code>	—	Num Generate X random rules -g 10000	An analyst would use the -g flag when encountering errors related to excessive rule usage, such as <code>clEnqueueCopyBuffer() -30</code> or <code>cuStreamSynchronize() 702</code> , to reduce the number of rules and resolve the issue.
<code>--generate-rules</code>	—	Num Generate X random rules -g 10000	An analyst would use the -g flag when encountering errors related to excessive rule usage, such as <code>clEnqueueCopyBuffer() -30</code> or <code>cuStreamSynchronize() 702</code> , to reduce the number of rules and resolve the issue.
<code>--generate-rules-func-min</code>	—	Num Force min X functions per rule	
<code>--generate-rules-func-max</code>	—	Num Force max X functions per rule	
<code>--generate-rules-func-sel</code>	—	Str Pool of rule operators valid for random rule engine -generate-rules-func-sel=ioTlc	
<code>--generate-rules-seed</code>	—	Num Force RNG seed set to X	

Flag	Argument	What it does	When you would use it
<code>-1</code>	—	CS User-defined charset ?1 -1 ?l?d?u	An analyst would use the --custom-charset1 flag when defining a custom character set (e.g., ?l?d) to reference in a mask with ?1, such as in a hashcat mask file line like "?l?d,?l?l?l?l?1" to specify a combination of lowercase letters and digits for password cracking.
<code>--custom-charset1</code>	—	CS User-defined charset ?1 -1 ?l?d?u	An analyst would use the --custom-charset1 flag when defining a custom character set (e.g., ?l?d) to reference in a mask with ?1, such as in a hashcat mask file line like "?l?d,?l?l?l?l?1" to specify a combination of lowercase letters and digits for password cracking.
<code>-2</code>	—	CS User-defined charset ?2 -2 ?l?d?s	
<code>--custom-charset2</code>	—	CS User-defined charset ?2 -2 ?l?d?s	
<code>-3</code>	—	CS User-defined charset ?3	
<code>--custom-charset3</code>	—	CS User-defined charset ?3	
<code>-4</code>	—	CS User-defined charset ?4	
<code>--custom-charset4</code>	—	CS User-defined charset ?4	
<code>--identify</code>	—	Shows all supported algorithms for input hashes --identify my.hash	
<code>-i</code>	—	Enable mask increment mode	
<code>--increment</code>	—	Enable mask increment mode	
<code>--increment-min</code>	—	Num Start mask incrementing at X --increment-min=4	
<code>--increment-max</code>	—	Num Stop mask incrementing at X --increment-max=8	
<code>-S</code>	—	Enable slower (but advanced) candidate generators	

Flag	Argument	What it does	When you would use it
<code>--slow-candidates</code>	—	Enable slower (but advanced) candidate generators	
<code>--brain-server</code>	—	Enable brain server	
<code>--brain-server-timer</code>	—	Num Update the brain server dump each X seconds (min:60) --brain-server-timer=300	
<code>-z</code>	—	Enable brain client, activates -S	
<code>--brain-client</code>	—	Enable brain client, activates -S	
<code>--brain-client-features</code>	—	Num Define brain client features, see below --brain-client-features=3	
<code>--brain-host</code>	—	Str Brain server host (IP or domain) --brain-host=127.0.0.1	
<code>--brain-port</code>	—	Port Brain server port --brain-port=13743	
<code>--brain-password</code>	—	Str Brain server authentication password --brain-password=bZfhCvGUSjRq	
<code>--brain-session</code>	—	Hex Overrides automatically calculated brain session --brain-session=0x2ae611db	
<code>--brain-session-whitelist</code>	—	Hex Allow given sessions only, separated with commas --brain-session-whitelist=0x2ae611db	

See also

[john](#), [hydra](#)

hydra

Kit	Kali Linux · SIFT Workstation
Capability	Crack passwords and hashes
Version	Hydra v9.4
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Test credentials against a network service across many protocols. In an authorised engagement it answers whether a recovered password works elsewhere; it is loud, it locks accounts, and it belongs nowhere near production without written permission.

When you'd reach for this

An analyst reaches for Hydra after enumeration and gathering web-form details from tools like Burp Suite, running it for online brute-force attacks on SSH or web forms; they choose it over similar tools like John the Ripper because Hydra operates online, making it suitable for live targets requiring real-time credential testing.

Sources: <https://crackerfrank.hashnode.dev/cracking-passwords-with-hydra-a-tryhackme-walkthrough> · <https://hackproofhacks.com/blog/password-cracking-with-hydra-hacking-series/> · <https://www.freecodecamp.org/news/how-to-use-hydra-pentesting-tutorial/>

Options

No option definitions could be parsed from this tool's help output. It may be subcommand-driven or have no flags; needs manual review.

See also

`hashcat` , `john`

john

Kit	Kali Linux
Capability	Crack passwords and hashes
Captured from	<code>cyberlab-aio</code> via <code>(no args)</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Copyright (c) 1996-2019 by Solar Designer

When you'd reach for this

An analyst reaches for John when attempting to crack password hashes, often after preparing a larger wordlist and configuring the tool's settings, as it supports multiple modes like single crack, wordlist with rules, and incremental cracking for thoroughness. They may run `john --show` afterward to display cracked passwords, preferring John over similar tools due to its flexibility in using custom charsets, filters, and incremental modes tailored to specific password patterns.

Sources: <https://www.openwall.com/john/doc/EXAMPLES.shtml>

Synopsis

```
john [OPTIONS] [PASSWORD-FILES]
```

Options

All 21 options parsed from the captured help text; 15 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>--single</code>	—	"single crack" mode	An analyst would use the <code>--single</code> flag when auditing Linux system passwords to quickly identify weak passwords in hash files during the initial phase of a security assessment.

Flag	Argument	What it does	When you would use it
<code>--wordlist</code>	FILE	wordlist mode, read words from FILE or stdin	An analyst would use the <code>--wordlist</code> flag when attempting to crack password hashes by feeding John the Ripper a file of potential passwords, such as the <code>rockyou.txt</code> wordlist, to compare against the target hash file.
<code>--stdin</code>	—	wordlist mode, read words from FILE or stdin	An analyst would use the <code>--wordlist</code> flag when attempting to crack password hashes by feeding John the Ripper a file of potential passwords, such as the <code>rockyou.txt</code> wordlist, to compare against the target hash file.
<code>--rules</code>	—	enable word mangling rules for wordlist mode	An analyst would use the <code>--rules</code> flag when applying word mangling rules to a wordlist to generate variations of passwords for cracking, as demonstrated in examples like <code>"john --wordlist=all.lst --rules mypasswd"</code> and similar commands in the documentation.
<code>--incremental</code>	MODE (optional)	"incremental" mode [using section MODE]	An analyst would use the <code>--incremental</code> flag when the wordlist has been exhausted and the hash remains uncracked, particularly for short passwords (5-7 characters) where incremental brute-force is feasible.
<code>--external</code>	MODE	external mode or word filter	An analyst would use the <code>--external</code> flag when generating a custom charset file with specific word filters to consider simpler passwords, as demonstrated in the example with <code>--external=filter_alpha</code> .

Flag	Argument	What it does	When you would use it
<code>--stdout</code>	LENGTH (optional)	just output candidate passwords [cut at LENGTH]	An analyst would use the <code>--stdout</code> flag when processing the output of John's password cracking through the 'unique' utility to eliminate duplicate candidate passwords, as demonstrated in the examples where mangled passwords are piped into 'unique' for deduplication.
<code>--restore</code>	NAME (optional)	restore an interrupted session [called NAME]	An analyst would use the <code>--restore</code> flag when resuming an interrupted session to continue cracking passwords from where it left off.
<code>--session</code>	NAME	give a new session the NAME	An analyst would use the <code>--session</code> flag when running multiple parallel cracking sessions or resuming an interrupted session to avoid conflicts and ensure proper restoration from a saved session state.
<code>--status</code>	NAME (optional)	print status of a session [called NAME]	An analyst would use the <code>--status</code> flag to check the status of a running or interrupted John session, such as when monitoring progress or resuming a paused cracking process.
<code>--make-charset</code>	FILE	make a charset, FILE will be overwritten	An analyst would use the <code>--make-charset</code> flag when generating a custom character set file based on character frequencies from a password file containing many already cracked passwords or multiple password files from the same organization or country.
<code>--show</code>	—	show cracked passwords	An analyst would use the <code>--show</code> flag after successfully cracking passwords to display the cracked credentials in a human-readable format for review or documentation.
<code>--test</code>	TIME (optional)	run tests and benchmarks for TIME seconds each	

Flag	Argument	What it does	When you would use it
<code>--users</code>	<code>[-]LOGIN UID[,...]</code>	[do not] load this (these) user(s) only	An analyst would use the <code>--users</code> flag when checking if cracked accounts correspond to specific UIDs, such as root (UID 0), or when isolating specific usernames like "root" in the output.
<code>--groups</code>	<code>[-]GID[,...]</code>	load users [not] of this (these) group(s) only	An analyst would use the <code>--groups</code> flag when checking for cracked accounts in specific privileged groups, such as those with group IDs 0 or 1, to identify compromised administrative or high-privilege user accounts.
<code>--shells</code>	<code>[-]SHELL[,...]</code>	load users with[out] this (these) shell(s) only	An analyst would use the <code>--shells</code> flag when excluding accounts with disabled shells from the cracked password report.
<code>--salts</code>	<code>[-]N</code>	load salts with[out] at least N passwords only	
<code>--save-memory</code>	LEVEL	enable memory saving, at LEVEL 1..3	
<code>--node</code>	MIN[-MAX]	this node's number range out of TOTAL count	
<code>--fork</code>	N	fork N processes	
<code>--format</code>	NAME	force hash type NAME: descrypt/bsdicrypt/md5crypt/ bcrypt/LM/AFS/tripcode/dum my/crypt	An analyst would use the <code>--format</code> flag when cracking hashes from specific sources like NTDS.dit or /etc/shadow, or when the hash type requires a specific format identifier such as NT or raw-md5 to ensure John the Ripper correctly interprets the hash structure.

See also

[hashcat](#) , [hydra](#)

openssl

Kit	Base OS — present on every Linux image
Capability	Decode, decrypt or transform encoded data
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

The general-purpose crypto toolkit: inspect certificates, compute digests, encrypt and decrypt, and speak TLS to a service. In analysis it is most often used to read a certificate a sample presented, or to decrypt a blob once the key is known.

When you'd reach for this

An analyst reaches for openssl when testing SSL/TLS connections to servers (e.g., using `s_client` to connect to ports like 993 or 995) or generating cryptographic digests (e.g., MD5 or SHA1) for file integrity checks. They may run these commands before verifying server configurations or after obtaining data for forensic analysis, as openssl provides direct command-line tools for these tasks without requiring additional software. They might prefer it over similar tools for its simplicity in quick tests or when specific functions like base64 encoding/decoding are needed.

Sources: <https://www.golinuxcloud.com/openssl-cheatsheet/> · <https://www.madboa.com/geek/openssl/>

Common invocations

```
# Verify certificate validity and trust chain
openssl verify cert.pem
```

Options

No option definitions could be parsed from this tool's help output. It may be subcommand-driven or have no flags; needs manual review.

See also

[cyberchef](#) , [base64dump.py](#) , [rax2](#) , [xxd](#) , [numbers-to-string.py](#)

rax2

Kit	REMnux · Kali Linux · SIFT Workstation
Capability	Decode, decrypt or transform encoded data
Version	rax2 6.1.9
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.radare.org/n/radare2.html

← [Capability index](#) · [Kit tool list](#)

Purpose

Examine binary files, including disassembling and debugging. Includes r2ai and decai plugins for LLM-powered analysis (API key or local Ollama required), plus the r2ghidra plugin for Ghidra decompilation via the pdg command.

Synopsis

```
rax2 [-h|...] [- | expr ...] # convert between numeric bases
int      -> hex           ; rax2 10
hex      -> int           ; rax2 0xa
-int     -> hex           ; rax2 -77
-hex     -> int           ; rax2 0xffffffffb3
int      -> bin           ; rax2 b30
```

Options

All 30 options parsed from the captured help text; 8 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-a</code>	—	show ascii table ; rax2 -a	
<code>-b</code>	base	output in ; rax2 -b 10 0x46	An analyst would use the -b flag when converting binary data into a string representation for analysis or documentation.
<code>-c</code>	—	output in C string ; rax2 -c 0x1234 # \x34\x12\x00\x00	
<code>-C</code>	—	dump as C byte array ; rax2 -C < bytes	

Flag	Argument	What it does	When you would use it
<code>-d</code>	—	force integer ; rax2 -d 3 -> 3 instead of 0x3	
<code>-e</code>	—	swap endianness ; rax2 -e 0x33	An analyst would use the -e flag when converting between different endianness representations, such as swapping byte order in hexadecimal values during data analysis.
<code>-D</code>	—	base64 decode ; rax2 -D "aGVsbG8="	An analyst would use the -D flag when decoding a base64 encoded string to retrieve its original binary or textual content.
<code>-E</code>	—	base64 encode ; rax2 -E "hello"	
<code>-f</code>	—	floating point ; rax2 -f 6.3+2.1	
<code>-F</code>	—	stdin slurp code hex ; rax2 -F < shellcode.[c/py/js]	An analyst would use the -F flag when processing hexadecimal data from standard input, such as converting shellcode from a file into another format for analysis.
<code>-h</code>	—	help ; rax2 -h	
<code>-H</code>	—	hash string ; rax2 -H linux osx	
<code>-i</code>	—	IP address <-> LONG ; rax2 -i 3530468537	
<code>-j</code>	—	json format output ; rax2 -j 0x1234 # same as r2 -c '?j 0x1234'	
<code>-k</code>	—	keep base ; rax2 -k 33+3 -> 36	An analyst would use the -k flag when performing calculations or conversions that require retaining the original numeric base representation of the input values.
<code>-K</code>	—	randomart ; rax2 -K 0x34 1020304050	An analyst would use the -K flag when generating a randomart visualization of binary data, such as for creating a visual representation of a hash or hexadecimal value in a forensic report.
<code>-n</code>	—	newline ; append newline to output (for -E/-D/-r/..)	

Flag	Argument	What it does	When you would use it
<code>-o</code>	—	octalstr -> raw ; rax2 -o \162 \62 # r2	
<code>-q</code>	—	quiet mode ; rax2 -qC < /etc/hosts # be quiet	
<code>-r</code>	—	r2 style output ; rax2 -r 0x1234 # same as r2 -c '? 0x1234'	
<code>-s</code>	—	hexstr -> raw ; rax2 -s 43 4a 50	An analyst would use the -s flag when converting a hexadecimal string into its corresponding raw byte representation for further analysis or processing.
<code>-S</code>	—	raw -> hexstr ; rax2 -S < /bin/l > ls.hex	An analyst would use the -S flag when converting raw binary data into a hexadecimal string representation for analysis or documentation.
<code>-t</code>	—	tstamp -> str ; rax2 -t 1234567890	
<code>-u</code>	—	units ; rax2 -u 389289238 # 317.0M	
<code>-v</code>	—	version ; rax2 -v	
<code>-w</code>	—	signed word ; rax2 -w 0xffff 0xffff_ffff '0xff&0xffff'	
<code>-x</code>	—	output in hexpairs ; rax2 -x 0x1234 # 34120000	
<code>-X</code>	—	bin -> hex(bignum) ; rax2 -X 111111111 # 0x1ff	
<code>-z</code>	—	str -> bin ; rax2 -z hello	
<code>-Z</code>	—	bin -> str ; rax2 -Z 01000101 01110110	

See also

[cyberchef](#) , [base64dump.py](#) , [xxd](#) , [openssl](#) , [numbers-to-string.py](#)

binwalk

Kit	REMnux · Kali Linux
Capability	Carve files out of unstructured data; Detect and reverse packing; Find hidden data
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://github.com/ReFirmLabs/binwalk

← [Capability index](#) · [Kit tool list](#)

Purpose

Find and extract embedded files and filesystems inside a binary blob or firmware image.

When you'd reach for this

An analyst reaches for binwalk when examining firmware images to identify embedded files, compressed data, or cryptographic keys, often after obtaining a firmware dump from a device; they may run it before deeper analysis to map contents or after extracting files for further inspection, preferring it for its entropy analysis and custom signature capabilities over tools lacking these specific features.

Sources: <https://github.com/ReFirmLabs/binwalk/wiki/Usage> · <https://www.hardbreak.wiki/hardware-hacking/basics/tools/software-tools/binwalk>

Synopsis

```
binwalk [OPTIONS] [FILE1] [FILE2] [FILE3] ...
```

Common invocations

```
# Scan firmware for embedded files and file systems
binwalk firmware.bin

# Extract embedded files from firmware image
binwalk -e firmware.bin

# Detect encrypted or compressed sections via entropy analysis
binwalk -E firmware.bin
```

Options

All 102 options parsed from the captured help text; 8 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-B</code>	—	Scan target file(s) for common file signatures	
<code>--signature</code>	—	Scan target file(s) for common file signatures	
<code>-R</code>	str	Scan target file(s) for the specified sequence of bytes	
<code>--raw</code>	str	Scan target file(s) for the specified sequence of bytes	
<code>-A</code>	—	Scan target file(s) for common executable opcode signatures	Scan for executable opcodes to identify architecture.
<code>--opcodes</code>	—	Scan target file(s) for common executable opcode signatures	Scan for executable opcodes to identify architecture.
<code>-m</code>	file	Specify a custom magic file to use	
<code>--magic</code>	file	Specify a custom magic file to use	
<code>-b</code>	—	Disable smart signature keywords	
<code>--dumb</code>	—	Disable smart signature keywords	
<code>-I</code>	—	Show results marked as invalid	
<code>--invalid</code>	—	Show results marked as invalid	
<code>-x</code>	str	Exclude results that match	Exclude signatures matching this string, to cut false hits.
<code>--exclude</code>	str	Exclude results that match	Exclude signatures matching this string, to cut false hits.
<code>-y</code>	str	Only show results that match	Only report signatures matching this string.
<code>--include</code>	str	Only show results that match	Only report signatures matching this string.
<code>-e</code>	—	Automatically extract known file types	Extract what is found, rather than only listing it.
<code>--extract</code>	—	Automatically extract known file types	Extract what is found, rather than only listing it.
<code>-D</code>	type[:ext[:cmd]]	Extract signatures (regular expression), give the files an extension of , and execute	
<code>--dd</code>	type[:ext[:cmd]]	Extract signatures (regular expression), give the files an extension of , and execute	

Flag	Argument	What it does	When you would use it
<code>-M</code>	—	Recursively scan extracted files	Recurse into extracted files (matryoshka) — for nested firmware.
<code>--matryoshka</code>	—	Recursively scan extracted files	Recurse into extracted files (matryoshka) — for nested firmware.
<code>-d</code>	int	Limit matryoshka recursion depth (default: 8 levels deep)	Limit recursion depth; unbounded <code>-M</code> can explode.
<code>--depth</code>	int	Limit matryoshka recursion depth (default: 8 levels deep)	Limit recursion depth; unbounded <code>-M</code> can explode.
<code>-C</code>	str	Extract files/folders to a custom directory (default: current working directory)	Choose the output directory for extractions.
<code>--directory</code>	str	Extract files/folders to a custom directory (default: current working directory)	Choose the output directory for extractions.
<code>-j</code>	int	Limit the size of each extracted file	
<code>--size</code>	int	Limit the size of each extracted file	
<code>-n</code>	int	Limit the number of extracted files	
<code>--count</code>	int	Limit the number of extracted files	
<code>-0</code>	str	Execute external extraction utilities with the specified user's privileges	
<code>--run-as</code>	str	Execute external extraction utilities with the specified user's privileges	
<code>-1</code>	—	Do not sanitize extracted symlinks that point outside the extraction directory (dangerous)	
<code>--preserve-symlinks</code>	—	Do not sanitize extracted symlinks that point outside the extraction directory (dangerous)	
<code>-r</code>	—	Delete carved files after extraction	
<code>--rm</code>	—	Delete carved files after extraction	
<code>-z</code>	—	Carve data from files, but don't execute extraction utilities	

Flag	Argument	What it does	When you would use it
<code>--carve</code>	—	Carve data from files, but don't execute extraction utilities	
<code>-V</code>	—	Extract into sub-directories named by the offset	
<code>--subdirs</code>	—	Extract into sub-directories named by the offset	
<code>-E</code>	—	Calculate file entropy	Entropy analysis — the fast way to spot encryption or compression.
<code>--entropy</code>	—	Calculate file entropy	Entropy analysis — the fast way to spot encryption or compression.
<code>-F</code>	—	Use faster, but less detailed, entropy analysis	
<code>--fast</code>	—	Use faster, but less detailed, entropy analysis	
<code>-J</code>	—	Save plot as a PNG	
<code>--save</code>	—	Save plot as a PNG	
<code>-Q</code>	—	Omit the legend from the entropy plot graph	
<code>--nlegend</code>	—	Omit the legend from the entropy plot graph	
<code>-N</code>	—	Do not generate an entropy plot graph	
<code>--nplot</code>	—	Do not generate an entropy plot graph	
<code>-H</code>	float	Set the rising edge entropy trigger threshold (default: 0.95)	
<code>--high</code>	float	Set the rising edge entropy trigger threshold (default: 0.95)	
<code>-L</code>	float	Set the falling edge entropy trigger threshold (default: 0.85)	
<code>--low</code>	float	Set the falling edge entropy trigger threshold (default: 0.85)	
<code>-W</code>	—	Perform a hexdump / diff of a file or files	
<code>--hexdump</code>	—	Perform a hexdump / diff of a file or files	
<code>-G</code>	—	Only show lines containing bytes that are the same among all files	

Flag	Argument	What it does	When you would use it
<code>--green</code>	—	Only show lines containing bytes that are the same among all files	
<code>-i</code>	—	Only show lines containing bytes that are different among all files	
<code>--red</code>	—	Only show lines containing bytes that are different among all files	
<code>-U</code>	—	Only show lines containing bytes that are different among some files	
<code>--blue</code>	—	Only show lines containing bytes that are different among some files	
<code>-u</code>	—	Only display lines that are the same between all files	
<code>--similar</code>	—	Only display lines that are the same between all files	
<code>-w</code>	—	Diff all files, but only display a hex dump of the first file	
<code>--terse</code>	—	Diff all files, but only display a hex dump of the first file	
<code>-X</code>	—	Scan for raw deflate compression streams	
<code>--deflate</code>	—	Scan for raw deflate compression streams	
<code>-Z</code>	—	Scan for raw LZMA compression streams	
<code>--lzma</code>	—	Scan for raw LZMA compression streams	
<code>-P</code>	—	Perform a superficial, but faster, scan	
<code>--partial</code>	—	Perform a superficial, but faster, scan	
<code>-S</code>	—	Stop after the first result	
<code>--stop</code>	—	Stop after the first result	
<code>-l</code>	int	Number of bytes to scan	
<code>--length</code>	int	Number of bytes to scan	
<code>-o</code>	int	Start scan at this file offset	
<code>--offset</code>	int	Start scan at this file offset	

Flag	Argument	What it does	When you would use it
<code>-O</code>	int	Add a base address to all printed offsets	
<code>--base</code>	int	Add a base address to all printed offsets	
<code>-K</code>	int	Set file block size	
<code>--block</code>	int	Set file block size	
<code>-g</code>	int	Reverse every n bytes before scanning	
<code>--swap</code>	int	Reverse every n bytes before scanning	
<code>-f</code>	file	Log results to file	
<code>--log</code>	file	Log results to file	
<code>-c</code>	—	Log results to file in CSV format	
<code>--csv</code>	—	Log results to file in CSV format	
<code>-t</code>	—	Format output to fit the terminal window	
<code>--term</code>	—	Format output to fit the terminal window	
<code>-q</code>	—	Suppress output to stdout	
<code>--quiet</code>	—	Suppress output to stdout	
<code>-v</code>	—	Enable verbose output	
<code>--verbose</code>	—	Enable verbose output	
<code>-h</code>	—	Show help output	
<code>--help</code>	—	Show help output	
<code>-a</code>	str	Only scan files whose names match this regex	
<code>--finclude</code>	str	Only scan files whose names match this regex	
<code>-p</code>	str	Do not scan files whose names match this regex	
<code>--fexclude</code>	str	Do not scan files whose names match this regex	
<code>-s</code>	int	Enable the status server on the specified port	
<code>--status</code>	int	Enable the status server on the specified port	

Gotchas

- **Always timeout-guard binwalk in a harness.** Signature scanning on a large or synthetic image can run effectively forever; this project has been bitten by exactly that.
- `-e -M` on an unknown blob can produce an enormous directory tree. Set `-d` and carve into a scratch filesystem, not your case directory.
- A signature hit is a guess based on magic bytes. Confirm with `file` or entropy before reporting an embedded filesystem as fact.

See also

`foremost`, `scalpel`, `bulk_extractor`, `tcpextract`, `upx`, `die`, `diec`, `7za`

blkls

Kit	REMnux · Kali Linux · SIFT Workstation
Capability	Recover deleted or lost files
Version	The Sleuth Kit ver 4.11.1
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.sleuthkit.org/sleuthkit

← [Capability index](#) · [Kit tool list](#)

Purpose

Extract filesystem blocks — by default the unallocated ones, which is the input a carver wants.

When you'd reach for this

An analyst uses blkls when recovering files from unallocated space after inodes are overwritten, running it after failed inode-based recovery attempts to extract raw unallocated data, then using carving tools like foremost or photorec on the output; they choose it over similar tools because it directly extracts unallocated space for carving when traditional file system metadata is unavailable.

Sources: <https://github.com/sleuthkit/sleuthkit/wiki/Body-file> · <https://github.com/sleuthkit/sleuthkit/wiki/Timelines> · <https://oneuptime.com/blog/post/2026-03-02-how-to-use-sleuth-kit-for-file-system-forensics-on-ubuntu/view>

Synopsis

```
blkls [-aAeLvV] [-f fstype] [-i imgtype] [-b dev_sector_size] [-o imgoffset] [-P pooltype] [-B pool_volume_block] image [images] [start-stop]
```

Common invocations

```
# Extract unallocated data fragments from image
blkls images/wd0e.dd > output/wd0e.blkls
# Extract unallocated space data from evidence
blkls -A -o 2048 "$EVIDENCE" > unallocated.raw
```

Options

All 13 options parsed from the captured help text; 12 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-e</code>	—	every block (including file system metadata blocks)	Every block, including filesystem metadata.
<code>-l</code>	—	print details in time machine list format	List block details rather than emitting their contents.
<code>-a</code>	—	Display allocated blocks	Allocated blocks only — the inverse, when isolating live data.
<code>-A</code>	—	Display unallocated blocks	Unallocated blocks. The default and the usual intent: pipe this into <code>foremost</code> or <code>scalpel</code> so the carver reads only free space instead of the whole image.
<code>-f</code>	fstype	File system type (use '-f list' for supported types)	Force the filesystem type (<code>-f list</code> shows the options).
<code>-i</code>	imgtype	The format of the image file (use '-i list' for supported types)	Image format for non-raw evidence such as E01 or AFF.
<code>-b</code>	dev_sector_size	The size (in bytes) of the device sectors	Device sector size; required on 4Kn drives.
<code>-o</code>	imgoffset	The offset of the file system in the image (in sectors)	Offset, in sectors, from <code>mm1s</code> .
<code>-P</code>	pooltype	Pool container type (use '-P list' for supported types)	Pool type, for APFS or LVM containers.
<code>-B</code>	pool_volume_block	Starting block (for pool volumes only)	Starting block within a pool volume.
<code>-s</code>	—	print slack space only (other flags are ignored)	Slack space only: the tail of the last block of each file, where fragments of previous contents survive. A distinct hunt from carving free space, and it ignores the other flags.
<code>-v</code>	—	verbose to stderr	Verbose diagnostics to stderr.
<code>-V</code>	—	print version	

Gotchas

- Output goes to stdout and is the size of the free space — redirect it to a file on a volume that can hold it, not into a pager.
- Block offsets in the extracted stream do **not** match offsets in the original image, because only unallocated blocks were written. Use `-l` if you need to map a hit back to its real location.

See also

tsk_recover, icat, photorec, testdisk

bulk_extractor

Kit	REMnux · Kali Linux · SIFT Workstation
Capability	Carve files out of unstructured data; Search raw data for a pattern; Recover encryption keys from memory
Version	bulk_extractor 2.2.0
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://github.com/simsong/bulk_extractor/

[← Capability index](#) · [Kit tool list](#)

Purpose

Scan an image for features — email addresses, URLs, credit card numbers, EXIF, keys — without parsing the filesystem at all, so deleted and unallocated content is included.

Synopsis

```
bulk_extractor [OPTION...] image_name
```

Common invocations

```
# Carve NTFS MFT records from volume E
bulk_extractor -E ntfsmft -o output \\.\\E:
# Extract files from disk image to output directory
bulk_extractor -o bulk-out xp-laptop-2005-07-04-1430.img
# Extract gzip and utmp data from Linux disk image
bulk_extractor -x all -e gzip -e utmp -o output Linux.E01
# Carve contiguous JPEGs from disk image
bulk_extractor -o out_folder -S jpeg_carve_mode=2 /evidence/disk.img
```

Options

All 69 options parsed from the captured help text; 58 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-A</code>	arg	Offset added (in bytes) to feature locations (default: 0)	Add an offset to reported feature locations — use it when scanning a carved fragment so offsets still refer to the original image.
<code>--offset_add</code>	arg	Offset added (in bytes) to feature locations (default: 0)	Add an offset to reported feature locations.
<code>-b</code>	arg	Path of file whose contents are prepended to top of all feature files	Prepend a banner file to every feature file, e.g. a case header.
<code>--banner_file</code>	arg	Path of file whose contents are prepended to top of all feature files	Prepend a banner file to every feature file.
<code>-C</code>	arg	Size of context window reported in bytes (default: 16)	Bytes of context stored around each hit. Raise it when a bare match is not enough to judge relevance.
<code>--context_window</code>	arg	Size of context window reported in bytes (default: 16)	Bytes of context stored around each hit.
<code>-d</code>	—	enable debug-level diagnostic logging	Debug-level diagnostic logging.
<code>--debug</code>	—	enable debug-level diagnostic logging	Debug-level diagnostic logging.
<code>-E</code>	arg	disable all scanners except the one specified. Same as <code>-x</code> all <code>-E</code> scanner.	Run exactly one scanner and disable the rest. The fastest way to answer a single question instead of a full sweep.
<code>--enable_exclusive</code>	arg	disable all scanners except the one specified. Same as <code>-x</code> all <code>-E</code> scanner.	Run exactly one scanner and disable the rest.
<code>-e</code>	arg	enable a scanner (can be repeated)	Enable a scanner that is off by default, repeatable.
<code>--enable</code>	arg	enable a scanner (can be repeated)	Enable a scanner that is off by default, repeatable.
<code>-x</code>	arg	disable a scanner (can be repeated)	Disable a scanner, repeatable. Turning off the noisy ones is usually a bigger speed win than adding threads.
<code>--disable</code>	arg	disable a scanner (can be repeated)	Disable a scanner, repeatable.
<code>-f</code>	arg	search for a pattern (can be repeated)	Search for a regex pattern, repeatable.

Flag	Argument	What it does	When you would use it
<code>--find</code>	arg	search for a pattern (can be repeated)	Search for a regex pattern, repeatable.
<code>-F</code>	arg	read patterns to search from a file (can be repeated)	Read search patterns from a file — the practical form when hunting a list of indicators.
<code>--find_file</code>	arg	read patterns to search from a file (can be repeated)	Read search patterns from a file.
<code>--find-case-sensitive</code>	—	make <code>-f</code> and <code>-F</code> patterns case-sensitive	Make <code>-f</code> / <code>-F</code> case-sensitive; they are not by default.
<code>-G</code>	arg	page size in bytes (default: 16777216)	Page size read per pass.
<code>--pagesize</code>	arg	page size in bytes (default: 16777216)	Page size read per pass.
<code>-g</code>	arg	margin size in bytes (default: 4194304)	Margin carried between pages, so a feature spanning a page boundary is still found.
<code>--marginsize</code>	arg	margin size in bytes (default: 4194304)	Margin carried between pages, so features spanning a page boundary are still found.
<code>-j</code>	arg	number of threads (default: 12)	Thread count. Defaults to the core count; lower it when the scan is competing with other work.
<code>--threads</code>	arg	number of threads (default: 12)	Thread count.
<code>-J</code>	—	read and process data in the primary thread	Single-threaded. Slow, but the first thing to try when a scan crashes or results look non-deterministic.
<code>--no_threads</code>	—	read and process data in the primary thread	Single-threaded — use when debugging a crash.
<code>-M</code>	arg	max recursion depth (default: 12)	Maximum recursion depth into nested containers. Lower it if a zip bomb or deeply nested archive stalls the scan.
<code>--max_depth</code>	arg	max recursion depth (default: 12)	Maximum recursion depth into nested containers.
<code>--max_bad_alloc_errors</code>	arg	max bad allocation errors (default: 3)	Allocation failures tolerated before aborting.
<code>--max_minute_wait</code>	arg	maximum number of minutes to wait until all data are read (default: 60)	How long to wait for all data to be read before giving up — raise it for slow or failing media.

Flag	Argument	What it does	When you would use it
<code>--log-level</code>	arg	diagnostic log level: trace, debug, info, warning, error, critical, or off	Diagnostic log level.
<code>--log-file</code>	arg	diagnostic log file (default: /bulk_extractor.log)	Diagnostic log file; defaults inside the output directory.
<code>--notify_main_thread</code>	—	Display notifications in the main thread after phase1 completes. Useful for running with ThreadSanitizer	
<code>--notify_async</code>	—	Display notificaitons asynchronously (default)	
<code>-o</code>	arg	output directory [REQUIRED]	Output directory. Required, and it must not already exist unless you also pass <code>-Z</code> .
<code>--outdir</code>	arg	output directory [REQUIRED]	Output directory. Required.
<code>-P</code>	arg	directories for scanner shared libraries (can be repeated). Default directories include /usr/local/lib/bulk_extractor, /usr/lib/bulk_extractor and any directories specified in the BE_PATH environment	Additional directories to load scanner plugins from.
<code>--scanner_dir</code>	arg	directories for scanner shared libraries (can be repeated). Default directories include /usr/local/lib/bulk_extractor, /usr/lib/bulk_extractor and any directories specified in the BE_PATH environment	Additional directories to load scanner plugins from.
<code>-p</code>	arg	print the value of [:length][h] [r] with optional length, hex output, or raw output.	Print the value at a path, with optional length and hex or raw output — inspection rather than scanning.
<code>--path</code>	arg	print the value of [:length][h] [r] with optional length, hex output, or raw output.	Print the value at a path, for inspection rather than scanning.
<code>-q</code>	—	no status or performance output	Suppress status and performance output.
<code>--quit</code>	—	no status or performance output	Suppress status and performance output.
<code>-r</code>	arg	file to read alert list from	Alert list: features to flag prominently — the inverse of a stop list, for known-bad indicators.

Flag	Argument	What it does	When you would use it
<code>--alert_list</code>	arg	file to read alert list from	Alert list of features to flag prominently.
<code>-R</code>	—	treat image file as a directory to recursively explore	Treat the input as a directory and recurse it, rather than as a disk image.
<code>--recurse</code>	—	treat image file as a directory to recursively explore	Treat the input as a directory and recurse it.
<code>-S</code>	arg	set a name=value option (can be repeated)	Set a scanner option as <code>name=value</code> , repeatable.
<code>--set</code>	arg	set a name=value option (can be repeated)	Set a scanner option as <code>name=value</code> , repeatable.
<code>-s</code>	arg	random sampling parameter <code>frac[:passes]</code>	Random sampling as <code>frac[:passes]</code> . Scans a fraction of a huge image to judge whether a full run is worth the hours.
<code>--sampling</code>	arg	random sampling parameter <code>frac[:passes]</code>	Random sampling as <code>frac[:passes]</code> .
<code>-V</code>	—	Display PACKAGE_VERSION (currently) 2.2.0-DEVELOP	
<code>--version</code>	—	Display PACKAGE_VERSION (currently) 2.2.0-DEVELOP	
<code>-w</code>	arg	file to read stop list from	Stop list: features to suppress. This is how you cut the known-good noise that otherwise buries the findings.
<code>--stop_list</code>	arg	file to read stop list from	Stop list of features to suppress.
<code>-Y</code>	arg	specify [-end] of area on disk to scan	Restrict the scan to a byte range, when <code>mmls</code> already told you which region matters.
<code>--scan</code>	arg	specify [-end] of area on disk to scan	Restrict the scan to a <code><start>[-end]</code> byte range.
<code>-z</code>	arg	specify a starting page number	Start at a given page number, to resume a long scan.
<code>--page_start</code>	arg	specify a starting page number	Start at a given page number.
<code>-Z</code>	—	wipe the output directory (recursively) before starting	Wipe the output directory first. Convenient for reruns and destructive by definition — never point it at a directory holding results you still need.
<code>--zap</code>	—	wipe the output directory (recursively) before starting	Wipe the output directory first — destructive.

Flag	Argument	What it does	When you would use it
<code>-0</code>	—	disable real-time notification	
<code>--no_notify</code>	—	disable real-time notification	
<code>-1</code>	—	version 1.0 notification (console-output)	
<code>--version1</code>	—	version 1.0 notification (console-output)	
<code>-H</code>	—	report information about each scanner	Report what each scanner does. Worth reading once; the scanner set is the tool.
<code>--info_scanners</code>	—	report information about each scanner	Report what each scanner does.
<code>-h</code>	—	print help screen	
<code>--help</code>	—	print help screen	

Gotchas

- It ignores the filesystem completely. That is the point — it finds content in unallocated space and slack that a filesystem-aware tool cannot reach — but it also means a hit carries no filename and no timestamp. Map the offset back with `ffind` / `istat` before naming a file in a report.
- Feature files are raw pattern matches, not verified findings. The credit-card scanner in particular flags anything passing a Luhn check, which includes plenty of ordinary numbers.
- Output is large and the scan is long. Sample with `-s` on a multi-terabyte image before committing to a full pass.

See also

`foremost`, `scalpel`, `binwalk`, `tcpxtract`, `rafind2`, `strings`, `grep`, `xxd`

ffind

Kit	REMnux · Kali Linux · SIFT Workstation
Capability	List files and directories, including deleted ones
Version	The Sleuth Kit ver 4.11.1
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.sleuthkit.org/sleuthkit

← [Capability index](#) · [Kit tool list](#)

Purpose

Find the file name that points at a given inode — the reverse of a directory lookup.

When you'd reach for this

An analyst reaches for `ffind` when searching for files based on string content or file signatures within a disk image, often after creating an image with tools like `dd`, as it efficiently locates files without requiring prior knowledge of inode numbers, making it preferable to manual searches or tools like `fls` for metadata-based queries.

Sources: <https://github.com/sleuthkit/sleuthkit/wiki/Body-file> · <https://github.com/sleuthkit/sleuthkit/wiki/Timelines> · <https://hackernoon.com/getting-started-with-digital-forensics-using-the-sleuth-kit-c34a3wkg>

Synopsis

```
ffind [-aduvV] [-f fstype] [-i imgtype] [-b dev_sector_size] [-o imgoffset] [-P pooltype] [-B pool_volume_block] image [images] inode
```

Common invocations

```
# Find deleted files associated with inode 493
ffind -a images/wd0e.dd 493

# Scripted analysis of evidence with ffind
ffind -o 2048 "$EVIDENCE" 12345
```

Options

All 11 options parsed from the captured help text; 10 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-a</code>	—	Find all occurrences	Show every name for the inode. Hard-linked files have more than one, and stopping at the first hides that.
<code>-d</code>	—	Find deleted entries ONLY	Deleted names only. The direct answer to "what was this inode called before it was removed?"
<code>-u</code>	—	Find undeleted entries ONLY	Undeleted names only, when a recycled inode is returning stale hits.
<code>-f</code>	<code>fstype</code>	Image file system type (use '-f list' for supported types)	Force the filesystem type (<code>-f list</code> shows the options).
<code>-i</code>	<code>imgtype</code>	The format of the image file (use '-i list' for supported types)	Image format for non-raw evidence such as E01 or AFF.
<code>-b</code>	<code>dev_sector_size</code>	The size (in bytes) of the device sectors	Device sector size; required on 4Kn drives.
<code>-o</code>	<code>imgoffset</code>	The offset of the file system in the image (in sectors)	Offset, in sectors, from <code>mmLs</code> .
<code>-P</code>	<code>pooltype</code>	Pool container type (use '-p list' for supported types)	Pool type, for APFS or LVM containers.
<code>-B</code>	<code>pool_volume_block</code>	Starting block (for pool volumes only)	Starting block within a pool volume.
<code>-v</code>	—	Verbose output to stderr	Verbose diagnostics to stderr.
<code>-V</code>	—	Print version	

Gotchas

- Inodes are reused. A name returned for a deleted inode may belong to whatever claimed it next, not to the file you are chasing — corroborate with `istat` timestamps before naming it in a report.
- This is the tool for the question `icat` provokes: you carved data out by inode and now need to say what it was called.

See also

`fls`, `ils`, `tsk_recover`

file

Kit	REMnux · FLARE-VM · SIFT Workstation
Capability	Inspect metadata for one file or inode; Identify what a file actually is
Version	file-5.44
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://github.com/file/file

← [Capability index](#) · [Kit tool list](#)

Purpose

Identify a file's type from its contents rather than its name, using magic signatures.

Synopsis

```
file [OPTION...] [FILE...]
```

Common invocations

```
# Identify file type and format
file app.py
# Identify file MIME type for scripting or automation
file -i script.py
# Identify file types and their MIME information
file -b document.pdf
# Identify file type and MIME for analysis
file -b -i photo.jpg
# Process a list of files to identify their types
file -f filelist.txt
# Identify the actual file type of a symbolic link target
file -L /usr/bin/python
```

Options

All 54 options parsed from the captured help text; 20 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>--help</code>	—	display this help and exit	
<code>-v</code>	—	output version information and exit	
<code>--version</code>	—	output version information and exit	
<code>-m</code>	LIST	use LIST as a colon-separated list of magic number files	Use an alternative magic database.
<code>--magic-file</code>	LIST	use LIST as a colon-separated list of magic number files	Use an alternative magic database.
<code>-z</code>	—	try to look inside compressed files	Look inside compressed files and report what they contain rather than reporting a compressed stream.
<code>--uncompress</code>	—	try to look inside compressed files	Look inside compressed files and report what they contain rather than reporting a compressed stream.
<code>-Z</code>	—	only print the contents of compressed files	Same, without reporting the compression itself.
<code>--uncompress-noreport</code>	—	only print the contents of compressed files	Same, without reporting the compression itself.
<code>-b</code>	—	do not prepend filenames to output lines	Omit the filename, leaving just the type — for pipelines and for-loops.
<code>--brief</code>	—	do not prepend filenames to output lines	Omit the filename, leaving just the type — for pipelines and for-loops.
<code>-c</code>	—	print the parsed form of the magic file, use in conjunction with -m to debug a new magic file before installing it	
<code>--checking-printout</code>	—	print the parsed form of the magic file, use in conjunction with -m to debug a new magic file before installing it	
<code>-e</code>	TEST	exclude TEST from the list of test to be performed for file. Valid tests are: apptype, ascii, cdf, compress, csv, elf, encoding, soft, tar, json, text, tokens	Exclude a test type, when one is misfiring on a corpus.

Flag	Argument	What it does	When you would use it
<code>--exclude</code>	TEST	exclude TEST from the list of test to be performed for file. Valid tests are: apptype, ascii, cdf, compress, csv, elf, encoding, soft, tar, json, text, tokens	Exclude a test type, when one is misfiring on a corpus.
<code>--exclude-quiet</code>	TEST	like exclude, but ignore unknown tests	
<code>-f</code>	FILE	read the filenames to be examined from FILE	Read the list of files to test from a file, for a corpus.
<code>--files-from</code>	FILE	read the filenames to be examined from FILE	Read the list of files to test from a file, for a corpus.
<code>-F</code>	STRING	use string as separator instead of `:`	
<code>--separator</code>	STRING	use string as separator instead of `:`	
<code>-i</code>	—	output MIME type strings (--mime-type and --mime-encoding)	Print a MIME type instead of prose. The form to use when the output feeds a script rather than a person.
<code>--mime</code>	—	output MIME type strings (--mime-type and --mime-encoding)	Print a MIME type instead of prose. The form to use when the output feeds a script rather than a person.
<code>--apple</code>	—	output the Apple CREATOR/TYPE	
<code>--extension</code>	—	output a slash-separated list of extensions	
<code>--mime-type</code>	—	output the MIME type	MIME type only, without the encoding.
<code>--mime-encoding</code>	—	output the MIME encoding	Character encoding only.
<code>-k</code>	—	don't stop at the first match	Keep going after the first match and print every rule that fired. Files crafted to defeat identification often match more than one signature, and the disagreement is the finding.
<code>--keep-going</code>	—	don't stop at the first match	Keep going after the first match and print every rule that fired. Files crafted to defeat identification often match more than one signature, and the disagreement is the finding.

Flag	Argument	What it does	When you would use it
<code>-l</code>	—	list magic strength	
<code>--list</code>	—	list magic strength	
<code>-L</code>	—	follow symlinks (default if POSIXLY_CORRECT is set)	Follow symlinks and report the target.
<code>--dereference</code>	—	follow symlinks (default if POSIXLY_CORRECT is set)	Follow symlinks and report the target.
<code>-h</code>	—	don't follow symlinks (default if POSIXLY_CORRECT is not set) (default)	Do not follow symlinks — report the link itself.
<code>--no-dereference</code>	—	don't follow symlinks (default if POSIXLY_CORRECT is not set) (default)	Do not follow symlinks — report the link itself.
<code>-n</code>	—	do not buffer output	
<code>--no-buffer</code>	—	do not buffer output	
<code>-N</code>	—	do not pad output	Do not pad filenames to align the output.
<code>--no-pad</code>	—	do not pad output	Do not pad filenames to align the output.
<code>-0</code>	—	terminate filenames with ASCII NUL	Print a NUL after the filename, for safe piping into <code>xargs -0</code> .
<code>--print0</code>	—	terminate filenames with ASCII NUL	Print a NUL after the filename, for safe piping into <code>xargs -0</code> .
<code>-p</code>	—	preserve access times on files	Preserve the access time on the files examined — the flag that stops a triage sweep from rewriting timestamps across the evidence.
<code>--preserve-date</code>	—	preserve access times on files	Preserve the access time on the files examined — the flag that stops a triage sweep from rewriting timestamps across the evidence.
<code>-P</code>	—	set file engine parameter limits bytes 7340032 max bytes to look inside file elf_notes 256 max ELF notes processed elf_phnum 2048 max ELF prog sections processed elf_shnum 32768 max ELF sections proce	Tune a parser limit, e.g. <code>bytes</code> or <code>indir</code> .

Flag	Argument	What it does	When you would use it
<code>--parameter</code>	—	set file engine parameter limits bytes 7340032 max bytes to look inside file elf_notes 256 max ELF notes processed elf_phnum 2048 max ELF prog sections processed elf_shnum 32768 max ELF sections proce	Tune a parser limit, e.g. <code>bytes</code> or <code>indir</code> .
<code>-r</code>	—	don't translate unprintable chars to \ooo	Print raw bytes rather than escaping unprintables.
<code>--raw</code>	—	don't translate unprintable chars to \ooo	Print raw bytes rather than escaping unprintables.
<code>-s</code>	—	treat special (block/char devices) files as ordinary ones	Read block and character devices too. Needed to type a raw disk, which <code>file</code> otherwise refuses.
<code>--special-files</code>	—	treat special (block/char devices) files as ordinary ones	Read block and character devices too. Needed to type a raw disk, which <code>file</code> otherwise refuses.
<code>-S</code>	—	disable system call sandboxing	Disable the sandbox. Only when seccomp blocks a legitimate test, and never on untrusted input if avoidable.
<code>--no-sandbox</code>	—	disable system call sandboxing	Disable the sandbox. Only when seccomp blocks a legitimate test, and never on untrusted input if avoidable.
<code>-C</code>	—	compile file specified by -m	Compile a magic file to its indexed form.
<code>--compile</code>	—	compile file specified by -m	Compile a magic file to its indexed form.
<code>-d</code>	—	print debugging messages	
<code>--debug</code>	—	print debugging messages	

Gotchas

- Magic reads the first few hundred bytes. Prepend a valid header to anything and `file` will report the header — it is a fast triage signal, not an authority on content.
- `-p` preserves atime. Without it, typing a directory tree updates access times across the evidence, which is the kind of avoidable contamination that gets noticed later.
- An 'ASCII text' verdict on something you expected to be binary usually means it is base64 or hex, not that it is harmless.

See also

istat, ils, stat, die, diec

fls

Kit: SIFT Workstation · REMnux · Kali (`sleuthkit`) **Category:** Filesystem analysis **Version:** The Sleuth Kit 4.11.1
Docs: <https://sleuthkit.org/sleuthkit/man/fls.html> **Verified:** 2026-07-29 from `cyberlab-aio:v1` — raw output in `capture/cyberlab-aio/help/fls.help.txt`

Worked example of the format. Every option below was read off the real binary, not written from memory.
See [docs/FORMAT.md](#).

Purpose

List file and directory names in a disk image, including deleted entries.

Synopsis

```
fls [-adDFlhpruvV] [-f fstype] [-i imgtype] [-b dev_sector_size] [-m dir/]  
    [-o imgoffset] [-z ZONE] [-s seconds] image [images] [inode]
```

If `[inode]` is not given, the root directory is used.

Note: the usage line above advertises only `[-adDFlhpruvV]`, but the tool also accepts `-P`, `-B`, `-S` and `-k`.
They are documented below.

Common invocations

```
# List the root directory of an image
fls {{path/to/image.dd}}

# List a partition that starts at a sector offset (the usual case for a full disk)
fls -o {{2048}} {{path/to/image.dd}}

# Show deleted entries only – the fast answer to "what was removed?"
fls -d -o {{2048}} {{path/to/image.dd}}

# Recurse the whole filesystem showing full paths, deleted files included
fls -rp -o {{2048}} {{path/to/image.dd}}

# Recurse showing ONLY deleted files with full paths
fls -rpd -o {{2048}} {{path/to/image.dd}}

# Long listing (sizes, MAC times) like ls -l, in a named time zone
fls -l -z {{EST5EDT}} -o {{2048}} {{path/to/image.dd}}

# Emit a body file for timelining, then build the timeline with mactime
fls -r -m {{/}} -o {{2048}} {{path/to/image.dd}} > {{path/to/body.txt}}

# List the contents of one directory by inode number
fls -o {{2048}} {{path/to/image.dd}} {{inode}}
```

Options — complete

Flag	Argument	What it does	When you would use it
<code>-a</code>	—	Display <code>.</code> and <code>..</code> entries	Rarely; they are hidden by default as noise
<code>-d</code>	—	Display deleted entries only	Recovery hunts — isolates what was removed
<code>-D</code>	—	Display only directories	Mapping structure without file noise
<code>-F</code>	—	Display only files	Excluding directories from a listing
<code>-l</code>	—	Display long version (like <code>ls -l</code>)	When you need sizes, UID/GID and MAC times
<code>-p</code>	—	Display full path for each file	Almost always with <code>-r</code> ; otherwise output is ambiguous
<code>-r</code>	—	Recurse on directory entries	Walking a whole filesystem rather than one directory
<code>-u</code>	—	Display undeleted entries only	The inverse of <code>-d</code> ; filtering live files

Flag	Argument	What it does	When you would use it
<code>-m</code>	<code>dir/</code>	Output in mactime input format, with <code>dir/</code> as the mount point	Building a super-timeline; feeds <code>mactime</code>
<code>-h</code>	—	Include MD5 checksum hash in mactime output	Timeline work where you also need file hashes
<code>-f</code>	<code>fstype</code>	Filesystem type (<code>-f list</code> for supported types)	When auto-detection fails or is wrong
<code>-i</code>	<code>imgtype</code>	Image format (<code>-i list</code> for supported types)	Non-raw images: E01, AFF, etc.
<code>-b</code>	<code>dev_sector_size</code>	Device sector size in bytes	4Kn drives, where the 512-byte assumption breaks
<code>-o</code>	<code>imgoffset</code>	Offset into the image, in sectors	The most common flag — targeting a partition in a full-disk image
<code>-P</code>	<code>pooltype</code>	Pool container type (<code>-P list</code> for supported types)	APFS / logical volume containers
<code>-B</code>	<code>pool_volume_block</code>	Starting block, pool volumes only	Selecting a volume inside a pool
<code>-S</code>	<code>snap_id</code>	Snapshot ID (APFS only)	Examining a specific APFS snapshot
<code>-k</code>	<code>password</code>	Decryption password for encrypted volumes	BitLocker/FileVault-style encrypted volumes
<code>-z</code>	<code>ZONE</code>	Time zone of the original machine (e.g. <code>EST5EDT</code> , <code>GMT</code>)	Only meaningful with <code>-l</code> ; wrong TZ silently skews every timestamp
<code>-s</code>	<code>seconds</code>	Time skew of the original machine, in seconds	Only with <code>-l</code> / <code>-m</code> ; correcting a known-bad system clock
<code>-v</code>	—	Verbose output to stderr	Debugging why an image will not parse
<code>-V</code>	—	Print version	Recording tool provenance in your notes

Gotchas

- `-o` **is in sectors, not bytes**. Get it from `mmls` first. A wrong offset usually yields "Cannot determine file system type", not an obvious error.
- `-z` **only affects `-l` output**. Setting it without `-l` changes nothing, which makes a wrong-timezone timeline easy to produce and hard to notice.
- **Deleted entries may have unrecoverable content**. `fls -d` listing a name does not mean `icat` will return its data — the blocks may be reallocated.
- `-r` **on a large image is slow and noisy**. Pipe to a file, then grep it, rather than scrolling.

See also

- `mmls` — partition table; where you get the `-o` value
- `icat` — extract the content of an inode `fls` found
- `istat` — detailed metadata for one inode
- `mactime` — consumes the `-m` body file to build the timeline
- `tsk_recover` — bulk export of allocated/deleted files

foremost

Kit	Kali Linux · SIFT Workstation
Capability	Carve files out of unstructured data; Extract files and payloads from traffic
Version	1.5.7
Captured from	<code>cyberlab-aio</code> via <code>-h</code> on 2026-08-11 — raw help output

← [Capability index](#) · [Kit tool list](#)

Purpose

Carve files out of an image or raw data by header and footer signatures.

When you'd reach for this

An analyst reaches for foremost when recovering lost files from disk images or drives, using command line switches to specify built-in file types or configuration files for headers and footers; they may run it after creating an image with tools like dd, and choose it over similar tools due to its reliability and speed from using internal data structures of file formats.

Sources: <http://foremost.sourceforge.net/> · <https://www.kali.org/tools/foremost/>

Common invocations

```
# Carve JPEGs, PDFs, DOCs, XLS from disk image
foremost -t doc,jpg,pdf,xls -i image.dd
```

Options

All 11 options parsed from the captured help text; 9 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-V</code>	—	- display copyright information and exit	
<code>-t</code>	—	- specify file type. (-t jpeg,pdf ...)	Restrict carving to given types (<code>jpg</code> , <code>pdf</code> , <code>all</code>). Narrowing this is the difference between a usable result and 40,000 files.

Flag	Argument	What it does	When you would use it
<code>-d</code>	—	- turn on indirect block detection (for UNIX file-systems)	An analyst would use the <code>-d</code> flag when examining UNIX file systems to enable indirect block detection for more thorough file recovery.
<code>-i</code>	—	- specify input file (default is stdin)	Input file or device to carve from.
<code>-a</code>	—	- Write all headers, perform no error detection (corrupted files)	Write all headers found, even without a valid footer.
<code>-w</code>	—	- Only write the audit file, do not write any detected files to the disk	Write the audit file only, carving nothing — a cheap dry run.
<code>-o</code>	—	- set output directory (defaults to output)	Output directory — must be empty or foremost refuses to run.
<code>-c</code>	—	- set configuration file to use (defaults to foremost.conf)	Use a custom configuration file to add signatures.
<code>-q</code>	—	- enables quick mode. Search are performed on 512 byte boundaries.	Quick mode: scan only sector boundaries. Much faster, misses embedded files.
<code>-Q</code>	—	- enables quiet mode. Suppress output messages.	
<code>-v</code>	—	- verbose mode. Logs all messages to screen	Verbose output.

Gotchas

- Carving recovers content but **not filenames or timestamps** — those live in filesystem metadata that carving bypasses. Use `tsk_recover` when the filesystem is intact and carve only what it cannot reach.
- The output directory must be empty; foremost aborts otherwise. This trips scripted reruns constantly.

See also

`scalpel`, `binwalk`, `bulk_extractor`, `tcpxtract`, `tcpflow`

fsstat

Kit	REMnux · Kali Linux · SIFT Workstation
Capability	See the partition and volume layout
Version	The Sleuth Kit ver 4.11.1
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.sleuthkit.org/sleuthkit

← [Capability index](#) · [Kit tool list](#)

Purpose

Report a filesystem's layout and parameters: type, block size, inode range, and the geometry every other TSK tool needs.

Synopsis

```
fsstat [-tvV] [-f fstype] [-i imgtype] [-b dev_sector_size] [-o imgoffset] image
```

Common invocations

```
# Identify inode group for deleted file recovery
fsstat images/hda1.dd

# Analyze file system structure of evidence
fsstat -o 2048 "$EVIDENCE"

# Determine fragment ranges in disk image for file system analysis
fsstat -t ufs images/wd0e.dd

# Analyze file system metadata structure
fsstat -o "$OFFSET" "$EVIDENCE" > "$OUTPUT/fsstat.txt"
```

Options

All 10 options parsed from the captured help text; 9 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-t</code>	—	display type only	Print only the filesystem type. The scriptable form when all you need is a yes/no on what this volume is.

Flag	Argument	What it does	When you would use it
<code>-i</code>	imgtype	The format of the image file (use '-i list' for supported types)	Set the image format for non-raw evidence such as E01 or AFF.
<code>-b</code>	dev_sector_size	The size (in bytes) of the device sectors	Device sector size. Needed on 4Kn drives, where the 512-byte default silently computes every offset wrong.
<code>-f</code>	fstype	File system type (use '-f list' for supported types)	Force the filesystem type when detection is wrong or the superblock is damaged (<code>-f list</code> shows the options).
<code>-o</code>	imgoffset	The offset of the file system in the image (in sectors)	Offset, in sectors , of the filesystem inside the image. Take it from <code>mmls</code> ; this is the flag that ties the two tools together.
<code>-P</code>	pooltype	Pool container type (use '-P list' for supported types)	Pool type, for APFS or LVM containers that hold the volume.
<code>-B</code>	pool_volume_block	Starting block (for pool volumes only)	Starting block within a pool volume.
<code>-v</code>	—	verbose output to stderr	Verbose diagnostics to stderr, useful when detection fails.
<code>-V</code>	—	Print version	
<code>-k</code>	password	Decryption password for encrypted volumes	Password for an encrypted volume.

Gotchas

- Run this first. Block size and inode range from `fsstat` are what make `blkls`, `icat` and `ils` output interpretable — starting anywhere else means guessing at the numbers they print.
- If it reports the wrong type or refuses the volume, the `-o` offset is wrong far more often than the image is corrupt.

See also

`mmls`, `img_stat`, `testdisk`

icat

Kit	REMnux · Kali Linux · SIFT Workstation
Capability	Recover deleted or lost files
Version	The Sleuth Kit ver 4.11.1
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.sleuthkit.org/sleuthkit

← [Capability index](#) · [Kit tool list](#)

Purpose

Extract the contents of a file by inode, including deleted files.

Synopsis

```
icat [-hrRsvV] [-f fstype] [-i imgtype] [-b dev_sector_size] [-o imgoffset] image [images] inum[-typ[-id]]
```

Common invocations

```
# Extract file content by inode from disk image
icat image.dd <inode> > file_recovered
# Generate MD5 hash of extracted file data
icat -o 2048 "$EVIDENCE" 12345 | md5sum
# Recover file from evidence by inode
icat -r -o 2048 "$EVIDENCE" 54321 > recovered_file.bin
```

Options

All 14 options parsed from the captured help text; 8 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	Do not display holes in sparse files	Skip holes in sparse files so the output is not padded with zeroes.
<code>-r</code>	—	Recover deleted file	Attempt recovery of a deleted file — the reason you usually reach for icat.

Flag	Argument	What it does	When you would use it
<code>-R</code>	—	Recover deleted file and suppress recovery errors	Recover with slack space included, when you want everything the blocks still hold.
<code>-s</code>	—	Display slack space at end of file	Include slack space in the output.
<code>-i</code>	imgtype	The format of the image file (use '-i list' for supported types)	Image format for non-raw evidence.
<code>-b</code>	dev_sector_size	The size (in bytes) of the device sectors	
<code>-f</code>	fstype	File system type (use '-f list' for supported types)	Force the filesystem type when detection is wrong.
<code>-o</code>	imgoffset	The offset of the file system in the image (in sectors)	Partition offset in sectors, from <code>mm1s</code> .
<code>-P</code>	pooltype	Pool container type (use '-P list' for supported types)	
<code>-B</code>	pool_volume_block	Starting block (for pool volumes only)	
<code>-S</code>	snap_id	Snapshot ID (for APFS only)	
<code>-v</code>	—	verbose to stderr	
<code>-V</code>	—	Print version	
<code>-k</code>	password	Decryption password for encrypted volumes	Supply a decryption password for an encrypted volume.

Gotchas

- Always redirect to a file (`icat ... > out.bin`). Binary content dumped to a terminal will corrupt your session.
- A deleted inode listed by `fls -d` may return nothing or garbage: the metadata survived but the blocks were reallocated. Empty output is evidence, not a tool failure.

See also

`tsk_recover`, `photorec`, `testdisk`, `blkls`

Kit	REMnux · Kali Linux · SIFT Workstation
Capability	List files and directories, including deleted ones; Inspect metadata for one file or inode
Version	The Sleuth Kit ver 4.11.1
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.sleuthkit.org/sleuthkit

← [Capability index](#) · [Kit tool list](#)

Purpose

List inode metadata, including inodes that no longer have a name pointing at them.

Synopsis

```
ils [-emOpvV] [-aAlLzZ] [-f fstype] [-i imgtype] [-b dev_sector_size] [-o imgoffset] [-P pooltype]
[-B pool_volume_block] [-s seconds] image [images] [inum[-end]]
```

Options

All 19 options parsed from the captured help text; 18 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-e</code>	—	Display all inodes	Every inode, allocated or not.
<code>-m</code>	—	Display output in the mactime format	mactime format — the form <code>mactime</code> consumes to build a timeline. This is how deleted-file metadata reaches the timeline at all.
<code>-O</code>	—	Display inodes that are unallocated, but were sill open (UFS/ExtX only)	Unallocated inodes that were still open at the time of imaging (UFS/ExtX). The same trick, caught mid-deletion.
<code>-p</code>	—	Display orphan inodes (unallocated with no file name)	Orphan inodes — allocated content with no directory entry. Files that were unlinked while still open, and a standard hiding place worth checking explicitly.

Flag	Argument	What it does	When you would use it
<code>-s</code>	seconds	Time skew of original machine (in seconds)	Correct for a known clock skew on the source machine, in seconds, so times line up with other evidence.
<code>-a</code>	—	Allocated inodes	Allocated inodes only.
<code>-A</code>	—	Unallocated inodes	Unallocated inodes only — deleted file metadata that often survives after the name is gone.
<code>-l</code>	—	Linked inodes	Linked inodes (a name still points at them).
<code>-L</code>	—	Unlinked inodes	Unlinked inodes (nothing does).
<code>-z</code>	—	Unused inodes	Unused inodes.
<code>-Z</code>	—	Used inodes	Used inodes.
<code>-i</code>	imgtype	The format of the image file (use '-i list' for supported types)	Image format for non-raw evidence such as E01 or AFF.
<code>-b</code>	dev_sector_size	The size (in bytes) of the device sectors	Device sector size; required on 4Kn drives.
<code>-f</code>	fstype	File system type (use '-f list' for supported types)	Force the filesystem type (<code>-f list</code> shows the options).
<code>-o</code>	imgoffset	The offset of the file system in the image (in sectors)	Offset, in sectors, from <code>mmfs</code> .
<code>-P</code>	pooltype	Pool container type (use '-p list' for supported types)	Pool type, for APFS or LVM containers.
<code>-B</code>	pool_volume_block	Starting block (for pool volumes only)	Starting block within a pool volume.
<code>-v</code>	—	verbose output to stderr	Verbose diagnostics to stderr.
<code>-V</code>	—	Display version number	

Gotchas

- `ils` finds metadata with no name; `ffind` turns an inode back into a name; `icat` extracts its content. Deleted-file work is usually all three in sequence.
- An inode surviving does not mean its data did. The blocks it points at may already be reallocated, so `icat` can return another file's contents entirely.

See also

`fls`, `ffind`, `tsk_recover`, `istat`, `file`, `stat`

istat

Kit	REMnux · Kali Linux · SIFT Workstation
Capability	Inspect metadata for one file or inode
Version	The Sleuth Kit ver 4.11.1
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.sleuthkit.org/sleuthkit

← [Capability index](#) · [Kit tool list](#)

Purpose

Show the full metadata for one inode: times, size, and the blocks it occupies.

Synopsis

```
istat [-N num] [-f fstype] [-i imgtype] [-b dev_sector_size] [-o imgoffset] [-P pooltype] [-B pool_volume_block] [-z zone] [-s seconds] [-rvV] image inum
```

Common invocations

```
# Retrieve inode details to identify deleted files
istat images/wd0e.dd 493
# Recover deleted directory entries via inode blocks
istat -b 2 images/hda9.dd 232
# Analyze file system metadata of evidence
istat -o 2048 "$EVIDENCE" 12345
# Inspect inode metadata from disk image
./istat -f linux-ext2 /root/able2/able2.part2.dd 2139 | less
```

Options

All 14 options parsed from the captured help text; 7 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-N</code>	num	force the display of NUM address of block pointers	Limit how many block addresses are printed for a large file.

Flag	Argument	What it does	When you would use it
<code>-r</code>	—	display run list instead of list of block addresses	Include recovery information for a deleted inode.
<code>-z</code>	zone	time zone of original machine (i.e. EST5EDT or GMT)	Set the time zone for the displayed timestamps.
<code>-s</code>	seconds	Time skew of original machine (in seconds)	Apply a clock skew in seconds for a known-bad system clock.
<code>-i</code>	imgtype	The format of the image file (use '-i list' for supported types)	
<code>-b</code>	dev_sector_size	The size (in bytes) of the device sectors	An analyst would use the -b flag when examining a deleted directory's inode with a size of 0 to force the display of block addresses and recover file names from the directory's data blocks.
<code>-f</code>	fstype	File system type (use '-f list' for supported types)	An analyst would use the -f flag with istat when examining a deleted file's inode on an ext2 file system to retrieve detailed metadata and block information, as demonstrated with inode 2139 in the /root/able2/able2.part2.dd disk image.
<code>-o</code>	imgoffset	The offset of the file system in the image (in sectors)	Partition offset in sectors, from <code>mm1s</code> .
<code>-P</code>	pooltype	Pool container type (use '-p list' for supported types)	
<code>-B</code>	pool_volume_block	Starting block (for pool volumes only)	
<code>-S</code>	snap_id	Snapshot ID (for APFS only)	
<code>-v</code>	—	verbose output to stderr	
<code>-V</code>	—	print version	
<code>-k</code>	password	Decryption password for encrypted volumes	

Gotchas

- The block list is what lets you prove whether a deleted file is still recoverable — cross-check it before promising a recovery.

See also

```
ils, file, stat
```

mmls

Kit	REMnux · Kali Linux · SIFT Workstation
Capability	See the partition and volume layout
Version	The Sleuth Kit ver 4.11.1
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.sleuthkit.org/sleuthkit

← [Capability index](#) · [Kit tool list](#)

Purpose

Display the partition layout of a disk image, including unallocated gaps.

When you'd reach for this

An analyst reaches for mmls after verifying the integrity of a disk image using hashing commands like md5sum to obtain details about the partition layout, which is critical before proceeding with further analysis. They run it to confirm the image is a physical disk copy rather than a logical one, ensuring accurate partition information for subsequent steps like fsstat. They choose mmls over similar tools because it specifically provides partition layout details necessary for forensic examination.

Sources: <https://hackernoon.com/getting-started-with-digital-forensics-using-the-sleuth-kit-c34a3wkg>

Common invocations

```
# First look at an unknown disk image
mmls {{image.dd}}

# The offset you need for every other TSK tool is the Start column, in sectors
mmls {{image.dd}} # then: fls -o {{2048}} {{image.dd}}

# Force the volume-system type when detection guesses wrong
mmls -t dos {{image.dd}}

# Read an E01 rather than a raw image
mmls -i ewf {{image.E01}}

# 4Kn drive, where the 512-byte default computes every offset wrong
mmls -b 4096 {{image.dd}}

# Show only the gaps, where a hidden partition would sit
mmls -A {{image.dd}}
```

Options

All 12 options parsed from the captured help text; 9 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-t</code>	vstype	The type of volume system (use '-t list' for list of supported types)	Force the volume-system type when auto-detection guesses wrong (<code>-t list</code> shows the options).
<code>-i</code>	imgtype	The format of the image file (use '-i list' for list supported types)	Set the image format for non-raw evidence such as E01 or AFF.
<code>-b</code>	dev_sector_size	The size (in bytes) of the device sectors	Set the device sector size; required on 4Kn drives where the 512-byte default is wrong.
<code>-o</code>	imgoffset	Offset to the start of the volume that contains the partition system (in sectors)	Read a volume system nested at an offset — rare, but needed for nested containers.
<code>-B</code>	—	print the rounded length in bytes	Print volume sizes in bytes rather than sectors, when reporting.
<code>-r</code>	—	recurse and look for other partition tables in partitions (DOS Only)	Recurse into nested volume systems (e.g. an extended partition).
<code>-v</code>	—	verbose output	Verbose diagnostics to stderr when an image will not parse.
<code>-V</code>	—	print the version	
<code>-a</code>	—	Show allocated volumes	Show allocated volumes only, when the gap entries are noise.
<code>-A</code>	—	Show unallocated volumes	Show unallocated space only — where a hidden or deleted partition would show up.
<code>-m</code>	—	Show metadata volumes	
<code>-M</code>	—	Hide metadata volumes	

Gotchas

- The **Start** column is in sectors. That value is what every other TSK tool wants for `-o`. Multiplying by the sector size here is the single most common mistake in a TSK workflow.
- If `mm1s` reports no partition table, the image may be a single volume rather than a whole disk — try `fsstat` on it directly at offset 0 before assuming the image is corrupt.

See also

`fsstat`, `img_stat`, `testdisk`

photorec

Kit	SIFT Workstation
Capability	Recover deleted or lost files
Version	PhotoRec 7.1
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

← [Capability index](#) · [Kit tool list](#)

Purpose

Christophe GRENIER grenier@cgsecurity.org

When you'd reach for this

An analyst reaches for PhotoRec when recovering deleted files from damaged or unbootable disks, disk images, or encrypted partitions, often after using TestDisk to repair partition tables; they run it with parameters like `/log` for logging or specifying raw devices for speed, preferring it over similar tools for its robust support of fragmented file recovery and diverse image formats like .dd, .E01, and split files.

Sources: <https://docslib.org/doc/9154809/photorec-step-by-step> · <https://oneuptime.com/blog/post/2026-01-15-recover-deleted-files-testdisk-ubuntu/view> · https://www.cgsecurity.org/wiki/PhotoRec_Step_By_Step

Synopsis

```
photorec [/log] [/debug] [/d recup_dir] [file.dd|file.e01|device]
photorec /version
```

Common invocations

```
# Recover files from raw disk image
photorec image.dd to carve a raw disk image
```

Options

All 2 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
<code>/log</code>	—	create a photorec.log file	

Flag	Argument	What it does	When you would use it
<code>/debug</code>	—	add debug information	

See also

`tsk_recover`, `icat`, `testdisk`, `blkls`

rafind2

Kit	REMnux · Kali Linux · SIFT Workstation
Capability	Search raw data for a pattern
Version	rafind2 6.1.9
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.radare.org/n/radare2.html

← [Capability index](#) · [Kit tool list](#)

Purpose

Examine binary files, including disassembling and debugging. Includes r2ai and decai plugins for LLM-powered analysis (API key or local Ollama required), plus the r2ghidra plugin for Ghidra decompilation via the pdg command.

When you'd reach for this

When an analyst needs to search for specific strings, hex patterns, or zero-terminated strings within a binary file, they use rafind2 to quickly locate offsets, then feed those results to radare2 for contextual analysis. They choose it over similar tools because it provides minimal, precise output that integrates seamlessly with radare2 commands for deeper inspection, and supports efficient workflows like counting results or displaying hex dumps.

Sources: <https://book.rada.re/tools/rafind2/intro.html>

Synopsis

```
rafind2 [-mBXnzZhqv] [-a align] [-b sz] [-f/t from/to] [-[e|s|S] str] [-x hex] [-R str] [-I str] [-g] -|file|dir ..
```

Options

No option definitions could be parsed from this tool's help output. It may be subcommand-driven or have no flags; needs manual review.

See also

`strings`, `grep`, `xxd`, `bulk_extractor`

scalpel

Kit	REMnux · Kali Linux · SIFT Workstation
Capability	Carve files out of unstructured data
Version	Scalpel version 1.60
Captured from	<code>cyberlab-aio</code> via <code>-h</code> on 2026-08-11 — raw help output
Documentation	https://github.com/sleuthkit/scalpel

← [Capability index](#) · [Kit tool list](#)

Purpose

Carve contents out of binary files, such as partitions.

When you'd reach for this

When an analyst needs to recover files from a disk image or raw device without relying on filesystem structure, they use Scalpel after imaging the drive, as it is filesystem-independent and can extract files from multiple formats. They may choose it over similar tools like Foremost because it is a faster, rewritten version designed for both digital forensics and file recovery.

Sources: <https://www.kali.org/tools/scalpel/>

Synopsis

```
scalpel [-b] [-c <config file>] [-d] [-h|V] [-i <file>]
[-m blocksize] [-n] [-o <outputdir>] [-O num] [-q clustersize]
[-r] [-s num] [-t <blockmap file>] [-u] [-v]
<imgfile> [<imgfile>] ...
```

Common invocations

```
# Extract embedded files from disk image
scalpel file.img -o output
```

Options

All 17 options parsed from the captured help text; 2 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-b</code>	—	Carve files even if defined footers aren't discovered within maximum carve size for file type [foremost 0.69 compat mode].	An analyst would use the <code>-b</code> flag when carving files from a disk image if defined footers aren't discovered within the maximum carve size for a file type.
<code>-c</code>	—	Choose configuration file.	An analyst would use the <code>-c</code> flag when they need to specify a custom configuration file to define or modify the header/footer database used for file carving.
<code>-d</code>	—	Generate header/footer database; will bypass certain optimizations and discover all footers, so performance suffers. Doesn't affect the set of files carved. EXPERIMENTAL	
<code>-h</code>	—	Print this help message and exit.	
<code>-i</code>	—	Read names of disk images from specified file.	
<code>-m</code>	—	Generate/update carve coverage blockmap file. The first 32bit unsigned int in the file identifies the block size. Thereafter each 32bit unsigned int entry in the blockmap file corresponds to one block	
<code>-n</code>	—	Don't add extensions to extracted files.	
<code>-o</code>	—	Set output directory for carved files.	
<code>-O</code>	—	Don't organize carved files by type. Default is to organize carved files into subdirectories.	
<code>-p</code>	—	Perform image file preview; audit log indicates which files would have been carved, but no files are actually carved.	
<code>-q</code>	—	Carve only when header is cluster-aligned.	
<code>-r</code>	—	Find only first of overlapping headers/footers [foremost 0.69 compat mode].	

Flag	Argument	What it does	When you would use it
<code>-s</code>	—	Skip n bytes in each disk image before carving.	
<code>-t</code>	—	Set directory for coverage blockmap. EXPERIMENTAL	
<code>-u</code>	—	Use carve coverage blockmap when carving. Carve only sections of the image whose entries in the blockmap are 0. These areas are treated as contiguous regions. EXPERIMENTAL	
<code>-V</code>	—	Print copyright information and exit.	
<code>-v</code>	—	Verbose mode.	

See also

`foremost`, `binwalk`, `bulk_extractor`, `tcpextract`

strings

Kit	Base OS — present on every Linux image
Capability	Search raw data for a pattern; Extract strings, including obfuscated ones
Version	GNU strings (GNU Binutils for Debian) 2.40
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Print sequences of printable characters found in a binary file.

Synopsis

```
strings [option(s)] [file(s)]
```

Common invocations

```
# Extract human-readable strings from binary files for analysis
strings /bin/ls
# Extract hidden text from binary files for analysis
strings -a firmware.bin
# Extract human-readable strings from binary files for analysis
strings -n 8 binary.exe
# Extract readable strings from binary files for analysis
strings $file > $file.txt
# Extract human-readable strings from binary with hex offsets
strings -tx /bin/ls | head
# Extract data section strings from binary files
strings -d executable_file
```

Options

All 14 options parsed from the captured help text; 6 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-d</code>	—	Only scan the data sections in the file	An analyst would use the <code>-d</code> flag when examining a binary file to extract only the strings from its data sections, ignoring other sections like debugging symbols or metadata.
<code>--data</code>	—	Only scan the data sections in the file	An analyst would use the <code>-d</code> flag when examining a binary file to extract only the strings from its data sections, ignoring other sections like debugging symbols or metadata.
<code>-f</code>	—	Print the name of the file before each string	An analyst would use the <code>-f</code> flag when processing multiple files to trace which file a particular string originated from.
<code>--print-file-name</code>	—	Print the name of the file before each string	An analyst would use the <code>-f</code> flag when processing multiple files to trace which file a particular string originated from.
<code>-n</code>	number	Locate & print any sequence of at least	Minimum length. Default 4 is noisy; 8–10 cuts most false hits.
<code>--bytes</code>	number	displayable characters. (The default is 4).	
<code>-t</code>	o,d,x	Print the location of the string in base 8, 10 or 16	An analyst would use the <code>-t</code> flag when they need to determine the exact location of strings within a binary file for further investigation or correlation with other data.
<code>--radix</code>	o,d,x	Print the location of the string in base 8, 10 or 16	An analyst would use the <code>-t</code> flag when they need to determine the exact location of strings within a binary file for further investigation or correlation with other data.
<code>-o</code>	—	An alias for <code>--radix=o</code>	Print the byte offset of each string — lets you seek back to it.
<code>-T</code>	—	Specify the binary file format	
<code>--target</code>	BFDNAME	Specify the binary file format	
<code>-U</code>	d s i x e h	Specify how to treat UTF-8 encoded unicode characters	Control how unicode is handled; needed for UTF-16 Windows strings.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	Display this information	
<code>--help</code>	—	Display this information	

Gotchas

- GNU `strings` reads only initialised, loaded sections by default. Use `-a` to scan the whole file — malware routinely hides outside them.
- It finds ASCII by default and will miss UTF-16LE strings that Windows binaries are full of. When a PE looks empty, that is usually why — reach for `floss` instead.

See also

`rafind2`, `grep`, `xxd`, `bulk_extractor`, `floss`, `base64dump.py`, `numbers-to-string.py`

tcpxtract

Kit	REMnux · SIFT Workstation
Capability	Carve files out of unstructured data; Extract files and payloads from traffic
Version	tcpxtract v1.0.1
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	http://tcpxtract.sourceforge.net/

[← Capability index](#) · [Kit tool list](#)

Purpose

Carve files out of network traffic by signature, without understanding the protocol that carried them. Useful when a transfer is not something a dissector recognises and you only need the payload.

When you'd reach for this

When an analyst needs to extract files from network traffic, they use tcpxtract on pcap capture files or live traffic, as it supports 26 file formats and allows custom configurations via its config file. They may run it after capturing traffic with tools that generate pcap files, preferring it over similar tools due to its flexibility in adding new formats and reliance on file signatures for accurate extraction.

Sources: <https://www.freshports.org/net/tcpxtract/>

Synopsis

```
tcpxtract [OPTIONS] [[-d <DEVICE>] [-f <FILE>]]
```

Options

All 12 options parsed from the captured help text; 1 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>--file</code>	FILE	to specify an input capture file instead of a device	
<code>-f</code>	FILE	to specify an input capture file instead of a device	
<code>--device</code>	DEVICE	to specify an input device (i.e. eth0)	

Flag	Argument	What it does	When you would use it
<code>-d</code>	DEVICE	to specify an input device (i.e. eth0)	
<code>--config</code>	FILE	use FILE as the config file	
<code>-c</code>	FILE	use FILE as the config file	
<code>--output</code>	DIRECTORY	dump files to DIRECTORY instead of current directory	An analyst would use the -o flag when performing a live capture from a network interface to specify the output directory for extracted files.
<code>-o</code>	DIRECTORY	dump files to DIRECTORY instead of current directory	An analyst would use the -o flag when performing a live capture from a network interface to specify the output directory for extracted files.
<code>--version</code>	—	display the version number of this program	
<code>-v</code>	—	display the version number of this program	
<code>--help</code>	—	display this lovely screen	
<code>-h</code>	—	display this lovely screen	

See also

`foremost`, `scalpel`, `binwalk`, `bulk_extractor`, `tcpflow`

testdisk

Kit	SIFT Workstation
Capability	See the partition and volume layout; Recover deleted or lost files
Version	TestDisk 7.1
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

← [Capability index](#) · [Kit tool list](#)

Purpose

Christophe GRENIER grenier@cgsecurity.org

When you'd reach for this

An analyst reaches for TestDisk when recovering lost partitions or repairing filesystems on physical devices, running it with administrative or root privileges after ensuring access rights; they choose it over similar tools because it specifically handles partition recovery and filesystem repair, unlike PhotoRec, which focuses on file recovery from unallocated space.

Sources: <https://www.cgsecurity.org/wiki/PhotoRec> · https://www.cgsecurity.org/wiki/TestDisk_Step_By_Step

Synopsis

```
testdisk [/log] [/debug] [file.dd|file.e01|device]
testdisk /list  [/log]  [file.dd|file.e01|device]
testdisk /version
```

Common invocations

```
# Recover partitions and repair filesystems from disk images
testdisk image.dd to work on a raw disk image
```

Options

All 3 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
<code>/log</code>	—	create a testdisk.log file	

Flag	Argument	What it does	When you would use it
<code>/debug</code>	—	add debug information	
<code>/list</code>	—	display current partitions	

See also

`mmls`, `fsstat`, `img_stat`, `tsk_recover`, `icat`, `photorec`, `blkls`

tsk_recover

Kit	REMnux · Kali Linux · SIFT Workstation
Capability	List files and directories, including deleted ones; Recover deleted or lost files
Version	The Sleuth Kit ver 4.11.1
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.sleuthkit.org/sleuthkit

[← Capability index](#) · [Kit tool list](#)

Purpose

Bulk-export files from an image to a directory.

When you'd reach for this

When an analyst needs to recover files from a disk image, they use `tsk_recover` after ensuring sufficient storage space in the destination folder. They run it following the creation of the image, as it is specifically designed for recovering files from disk images.

Sources: <https://github.com/sleuthkit/sleuthkit/wiki/Body-file> · <https://github.com/sleuthkit/sleuthkit/wiki/Timelines> · <https://hackernoon.com/getting-started-with-digital-forensics-using-the-sleuth-kit-c34a3wkg>

Synopsis

```
tsk_recover [-vVae] [-f fstype] [-i imgtype] [-b dev_sector_size] [-o sector_offset] [-P pooltype]
[-B pool_volume_block] [-d dir_inum] image [image] output_dir
```

Common invocations

```
# Recover deleted files from disk image
tsk_recover image.dd output_dir/
```

Options

All 11 options parsed from the captured help text; 6 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-i</code>	imgtype	The format of the image file (use '-i list' for supported types)	Image format for non-raw evidence.
<code>-b</code>	dev_sector_size	The size (in bytes) of the device sectors	
<code>-f</code>	fstype	The file system type (use '-f list' for supported types)	Force the filesystem type.
<code>-v</code>	—	verbose output to stderr	
<code>-V</code>	—	Print version	
<code>-a</code>	—	Recover allocated files only	Recover allocated (live) files only.
<code>-e</code>	—	Recover all files (allocated and unallocated)	Recover every file, allocated and deleted — the usual choice.
<code>-o</code>	sector_offset	sector offset for a volume to recover (recovers only that volume)	Partition offset in sectors, from <code>mm1s</code> .
<code>-P</code>	pooltype	Pool container type (use '-P list' for supported types)	
<code>-B</code>	pool_volume_block	Starting block (for pool volumes only)	
<code>-d</code>	dir_inum	Directory inum to recover from (must also specify a specific partition using -o or there must not be a volume system)	Recover from a specified directory inode rather than the root.

Gotchas

- Default behaviour recovers only *deleted* files, which surprises people expecting a full export. Use `-e` for everything.
- This preserves paths and names, unlike carving. Prefer it whenever the filesystem metadata is intact, and carve only what it cannot reach.

See also

`fls`, `ffind`, `ils`, `icat`, `photorec`, `testdisk`, `blkls`

xxd

Kit	Base OS — present on every Linux image
Capability	Search raw data for a pattern; Decode, decrypt or transform encoded data; Inspect files by hand
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Produce a hex dump, or reverse one back into binary with `-r`. The reverse direction is what distinguishes it from a viewer: edit the hex, convert it back, and you have carved or patched a file without a hex editor.

Synopsis

```
xxd [options] [infile [outfile]]
or
xxd -r [-s [-]offset] [-c cols] [-ps] [infile [outfile]]
```

Common invocations

```
# Convert hex dumps back to original binary data
xxd -r
# Recover original data from hexdump
xxd -r -i
# Display binary file contents as hex for analysis
xxd file.bin
# Analyze binary data at bit level
xxd -b file.bin
# Convert binary file to uppercase hexadecimal dump for analysis
xxd -u file.bin
# Inspect binary file contents in detailed hex format
xxd -g 1 file.bin
```

Options

All 16 options parsed from the captured help text; 4 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-a</code>	—	toggle autoskip: A single '*' replaces nul-lines. Default off.	
<code>-b</code>	—	binary digit dump (incompatible with -ps,-i,-r). Default hex.	
<code>-C</code>	—	capitalize variable names in C include file style (-i).	
<code>-c</code>	cols	format octets per line. Default 16 (-i: 12, -ps: 30).	
<code>-E</code>	—	show characters in EBCDIC. Default ASCII.	
<code>-e</code>	—	little-endian dump (incompatible with -ps,-i,-r).	
<code>-g</code>	bytes	number of octets per group in normal output. Default 2 (-e: 4).	An analyst would use the -g flag when adjusting the number of bytes per group in a hex dump to improve readability or align with specific data formats, such as when working with little-endian structures.
<code>-h</code>	—	print this summary.	
<code>-i</code>	—	output in C include file style.	An analyst would use the -i flag when generating a C include file (array) from a binary file to embed the data into a program or for forensic analysis requiring textual representation of binary content.
<code>-l</code>	len	stop after octets.	An analyst would use the -l flag when they need to examine a specific number of bytes from a file, such as inspecting the first 64 bytes of a binary to identify its header or a particular segment without processing the entire file.
<code>-n</code>	name	set the variable name used in C include output (-i).	
<code>-o</code>	off	add to the displayed file position.	

Flag	Argument	What it does	When you would use it
<code>-r</code>	—	reverse operation: convert (or patch) hexdump into binary.	An analyst would use the -r flag with xxd when they need to reconstruct a binary file from a properly formatted hex dump that includes offsets and correct hexadecimal data.
<code>-d</code>	—	show offset in decimal instead of hex.	
<code>-u</code>	—	use upper case hex letters.	
<code>-v</code>	—	show version: "xxd 2022-01-14 by Juergen Weigert et al.".	

See also

[rafind2](#), [strings](#), [grep](#), [bulk_extractor](#), [cyberchef](#), [base64dump.py](#), [rax2](#), [openssl](#)

mraptor

Kit	REMnux
Capability	Analyse Office documents and macros
Version	MacroRaptor 0.56.2
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.decalage.info/python/oletools

← [Capability index](#) · [Kit tool list](#)

Purpose

Microsoft Office OLE2 compound documents.

Synopsis

```
mraptor [options] <filename> [filename2 ...]
```

Common invocations

```
# Analyze files for malicious macro content
mraptor file.doc
# Scan zipped files for malicious macros
mraptor malicious_file.xls.zip -z infected
```

Options

All 11 options parsed from the captured help text; 3 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	show this help message and exit	
<code>--help</code>	—	show this help message and exit	
<code>-r</code>	—	find files recursively in subdirectories.	An analyst would use the <code>-r</code> flag when scanning multiple files across subdirectories to check for suspicious macro behaviors recursively.

Flag	Argument	What it does	When you would use it
<code>-z</code>	ZIP_PASSWORD	if the file is a zip archive, open all files from it, using the provided password (requires Python 2.6+)	An analyst would use the <code>-z</code> flag when examining a password-protected zip archive containing files to be analyzed by mraptor.
<code>--zip</code>	ZIP_PASSWORD	if the file is a zip archive, open all files from it, using the provided password (requires Python 2.6+)	When an analyst needs to scan a file contained within a password-protected ZIP archive, such as "malicious_file.xls" with the password "infected".
<code>-f</code>	ZIP_FNAME	if the file is a zip archive, file(s) to be opened within the zip. Wildcards * and ? are supported. (default:*)	
<code>--zipfname</code>	ZIP_FNAME	if the file is a zip archive, file(s) to be opened within the zip. Wildcards * and ? are supported. (default:*)	
<code>-l</code>	LOGLEVEL	logging level debug/info/warning/error/critical (default=warning)	
<code>--loglevel</code>	LOGLEVEL	logging level debug/info/warning/error/critical (default=warning)	
<code>-m</code>	—	Show matched strings.	
<code>--matches</code>	—	Show matched strings.	

See also

`olevba`, `oleid`, `oledump.py`, `olemeta`, `oletimes`, `olemap`, `oledir`, `olebrowse`

msodde

Kit	REMnux
Capability	Analyse Office documents and macros
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.decalage.info/python/oletools

← [Capability index](#) · [Kit tool list](#)

Purpose

Microsoft Office OLE2 compound documents.

When you'd reach for this

An analyst reaches for msodde when examining Office documents for DDE (Dynamic Data Exchange) or malicious field commands linked to exploitation techniques, often after decrypting password-protected files or alongside tools like olevba for macro analysis. They choose it over similar tools because it specifically filters and extracts DDE-related fields, which are critical for detecting vulnerabilities like those in CSV injection or macro-less code execution mentioned in the documentation.

Sources: <https://github.com/decalage2/oletools/blob/master/oletools/msodde.py> · <https://github.com/decalage2/oletools/wiki/msodde>

Synopsis

```
msodde [-h] [-j] [--nounquote] [-l LOGLEVEL] [-p PASSWORD] [-d] [-f]
[-a]
FILE
```

Common invocations

```
# Detect malicious content in OLE documents
msodde file.doc

# Extract all fields from a Word document for analysis
msodde -a file.doc
```

Options

All 15 options parsed from the captured help text; 5 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	show this help message and exit	
<code>--help</code>	—	show this help message and exit	
<code>-j</code>	—	Output in json format. Do not use with -ldebug	
<code>--json</code>	—	Output in json format. Do not use with -ldebug	
<code>--nounquote</code>	—	don't unquote values	
<code>-l</code>	LOGLEVEL	logging level debug/info/warning/error/critical (default=warning)	An analyst would use the -l flag to adjust the logging level when analyzing MS Office files for DDE links, allowing more detailed or less verbose output depending on the investigation's needs.
<code>--loglevel</code>	LOGLEVEL	logging level debug/info/warning/error/critical (default=warning)	An analyst would use the -l flag to adjust the logging level when analyzing MS Office files for DDE links, allowing more detailed or less verbose output depending on the investigation's needs.
<code>-p</code>	PASSWORD	if encrypted office files are encountered, try decryption with this password. May be repeated.	An analyst would use the -p flag when analyzing encrypted Office files (e.g., .docx or .rtf) to attempt decryption using a provided password in order to extract and analyze DDE links.
<code>--password</code>	PASSWORD	if encrypted office files are encountered, try decryption with this password. May be repeated.	An analyst would use the -p flag when analyzing encrypted Office files (e.g., .docx or .rtf) to attempt decryption using a provided password in order to extract and analyze DDE links.
<code>-d</code>	—	Return only DDE and DDEAUTO fields	An analyst would use the -d flag when analyzing OpenXML files to filter specific field commands during the detection of DDE links.
<code>--dde-only</code>	—	Return only DDE and DDEAUTO fields	An analyst would use the -d flag when analyzing OpenXML files to filter specific field commands during the detection of DDE links.

Flag	Argument	What it does	When you would use it
<code>-f</code>	—	Return all fields except harmless ones	When analyzing OpenXML files (e.g., docx) to filter specific field commands, an analyst would use the -f flag with msodde.
<code>--filter</code>	—	Return all fields except harmless ones	When analyzing OpenXML files (e.g., docx) to filter specific field commands, an analyst would use the -f flag with msodde.
<code>-a</code>	—	Return all fields, irrespective of their contents	An analyst would use the -a flag when examining a Word document to extract all field commands for comprehensive analysis, as demonstrated in the example "Scan a Word document, extracting all fields: msodde -a file.doc."
<code>--all-fields</code>	—	Return all fields, irrespective of their contents	An analyst would use the -a flag when examining a Word document to extract all field commands for comprehensive analysis, as demonstrated in the example "Scan a Word document, extracting all fields: msodde -a file.doc."

See also

[olevba](#), [oleid](#), [oledump.py](#), [olemeta](#), [oletimes](#), [olemap](#), [oledir](#), [olebrowse](#)

msoffcrypto-tool

Kit	REMnux
Capability	Analyse other container formats
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://github.com/nolze/msoffcrypto-tool

← [Capability index](#) · [Kit tool list](#)

Purpose

Decrypt a Microsoft Office file with password, intermediate key, or private key which generated its escrow key.

When you'd reach for this

An analyst reaches for msoffcrypto-tool when decrypting password-protected or encrypted Microsoft Office files, often after using Oletools to extract initial artifacts, as it specifically handles decryption with passwords, intermediate keys, or private keys generated for escrow, making it preferable over tools like SSView, which focuses on structured storage analysis rather than decryption.

Sources: <https://docs.remnux.org/discover-the-tools/analyze+documents/microsoft+office> · <https://medium.com/@m01z/dissecting-malicious-office-docs-a-quick-guide-3884732804e7>

Synopsis

```
msoffcrypto-tool [-h] (-p [PASSWORD] | -t) [-e] [-v] [infile] [outfile]
```

Options

All 8 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	show this help message and exit	
<code>--help</code>	—	show this help message and exit	
<code>-p</code>	PASSWORD	password text	
<code>--password</code>	PASSWORD	password text	
<code>-t</code>	—	test if the file is encrypted	

Flag	Argument	What it does	When you would use it
<code>--test</code>	—	test if the file is encrypted	
<code>-e</code>	—	encryption mode (default is false)	
<code>-v</code>	—	print verbose information	

See also

[7za](#), [unzip](#)

OffVis (GUI)

Capability	malware triage documents
Window title	OffVis:
Captured from	C:\Tools\OffVis\OffVis.exe on 2026-08-02 — control tree in capture/gui/OffVis/OffVis.tree.txt

[← Capability index](#) · [Kit tool list](#)

Purpose

Show a legacy Office file (`.doc` , `.xls` , `.ppt`) as three linked views: the raw bytes, the record structure Microsoft's own parser derives from them, and notes on where parsing failed. Those formats are a chain of length-prefixed records, and the classic Office exploits work by lying in a length or pointer field so the reader walks off the end of a buffer. Selecting a record highlights the bytes it came from, so a field claiming 4000 bytes inside a 200-byte record is visible rather than inferred — which is the thing you cannot see with `strings` or a hex editor alone. Microsoft published it alongside the MS10-087 advisory to teach exactly that reading.

When you'd reach for this

Reach for OffVis on **legacy** Office files - `.doc` , `.xls` , `.ppt` - not the modern XML formats. Those older formats are a chain of length-prefixed records, and the classic Office exploits work by lying in a length or pointer field so the parser walks off the end of a buffer.

It shows three linked views: the raw bytes, the record structure Microsoft's own parser derives from them, and where parsing failed. Selecting a record highlights the bytes it came from, so a field claiming 4000 bytes inside a 200-byte record becomes visible rather than inferred - which is precisely what `strings` or a hex editor cannot show you.

For macro-bearing documents reach for `olevba` instead. This is for structural corruption, not for reading VBA.

Sources: <https://learn.microsoft.com/en-us/security-updates/securitybulletins/2010/ms10-087>

Controls

The parts of this window you will actually touch, read from the application's own accessibility tree rather than from a screenshot. The full node list is in [capture/gui/OffVis/OffVis.tree.txt](#) .

Control	Type	What it does
Raw File Contents	Pane	The bytes, as a hex view. Selecting a record in the results highlights the bytes it was read from — the link between the two panes is the whole point of the tool.

Control	Type	What it does
Parsing Results	Pane	The record tree Microsoft's own parser derived. Walk it looking for a length or offset that cannot be true given the size of the record containing it.
Parsing Notes	Pane	Where the parser struggled. On a malformed file this is the first place to look, because the point at which parsing breaks is usually the point the exploit targets.
Parse	Pane	Run the parse. Nothing populates until this is pressed.
Parser:	Pane	Which file format to parse as. Set it to match the document; parsing a .doc as .xls produces nonsense rather than an error.

Using it

1. Open the legacy Office binary — `.doc`, `.xls`, `.ppt`.
2. Read the parsed structure against the raw bytes shown alongside it.
3. Compare what the record headers claim against what is actually present; the mismatch is the exploit.

Gotchas

- It only understands the *binary* formats. OOXML files — `.docx`, `.xlsx` — are ZIP archives and this tool cannot read them; unzip and use `oledump.py` on the extracted `vbaProject.bin`.
- It is a structure visualiser, not a detector. It shows you the record layout so you can see the malformation; it does not tell you one is present.
- The tool is old and unmaintained. Treat a crash as information about the file rather than a reason to stop.

olebrowse

Kit	REMnux
Capability	Analyse Office documents and macros
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.decalage.info/python/oletools

← [Capability index](#) · [Kit tool list](#)

Purpose

Microsoft Office OLE2 compound documents.

When you'd reach for this

An analyst reaches for olebrowse when they need to interactively explore the structure of an OLE file, such as viewing or extracting individual data streams from MS Office documents, as it provides a simple GUI interface for this purpose. They may use it alongside tools like oledir or olemap for structural analysis, but prefer olebrowse for its visual inspection capabilities over command-line alternatives.

Sources: <https://github.com/decalage2/oletools/wiki> · <https://github.com/decalage2/oletools/wiki/olevba>

Options

No option definitions could be parsed from this tool's help output. It may be subcommand-driven or have no flags; needs manual review.

See also

[olevba](#) , [oleid](#) , [oledump.py](#) , [olemeta](#) , [oletimes](#) , [olemap](#) , [oledir](#) , [olefile](#)

oledir

Kit	REMnux
Capability	Analyse Office documents and macros
Version	oledir 0.54
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.decorage.info/python/oletools

← [Capability index](#) · [Kit tool list](#)

Purpose

Microsoft Office OLE2 compound documents.

Synopsis

```
oledir [options] <filename> [filename2 ...]
```

Common invocations

```
# List embedded objects in OLE files
oledir file.doc
```

Options

All 7 options parsed from the captured help text; 5 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	show this help message and exit	
<code>--help</code>	—	show this help message and exit	
<code>-r</code>	—	find files recursively in subdirectories.	An analyst would use the <code>-r</code> flag when they need to recursively search for OLE files in all subdirectories of a given directory.

Flag	Argument	What it does	When you would use it
<code>-z</code>	ZIP_PASSWORD	if the file is a zip archive, open all files from it, using the provided password (requires Python 2.6+)	An analyst would use the <code>-z</code> flag when extracting OLE file entries from a password-protected ZIP archive to analyze its contents.
<code>--zip</code>	ZIP_PASSWORD	if the file is a zip archive, open all files from it, using the provided password (requires Python 2.6+)	An analyst would use the <code>--zip</code> flag when processing a password-protected zip archive containing OLE files that need to be extracted and analyzed.
<code>-f</code>	ZIP_FNAME	if the file is a zip archive, file(s) to be opened within the zip. Wildcards * and ? are supported. (default:*)	An analyst would use the <code>-f</code> flag when extracting specific files from a zip archive that is contained within an OLE file.
<code>--zipfname</code>	ZIP_FNAME	if the file is a zip archive, file(s) to be opened within the zip. Wildcards * and ? are supported. (default:*)	An analyst would use the <code>--zipfname</code> flag when examining a zip archive to specify particular files within it for processing by oledir.

See also

[olevba](#), [oleid](#), [oledump.py](#), [olemeta](#), [oletimes](#), [olemap](#), [olebrowse](#), [olefile](#)

oledump.py

Kit	REMnux
Capability	Analyse Office documents and macros
Version	oledump.py 0.0.85
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.decorage.info/python/oletools

← [Capability index](#) · [Kit tool list](#)

Purpose

List and extract the streams inside an OLE2 file — the container behind legacy Office documents and many malicious attachments.

When you'd reach for this

An analyst reaches for oledump.py when examining OLE files (e.g., .xls) for embedded VBA macros or obfuscated content, often after extracting a file from a password-protected zip (using the password "infected") to avoid manual extraction. They may run it with options like -m to list streams, -v to decompress macros, or plugins like plugin_http_heuristics.py to extract URLs, preferring it over similar tools due to its ability to analyze zipped files in-place and integrate with YARA for rule-based scanning.

Sources: <https://blog.didierstevens.com/programs/oledump-py/>

Synopsis

```
oledump.py [options] [file]
```

Options

All 64 options parsed from the captured help text; 17 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>--version</code>	—	show program's version number and exit	
<code>-h</code>	—	show this help message and exit	
<code>--help</code>	—	show this help message and exit	

Flag	Argument	What it does	When you would use it
<code>-m</code>	—	Print manual	Print the full manual, which documents the plugin and selection syntax this summary cannot cover.
<code>--man</code>	—	Print manual	Print the full manual, which documents the plugin and selection syntax this summary cannot cover.
<code>-s</code>	SELECT	select item nr for dumping (a for all)	Select a stream by number, or <code>a</code> for all. Everything else operates on the selection, so this usually comes first.
<code>--select</code>	SELECT	select item nr for dumping (a for all)	Select a stream by number, or <code>a</code> for all. Everything else operates on the selection, so this usually comes first.
<code>-d</code>	—	perform dump	Dump the selected stream raw, for carving an embedded payload out to a file.
<code>--dump</code>	—	perform dump	Dump the selected stream raw, for carving an embedded payload out to a file.
<code>-x</code>	—	perform hex dump	Hex dump, when the stream is binary and you need to see structure.
<code>--hexdump</code>	—	perform hex dump	Hex dump, when the stream is binary and you need to see structure.
<code>-a</code>	—	perform ascii dump	ASCII dump, for a quick look at mostly-text content.
<code>--asciidump</code>	—	perform ascii dump	ASCII dump, for a quick look at mostly-text content.
<code>-A</code>	—	perform ascii dump with RLE	ASCII dump with run-length encoding, which collapses long runs of padding that otherwise bury the content.
<code>--asciidumprle</code>	—	perform ascii dump with RLE	ASCII dump with run-length encoding, which collapses long runs of padding that otherwise bury the content.
<code>-S</code>	—	perform strings dump	Strings dump — the fastest way to see whether a stream holds anything readable.

Flag	Argument	What it does	When you would use it
<code>--strings</code>	—	perform strings dump	Strings dump — the fastest way to see whether a stream holds anything readable.
<code>-T</code>	—	do head & tail	Head and tail only, to glance at a large stream.
<code>--headtail</code>	—	do head & tail	Head and tail only, to glance at a large stream.
<code>-v</code>	—	VBA decompression	Decompress VBA. Macro streams are stored compressed, so without this the source looks like binary noise. This is the flag the tool exists for.
<code>--vbadecompress</code>	—	VBA decompression	Decompress VBA. Macro streams are stored compressed, so without this the source looks like binary noise. This is the flag the tool exists for.
<code>--vbadecompressskipattributes</code>	—	VBA decompression, skipping initial attributes	Skip the attribute preamble and show only the macro body.
<code>--vbadecompresscorrupt</code>	—	VBA decompression, display beginning if corrupted	Decompress as far as possible and show it, for a deliberately corrupted macro stream that defeats a clean decompress.
<code>-r</code>	—	read raw file (use with options <code>-v</code> or <code>-p</code>)	Treat the input as a raw stream rather than an OLE file, for a stream already carved out elsewhere.
<code>--raw</code>	—	read raw file (use with options <code>-v</code> or <code>-p</code>)	Treat the input as a raw stream rather than an OLE file, for a stream already carved out elsewhere.
<code>-t</code>	TRANSLATE	string translation, like utf16 or <code>.decode("utf8")</code>	Apply a translation such as <code>utf16</code> when the stream is wide-character text.
<code>--translate</code>	TRANSLATE	string translation, like utf16 or <code>.decode("utf8")</code>	Apply a translation such as <code>utf16</code> when the stream is wide-character text.
<code>-e</code>	—	extract OLE embedded file	
<code>--extract</code>	—	extract OLE embedded file	

Flag	Argument	What it does	When you would use it
<code>-i</code>	—	print extra info for selected item	An analyst would use the <code>-i</code> flag when examining OLE files to display additional details about modules, such as their sizes and potential structure, aiding in the analysis of embedded components like VBA projects.
<code>--info</code>	—	print extra info for selected item	An analyst would use the <code>-i</code> flag when examining OLE files to display additional details about modules, such as their sizes and potential structure, aiding in the analysis of embedded components like VBA projects.
<code>-p</code>	PLUGINS	plugins to load (separate plugins with a comma , ; @file supported)	An analyst would use the <code>-p</code> flag when analyzing a malicious Office document to extract hidden data, such as URLs, by applying specific plugins like <code>plugin_http_heuristics</code> or <code>plugin_dridex</code> .
<code>--plugins</code>	PLUGINS	plugins to load (separate plugins with a comma , ; @file supported)	An analyst would use the <code>-p</code> flag when analyzing a malicious Office document to extract hidden data, such as URLs, by applying specific plugins like <code>plugin_http_heuristics</code> or <code>plugin_dridex</code> .
<code>--pluginoptions</code>	PLUGINOPTIONS	options for the plugin	
<code>--plugindir</code>	PLUGINDIR	directory for the plugin	
<code>-q</code>	—	only print output from plugins	
<code>--quiet</code>	—	only print output from plugins	
<code>-y</code>	YARA	YARA rule-file, @file, directory or #rule to check streams (YARA search doesn't work with <code>-s</code> option)	
<code>--yara</code>	YARA	YARA rule-file, @file, directory or #rule to check streams (YARA search doesn't work with <code>-s</code> option)	
<code>-D</code>	DECODERS	decoders to load (separate decoders with a comma , ; @file supported)	

Flag	Argument	What it does	When you would use it
<code>--decoders</code>	DECODERS	decoders to load (separate decoders with a comma , ; @file supported)	
<code>--decoderoptions</code>	DECODEROPTIONS	options for the decoder	
<code>--decoderdir</code>	DECODERDIR	directory for the decoder	
<code>--yarastrings</code>	—	Print YARA strings	
<code>-M</code>	—	Print metadata	
<code>--metadata</code>	—	Print metadata	
<code>-c</code>	—	Add extra calculated data to output, like hashes	
<code>--calc</code>	—	Add extra calculated data to output, like hashes	
<code>--decompress</code>	—	Search for compressed data in the stream and decompress it	
<code>-V</code>	—	verbose output with decoder errors and YARA rules	
<code>--verbose</code>	—	verbose output with decoder errors and YARA rules	
<code>-C</code>	CUT	cut data	An analyst would use the --cut flag when they need to extract a specific section of a stream's content for focused analysis.
<code>--cut</code>	CUT	cut data	An analyst would use the --cut flag when they need to extract a specific section of a stream's content for focused analysis.
<code>-E</code>	EXTRA	add extra info (environment variable: OLEDUMP_EXTRA)	
<code>--extra</code>	EXTRA	add extra info (environment variable: OLEDUMP_EXTRA)	
<code>--storages</code>	—	Include storages in report	
<code>-f</code>	FIND	Find D0CF11E0 MAGIC sequence (use l for listing, number for selecting)	An analyst would use the -f flag when examining files like AutoCAD .dwg files to locate embedded OLE objects such as VBA macros by scanning for the OLE MAGIC sequence (D0CF11E0).

Flag	Argument	What it does	When you would use it
<code>--find</code>	FIND	Find D0CF11E0 MAGIC sequence (use l for listing, number for selecting)	An analyst would use the -f flag when examining files like AutoCAD .dwg files to locate embedded OLE objects such as VBA macros by scanning for the OLE MAGIC sequence (D0CF11E0).
<code>-j</code>	—	produce json output	
<code>--jsonoutput</code>	—	produce json output	
<code>-u</code>	—	Include unused data after end of stream	
<code>--unuseddata</code>	—	Include unused data after end of stream	
<code>--password</code>	PASSWORD	The ZIP password to be used (default infected)	
<code>--trimnull</code>	—	Trim starting at first null byte	

Gotchas

- The stream letters in the listing are the finding: `M` marks a macro stream and `O` an embedded object. A document with neither is not carrying either, whatever else it contains.
- OOXML files — .docx, .xlsx — are ZIP archives, not OLE2. Unzip first and run this against the extracted `vbaProject.bin`.
- Reading a decompressed macro is not analysing it. `olevba` adds the keyword and IOC pass; this tool gets you the bytes.

See also

`olevba`, `oleid`, `olemeta`, `oletimes`, `olemap`, `oledir`, `olebrowse`, `olefile`

olefile

Kit	REMnux
Capability	Analyse Office documents and macros
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://github.com/decalage2/olefile

← [Capability index](#) · [Kit tool list](#)

Purpose

Python package to parse, read and write MS OLE2 files.

Synopsis

```
olefile [options] <filename> [filename2 ...]
```

Options

All 7 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	show this help message and exit	
<code>--help</code>	—	show this help message and exit	
<code>-c</code>	—	check all streams (for debugging purposes)	
<code>-p</code>	—	extract all user-defined properties	
<code>-d</code>	—	debug mode, shortcut for <code>-l debug</code> (displays a lot of debug information, for developers only)	
<code>-l</code>	LOGLEVEL	logging level debug/info/warning/error/critical (default=warning)	
<code>--loglevel</code>	LOGLEVEL	logging level debug/info/warning/error/critical (default=warning)	

See also

[olevba](#), [oleid](#), [oledump.py](#), [olemeta](#), [oletimes](#), [olemap](#), [oledir](#), [olebrowse](#)

oleid

Kit	REMnux
Capability	Analyse Office documents and macros
Version	oleid 0.60.1
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.decorage.info/python/oletools

← [Capability index](#) · [Kit tool list](#)

Purpose

Microsoft Office OLE2 compound documents.

Synopsis

```
oleid [-h] [FILE ...]
```

Options

All 2 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	show this help message and exit	
<code>--help</code>	—	show this help message and exit	

See also

[olevba](#) , [oledump.py](#) , [olemeta](#) , [oletimes](#) , [olemap](#) , [oledir](#) , [olebrowse](#) , [olefile](#)

olemap

Kit	REMnux
Capability	Analyse Office documents and macros
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.decalage.info/python/oletools

[← Capability index](#) · [Kit tool list](#)

Purpose

Microsoft Office OLE2 compound documents.

Options

No option definitions could be parsed from this tool's help output. It may be subcommand-driven or have no flags; needs manual review.

See also

`olevba` , `oleid` , `oledump.py` , `olemeta` , `oletimes` , `oledir` , `olebrowse` , `olefile`

olemeta

Kit	REMnux
Capability	Analyse Office documents and macros
Version	olemeta 0.54
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.decalage.info/python/oletools

← [Capability index](#) · [Kit tool list](#)

Purpose

Microsoft Office OLE2 compound documents.

When you'd reach for this

The analyst uses **olemeta** when analyzing **OLE files** to extract **standard properties** present in the **OLE file**, and it is part of the **python-oletools** package. ### Summary: - **Tool:** `olemeta` - **Purpose:** Extract standard properties from OLE files. - **Part of:** `python-oletools` package. This is a concise and accurate summary of the information provided.

Sources: <https://github.com/decalage2/oletools/blob/master/oletools/doc/olemeta.md> · <https://www.redsecuretech.co.uk/blog/post/olevba-oletools-guide-practical-examples-and-exercises/871>

Synopsis

```
olemeta [options] <filename> [filename2 ...]
```

Options

All 7 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	show this help message and exit	
<code>--help</code>	—	show this help message and exit	
<code>-r</code>	—	find files recursively in subdirectories.	

Flag	Argument	What it does	When you would use it
<code>-z</code>	ZIP_PASSWORD	if the file is a zip archive, open all files from it, using the provided password (requires Python 2.6+)	
<code>--zip</code>	ZIP_PASSWORD	if the file is a zip archive, open all files from it, using the provided password (requires Python 2.6+)	
<code>-f</code>	ZIP_FNAME	if the file is a zip archive, file(s) to be opened within the zip. Wildcards * and ? are supported. (default:*)	
<code>--zipfname</code>	ZIP_FNAME	if the file is a zip archive, file(s) to be opened within the zip. Wildcards * and ? are supported. (default:*)	

See also

`olevba`, `oleid`, `oledump.py`, `oletimes`, `olemap`, `oledir`, `olebrowse`, `olefile`

oleobj

Kit	REMnux
Capability	Analyse Office documents and macros; Analyse RTF documents and embedded objects
Version	oleobj 0.60.1
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.decalage.info/python/oletools

[← Capability index](#) · [Kit tool list](#)

Purpose

Microsoft Office OLE2 compound documents.

Synopsis

```
usage: oleobj [options] <filename> [filename2 ...]
```

Options

All 14 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	show this help message and exit	
<code>--help</code>	—	show this help message and exit	
<code>-r</code>	—	find files recursively in subdirectories.	
<code>-d</code>	OUTPUT_DIR	use specified directory to output files.	
<code>-z</code>	ZIP_PASSWORD	if the file is a zip archive, open first file from it, using the provided password (requires Python 2.6+)	
<code>--zip</code>	ZIP_PASSWORD	if the file is a zip archive, open first file from it, using the provided password (requires Python 2.6+)	

Flag	Argument	What it does	When you would use it
<code>-f</code>	ZIP_FNAME	if the file is a zip archive, file(s) to be opened within the zip. Wildcards * and ? are supported. (default:*)	
<code>--zipfname</code>	ZIP_FNAME	if the file is a zip archive, file(s) to be opened within the zip. Wildcards * and ? are supported. (default:*)	
<code>-l</code>	LOGLEVEL	logging level debug/info/warning/error/critical (default=warning)	
<code>--loglevel</code>	LOGLEVEL	logging level debug/info/warning/error/critical (default=warning)	
<code>-i</code>	FILE	Additional file to parse (same as positional arguments)	
<code>--more-input</code>	FILE	Additional file to parse (same as positional arguments)	
<code>-v</code>	—	verbose mode, set logging to DEBUG (overwrites -l)	
<code>--verbose</code>	—	verbose mode, set logging to DEBUG (overwrites -l)	

See also

[olevba](#) , [oleid](#) , [oledump.py](#) , [olemeta](#) , [oletimes](#) , [olemap](#) , [oledir](#) , [olebrowse](#)

oletimes

Kit	REMnux
Capability	Analyse Office documents and macros
Version	oletimes 0.54
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.decalage.info/python/oletools

← [Capability index](#) · [Kit tool list](#)

Purpose

Microsoft Office OLE2 compound documents.

When you'd reach for this

When analyzing OLE files for timestamps, an analyst uses oletimes to extract creation and modification timestamps of all streams and storages, as it provides precise timing data crucial for forensic timelines.

Sources: <https://decalage.info/python/oletools/> · <https://github.com/decalage2/oletools/wiki> · <https://github.com/decalage2/oletools/wiki/oleid>

Synopsis

```
oletimes [options] <filename> [filename2 ...]
```

Options

All 7 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	show this help message and exit	
<code>--help</code>	—	show this help message and exit	
<code>-r</code>	—	find files recursively in subdirectories.	

Flag	Argument	What it does	When you would use it
<code>-z</code>	ZIP_PASSWORD	if the file is a zip archive, open all files from it, using the provided password (requires Python 2.6+)	
<code>--zip</code>	ZIP_PASSWORD	if the file is a zip archive, open all files from it, using the provided password (requires Python 2.6+)	
<code>-f</code>	ZIP_FNAME	if the file is a zip archive, file(s) to be opened within the zip. Wildcards * and ? are supported. (default:*)	
<code>--zipfname</code>	ZIP_FNAME	if the file is a zip archive, file(s) to be opened within the zip. Wildcards * and ? are supported. (default:*)	

See also

`olevba`, `oleid`, `oledump.py`, `olemeta`, `olemap`, `oledir`, `olebrowse`, `olefile`

olevba

Kit	REMnux
Capability	Analyse Office documents and macros
Version	olevba 0.60.2
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.decorage.info/python/oletools

[← Capability index](#) · [Kit tool list](#)

Purpose

Extract and analyse VBA macros from Office documents.

Synopsis

```
usage: olevba [options] <filename> [filename2 ...]
```

Common invocations

```
# Analyze VBA macros in documents for malicious code
olevba file.doc
# Decode obfuscated strings in VBA macro for analysis
olevba file.doc --decode
# Analyze document macros by revealing deobfuscated VBA code
olevba file.doc --reveal
```

Options

All 29 options parsed from the captured help text; 11 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	show this help message and exit	
<code>--help</code>	—	show this help message and exit	

Flag	Argument	What it does	When you would use it
<code>-r</code>	—	find files recursively in subdirectories.	An analyst would use the <code>-r</code> flag when scanning a directory structure to recursively process all <code>.doc</code> and <code>.xls</code> files in subfolders.
<code>-z</code>	ZIP_PASSWORD	if the file is a zip archive, open all files from it, using the provided password.	Read documents from inside a zip archive.
<code>--zip</code>	ZIP_PASSWORD	if the file is a zip archive, open all files from it, using the provided password.	Read documents from inside a zip archive.
<code>-p</code>	PASSWORD	if encrypted office files are encountered, try decryption with this password. May be repeated.	Supply a password for an encrypted document.
<code>--password</code>	PASSWORD	if encrypted office files are encountered, try decryption with this password. May be repeated.	Supply a password for an encrypted document.
<code>-f</code>	ZIP_FNAME	if the file is a zip archive, file(s) to be opened within the zip. Wildcards <code>*</code> and <code>?</code> are supported. (default: <code>*</code>)	
<code>--zipfname</code>	ZIP_FNAME	if the file is a zip archive, file(s) to be opened within the zip. Wildcards <code>*</code> and <code>?</code> are supported. (default: <code>*</code>)	
<code>-a</code>	—	display only analysis results, not the macro source code	Show only the analysis results, skipping the macro source.
<code>--analysis</code>	—	display only analysis results, not the macro source code	Show only the analysis results, skipping the macro source.
<code>-c</code>	—	display only VBA source code, do not analyze it	Show only the macro source code.
<code>--code</code>	—	display only VBA source code, do not analyze it	Show only the macro source code.
<code>--decode</code>	—	display all the obfuscated strings with their decoded content (Hex, Base64, StrReverse, Dridex, VBA).	Display all the decoded strings the analyser recovered.
<code>--attr</code>	—	display the attribute lines at the beginning of VBA source code	

Flag	Argument	What it does	When you would use it
<code>--reveal</code>	—	display the macro source code after replacing all the obfuscated strings by their decoded content.	Substitute deobfuscated values back into the source, which is the most readable view of a hostile macro.
<code>-l</code>	LOGLEVEL	logging level debug/info/warning/error/critical (default=warning)	Set log level when diagnosing a parse failure.
<code>--loglevel</code>	LOGLEVEL	logging level debug/info/warning/error/critical (default=warning)	Set log level when diagnosing a parse failure.
<code>--deobf</code>	—	Attempt to deobfuscate VBA expressions (slow)	
<code>--relaxed</code>	—	Do not raise errors if opening of substream fails (this option is now deprecated, enabled by default)	
<code>--show-pcode</code>	—	Show disassembled P-code (using pcodedmp)	Disassemble the stored p-code — catches macros where the VBA source and compiled p-code disagree.
<code>--no-pcode</code>	—	Disable extraction and analysis of pcode	
<code>--no-xlm</code>	—	Do not extract XLM Excel macros. This may speed up analysis of large files.	
<code>-t</code>	—	triage mode, display results as a summary table (default for multiple files)	Triage mode — one compact line per file, ideal for a batch.
<code>--triage</code>	—	triage mode, display results as a summary table (default for multiple files)	Triage mode — one compact line per file, ideal for a batch.
<code>-d</code>	—	detailed mode, display full results (default for single file)	Show the deobfuscated macro source.
<code>--detailed</code>	—	detailed mode, display full results (default for single file)	Show the deobfuscated macro source.
<code>-j</code>	—	json mode, detailed in json format (never default)	
<code>--json</code>	—	json mode, detailed in json format (never default)	

Gotchas

- **VBA source and p-code can differ.** Malware abuses this so the readable source looks benign while the p-code executes. If a document is suspicious but the source is clean, check `--show-pcode`.

- Run this on a copy, in an isolated VM. Nothing here executes the macro, but the sample itself is still live malware.

See also

[oleid](#), [oledump.py](#), [olemeta](#), [oletimes](#), [olemap](#), [oledir](#), [olebrowse](#), [olefile](#)

pcodedmp

Kit	REMnux
Capability	Analyse Office documents and macros
Version	pcodedmp version 1.2.6
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://github.com/bontchev/pcodedmp

← [Capability index](#) · [Kit tool list](#)

Purpose

Disassemble the VBA p-code stored alongside macro source in an Office document. The two can disagree: an attacker can leave innocuous source in place while the p-code that actually executes does something else, and this is how you see the difference.

Synopsis

```
pcodedmp [-h] [-v] [-n] [-d] [-b] [-o OUTPUTFILE]
fileOrDir [fileOrDir ...]
```

Options

All 12 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	show this help message and exit	
<code>--help</code>	—	show this help message and exit	
<code>-v</code>	—	show program's version number and exit	
<code>--version</code>	—	show program's version number and exit	
<code>-n</code>	—	Don't recurse into directories	
<code>--norecurse</code>	—	Don't recurse into directories	
<code>-d</code>	—	Only disassemble, no stream dumps	

Flag	Argument	What it does	When you would use it
<code>--disasmonly</code>	—	Only disassemble, no stream dumps	
<code>-b</code>	—	Dump the stream contents	
<code>--verbose</code>	—	Dump the stream contents	
<code>-o</code>	OUTPUTFILE	Output file name	
<code>--output</code>	OUTPUTFILE	Output file name	

See also

`olevba`, `oleid`, `oledump.py`, `olemeta`, `oletimes`, `olemap`, `oledir`, `olebrowse`

pdf-parser

Kit	Kali Linux
Capability	Analyse PDFs
Version	pdf-parser.py 0.7.14
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Walk a PDF's object graph and show what each object contains, decoding streams on request. Where `pdfid` counts suspicious keywords, this resolves references and shows the actual content — the step from 'this PDF contains JavaScript' to 'here is the JavaScript'.

Synopsis

```
pdf-parser.py [options] pdf-file|zip-file|url
```

Common invocations

```
# Extract malicious PDF content for analysis
pdf-parser.py -f -w malpdf.1 > mal.1
# Search PDF for JavaScript to detect malware
pdf-parser.py -search javascript malware.pdf
# Extract PDF encryption details for analysis
python pdf-parser.py -o 16 Project#1542292355.pdf
# Extract encrypted URLs from PDF file
python pdf-parser.py -s /URI Project#1542292355.pdf
```

Options

All 54 options parsed from the captured help text; 1 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>--version</code>	—	show program's version number and exit	
<code>-h</code>	—	show this help message and exit	

Flag	Argument	What it does	When you would use it
<code>--help</code>	—	show this help message and exit	
<code>-m</code>	—	Print manual	
<code>--man</code>	—	Print manual	
<code>-s</code>	SEARCH	string to search in indirect objects (except streams)	
<code>--search</code>	SEARCH	string to search in indirect objects (except streams)	
<code>-f</code>	—	pass stream object through filters (FlateDecode, ASCIIHexDecode, ASCII85Decode, LZWDecode and RunLengthDecode only)	
<code>--filter</code>	—	pass stream object through filters (FlateDecode, ASCIIHexDecode, ASCII85Decode, LZWDecode and RunLengthDecode only)	
<code>-o</code>	OBJECT	id(s) of indirect object(s) to select, use comma (,) to separate ids (version independent)	
<code>--object</code>	OBJECT	id(s) of indirect object(s) to select, use comma (,) to separate ids (version independent)	
<code>-r</code>	REFERENCE	id of indirect object being referenced (version independent)	
<code>--reference</code>	REFERENCE	id of indirect object being referenced (version independent)	
<code>-e</code>	ELEMENTS	type of elements to select (cxtsi)	
<code>--elements</code>	ELEMENTS	type of elements to select (cxtsi)	
<code>-w</code>	—	raw output for data and filters	An analyst would use the <code>-w</code> flag when processing PDFs that contain embedded malicious content with complex filters like <code>/ASCIIHexDecode</code> and <code>/FlateDecode</code> , as demonstrated in the example where decompression failed.

Flag	Argument	What it does	When you would use it
<code>--raw</code>	—	raw output for data and filters	An analyst would use the <code>-w</code> flag when processing PDFs that contain embedded malicious content with complex filters like <code>/ASCIIHexDecode</code> and <code>/FlateDecode</code> , as demonstrated in the example where decompression failed.
<code>-a</code>	—	display stats for pdf document	
<code>--stats</code>	—	display stats for pdf document	
<code>-t</code>	TYPE	type of indirect object to select	
<code>--type</code>	TYPE	type of indirect object to select	
<code>-0</code>	—	parse stream of <code>/ObjStm</code> objects	
<code>--objstm</code>	—	parse stream of <code>/ObjStm</code> objects	
<code>-v</code>	—	display malformed PDF elements	
<code>--verbose</code>	—	display malformed PDF elements	
<code>-x</code>	EXTRACT	filename to extract malformed content to	
<code>--extract</code>	EXTRACT	filename to extract malformed content to	
<code>-H</code>	—	display hash of objects	
<code>--hash</code>	—	display hash of objects	
<code>-n</code>	—	do not canonicalize the output	
<code>--nocanonicalizedoutput</code>	—	do not canonicalize the output	
<code>-d</code>	DUMP	filename to dump stream content to	
<code>--dump</code>	DUMP	filename to dump stream content to	
<code>-D</code>	—	display debug info	
<code>--debug</code>	—	display debug info	
<code>-c</code>	—	display the content for objects without streams or with streams without filters	
<code>--content</code>	—	display the content for objects without streams or with streams without filters	

Flag	Argument	What it does	When you would use it
<code>--searchstream</code>	SEARCHSTREAM	string to search in streams	
<code>--unfiltered</code>	—	search in unfiltered streams	
<code>--casesensitive</code>	—	case sensitive search in streams	
<code>--regex</code>	—	use regex to search in streams	
<code>--overridingfilters</code>	OVERRIDINGFILTERS	override filters with given filters (use raw for the raw stream content)	
<code>-g</code>	—	generate a Python program that creates the parsed PDF file	
<code>--generate</code>	—	generate a Python program that creates the parsed PDF file	
<code>--generateembedded</code>	GENERATEEMBEDDED	generate a Python program that embeds the selected indirect object as a file	
<code>-y</code>	YARA	YARA rule (or directory or @file) to check streams (can be used with option --unfiltered)	
<code>--yara</code>	YARA	YARA rule (or directory or @file) to check streams (can be used with option --unfiltered)	
<code>--yarastrings</code>	—	Print YARA strings	
<code>--decoders</code>	DECODERS	decoders to load (separate decoders with a comma , ; @file supported)	
<code>--decoderoptions</code>	DECODEROPTIONS	options for the decoder	
<code>-k</code>	KEY	key to search in dictionaries	
<code>--key</code>	KEY	key to search in dictionaries	
<code>-j</code>	—	produce json output	
<code>--jsonoutput</code>	—	produce json output	

See also

`pdfid` , `pdfid.py` , `pdf-parser.py`

pdf-parser.py

Kit	REMnux · Kali Linux
Capability	Analyse PDFs
Version	pdf-parser.py 0.7.14
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://blog.didierstevens.com/programs/pdf-tools/

← [Capability index](#) · [Kit tool list](#)

Purpose

Walk a PDF's object graph and show what each object contains, decoding streams on request. Where `pdfid` counts suspicious keywords, this resolves references and shows the actual content — the step from 'this PDF contains JavaScript' to 'here is the JavaScript'.

Synopsis

```
pdf-parser.py [options] pdf-file|zip-file|url
```

Options

All 54 options parsed from the captured help text; 1 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>--version</code>	—	show program's version number and exit	
<code>-h</code>	—	show this help message and exit	
<code>--help</code>	—	show this help message and exit	
<code>-m</code>	—	Print manual	
<code>--man</code>	—	Print manual	
<code>-s</code>	SEARCH	string to search in indirect objects (except streams)	
<code>--search</code>	SEARCH	string to search in indirect objects (except streams)	

Flag	Argument	What it does	When you would use it
<code>-f</code>	—	pass stream object through filters (FlateDecode, ASCIIHexDecode, ASCII85Decode, LZWDecode and RunLengthDecode only)	
<code>--filter</code>	—	pass stream object through filters (FlateDecode, ASCIIHexDecode, ASCII85Decode, LZWDecode and RunLengthDecode only)	
<code>-o</code>	OBJECT	id(s) of indirect object(s) to select, use comma (,) to separate ids (version independent)	
<code>--object</code>	OBJECT	id(s) of indirect object(s) to select, use comma (,) to separate ids (version independent)	
<code>-r</code>	REFERENCE	id of indirect object being referenced (version independent)	
<code>--reference</code>	REFERENCE	id of indirect object being referenced (version independent)	
<code>-e</code>	ELEMENTS	type of elements to select (cxtsi)	
<code>--elements</code>	ELEMENTS	type of elements to select (cxtsi)	
<code>-w</code>	—	raw output for data and filters	An analyst would use the -w flag when processing a PDF containing embedded malicious content with complex filters like ASCIIHexDecode/FlateDecode, as demonstrated by the user's attempt to decode a malicious file that resulted in a decompression error.

Flag	Argument	What it does	When you would use it
<code>--raw</code>	—	raw output for data and filters	An analyst would use the <code>-w</code> flag when processing a PDF containing embedded malicious content with complex filters like <code>ASCIIHexDecode/FlateDecode</code> , as demonstrated by the user's attempt to decode a malicious file that resulted in a decompression error.
<code>-a</code>	—	display stats for pdf document	
<code>--stats</code>	—	display stats for pdf document	
<code>-t</code>	TYPE	type of indirect object to select	
<code>--type</code>	TYPE	type of indirect object to select	
<code>-0</code>	—	parse stream of /ObjStm objects	
<code>--objstm</code>	—	parse stream of /ObjStm objects	
<code>-v</code>	—	display malformed PDF elements	
<code>--verbose</code>	—	display malformed PDF elements	
<code>-x</code>	EXTRACT	filename to extract malformed content to	
<code>--extract</code>	EXTRACT	filename to extract malformed content to	
<code>-H</code>	—	display hash of objects	
<code>--hash</code>	—	display hash of objects	
<code>-n</code>	—	do not canonicalize the output	
<code>--nocanonicaledoutput</code>	—	do not canonicalize the output	
<code>-d</code>	DUMP	filename to dump stream content to	
<code>--dump</code>	DUMP	filename to dump stream content to	
<code>-D</code>	—	display debug info	
<code>--debug</code>	—	display debug info	
<code>-c</code>	—	display the content for objects without streams or with streams without filters	

Flag	Argument	What it does	When you would use it
<code>--content</code>	—	display the content for objects without streams or with streams without filters	
<code>--searchstream</code>	SEARCHSTREAM	string to search in streams	
<code>--unfiltered</code>	—	search in unfiltered streams	
<code>--casesensitive</code>	—	case sensitive search in streams	
<code>--regex</code>	—	use regex to search in streams	
<code>--overridingfilters</code>	OVERRIDINGFILTERS	override filters with given filters (use raw for the raw stream content)	
<code>-g</code>	—	generate a Python program that creates the parsed PDF file	
<code>--generate</code>	—	generate a Python program that creates the parsed PDF file	
<code>--generateembedded</code>	GENERATEEMBEDDED	generate a Python program that embeds the selected indirect object as a file	
<code>-y</code>	YARA	YARA rule (or directory or @file) to check streams (can be used with option --unfiltered)	
<code>--yara</code>	YARA	YARA rule (or directory or @file) to check streams (can be used with option --unfiltered)	
<code>--yarastrings</code>	—	Print YARA strings	
<code>--decoders</code>	DECODERS	decoders to load (separate decoders with a comma , ; @file supported)	
<code>--decoderoptions</code>	DECODEROPTIONS	options for the decoder	
<code>-k</code>	KEY	key to search in dictionaries	
<code>--key</code>	KEY	key to search in dictionaries	
<code>-j</code>	—	produce json output	
<code>--jsonoutput</code>	—	produce json output	

See also

`pdfid` , `pdfid.py` , `pdf-parser`

pdfid

Kit	Kali Linux
Capability	Analyse PDFs
Version	pdfid.py 0.2.10
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Count the PDF tags that make a document *do* something, and print the tally. A PDF is a set of objects, and a handful of tag names are the ones that can execute or fetch: `/JavaScript` and `/JS` carry script, `/OpenAction` and `/AA` run something when the file opens or on an event, `/Launch` starts an external program, `/EmbeddedFile` carries another file inside this one, and `/URI` reaches the network. A normal invoice has none of them. Output is one line per tag with a count, so the judgement is simply: are any non-zero, and does this document have any business containing them? It parses nothing and decodes nothing -- it is the ten-second look that tells you whether to spend an hour with pdf-parser.

When you'd reach for this

An analyst reaches for pdfid when triaging PDF documents to quickly identify potential threats, such as those containing JavaScript or obfuscation, before conducting deeper analysis with a parser; they may run it initially to screen files for suspicious content, preferring it over more complex parsers due to its simplicity and reduced risk of exploitation.

Sources: <https://blog.didierstevens.com/programs/pdf-tools/>

Synopsis

```
pdfid.py [options] [pdf-file|zip-file|url|@file] ...
```

Options

All 31 options parsed from the captured help text; 7 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>--version</code>	—	show program's version number and exit	

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	show this help message and exit	
<code>--help</code>	—	show this help message and exit	
<code>-s</code>	—	scan the given directory	Scan a directory of PDFs rather than a single file.
<code>--scan</code>	—	scan the given directory	Scan a directory of PDFs rather than a single file.
<code>-a</code>	—	display all the names	Display all keyword counts, including zero entries.
<code>--all</code>	—	display all the names	Display all keyword counts, including zero entries.
<code>-e</code>	—	display extra data, like dates	Include extra keywords beyond the default set.
<code>--extra</code>	—	display extra data, like dates	Include extra keywords beyond the default set.
<code>-f</code>	—	force the scan of the file, even without proper %PDF header	
<code>--force</code>	—	force the scan of the file, even without proper %PDF header	
<code>-d</code>	—	disable JavaScript and auto launch	Disarm the PDF — neutralise /JS and /JavaScript. Produces a safe-to-open copy; never overwrite the original.
<code>--disarm</code>	—	disable JavaScript and auto launch	Disarm the PDF — neutralise /JS and /JavaScript. Produces a safe-to-open copy; never overwrite the original.
<code>-p</code>	PLUGINS	plugins to load (separate plugins with a comma , ; @file supported)	
<code>--plugins</code>	PLUGINS	plugins to load (separate plugins with a comma , ; @file supported)	
<code>-c</code>	—	output csv data when using plugins	CSV output for batch triage.
<code>--csv</code>	—	output csv data when using plugins	CSV output for batch triage.
<code>-m</code>	MINIMUMSCORE	minimum score for plugin results output	Only report files scoring above a minimum, for a large sweep.
<code>--minimumscore</code>	MINIMUMSCORE	minimum score for plugin results output	Only report files scoring above a minimum, for a large sweep.

Flag	Argument	What it does	When you would use it
<code>-v</code>	—	verbose (will also raise caught exceptions)	
<code>--verbose</code>	—	verbose (will also raise caught exceptions)	
<code>-S</code>	SELECT	selection expression	
<code>--select</code>	SELECT	selection expression	
<code>-n</code>	—	supress output for counts equal to zero	Hide zero-count keywords to shorten the output.
<code>--nozero</code>	—	supress output for counts equal to zero	Hide zero-count keywords to shorten the output.
<code>-o</code>	OUTPUT	output to log file	
<code>--output</code>	OUTPUT	output to log file	
<code>--pluginoptions</code>	PLUGINOPTIONS	options for the plugin	
<code>-l</code>	—	take filenames literally, no wildcard matching	
<code>--literalfilenames</code>	—	take filenames literally, no wildcard matching	
<code>--recursedir</code>	—	Recurse directories (wildcards and here files (@...) allowed)	

Gotchas

- `pdfid` **counts keywords; it does not parse**. A high `/JS` count is a reason to look, not proof of malice — and a crafted PDF can hide content from it. Follow up with `pdf-parser`.
- `/OpenAction` plus `/JS` is the classic auto-execute pair worth prioritising in triage.

See also

`pdfid.py`, `pdf-parser`, `pdf-parser.py`

pdfid.py

Kit	REMnux · Kali Linux
Capability	Analyse PDFs
Version	pdfid.py 0.2.10
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://blog.didierstevens.com/programs/pdf-tools/

← [Capability index](#) · [Kit tool list](#)

Purpose

Identify suspicious elements of the PDF file.

When you'd reach for this

An analyst reaches for pdfid.py when triaging PDF documents to quickly identify potential threats like embedded JavaScript or suspicious actions, running it before deeper analysis with a full parser due to its simplicity and effectiveness in scanning for keywords without complex parsing.

Sources: <https://blog.didierstevens.com/programs/pdf-tools/>

Synopsis

```
pdfid.py [options] [pdf-file|zip-file|url|@file] ...
```

Options

All 31 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
<code>--version</code>	—	show program's version number and exit	
<code>-h</code>	—	show this help message and exit	
<code>--help</code>	—	show this help message and exit	
<code>-s</code>	—	scan the given directory	
<code>--scan</code>	—	scan the given directory	

Flag	Argument	What it does	When you would use it
<code>-a</code>	—	display all the names	
<code>--all</code>	—	display all the names	
<code>-e</code>	—	display extra data, like dates	
<code>--extra</code>	—	display extra data, like dates	
<code>-f</code>	—	force the scan of the file, even without proper %PDF header	
<code>--force</code>	—	force the scan of the file, even without proper %PDF header	
<code>-d</code>	—	disable JavaScript and auto launch	
<code>--disarm</code>	—	disable JavaScript and auto launch	
<code>-p</code>	PLUGINS	plugins to load (separate plugins with a comma , ; @file supported)	
<code>--plugins</code>	PLUGINS	plugins to load (separate plugins with a comma , ; @file supported)	
<code>-c</code>	—	output csv data when using plugins	
<code>--csv</code>	—	output csv data when using plugins	
<code>-m</code>	MINIMUMSCORE	minimum score for plugin results output	
<code>--minimumscore</code>	MINIMUMSCORE	minimum score for plugin results output	
<code>-v</code>	—	verbose (will also raise caught exceptions)	
<code>--verbose</code>	—	verbose (will also raise caught exceptions)	
<code>-S</code>	SELECT	selection expression	
<code>--select</code>	SELECT	selection expression	
<code>-n</code>	—	supress output for counts equal to zero	
<code>--nozero</code>	—	supress output for counts equal to zero	
<code>-o</code>	OUTPUT	output to log file	
<code>--output</code>	OUTPUT	output to log file	
<code>--pluginoptions</code>	PLUGINOPTIONS	options for the plugin	

Flag	Argument	What it does	When you would use it
<code>-l</code>	—	take filenames literally, no wildcard matching	
<code>--literalfilenames</code>	—	take filenames literally, no wildcard matching	
<code>--recursedir</code>	—	Recurse directories (wildcards and here files (@...) allowed)	

See also

`pdfid`, `pdf-parser`, `pdf-parser.py`

PDFStreamDumper (GUI)

Capability	malware triage documents
Window title	PDF Stream Dumper - http://sandsprite.com
Captured from	C:\Tools\PDFStreamDumper\PDFStreamDumper.exe on 2026-08-02 — control tree in capture/gui/PDFStreamDumper/PDFStreamDumper.tree.txt

← [Capability index](#) · [Kit tool list](#)

Purpose

Enumerate the objects and streams inside a PDF and decode them, applying the filters so JavaScript, embedded files and launch actions are readable rather than compressed noise.

When you'd reach for this

Reach for PDFStreamDumper when `pdfid` has flagged a PDF - a non-zero `/JS`, `/OpenAction` or `/Launch` count - and you need to read what is actually inside those objects. It enumerates every object and stream and applies the filters, so compressed and hex-encoded content becomes readable instead of noise.

It occupies the same point in the workflow as `pdf-parser`, and the choice between them is browsing versus scripting. Use this when you are exploring an unfamiliar document and want to click through its object graph; use `pdf-parser` when you already know what you are extracting, or need the extraction repeatable.

Treat any JavaScript it shows you as hostile input to be read, never executed.

Sources: <https://blog.didierstevens.com/programs/pdf-tools/>

Controls

The parts of this window you will actually touch, read from the application's own accessibility tree rather than from a screenshot. The full node list is in [capture/gui/PDFStreamDumper/PDFStreamDumper.tree.txt](#).

The window exposes 5 further named controls: **Load**, **...**, **Shell**, **Abort**, **Progress Bar**. The full tree, with every automation id, is in [the capture](#).

Using it

1. Open the PDF. Objects load into the list on the left.
2. Look for streams with `/JavaScript`, `/OpenAction`, `/Launch` or `/EmbeddedFile` first — those are where behaviour hides.
3. Select a stream to see its decoded content; the tool applies the filters so you do not have to.

4. Extract an embedded file rather than executing anything.

Gotchas

- A PDF that parses cleanly here can still be malicious, and one that fails to parse is often *more* interesting — malformed structure is used deliberately to defeat parsers.
- It decodes streams for you, which means it also decodes hostile content. Run it on the analysis VM with no network path, not on a workstation.
- Object numbers are not stable across a rewrite of the same document, so cite content, not object numbers, in a report.

pyxswf

Kit	REMnux
Capability	Analyse Office documents and macros
Version	pyxswf 0.54
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.decalage.info/python/oletools

← [Capability index](#) · [Kit tool list](#)

Purpose

Microsoft Office OLE2 compound documents.

When you'd reach for this

An analyst reaches for pyxswf when examining files like MS Office documents or RTF files suspected of containing embedded Flash (SWF) objects, particularly when SWF streams are fragmented within OLE structures or encoded in hexadecimal within RTF; they may run it after initial file inspection to extract SWF content for further analysis, preferring it over similar tools due to its specific handling of OLE fragmentation and RTF hex-encoded SWF extraction.

Sources: <https://github.com/decalage2/oletools/wiki> · <https://github.com/decalage2/oletools/wiki/oleid> · <https://github.com/decalage2/oletools/wiki/olevba>

Synopsis

```
pyxswf.py
```

Options

All 20 options parsed from the captured help text; 3 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	show this help message and exit	
<code>--help</code>	—	show this help message and exit	

Flag	Argument	What it does	When you would use it
<code>-x</code>	—	Extracts the embedded SWF(s), names it MD5HASH.swf & saves it in the working dir. No addition args needed	An analyst would use the -x flag when extracting embedded SWF files from OLE or RTF documents to save them as MD5HASH.swf in the working directory for further analysis.
<code>--extract</code>	—	Extracts the embedded SWF(s), names it MD5HASH.swf & saves it in the working dir. No addition args needed	An analyst would use the -x flag when extracting embedded SWF files from OLE or RTF documents to save them as MD5HASH.swf in the working directory for further analysis.
<code>-y</code>	—	Scans the SWF(s) with yara. If the SWF(s) is compressed it will be deflated. No addition args needed	
<code>--yara</code>	—	Scans the SWF(s) with yara. If the SWF(s) is compressed it will be deflated. No addition args needed	
<code>-s</code>	—	Scans the SWF(s) for MD5 signatures. Please see func checkMD5 to define hashes. No addition args needed	
<code>--md5scan</code>	—	Scans the SWF(s) for MD5 signatures. Please see func checkMD5 to define hashes. No addition args needed	
<code>-H</code>	—	Displays the SWFs file header. No addition args needed	
<code>--header</code>	—	Displays the SWFs file header. No addition args needed	
<code>-d</code>	—	Deflates compressed SWFS(s)	
<code>--decompress</code>	—	Deflates compressed SWFS(s)	
<code>-r</code>	PATH	Will recursively scan a directory for files that contain SWFs. Must provide path in quotes	
<code>--recdir</code>	PATH	Will recursively scan a directory for files that contain SWFs. Must provide path in quotes	
<code>-c</code>	—	Compresses the SWF using Zlib	
<code>--compress</code>	—	Compresses the SWF using Zlib	

Flag	Argument	What it does	When you would use it
<code>-o</code>	—	Parse an OLE file (e.g. Word, Excel) to look for SWF in each stream	An analyst would use the <code>-o</code> flag when examining OLE files like MS Office documents to extract embedded SWF streams that may be fragmented or obfuscated within the file's structure.
<code>--ole</code>	—	Parse an OLE file (e.g. Word, Excel) to look for SWF in each stream	An analyst would use the <code>-o</code> flag when examining OLE files like MS Office documents to extract embedded SWF streams that may be fragmented or obfuscated within the file's structure.
<code>-f</code>	—	Parse an RTF file to look for SWF in each embedded object	An analyst would use the <code>-f</code> flag when examining an RTF file containing Flash objects embedded in hexadecimal format to extract and analyze the embedded SWF content.
<code>--rtf</code>	—	Parse an RTF file to look for SWF in each embedded object	An analyst would use the <code>-f</code> flag when examining an RTF file containing Flash objects embedded in hexadecimal format to extract and analyze the embedded SWF content.

See also

`olevba`, `oleid`, `oledump.py`, `olemeta`, `oletimes`, `olemap`, `oledir`, `olebrowse`

rtfobj

Kit	REMnux
Capability	Analyse RTF documents and embedded objects
Version	rtfobj 0.60.1
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.decalage.info/python/oletools

← [Capability index](#) · [Kit tool list](#)

Purpose

Microsoft Office OLE2 compound documents.

Synopsis

```
rtfobj [options] <filename> [filename2 ...]
```

Options

All 12 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	show this help message and exit	
<code>--help</code>	—	show this help message and exit	
<code>-r</code>	—	find files recursively in subdirectories.	
<code>-z</code>	ZIP_PASSWORD	if the file is a zip archive, open first file from it, using the provided password (requires Python 2.6+)	
<code>--zip</code>	ZIP_PASSWORD	if the file is a zip archive, open first file from it, using the provided password (requires Python 2.6+)	
<code>-f</code>	ZIP_FNAME	if the file is a zip archive, file(s) to be opened within the zip. Wildcards * and ? are supported. (default:*)	

Flag	Argument	What it does	When you would use it
<code>--zipfname</code>	ZIP_FNAME	if the file is a zip archive, file(s) to be opened within the zip. Wildcards * and ? are supported. (default:*)	
<code>-l</code>	LOGLEVEL	logging level debug/info/warning/error/critical (default=warning)	
<code>--loglevel</code>	LOGLEVEL	logging level debug/info/warning/error/critical (default=warning)	
<code>-s</code>	SAVE_OBJECT	Save the object corresponding to the provided number to a file, for example "-s 2". Use "-s all" to save all objects at once.	
<code>--save</code>	SAVE_OBJECT	Save the object corresponding to the provided number to a file, for example "-s 2". Use "-s all" to save all objects at once.	
<code>-d</code>	OUTPUT_DIR	use specified directory to save output files.	

See also

[oleobj](#)

xlmdeobfuscator

Kit	REMnux
Capability	Analyse Office documents and macros; Break simple obfuscation
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://github.com/DissectMalware/XLMMacroDeobfuscator

← [Capability index](#) · [Kit tool list](#)

Purpose

Deobfuscate XLM macros (also known as Excel 4.0 macros) from Microsoft Office files.

When you'd reach for this

An analyst reaches for xlmdeobfuscator when examining obfuscated XLM macros in xls, xlsx, or xlsm files, often using it as a Python library with parameters like `process_file` to extract and deobfuscate macros; they may run it after extracting the files or before analyzing the deobfuscated code, preferring it over similar tools due to its ability to function without MS Excel and its integration into projects like CAPE Sandbox and IntelOwl.

Sources: <https://github.com/DissectMalware/XLMMacroDeobfuscator/blob/master/README.md> · <https://github.com/dissectmalware/xlmmacrodeobfuscator>

Synopsis

```
xlmdeobfuscator [-h] [-c FILE_PATH] [-f FILE_PATH] [-n] [-x]
[--sort-formulas] [--defined-names] [-2]
[--with-ms-excel] [-s] [-d DAY]
[--output-formula-format OUTPUT_FORMULA_FORMAT]
[--extract-formula-format EXTRACT_FORMULA_FORMAT]
[--no-indent] [--silent] [--export-json FILE_PATH]
```

Common invocations

```
# Deobfuscate macros in Excel documents for analysis
xlmdeobfuscator --file document.xlsm
# Extract macros from Excel document without deobfuscation
xlmdeobfuscator --file document.xlsm -x
# Deobfuscate Excel macros using configuration file
xlmdeobfuscator --file document.xlsm -c default.config
# Deobfuscate Excel macro code and export to JSON
xlmdeobfuscator --file document.xlsm --export-json result.json
# Deobfuscate Excel macro code for analysis
xlmdeobfuscator --file document.xlsm --no-indent --output-formula-format "[[INT-FORMULA]]"
```

Options

All 30 options parsed from the captured help text; 6 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	show this help message and exit	
<code>--help</code>	—	show this help message and exit	
<code>-c</code>	FILE_PATH	Specify a config file (must be a valid JSON file)	
<code>--config-file</code>	FILE_PATH	Specify a config file (must be a valid JSON file)	
<code>-f</code>	FILE_PATH	The path of a XLSM file	An analyst would use the --file flag when processing an Excel document (such as .xlsm) to deobfuscate or extract macros from it.
<code>--file</code>	FILE_PATH	The path of a XLSM file	An analyst would use the --file flag when processing an Excel document (such as .xlsm) to deobfuscate or extract macros from it.
<code>-n</code>	—	Disable interactive shell	
<code>--noninteractive</code>	—	Disable interactive shell	
<code>-x</code>	—	Only extract cells without any emulation	An analyst would use the -x flag when they need to extract macros from Excel documents without performing any deobfuscation.

Flag	Argument	What it does	When you would use it
<code>--extract-only</code>	—	Only extract cells without any emulation	An analyst would use the -x flag when they need to extract macros from Excel documents without performing any deobfuscation.
<code>--sort-formulas</code>	—	Sort extracted formulas based on their cell address (requires -x)	
<code>--defined-names</code>	—	Extract all defined names	
<code>-2</code>	—	[Deprecated] Do not use MS Excel to process XLS files	
<code>--no-ms-excel</code>	—	[Deprecated] Do not use MS Excel to process XLS files	
<code>--with-ms-excel</code>	—	Use MS Excel to process XLS files	An analyst would use the --with-ms-excel flag when processing Excel files on Windows and needing to leverage MS Excel for deobfuscation, as the tool first attempts to load the file with MS Excel before falling back to xldr2.
<code>-s</code>	—	Open an XLM shell before interpreting the macros in the input	
<code>--start-with-shell</code>	—	Open an XLM shell before interpreting the macros in the input	
<code>-d</code>	DAY	Specify the day of month	
<code>--day</code>	DAY	Specify the day of month	
<code>--output-formula-format</code>	OUTPUT_FORMULA_FORMAT	Specify the format for output formulas ([[CELL-ADDR]], [[INT-FORMULA]], and [[STATUS]])	An analyst would use the --output-formula-format flag when they need to deobfuscate macros in Excel documents and want the output to display only the integer-formula representation without any indentation or additional formatting.
<code>--extract-formula-format</code>	EXTRACT_FORMULA_FORMAT	Specify the format for extracted formulas ([[CELL-ADDR]], [[CELL-FORMULA]], and [[CELL-VALUE]])	

Flag	Argument	What it does	When you would use it
<code>--no-indent</code>	—	Do not show indent before formulas	An analyst would use the <code>--no-indent</code> flag when they need to extract deobfuscated macros from an Excel document without any formatting indentation to simplify analysis or processing.
<code>--silent</code>	—	Do not print output	
<code>--export-json</code>	FILE_PATH	Export the output to JSON	An analyst would use the <code>--export-json</code> flag when they need to save the deobfuscated macro output in a structured JSON format for further analysis or reporting.
<code>--start-point</code>	CELL_ADDR	Start interpretation from a specific cell address	
<code>-p</code>	PASSWORD	Password to decrypt the protected document	
<code>--password</code>	PASSWORD	Password to decrypt the protected document	
<code>-o</code>	OUTPUT_LEVEL	Set the level of details to be shown (0:all commands, 1: commands no jump 2:important commands 3:strings in important commands).	
<code>--output-level</code>	OUTPUT_LEVEL	Set the level of details to be shown (0:all commands, 1: commands no jump 2:important commands 3:strings in important commands).	
<code>--timeout</code>	N	stop emulation after N seconds (0: not interruption N>0: stop emulation after N seconds)	

See also

[olevba](#), [oleid](#), [oledump.py](#), [olemeta](#), [oletimes](#), [olemap](#), [oledir](#), [olebrowse](#)

7za

Kit	REMnux
Capability	Detect and reverse packing; Analyse other container formats
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.7-zip.org

← [Capability index](#) · [Kit tool list](#)

Purpose

Compress and decompress files using a variety of algorithms.

When you'd reach for this

An analyst reaches for 7za when needing to create or manipulate archives with precise control over case sensitivity, locked files, or exclusion of specific files, often running it after collecting digital evidence to securely compress and organize data. They may use switches like `-ssc`, `-ssw`, or `-x!` to tailor compression settings, preferring 7za over similar tools for its detailed command-line options that align with forensic requirements for accuracy and customization.

Sources: <https://thedeveloperblog.com/7-zip-examples> · <https://www.tecmint.com/7zip-command-examples-in-linux/>

Synopsis

```
7za <command> [<switches>...] <archive_name> [<file_names>...] [@listfile]
```

Common invocations

```
# Create password-protected archive with text files
7za a pw.7z *.txt -pSECRET
# Add Z files to 7z archive
7za.exe a archive.7z Z*. * -ssc
# Restore backup from 7z archive containing tar file
7za x -so tecmint_files.tar.7z | tar xf -
# Create password-protected archive to secure sensitive data
7za a -p{password_here} tecmint_secrets.tar.7z
```

Options

All 1 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
-y	—	assume Yes on all queries	

See also

[upx](#), [die](#), [diec](#), [binwalk](#), [unzip](#), [msoffcrypto-tool](#)

base64dump.py

Kit	REMnux
Capability	Extract strings, including obfuscated ones; Decode, decrypt or transform encoded data
Version	base64dump.py 0.0.30
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://blog.didierstevens.com/2020/07/03/update-base64dump-py-version-0-0-12/

[← Capability index](#) · [Kit tool list](#)

Purpose

Locate and decode strings encoded in Base64 and other common encodings.

When you'd reach for this

An analyst reaches for base64dump.py when encountering malformed base64 or hexadecimal strings that require length adjustment or specific decoding, such as after initial detection using regular expressions. They may run it with options like `-p` (e.g., L4 or custom lambdas) to preprocess strings before decoding or `-P` to postprocess decoded data, as it allows handling of non-standard encodings and integrates built-in functions for tasks like UTF16-to-ASCII conversion, which other tools may not natively support.

Sources: <https://github.com/DidierStevens/DidierStevensSuite/blob/master/base64dump.py>

Synopsis

```
base64dump.py [options] [file]
```

Options

All 54 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
<code>--version</code>	—	show program's version number and exit	
<code>-h</code>	—	show this help message and exit	

Flag	Argument	What it does	When you would use it
<code>--help</code>	—	show this help message and exit	
<code>-m</code>	—	Print manual	
<code>--man</code>	—	Print manual	
<code>-e</code>	ENCODING	select encoding to use, use "all" to try all encodings (default base64)	
<code>--encoding</code>	ENCODING	select encoding to use, use "all" to try all encodings (default base64)	
<code>-s</code>	SELECT	select item nr for dumping (a for all, L for largest)	
<code>--select</code>	SELECT	select item nr for dumping (a for all, L for largest)	
<code>-d</code>	—	perform dump	
<code>--dump</code>	—	perform dump	
<code>-x</code>	—	perform hex dump	
<code>--hexdump</code>	—	perform hex dump	
<code>-a</code>	—	perform ascii dump	
<code>--asciidump</code>	—	perform ascii dump	
<code>-A</code>	—	perform ascii dump with RLE	
<code>--asciidumprle</code>	—	perform ascii dump with RLE	
<code>-S</code>	—	perform strings dump	
<code>--strings</code>	—	perform strings dump	
<code>-t</code>	TRANSLATE	string translation, like utf16 or .decode("utf8")	
<code>--translate</code>	TRANSLATE	string translation, like utf16 or .decode("utf8")	
<code>-n</code>	NUMBER	minimum number of bytes in decoded data	
<code>--number</code>	NUMBER	minimum number of bytes in decoded data	
<code>-c</code>	CUT	cut data	
<code>--cut</code>	CUT	cut data	
<code>-w</code>	—	ignore whitespace	
<code>--ignorewhitespace</code>	—	ignore whitespace	
<code>-u</code>	—	do not repeat identical decoded data	

Flag	Argument	What it does	When you would use it
<code>--unique</code>	—	do not repeat identical decoded data	
<code>-z</code>	—	ignore null (zero) bytes	
<code>--ignorenullbytes</code>	—	ignore null (zero) bytes	
<code>-i</code>	IGNORE	characters to ignore	
<code>--ignore</code>	IGNORE	characters to ignore	
<code>-I</code>	IGNOREHEX	characters to ignore (hexadecimal)	
<code>--ignorehex</code>	IGNOREHEX	characters to ignore (hexadecimal)	
<code>-y</code>	YARA	YARA rule-file, @file, directory or #rule to check data (YARA search doesn't work with -s option)	
<code>--yara</code>	YARA	YARA rule-file, @file, directory or #rule to check data (YARA search doesn't work with -s option)	
<code>-D</code>	DECODERS	decoders to load (separate decoders with a comma , ; @file supported)	
<code>--decoders</code>	DECODERS	decoders to load (separate decoders with a comma , ; @file supported)	
<code>--decoderoptions</code>	DECODEROPTIONS	options for the decoder	
<code>--decoderdir</code>	DECODERDIR	directory for the decoder	
<code>--yarastrings</code>	—	Print YARA strings	
<code>-V</code>	—	verbose output with decoder errors and YARA rules	
<code>--verbose</code>	—	verbose output with decoder errors and YARA rules	
<code>--jsonoutput</code>	—	produce json output	
<code>--jsoninput</code>	—	consume json input	
<code>-T</code>	—	do head & tail	
<code>--headtail</code>	—	do head & tail	
<code>-p</code>	PREPROCESS	Python function to process encodings prior to decoding (like L4, lambda bytes: bytes[:-1], ...)	

Flag	Argument	What it does	When you would use it
<code>--preprocess</code>	PREPROCESS	Python function to process encodings prior to decoding (like L4, lambda bytes: bytes[:-1], ...)	
<code>-P</code>	POSTPROCESS	Python function to post-process decodings	
<code>--postprocess</code>	POSTPROCESS	Python function to post-process decodings	
<code>--sort</code>	SORT	Sort output	
<code>--stats</code>	—	provide statistics for encoded data	

See also

[floss](#), [strings](#), [numbers-to-string.py](#), [cyberchef](#), [rax2](#), [xxd](#), [openssl](#)

capa

Kit	REMnux · FLARE-VM
Capability	Identify capabilities in a binary
Version	capa 9.4.0
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://github.com/mandiant/capa

← [Capability index](#) · [Kit tool list](#)

Purpose

Identify the capabilities of a binary by matching rules against its disassembly, mapped to MITRE ATT&CK.

When you'd reach for this

An analyst reaches for capa after submitting a sample to CAPE for dynamic analysis, running it against the generated JSON report to extract capabilities, particularly when dealing with packed or obfuscated binaries where static analysis may fail. They may use it following unpacking or sandbox execution to bypass obfuscation limitations, preferring it over standalone static analysis tools due to its integration with dynamic reports and support for IDA Pro/Ghidra for enhanced feature extraction.

Sources: <https://github.com/mandiant/capa> · <https://github.com/xuguowong/capa-SAST>

Synopsis

```
capa [-h] [--version] [-v] [-vv] [-d] [-q]
[--color {auto,always,never}]
[-f {auto,pe,dotnet,elf,sc32,sc64,cape,drakvuf,vmray,freeze,binexport2,binja_database}]
[-b {auto,vivisect,ida,pefile,binja,dotnet,binexport2,ghidra,freeze,cape,drakvuf,vmray}]
[--restrict-to-functions RESTRICT_TO_FUNCTIONS]
[--restrict-to-processes RESTRICT_TO_PROCESSES]
```

Options

All 21 options parsed from the captured help text; 11 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	show this help message and exit	

Flag	Argument	What it does	When you would use it
<code>--help</code>	—	show this help message and exit	
<code>--version</code>	—	show program's version number and exit	
<code>-v</code>	—	enable verbose result document (no effect with <code>--json</code>)	Verbose: show which rules matched and where.
<code>--verbose</code>	—	enable verbose result document (no effect with <code>--json</code>)	Verbose: show which rules matched and where.
<code>-d</code>	—	enable debugging output on STDERR	
<code>--debug</code>	—	enable debugging output on STDERR	
<code>-q</code>	—	disable all output but errors	Quiet output for scripted use.
<code>--quiet</code>	—	disable all output but errors	An analyst would use the <code>--quiet</code> flag when they want to disable all output from capa except for error messages.
<code>--color</code>	auto,always,never	enable ANSI color codes in results, default: only during interactive session	
<code>--restrict-to-functions</code>	RESTRICT_TO_FUNCTIONS	provide a list of comma-separated function virtual addresses to analyze (static analysis).	Analyse only the named functions — useful on a large binary you have already triaged.
<code>--restrict-to-processes</code>	RESTRICT_TO_PROCESSES	provide a list of comma-separated process IDs to analyze (dynamic analysis).	
<code>--os</code>	auto,linux,macos,windows	select sample OS: auto (detect OS automatically - default), linux, macos, windows	Force the target OS when auto-detection is wrong.
<code>-r</code>	RULES	path to rule file or directory, use embedded rules by default	Use an alternate rules directory.
<code>--rules</code>	RULES	path to rule file or directory, use embedded rules by default	An analyst would use the <code>--rules</code> flag when they need to apply a custom set of rules for analyzing a binary instead of the default embedded rules.
<code>-s</code>	SIGNATURES	path to .sig/.pat file or directory used to identify library functions, use embedded signatures by default	Point at the library-identification signatures.

Flag	Argument	What it does	When you would use it
<code>--signatures</code>	SIGNATURES	path to .sig/.pat file or directory used to identify library functions, use embedded signatures by default	Point at the library-identification signatures.
<code>-t</code>	TAG	filter on rule meta field values	Only run rules carrying a given tag.
<code>--tag</code>	TAG	filter on rule meta field values	An analyst would use the <code>--tag</code> flag when they need to filter the identified capabilities based on specific values in the rule's metadata fields.
<code>-j</code>	—	emit JSON instead of text	JSON output, for feeding a triage pipeline.
<code>--json</code>	—	emit JSON instead of text	JSON output, for feeding a triage pipeline.

Gotchas

- capa reports *capabilities*, not verdicts. 'Can encrypt data' describes a backup tool as readily as ransomware — it narrows where to look.
- Packed samples yield almost nothing useful. Unpack first, or capa will just describe the packer stub.

See also

[floss](#)

CFF-Explorer (GUI)

Capability	malware triage static
Window title	CFF Explorer VIII
Captured from	C:\Tools\Explorer Suite\CFF Explorer.exe on 2026-08-02 — control tree in capture/gui/CFF-Explorer/CFF-Explorer.tree.txt

[← Capability index](#) · [Kit tool list](#)

Purpose

Inspect and edit every structure in a PE file — headers, sections, imports, exports, resources — with a built-in hex editor. The import table and the gap between virtual and raw section sizes are the two fastest reads on what a binary can do and whether it is packed.

When you'd reach for this

Reach for CFF Explorer when you need to read the PE itself rather than ask a signature database what built it. It parses every structure - headers, sections, imports, exports, resources - with a hex editor alongside, so it answers questions no signature can.

Two reads carry most of the weight on a first pass. The **import table** bounds what the binary is able to do: no network imports means it is not calling home in that form. And the gap between a section's **VirtualSize** and **SizeOfRawData** is the classic packing tell - a section occupying far more memory than disk unpacks itself at runtime. A packed sample often imports almost nothing beyond LoadLibrary and GetProcAddress, because it resolves everything else after unpacking; an import table that short is itself the finding.

Where PE Detective answers *what produced this*, CFF Explorer answers *what it is made of* - and it still works when no signature matches at all.

Sources: <https://tech-zealots.com/malware-analysis/pe-portable-executable-structure-malware-analysis-part-2/> · <https://ccdcoe.org/uploads/2020/07/Malware-Reverse-Engineering-Handbook-final.pdf>

Controls

The parts of this window you will actually touch, read from the application's own accessibility tree rather than from a screenshot. The full node list is in [capture/gui/CFF-Explorer/CFF-Explorer.tree.txt](#).

The window exposes 1 further named controls: **Tree**. The full tree, with every automation id, is in [the capture](#).

Using it

1. Drag the binary onto the window, or use the file menu.

2. Work through the **Tree** on the left: headers, sections, imports, resources.
3. Read the import table first — what a binary asks the OS for is the fastest read on what it can do.
4. Compare section virtual size against raw size; a large gap is the classic packed-binary signal.
5. Use the hex editor for a targeted look rather than a general browse.

Gotchas

- It is an *editor*, not just a viewer. It will happily write to the file you opened, so work on a copy — never the evidence.
- It reports the headers as they are written. Malware edits headers freely, so a field saying DLL means the field says DLL.
- The UI is an old Win32 dialog and exposes almost nothing to automation: the whole window comes back as four unnamed panes, which is why this page has a small control table and a screenshot doing the work.

clamscan

Kit	REMnux · SIFT Workstation
Capability	Scan with signatures for known-bad
Version	ClamAV 1.4.3
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.clamav.net

← [Capability index](#) · [Kit tool list](#)

Purpose

Scan files and directories with the ClamAV signature engine.

Common invocations

```
# Scan directory and subdirectories for malware
clamscan --recursive .
# Scan directory for malware detection
clamscan ~/Downloads/malware/
# Scan directory to detect malware
clamscan -r ~/Downloads/malware/
# Scan malware directory for infected files
clamscan -i -r ~/Downloads/malware/
```

Options

All 53 options parsed from the captured help text; 14 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>--help</code>	—	Show this help	
<code>-h</code>	—	Show this help	
<code>--version</code>	—	Print version number	
<code>-V</code>	—	Print version number	
<code>--verbose</code>	—	Be verbose	
<code>-v</code>	—	Be verbose	
<code>--archive-verbose</code>	—	Show filenames inside scanned archives	

Flag	Argument	What it does	When you would use it
<code>-a</code>	—	Show filenames inside scanned archives	
<code>--debug</code>	—	Enable libclamav's debug messages	
<code>--quiet</code>	—	Only output error messages	Only report errors, for scripted runs.
<code>--stdout</code>	—	Write to stdout instead of stderr. Does not affect 'debug' messages.	
<code>--no-summary</code>	—	Disable summary at end of scanning	Suppress the trailing summary block when parsing output.
<code>--infected</code>	—	Only print infected files	An analyst would use the <code>--infected</code> flag when scanning directories or files to quickly identify and isolate only the infected items without displaying clean files, as demonstrated in the examples of showing infected files during recursive scans or automated upload checks.
<code>-i</code>	—	Only print infected files	Print infected files only — the flag that makes output usable.
<code>--bell</code>	—	Sound bell on virus detection	Audible alert on detection; useful during a long live scan.
<code>--tempdir</code>	DIRECTORY	Create temporary files in DIRECTORY	Redirect temp extraction; important when scanning large archives on a small root filesystem.
<code>--database</code>	FILE	<code>-d FILE/DIR</code> Load virus database from FILE or load all supported db files from DIR	
<code>--fail-if-cvd-older-than</code>	days	Return with a nonzero error code if virus database outdated.	
<code>--log</code>	FILE	<code>-l FILE</code> Save scan report to FILE	Write results to a log file for the case record.
<code>--file-list</code>	FILE	<code>-f FILE</code> Scan files from FILE	Scan exactly the paths listed in a file.
<code>--move</code>	DIRECTORY	Move infected files into DIRECTORY	Quarantine detections into a directory. Never point this at evidence you must preserve.
<code>--copy</code>	DIRECTORY	Copy infected files into DIRECTORY	Copy rather than move detections, preserving the original.

Flag	Argument	What it does	When you would use it
<code>--exclude</code>	REGEX	Don't scan file names matching REGEX	Skip paths matching a regex — noisy or irrelevant trees.
<code>--exclude-dir</code>	REGEX	Don't scan directories matching REGEX	An analyst would use the <code>--exclude-dir</code> flag when scanning a system to skip over directories like <code>/proc</code> , <code>/sys</code> , and <code>/dev</code> that contain non-file system data and are not relevant to virus scanning.
<code>--include</code>	REGEX	Only scan file names matching REGEX	Restrict the scan to paths matching a regex.
<code>--include-dir</code>	REGEX	Only scan directories matching REGEX	
<code>--bytecode-timeout</code>	N	Set bytecode timeout (in milliseconds)	
<code>--exclude-pua</code>	CAT	Skip PUA sigs of category CAT	
<code>--include-pua</code>	CAT	Load PUA sigs of category CAT	
<code>--structured-ssn-format</code>	X	SSN format (0=normal,1=stripped,2=both)	
<code>--structured-ssn-count</code>	N	Min SSN count to generate a detect	
<code>--structured-cc-count</code>	N	Min CC count to generate a detect	
<code>--structured-cc-mode</code>	X	CC mode (0=credit debit and private label, 1=credit cards only)	
<code>--nocerts</code>	—	Disable authenticode certificate chain verification in PE files	
<code>--dumpcerts</code>	—	Dump authenticode certificate chain in PE files	
<code>--max-scantime</code>	#n	Scan time longer than this will be skipped and assumed clean (milliseconds)	
<code>--max-filesize</code>	#n	Files larger than this will be skipped and assumed clean	An analyst would use the <code>--max-filesize</code> flag when scanning directories containing large files to skip scanning files exceeding a specified size, such as 100MB, to avoid unnecessary processing.
<code>--max-scansize</code>	#n	The maximum amount of data to scan for each container file (**)	

Flag	Argument	What it does	When you would use it
<code>--max-files</code>	#n	The maximum number of files to scan for each container file (**)	
<code>--max-recursion</code>	#n	Maximum archive recursion level for container file (**)	
<code>--max-dir-recursion</code>	#n	Maximum directory recursion level	
<code>--max-embeddedpe</code>	#n	Maximum size file to check for embedded PE	
<code>--max-htmlnormalize</code>	#n	Maximum size of HTML file to normalize	
<code>--max-htmlnotags</code>	#n	Maximum size of normalized HTML file to scan	
<code>--max-scriptnormalize</code>	#n	Maximum size of script file to normalize	
<code>--max-ziptypercg</code>	#n	Maximum size zip to type reanalyze	
<code>--max-partitions</code>	#n	Maximum number of partitions in disk image to be scanned	
<code>--max-iconspe</code>	#n	Maximum number of icons in PE file to be scanned	
<code>--max-rechwp3</code>	#n	Maximum recursive calls to HWP3 parsing function	
<code>--pcre-match-limit</code>	#n	Maximum calls to the PCRE match function.	
<code>--pcre-recmatch-limit</code>	#n	Maximum recursive calls to the PCRE match function.	
<code>--pcre-max-filesize</code>	#n	Maximum size file to perform PCRE subsig matching.	
<code>--disable-cache</code>	—	Disable caching and cache checks for hash sums of scanned files.	

Gotchas

- Without `-i` the output lists every clean file too, which buries the hits in a large tree.
- `--move` and `--remove` MUTATE evidence. On an investigation, scan a copy or use `--copy` — a quarantine action on original evidence is not defensible.
- Signatures must be current for the result to mean anything; a stale database yields a comfortable and worthless clean result.

See also

yara, yarac, freshclam, sigtool

die (CLI)

Kit	FLARE-VM
Capability	Identify what a file actually is; Detect and reverse packing
Version	Detect It Easy v3.10
Graphical version	The same tool has a window-based version, die (GUI) , which is the better choice when you are exploring one sample rather than scripting
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

qt.qpa.xcb: could not connect to display

Options

No option definitions could be parsed from this tool's help output. It may be subcommand-driven or have no flags; needs manual review.

See also

`file`, `diec`, `upx`, `binwalk`, `7za`, `unzip`

Detect It Easy (GUI)

Kit	REMnux · FLARE-VM
Capability	Identify a file's format, compiler and packer
Version	Detect It Easy v3.10 (x86_64)
Command line	This tool also ships a command-line version, <code>diec</code> , which performs the same detection without opening a window
Captured from	<code>C:\Tools\die\die.exe</code> on FLARE-VM, 2026-08-02 — control tree in <code>capture/gui/die/die.tree.txt</code>

Worked example of the GUI format. Every control below was read from the application's accessibility tree, not from a screenshot or from memory. See <docs/GUI-FORMAT.md>.

Purpose

Identify what a binary actually is — its format, architecture, compiler, linker and any packer or protector — before committing to a deeper analysis path.

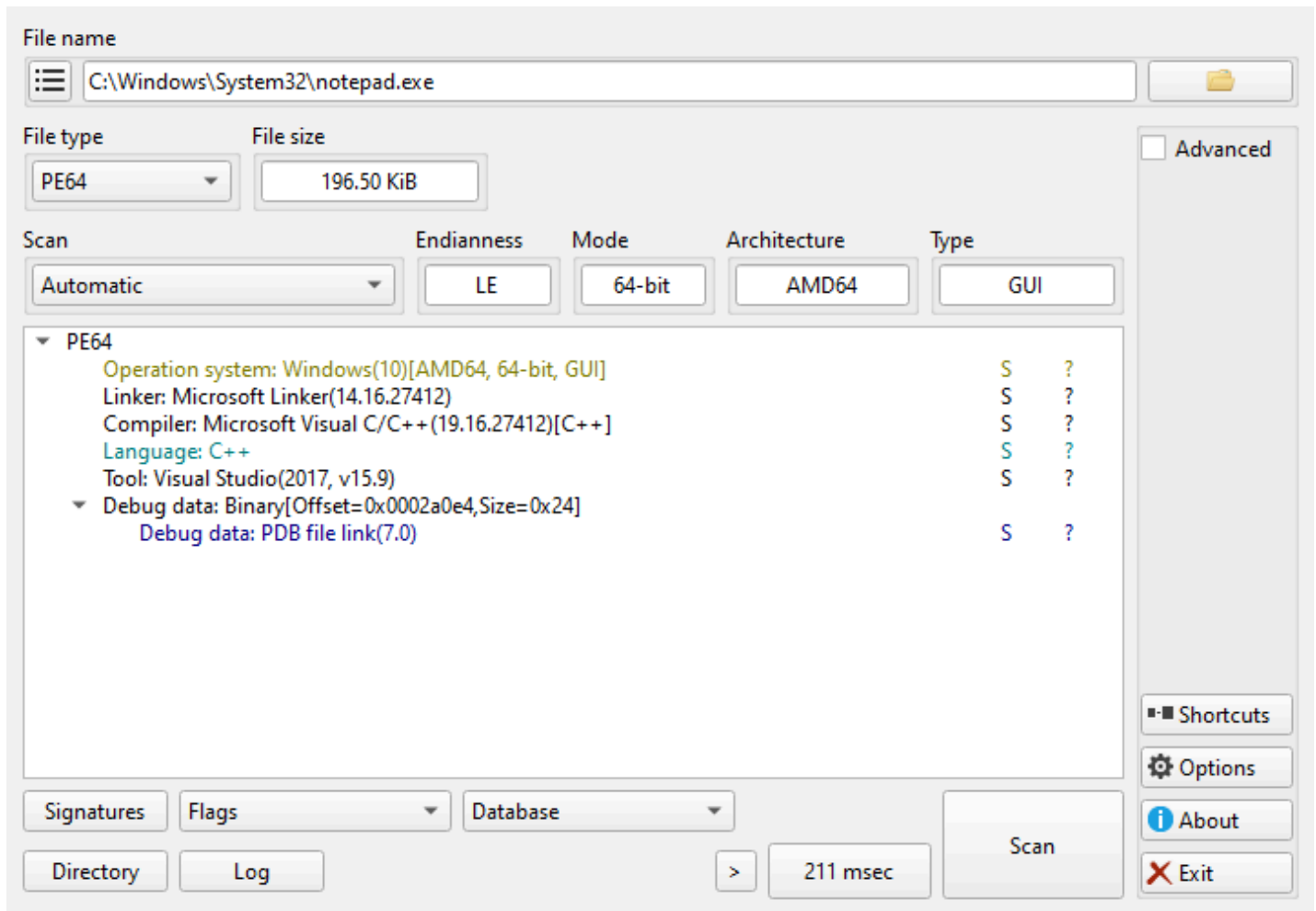
Use the GUI or the CLI?

`diec` does the same detection and is the better choice for scripting, batch work and anything reproducible. The GUI earns its place for three things the CLI cannot do:

- browsing the **Signatures** that produced a verdict, when you need to know *why* something was flagged
- switching detection engines and re-scanning interactively to compare results
- the **Advanced** panes — entropy, section and hex views — which are exploratory by nature

Anything you would put in a report or a pipeline belongs in `diec`.

Window map



Captured against `C:\Windows\System32\notepad.exe` — a benign, signed, universally present PE — so the panes show real output rather than an empty window. The result tree is the point of the tool: operating system, linker, compiler, build tool and debug data, each with the signature that produced it.

One window, `GuiMainWindow`, in three regions:

Region	What it is for
File name	Choosing the target file
Formats	The detection result, and the scan controls
Extra	Advanced views, options and about

Controls

All 72 nodes come from the capture; the leaf controls are listed here.

Control	Type	What it does
File name	Edit	Path of the file being examined
...	Button	Open the file chooser
>	Button	Recently opened files

Control	Type	What it does
File type	ComboBox	Detected container format; selectable when a file has more than one interpretation
File size	Edit	Size of the target, read-only
Scan	ComboBox	Which detection engine runs — see below
Endianness	Edit	Byte order, read-only
Mode	Edit	32- or 64-bit, read-only
Architecture	Edit	Target CPU, read-only
Type	Edit	Executable, DLL, driver, read-only
Signatures	Button	Open the signature browser — the "why was this flagged" view
Directory	Button	Scan a whole directory instead of one file
Log	Button	Scan log
Scan	Button	Run the scan
211 msec	Button	Elapsed time of the last scan — the label <i>is</i> the timing, so it changes with every run
Advanced	CheckBox	Reveal entropy, sections and hex views
Shortcuts	Button	Keyboard shortcut list
Options	Button	Settings
About	Button	Version and credits
Exit	Button	Close

Two combo boxes carry no label in the tree and sit beside **Signatures**: `comboBoxFlags` and `comboBoxDatabases`, which select the signature flags and the signature database in use.

Which scan engine should I pick?

The **Scan** dropdown is the one control here that changes your answer rather than your view, so it is worth knowing what is behind each choice. These are the actual options in this build, read from the control itself:

Value	When to use it
Automatic	The default; picks an engine from the detected format
Detect It Easy (DiE)	DiE's own signature set — the packer and compiler detection it is known for
Nauz File Detector (NFD)	A second opinion from a different signature set, when DiE returns nothing
Yara rules	Run YARA rules instead of signatures, when you have rules of your own

Identify an unknown binary

1. Select the file with ..., or drag it onto **File name**.
2. Leave **Scan** on **Automatic** for a first pass.
3. Press **Scan**.
4. Read **Architecture**, **Mode** and **Type** — these are structural facts, not guesses.
5. Press **Signatures** to see which signature produced the verdict.
6. If nothing is detected, switch **Scan** to **Nauz File Detector (NFD)** and scan again.
7. Tick **Advanced** and check entropy before concluding a file is unpacked.

Gotchas

- A packer verdict is a **signature match, not proof**. Packers are routinely misidentified, and a custom packer will not match anything at all. High entropy with no signature is the interesting case, not the boring one.
- "Nothing detected" is a statement about the signature set, not about the file. Try the other engine before concluding anything.
- The read-only fields are parsed from headers, which malware edits freely. **Type** saying `DLL` means the header says DLL.
- The GUI scans one file at a time unless you use **Directory**. For a corpus, use `diec` — this window is for looking at one sample closely.

diec (CLI)

Kit	REMnux · FLARE-VM
Capability	Identify what a file actually is; Detect and reverse packing
Version	die 3.10
Graphical version	The same tool has a window-based version, die (GUI) , which is the better choice when you are exploring one sample rather than scripting
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://github.com/horsicq/Detect-It-Easy

[← Capability index](#) · [Kit tool list](#)

Purpose

Identify a file's format, compiler, linker and packer from the command line — the scriptable half of Detect It Easy.

Synopsis

```
diec [options] target
```

Options

All 47 options parsed from the captured help text; 12 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	Displays help on commandline options.	
<code>--help</code>	—	Displays help on commandline options.	
<code>--help-all</code>	—	Displays help including Qt specific options.	
<code>-v</code>	—	Displays version information.	
<code>--version</code>	—	Displays version information.	
<code>-r</code>	—	Recursive scan.	Recurse a directory. This is the flag that makes <code>diec</code> the right tool for a corpus, where the GUI handles one sample.

Flag	Argument	What it does	When you would use it
<code>--recursivescan</code>	—	Recursive scan.	Recurse a directory. This is the flag that makes <code>diec</code> the right tool for a corpus, where the GUI handles one sample.
<code>-d</code>	—	Deep scan.	Deep scan: look past the entry point for signatures a quick pass misses. Slower, and worth it on anything suspicious.
<code>--deepscan</code>	—	Deep scan.	Deep scan: look past the entry point for signatures a quick pass misses. Slower, and worth it on anything suspicious.
<code>-u</code>	—	Heuristic scan.	Heuristic scan, for packers with no exact signature. Raises false positives, so treat a heuristic-only hit as a lead.
<code>--heuristicscan</code>	—	Heuristic scan.	Heuristic scan, for packers with no exact signature. Raises false positives, so treat a heuristic-only hit as a lead.
<code>-b</code>	—	Verbose.	Verbose output, including which signature matched.
<code>--verbose</code>	—	Verbose.	Verbose output, including which signature matched.
<code>-g</code>	—	Aggressive scan.	Aggressive scan — the most thorough and the noisiest.
<code>--aggressivescan</code>	—	Aggressive scan.	Aggressive scan — the most thorough and the noisiest.
<code>-a</code>	—	Scan all types.	Scan every type rather than stopping at the detected format.
<code>--alltypes</code>	—	Scan all types.	Scan every type rather than stopping at the detected format.
<code>-l</code>	—	Profiling signatures.	Profile signature performance — for tuning a slow scan, not for analysis.
<code>--profiling</code>	—	Profiling signatures.	Profile signature performance — for tuning a slow scan, not for analysis.
<code>-U</code>	—	Hide unknown.	Hide unknown results, to cut noise across a large scan.
<code>--hideunknown</code>	—	Hide unknown.	Hide unknown results, to cut noise across a large scan.

Flag	Argument	What it does	When you would use it
<code>-e</code>	—	Show entropy.	Show entropy. High entropy with no packer signature is the interesting case: something is compressed or encrypted and nothing recognises it.
<code>--entropy</code>	—	Show entropy.	Show entropy. High entropy with no packer signature is the interesting case: something is compressed or encrypted and nothing recognises it.
<code>-i</code>	—	Show file info.	Show file info — size, format, architecture.
<code>--info</code>	—	Show file info.	Show file info — size, format, architecture.
<code>-S</code>	method	Special file info for . For example <code>-S "Hash"</code> or <code>-S "Hash#MD5"</code> .	Ask for one specific piece of info, e.g. <code>-S Hash#MD5</code> . The form to use when scripting rather than reading.
<code>--special</code>	method	Special file info for . For example <code>-S "Hash"</code> or <code>-S "Hash#MD5"</code> .	Ask for one specific piece of info, e.g. <code>-S Hash#MD5</code> . The form to use when scripting rather than reading.
<code>-x</code>	—	Result as XML.	
<code>--xml</code>	—	Result as XML.	
<code>-j</code>	—	Result as JSON.	JSON output, for a pipeline.
<code>--json</code>	—	Result as JSON.	JSON output, for a pipeline.
<code>-c</code>	—	Result as CSV.	
<code>--csv</code>	—	Result as CSV.	
<code>-t</code>	—	Result as TSV.	
<code>--tsv</code>	—	Result as TSV.	
<code>-p</code>	—	Result as Plain Text.	
<code>--plaintext</code>	—	Result as Plain Text.	
<code>-D</code>	path	Set database.	
<code>--database</code>	path	Set database.	
<code>-E</code>	path	Set extra database.	
<code>--extradatabase</code>	path	Set extra database.	
<code>-C</code>	path	Set custom database.	
<code>--customdatabase</code>	path	Set custom database.	
<code>-s</code>	—	Show database.	
<code>--showdatabase</code>	—	Show database.	

Flag	Argument	What it does	When you would use it
<code>-m</code>	—	Show all special methods for the file.	
<code>--showmethods</code>	—	Show all special methods for the file.	

Gotchas

- `diec` and the GUI share a signature set, so they agree by construction. Use this for corpora and reports; use [the GUI](#) when you need to browse *why* something matched.
- A packer name is a signature match, not proof. Custom and modified packers match nothing, so silence is not the same as clean.
- Deep and aggressive scans cost real time on a large directory. Sample before committing to a full corpus run.

See also

`file`, `die`, `upx`, `binwalk`, `7za`, `unzip`

floss

Kit	REMnux · FLARE-VM
Capability	Identify capabilities in a binary; Extract strings, including obfuscated ones; Break simple obfuscation
Version	floss 3.1.1
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://github.com/mandiant/flare-floss

← [Capability index](#) · [Kit tool list](#)

Purpose

Extract strings from a binary, including obfuscated strings that `strings` cannot see.

When you'd reach for this

An analyst reaches for FLOSS when examining an executable file containing obfuscated strings, as it automatically emulates decoding routines and extracts human-readable strings from memory differences; they may run it after initial static analysis to uncover hidden data, preferring it over manual emulation or other tools due to its automated comparison of memory states before and after decoding.

Sources: <https://cloud.google.com/blog/topics/threat-intelligence/automatically-extracting-obfuscated-strings>

Synopsis

```
floss [-h] [-H] [-n MIN_LENGTH]
      [--no {static,stack,tight,decoded} [{static,stack,tight,decoded} ...]]
      [--only {static,stack,tight,decoded} [{static,stack,tight,decoded} ...]]
      [-j] [-v] [-d] [-q] [--color {auto,always,never}]
sample
```

Options

All 14 options parsed from the captured help text; 5 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	show this help message and exit	

Flag	Argument	What it does	When you would use it
<code>--help</code>	—	show this help message and exit	
<code>-H</code>	—	show advanced options and exit	
<code>-n</code>	MIN_LENGTH	minimum string length	Minimum string length; raise it to cut noise on large binaries.
<code>--minimum-length</code>	MIN_LENGTH	minimum string length	Minimum string length; raise it to cut noise on large binaries.
<code>-j</code>	—	emit JSON instead of text	Emit JSON, for feeding results into other tooling.
<code>--json</code>	—	emit JSON instead of text	Emit JSON, for feeding results into other tooling.
<code>-v</code>	—	enable verbose results, e.g. including function offsets (does not affect JSON output)	Verbose progress; FLOSS emulation can be slow and silent otherwise.
<code>--verbose</code>	—	enable verbose results, e.g. including function offsets (does not affect JSON output)	Verbose progress; FLOSS emulation can be slow and silent otherwise.
<code>-d</code>	—	enable debugging output on STDERR, specify multiple times to increase verbosity	
<code>--debug</code>	—	enable debugging output on STDERR, specify multiple times to increase verbosity	
<code>-q</code>	—	disable all status output on STDOUT except fatal errors	Quiet mode — just the strings, for piping.
<code>--quiet</code>	—	disable all status output on STDOUT except fatal errors	Quiet mode — just the strings, for piping.
<code>--color</code>	auto,always,never	enable ANSI color codes in results, default: only during interactive session	Control colour output when redirecting to a file.

Gotchas

- FLOSS emulates code to recover decoded strings, so it is far slower than `strings` and can take minutes on a large sample. Budget for it.
- It is built for Windows PE files. Pointing it at an ELF or a script gives you little more than plain `strings` would.

See also

`capa`, `strings`, `base64dump.py`, `numbers-to-string.py`, `xortool`, `xlmdeobfuscator`

freshclam

Kit	REMnux · SIFT Workstation
Capability	Scan with signatures for known-bad
Version	ClamAV 1.4.3
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.clamav.net

← [Capability index](#) · [Kit tool list](#)

Purpose

Update ClamAV's signature databases. Worth knowing in an air-gapped lab, where it will fail silently and leave `clamscan` quietly scanning with signatures that are months old.

Common invocations

```
# Maintain up-to-date virus databases with regular checks
freshclam -d -c 2
# Update virus database automatically
freshclam --quiet
# Update ClamAV virus database locally
freshclam --datadir=.
```

Options

All 27 options parsed from the captured help text; 5 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>--help</code>	—	Show this help	
<code>-h</code>	—	Show this help	
<code>--version</code>	—	Print version number and exit	
<code>-V</code>	—	Print version number and exit	
<code>--verbose</code>	—	Be verbose	An analyst would use the <code>-v</code> flag when needing detailed output to monitor the ClamAV database update process, such as verifying download progress and confirming successful updates.

Flag	Argument	What it does	When you would use it
<code>-v</code>	—	Be verbose	An analyst would use the <code>-v</code> flag when needing detailed output to monitor the ClamAV database update process, such as verifying download progress and confirming successful updates.
<code>--debug</code>	—	Enable debug messages	
<code>--quiet</code>	—	Only output error messages	An analyst would use the <code>--quiet</code> flag when automating ClamAV database updates via scripts or cron jobs to suppress output and check exit codes for success or failure without generating unnecessary log entries.
<code>--no-warnings</code>	—	Don't print and log warnings	
<code>--stdout</code>	—	Write to stdout instead of stderr. Does not affect 'debug' messages.	
<code>--show-progress</code>	—	Show download progress percentage	
<code>--config-file</code>	FILE	Read configuration from FILE.	
<code>--log</code>	FILE	-l FILE Log into FILE	
<code>--daemon</code>	—	Run in daemon mode	An analyst would use the <code>-d</code> flag when they need freshclam to run continuously in the background to periodically check for and download virus database updates without manual intervention.
<code>-d</code>	—	Run in daemon mode	An analyst would use the <code>-d</code> flag when they need freshclam to run continuously in the background to periodically check for and download virus database updates without manual intervention.
<code>--pid</code>	FILE	-p FILE Write the daemon's pid to FILE	
<code>--foreground</code>	—	Don't fork into background (for use in daemon mode).	
<code>-F</code>	—	Don't fork into background (for use in daemon mode).	
<code>--user</code>	USER	-u USER Run as USER	

Flag	Argument	What it does	When you would use it
<code>--no-dns</code>	—	Force old non-DNS verification method	
<code>--checks</code>	#n	-c #n Number of checks per day, 1 <= n <= 50	An analyst would use the --checks flag to specify a custom number of daily database update checks when the default of 12 checks per day is insufficient for their system's needs.
<code>--datadir</code>	DIRECTORY	Download new databases into DIRECTORY NOTE: DIRECTORY must already exist, be an absolute path, and be writeable by freshclam and readable by clamd/clamscan. --daemon-notify[=/path/clamd.conf] Send REL	An analyst would use the --datadir flag when they need to install the new ClamAV database in a specific directory that is writable, already exists, and is an absolute path, rather than the default location.
<code>--local-address</code>	IP	-a IP Bind to IP for HTTP downloads	
<code>--on-update-execute</code>	COMMAND	Execute COMMAND after successful update. Use EXIT_1 to return 1 after successful database update.	
<code>--on-error-execute</code>	COMMAND	Execute COMMAND if errors occurred	
<code>--on-outdated-execute</code>	COMMAND	Execute COMMAND when software is outdated	
<code>--update-db</code>	DBNAME	Only update database DBNAME	

See also

`yara`, `yarac`, `clamscan`, `sigtool`

numbers-to-string.py

Kit	REMnux
Capability	Extract strings, including obfuscated ones; Decode, decrypt or transform encoded data
Version	numbers-to-string.py 0.0.11
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://blog.didierstevens.com/2020/12/12/update-numbers-to-string-py-version-0-0-11/

[← Capability index](#) · [Kit tool list](#)

Purpose

Translate number sequences into ASCII characters.

When you'd reach for this

An analyst reaches for numbers-to-string.py when processing files containing numeric sequences that need conversion to readable strings, such as ASCII or custom-encoded data; they may run it after extracting numbers from a file or before analyzing the resulting text, and they choose it over similar tools due to its support for custom translation tables, statistical analysis, and binary output handling as described in the documentation.

Sources: <https://github.com/DidierStevens/DidierStevensSuite/blob/master/numbers-to-string.py>

Synopsis

```
numbers-to-string.py [options] [expression [[@]file ...]]
```

Options

All 31 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
<code>--version</code>	—	show program's version number and exit	
<code>-h</code>	—	show this help message and exit	
<code>--help</code>	—	show this help message and exit	

Flag	Argument	What it does	When you would use it
<code>-m</code>	—	Print manual	
<code>--man</code>	—	Print manual	
<code>-o</code>	OUTPUT	Output to file	
<code>--output</code>	OUTPUT	Output to file	
<code>-e</code>	—	Generate error when error occurs in Python expression	
<code>--error</code>	—	Generate error when error occurs in Python expression	
<code>-i</code>	—	Ignore numbers greater than 255	
<code>--ignore</code>	—	Ignore numbers greater than 255	
<code>-n</code>	NUMBER	Minimum number of numbers (3 by default)	
<code>--number</code>	NUMBER	Minimum number of numbers (3 by default)	
<code>-v</code>	—	Verbose	
<code>--verbose</code>	—	Verbose	
<code>-j</code>	—	Join output	
<code>--join</code>	—	Join output	
<code>-S</code>	—	Generate statistics	
<code>--statistics</code>	—	Generate statistics	
<code>-t</code>	TABLE	Translation table	
<code>--table</code>	TABLE	Translation table	
<code>-T</code>	BEGINTABLE	Begin translation table	
<code>--begintable</code>	BEGINTABLE	Begin translation table	
<code>-b</code>	—	Produce binary output	
<code>--binary</code>	—	Produce binary output	
<code>-l</code>	LINE	Select line with given number	
<code>--line</code>	LINE	Select line with given number	
<code>--grep</code>	GREP	Grep expression	
<code>--grepoptions</code>	GREPOPTIONS	Grep options	
<code>--begin</code>	BEGIN	Begin substring	
<code>--end</code>	END	End substring	

See also

`floss`, `strings`, `base64dump.py`, `cyberchef`, `rax2`, `xxd`, `openssl`

objdump

Kit	REMnux
Capability	Inspect PE / ELF structure; Disassemble and explore a binary
Version	GNU objdump (GNU Binutils for Debian) 2.40
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://en.wikipedia.org/wiki/Objdump

← [Capability index](#) · [Kit tool list](#)

Purpose

Disassemble and dump the contents of an object file or executable — sections, symbols, relocations and code.

When you'd reach for this

An analyst reaches for objdump when examining raw binary files like BIOS dumps or ELF binaries to inspect assembly code, section headers, or memory layouts, often after capturing memory or firmware data; they may use it alongside tools like ndisasm or Ghidra for deeper analysis, preferring objdump for its integration with ELF metadata and ability to display low-level code structures.

Sources: <https://can-ozkan.medium.com/learning-elf-the-foundation-of-linux-binary-analysis-c4f1f8df83e4> · <https://github.com/globa101/intel-codex/blob/main/Security/Analysis/sop-malware-analysis.md> · <https://hacktricks.wiki/en/binary-exploitation/basic-stack-binary-exploitation-methodology/elf-tricks.html>

Synopsis

```
objdump <option(s)> <file(s)>
```

Common invocations

```
# Show objdump version and supported formats
objdump -v -i
# Inspect executable's overall file header metadata
objdump -f factorial
# Display executable file header information for analysis
objdump -p factorial
# Display section headers to analyze object file structure
objdump -h factorial
# Display all object file headers
objdump -x factorial
# Analyze machine code instructions in executable sections
objdump -d factorial
```

Options

All 78 options parsed from the captured help text; 19 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-a</code>	—	Display archive header information	
<code>--archive-headers</code>	—	Display archive header information	
<code>-f</code>	—	Display the contents of the overall file header	The file header alone: format, architecture, entry point.
<code>--file-headers</code>	—	Display the contents of the overall file header	The file header alone: format, architecture, entry point.
<code>-p</code>	—	Display object format specific file header contents	Format-specific private headers. On a PE this is where the import table and data directories live.
<code>--private-headers</code>	—	Display object format specific file header contents	Format-specific private headers. On a PE this is where the import table and data directories live.
<code>-x</code>	—	Display the contents of all headers	All headers at once — the fastest orientation on an unknown object.
<code>--all-headers</code>	—	Display the contents of all headers	All headers at once — the fastest orientation on an unknown object.
<code>-d</code>	—	Display assembler contents of executable sections	Disassemble the executable sections. The common case.
<code>--disassemble</code>	—	Display assembler contents of executable sections	Disassemble the executable sections. The common case.

Flag	Argument	What it does	When you would use it
<code>-D</code>	—	Display assembler contents of all sections	Disassemble everything, including data. Use it when code has been hidden in a section not marked executable.
<code>--disassemble-all</code>	—	Display assembler contents of all sections	Disassemble everything, including data. Use it when code has been hidden in a section not marked executable.
<code>-S</code>	—	Intermix source code with disassembly --source-comment[=] Prefix lines of source code with	Interleave source with disassembly. Only useful when debug info survived, which for malware it has not.
<code>--source</code>	—	Intermix source code with disassembly --source-comment[=] Prefix lines of source code with	Interleave source with disassembly. Only useful when debug info survived, which for malware it has not.
<code>-s</code>	—	Display the full contents of all sections requested	Full contents of each section as a hex dump.
<code>--full-contents</code>	—	Display the full contents of all sections requested	Full contents of each section as a hex dump.
<code>-g</code>	—	Display debug information in object file	Debug information, if present.
<code>--debugging</code>	—	Display debug information in object file	Debug information, if present.
<code>-e</code>	—	Display debug information using ctags style	
<code>--debugging-tags</code>	—	Display debug information using ctags style	
<code>-G</code>	—	Display (in raw form) any STABS info in the file	
<code>--stabs</code>	—	Display (in raw form) any STABS info in the file	
<code>-L</code>	—	Display the contents of non-debug sections in separate debuginfo files. (Implies -WK)	
<code>--process-links</code>	—	Display the contents of non-debug sections in separate debuginfo files. (Implies -WK)	
<code>--ctf</code>	SECTION (optional)	Display CTF info from SECTION, (default '.ctf')	
<code>--sframe</code>	SECTION (optional)	Display SFrame info from SECTION, (default '.sframe')	
<code>-t</code>	—	Display the contents of the symbol table(s)	The symbol table, when the binary is not stripped.

Flag	Argument	What it does	When you would use it
<code>--syms</code>	—	Display the contents of the symbol table(s)	The symbol table, when the binary is not stripped.
<code>-T</code>	—	Display the contents of the dynamic symbol table	Dynamic symbols — what a shared object exports and imports.
<code>--dynamic-syms</code>	—	Display the contents of the dynamic symbol table	Dynamic symbols — what a shared object exports and imports.
<code>-r</code>	—	Display the relocation entries in the file	Relocations.
<code>--reloc</code>	—	Display the relocation entries in the file	Relocations.
<code>-R</code>	—	Display the dynamic relocation entries in the file	Dynamic relocations, which reveal the PLT/GOT layout.
<code>--dynamic-reloc</code>	—	Display the dynamic relocation entries in the file	Dynamic relocations, which reveal the PLT/GOT layout.
<code>-v</code>	—	Display this program's version number	
<code>--version</code>	—	Display this program's version number	
<code>-i</code>	—	List object formats and architectures supported	
<code>--info</code>	—	List object formats and architectures supported	
<code>-H</code>	—	Display this information	
<code>--help</code>	—	Display this information	
<code>-b</code>	BFDNAME	Specify the target object format as BFDNAME	Force the file format when detection fails.
<code>--target</code>	BFDNAME	Specify the target object format as BFDNAME	Force the file format when detection fails.
<code>-m</code>	MACHINE	Specify the target architecture as MACHINE	Force the architecture, needed for raw shellcode with no container to describe it.
<code>--architecture</code>	MACHINE	Specify the target architecture as MACHINE	Force the architecture, needed for raw shellcode with no container to describe it.
<code>-j</code>	NAME	Only display information for section NAME	Restrict the operation to one section, so a <code>-s</code> or <code>-d</code> on a large binary stays readable.
<code>--section</code>	NAME	Only display information for section NAME	Restrict the operation to one section, so a <code>-s</code> or <code>-d</code> on a large binary stays readable.

Flag	Argument	What it does	When you would use it
<code>--file-start-context</code>	—	Include context from start of file (with -S)	
<code>-I</code>	DIR	Add DIR to search list for source files	
<code>--include</code>	DIR	Add DIR to search list for source files	
<code>-l</code>	—	Include line numbers and filenames in output	Annotate with file and line numbers from debug info.
<code>--line-numbers</code>	—	Include line numbers and filenames in output	Annotate with file and line numbers from debug info.
<code>-F</code>	—	Include file offsets when displaying information	
<code>--file-offsets</code>	—	Include file offsets when displaying information	
<code>-C</code>	STYLE (optional)	Decode mangled/processed symbol names STYLE can be "none", "auto", "gnu-v3", "java", "gnat", "dlang", "rust"	Demangle C++ symbols into something readable.
<code>--demangle</code>	STYLE (optional)	Decode mangled/processed symbol names STYLE can be "none", "auto", "gnu-v3", "java", "gnat", "dlang", "rust"	Demangle C++ symbols into something readable.
<code>--recurse-limit</code>	—	Enable a limit on recursion whilst demangling (default)	
<code>--no-recurse-limit</code>	—	Disable a limit on recursion whilst demangling	
<code>-w</code>	—	Format output for more than 80 columns	Wide output that does not truncate long symbol names.
<code>--wide</code>	—	Format output for more than 80 columns	Wide output that does not truncate long symbol names.
<code>-z</code>	—	Do not skip blocks of zeroes when disassembling	Do not collapse blocks of zero bytes, when the padding itself matters.
<code>--disassemble-zeroes</code>	—	Do not skip blocks of zeroes when disassembling	Do not collapse blocks of zero bytes, when the padding itself matters.
<code>--start-address</code>	ADDR	Only process data whose address is >= ADDR	
<code>--stop-address</code>	ADDR	Only process data whose address is < ADDR	
<code>--no-addresses</code>	—	Do not print address alongside disassembly	

Flag	Argument	What it does	When you would use it
<code>--prefix-addresses</code>	—	Print complete address alongside disassembly	
<code>--insn-width</code>	WIDTH	Display WIDTH bytes on a single line for -d	
<code>--adjust-vma</code>	OFFSET	Add OFFSET to all displayed section addresses	
<code>--show-all-symbols</code>	—	When disassembling, display all symbols at a given address	
<code>--special-syms</code>	—	Include special symbols in symbol dumps	
<code>--inlines</code>	—	Print all inlines for source line (with -l)	
<code>--prefix</code>	PREFIX	Add PREFIX to absolute paths for -S	
<code>--prefix-strip</code>	LEVEL	Strip initial directory names for -S	
<code>--dwarf-depth</code>	N	Do not display DIEs at depth N or greater	
<code>--dwarf-start</code>	N	Display DIEs starting at offset N	
<code>--dwarf-check</code>	—	Make additional dwarf consistency checks.	
<code>--ctf-parent</code>	NAME	Use CTF archive member NAME as the CTF parent	
<code>--visualize-jumps</code>	—	Visualize jumps by drawing ASCII art lines	
<code>--disassembler-color</code>	off	Disable disassembler color output. (default)	

Gotchas

- Disassembly starts where the section says code starts. Packed and self-modifying binaries decrypt themselves at runtime, so a static pass shows the unpacking stub and nothing about the payload.
- Default output is AT&T syntax. Most Windows malware documentation is Intel, so `-M intel` avoids constant mental translation.
- `-d` skips sections not marked executable, which is precisely where code gets hidden. `-D` is slower and sees them.

See also

[rabin2](#), [readelf](#), [readelf.py](#), [r2](#), [rasm2](#), [vivbin](#), [vdbbin](#)

PE-Detective (GUI)

Capability	malware triage static
Window title	PE Detective
Captured from	C:\Tools\Explorer Suite\PE Detective.exe on 2026-08-02 — control tree in capture/gui/PE-Detective/PE-Detective.tree.txt

[← Capability index](#) · [Kit tool list](#)

Purpose

Scan a PE, or a directory of them, against a signature database to name the compiler, linker or packer that produced it. Answers "what built this?" before you commit to unpacking.

When you'd reach for this

Reach for PE Detective to answer *what built this* before committing to unpacking. It matches the binary against a signature database of compilers, linkers and packers - a few seconds of work that can save an hour spent unpacking the wrong way.

Tick **Deep Scan**. The default pass inspects the entry point only, so any packer that relocates the entry point is missed, and that is most of the ones worth detecting. Prefer **All Matches** over **Best Match** while triaging: disagreement between signatures tells you an identification is shaky, and a single best match hides exactly that.

A clean result is not an all-clear. Signatures only recognise what someone has already catalogued, so a custom or modified packer shows nothing at all. That is the moment to open CFF Explorer and read the section sizes and imports yourself.

Sources: <https://www.eyehatemailwares.com/malware-analysis/static-analysis/ma-peid/> · <https://ccdcoe.org/uploads/2020/07/Malware-Reverse-Engineering-Handbook-final.pdf>

Controls

The parts of this window you will actually touch, read from the application's own accessibility tree rather than from a screenshot. The full node list is in [capture/gui/PE-Detective/PE-Detective.tree.txt](#).

The window exposes 9 further named controls: **File Name:**, **Browse**, **Scan**, **About**, **Directory Scan**, **Recursive**, **Deep Scan**, **Best Match**, **All Matches**. The full tree, with every automation id, is in [the capture](#).

Using it

1. Press **Browse** and choose the binary.

2. Tick **Deep Scan** — the default pass only checks the entry point, and packers that relocate it are missed without this.
3. Choose **All Matches** rather than **Best Match** when triaging; a single best match hides the disagreement that tells you a signature is shaky.
4. Press **Scan**.
5. For a corpus, tick **Directory Scan** and **Recursive** instead of selecting one file.

Gotchas

- It reports signature matches, not facts. A packer name is a hypothesis; a custom or modified packer matches nothing at all, so silence never means clean.
- **Best Match** is the default and is the wrong setting for triage. Two signatures disagreeing is information.
- It shares a signature format with Signature Explorer, so an unrecognised sample can be turned into a new signature there.

rabin2

Kit	REMnux · Kali Linux · SIFT Workstation
Capability	Inspect PE / ELF structure; Disassemble and explore a binary
Version	rabin2 6.1.9
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.radare.org/n/radare2.html

← [Capability index](#) · [Kit tool list](#)

Purpose

Report the structure of a binary — headers, sections, imports, exports, strings, symbols — for any format radare2 understands.

When you'd reach for this

An analyst reaches for rabin2 when examining binary files to extract structured information about ELF/PE/MZ/CLASS files, often after extracting them from disk images or PCAPs, as it provides detailed insights into binary structure and security features (e.g., `rabin2 -I` for security checks or `rabin2 -z` for strings) that may be more straightforward or comprehensive than alternatives like `readelf` or `checksec`.

Sources: <https://gist.github.com/52617365/95baed8b731c3effdad04b1d6ccf4831> · <https://github.com/Adamkadaban/CTFs>

Synopsis

```
rabin2 [-AcdeEghHiIjJlLMqrRsSuvVxzZ] [-@ at] [-a arch] [-b bits] [-B addr]
[-C F:C:D] [-f str] [-m addr] [-n str] [-N m:M] [-P[-P] pdb]
[-o str] [-O help] [-k query] [-D lang mangledsymbol] file
```

Options

All 47 options parsed from the captured help text; 26 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-@</code>	addr	show section, symbol or import at addr	
<code>-A</code>	—	list sub-binaries and their arch-bits pairs	List all sub-binaries in a fat/universal file.

Flag	Argument	What it does	When you would use it
-a	arch	set arch (x86, arm, .. or _)	Force the architecture when detection is wrong.
-b	bits	set bits (32, 64 ...)	Force the bit width, 32 or 64.
-B	addr	override base address (pie bins)	An analyst would use the -B flag when examining a PIE binary to override its base address for accurate symbol and section analysis.
-c	—	list classes	Class information, for Objective-C, Java and .NET binaries.
-C	fmt:C:D	create [elf,mach0,pe] with Code and Data hexpairs (see -a)	An analyst would use the -C flag when creating a binary file (such as ELF, Mach-O, or PE) by specifying code and data hexpairs to reconstruct or analyze the binary structure.
-d	—	show debug/dwarf information	
-e	—	program entrypoint	Entrypoint address.
-E	—	globally exportable symbols	Exports — the entry points a DLL offers.
-f	str	select sub-bin named str	An analyst would use the -f flag when they need to select and analyze a specific sub-binary within a larger file by specifying its name.
-F	binfmt	force to use that bin plugin (ignore header check)	
-g	—	same as -SMZIHVResizcld -SS -SSS -ee (show all info)	
-G	addr	load address . address to header	
-h	—	this help message	
-H	—	header fields	Headers in full detail.
-i	—	imports (symbols imported from libraries)	Imports. What a binary asks the OS for is the fastest read on what it can do.
-I	—	binary info	The header summary: format, architecture, bits, endianness, stripped, and whether NX, PIE and canaries are on. The usual first command on an unknown binary.
-j	—	output in json	JSON output, for a pipeline.

Flag	Argument	What it does	When you would use it
<code>-k</code>	sdb-query	run sdb query. for example: '*'	Query a specific field rather than printing everything.
<code>-K</code>	algo	calculate checksums (md5, sha1, ..)	An analyst would use the <code>-K</code> flag when needing to compute a checksum for sections of a binary file using a specified rahash2 algorithm, such as md5, to verify integrity or analyze file components.
<code>-l</code>	—	linked libraries	Linked libraries.
<code>-L</code>	plugin	list supported bin plugins or plugin details	
<code>-m</code>	addr	show source line at addr	An analyst would use the <code>-m</code> flag when examining a binary to retrieve source line references corresponding to a specific memory address.
<code>-M</code>	—	main (show address of main symbol)	
<code>-n</code>	str	show section, symbol or import named str	An analyst would use the <code>-n</code> flag with <code>rabin2</code> to retrieve the offset of a specific symbol, such as when examining the location of a function like <code>_main</code> in a binary file.
<code>-N</code>	min:max	force min:max number of chars per string (see <code>-z</code> and <code>-zz</code>)	
<code>-o</code>	str	output file or extraction directory for write operations	
<code>-O</code>	str	write/extract operations (<code>-O</code> help)	Patch the binary. Read-only work never needs this, and using it on evidence modifies it.
<code>-p</code>	—	show always physical addresses	
<code>-P</code>	—	show debug/pdb information	
<code>-q</code>	—	be quiet, just show fewer data	Quiet, minimal formatting — easier to parse in a shell.
<code>-Q</code>	—	show load address used by dlopen (non-aslr libs)	
<code>-r</code>	—	radare output	
<code>-R</code>	—	relocations	Relocations.
<code>-s</code>	—	symbols	Symbols, when the binary is not stripped.

Flag	Argument	What it does	When you would use it
<code>-S</code>	—	sections	Sections with their sizes and permissions. A section that is both writable and executable is worth a second look.
<code>-t</code>	—	display file hashes	
<code>-T</code>	—	display file signature	
<code>-u</code>	—	resources	
<code>-v</code>	—	display version and quit	Version.
<code>-V</code>	—	show binary version information	
<code>-w</code>	—	display try/catch blocks	
<code>-x</code>	—	extract sub-binaries (-xS sections, -xSS segments, -xu resources)	Extract the sub-binaries of a fat file.
<code>-y</code>	—	show types (structs, enums, function signatures)	
<code>-z</code>	—	strings (from data section)	Strings from the data sections.
<code>-Z</code>	—	guess size of binary program	

Gotchas

- This is the static, scriptable half of radare2. Nothing here executes the binary, which makes it safe to run over a corpus of samples — unlike opening them in a debugger.
- The security flags in `-I` describe the *binary*, not the running process. NX and ASLR also depend on the loader and the OS.
- `-z` misses strings outside the data sections, which is where packed and obfuscated samples hide theirs. Try `-zz` before concluding a sample has none.

See also

`readelf`, `readelf.py`, `objdump`, `r2`, `rasm2`, `vivbin`, `vdbbin`

radiff2

Kit	REMnux · Kali Linux · SIFT Workstation
Capability	Compare or cluster samples; Diff two binaries
Version	radiff2 6.1.9
Captured from	<code>cyberlab-aio</code> via <code>-h</code> on 2026-08-11 — raw help output
Documentation	https://www.radare.org/n/radare2.html

[← Capability index](#) · [Kit tool list](#)

Purpose

Examine binary files, including disassembling and debugging. Includes r2ai and decal plugins for LLM-powered analysis (API key or local Ollama required), plus the r2ghidra plugin for Ghidra decompilation via the pdg command.

When you'd reach for this

An analyst reaches for radiff2 when comparing two binaries to identify byte-level differences, such as when examining modified or cracked versions of a file; they may run it after obtaining the binaries to analyze changes or similarities, using options like `-s` for similarity metrics or `-c` to count differences, as it provides precise offsets and matching function data that other tools might not explicitly detail.

Sources: https://book.rada.re/tools/radiff2/binary_diffing.html

Synopsis

```
radiff2 [-options] [-A[A]] [-B #] [-g sym] [-m graph_mode][-t %] [file] [file]
```

Options

All 29 options parsed from the captured help text; 9 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-a</code>	arch	specify architecture plugin to use (x86, arm, ..)	An analyst would use the <code>-a</code> flag when specifying the architecture plugin (e.g., x86, arm) to ensure accurate binary analysis during code or graph diffing operations.

Flag	Argument	What it does	When you would use it
<code>-A</code>	-A	run aaa or aaaa after loading each binary (see -C)	An analyst would use the -A flag when they need to automatically run the 'aaa' or 'aaaa' analysis commands on each binary after loading to ensure both files are fully analyzed before performing a differential comparison.
<code>-b</code>	bits	specify register size for arch (16 (thumb), 32, 64, ..)	
<code>-c</code>	cmd	run given command on every RCore instance	An analyst would use the -c flag when they need a concrete count of the number of differences between two binaries.
<code>-C</code>	—	graphdiff code (columns: off-A, match-ratio, off-B) (see -A)	When an analyst is unsure whether two binaries are similar and needs to check for matching functions between them.
<code>-d</code>	—	use delta diffing	
<code>-D</code>	—	show disasm instead of hexpairs	
<code>-e</code>	k=v	set eval config var value for all RCore instances	
<code>-f</code>	help	select output format (see '-f help' for details)	
<code>-g</code>	arg	graph diff of [sym] or functions in [off1,off2]	An analyst would use the -g flag when comparing the control flow graphs of specific functions between two binaries to visually identify structural or code differences, such as analyzing security updates or infected files.
<code>-i</code>	help	compare bin information (symbols, strings, classes, ..)	
<code>-j</code>	—	output in json format (see -f json)	
<code>-l</code>	addr	specify final offset (length) for diffing, same format as -o	
<code>-m</code>	mode	choose the graph output mode (aditsjl)	
<code>-n</code>	—	count of changes	

Flag	Argument	What it does	When you would use it
<code>-o</code>	addr	specify initial offset for diffing, can use A:B format	
<code>-O</code>	—	code diffing with opcode bytes only	An analyst would use the <code>-O</code> flag when comparing the opcodes of two functions in different binaries to identify differences in their machine code instructions.
<code>-p</code>	—	use physical addressing (io.va=false) (only for radiff2 - AC)	
<code>-q</code>	—	quiet mode (disable colors, reduce output)	
<code>-r</code>	—	output in radare commands	
<code>-s</code>	—	compute edit distance (no substitution, Eugene W. Myers O(ND) diff algorithm)	An analyst would use the <code>-s</code> flag when comparing two binaries to quickly determine their overall similarity percentage and distance for a high-level overview of differences.
<code>-S</code>	name	sort code diff (name, namelen, addr, size, type, dist) (only for -C or -g)	An analyst would use the <code>-S</code> flag when sorting code diff output by attributes such as name, address, or size to organize and analyze differences between binary files more effectively.
<code>-T</code>	—	analyze files in threads (EXPERIMENTAL, 30% faster and crashy)	
<code>-x</code>	—	show two column hexdump diffing	
<code>-X</code>	—	use xpatch format for the diffing output	
<code>-u</code>	—	unified output (---++)	An analyst would use the <code>-u</code> flag when comparing two binaries to output the differences in a unified format similar to the system 'diff' tool.
<code>-U</code>	—	unified output using system 'diff'	
<code>-v</code>	—	show version information	
<code>-V</code>	—	be verbose (current only for -s)	

See also

[ssdeep](#)

readelf

Kit	Base OS — present on every Linux image
Capability	Inspect PE / ELF structure
Version	GNU readelf (GNU Binutils for Debian) 2.40
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

← [Capability index](#) · [Kit tool list](#)

Purpose

Display the structure of an ELF file — headers, sections, segments, symbols, notes and dynamic linkage.

When you'd reach for this

An analyst reaches for readelf when examining stripped binaries or analyzing ELF headers to identify architecture, sections, or security features like CET; they may run it after using strings or before deeper disassembly to understand the binary's structure and protections, preferring it over similar tools for its precise ELF-specific insights into headers, sections, and dynamic symbols.

Sources: <https://hacktricks.wiki/en/binary-exploitation/basic-stack-binary-exploitation-methodology/elf-tricks.html> · <https://intezer.com/blog/elf-malware-analysis-101-initial-analysis/> · <https://w00tsec.blogspot.com/2015/02/firmware-forensics-diffs-timelines-elfs.html>

Synopsis

```
readelf <option(s)> elf-file(s)
```

Common invocations

```
# Display ELF file header metadata
readelf -h /bin/ls

# Examine ELF file section headers and their details
readelf -S /bin/ls

# Inspect symbol table entries in /bin/ls binary
readelf -s /bin/ls

# View program headers of /bin/ls executable
readelf -l /bin/ls

# Examine dynamic linking information in executable
readelf -d /bin/ls
```

Options

All 68 options parsed from the captured help text; 17 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-a</code>	—	Equivalent to: <code>-h -l -S -s -r -d -V -A -l</code>	Everything. Verbose, and the right first move when you do not yet know what you are looking for.
<code>--all</code>	—	Equivalent to: <code>-h -l -S -s -r -d -V -A -l</code>	Everything. Verbose, and the right first move when you do not yet know what you are looking for.
<code>-h</code>	—	Display the ELF file header	The ELF header: type, architecture, entry point. First thing to run on an unknown Linux binary.
<code>--file-header</code>	—	Display the ELF file header	The ELF header: type, architecture, entry point. First thing to run on an unknown Linux binary.
<code>-l</code>	—	Display the program headers	Program headers, which describe what the loader maps. A section table can be stripped or faked; the program headers must be right or the binary will not run.
<code>--program-headers</code>	—	Display the program headers	Program headers, which describe what the loader maps. A section table can be stripped or faked; the program headers must be right or the binary will not run.
<code>--segments</code>	—	An alias for <code>--program-headers</code>	
<code>-S</code>	—	Display the sections' header	Section headers.
<code>--section-headers</code>	—	Display the sections' header	Section headers.
<code>--sections</code>	—	An alias for <code>--section-headers</code>	
<code>-g</code>	—	Display the section groups	
<code>--section-groups</code>	—	Display the section groups	
<code>-t</code>	—	Display the section details	Section details in full.
<code>--section-details</code>	—	Display the section details	Section details in full.
<code>-e</code>	—	Equivalent to: <code>-h -l -S</code>	Header summary: file, section and program headers together.
<code>--headers</code>	—	Equivalent to: <code>-h -l -S</code>	Header summary: file, section and program headers together.

Flag	Argument	What it does	When you would use it
<code>-s</code>	—	Display the symbol table	The symbol table.
<code>--syms</code>	—	Display the symbol table	The symbol table.
<code>--symbols</code>	—	An alias for <code>--syms</code>	
<code>--dyn-syms</code>	—	Display the dynamic symbol table	Dynamic symbols only — what the binary imports and exports at runtime.
<code>--lto-syms</code>	—	Display LTO symbol tables	
<code>--sym-base</code>	0 8 10 16	Force base for symbol sizes. The options are mixed (the default), octal, decimal, hexadecimal.	
<code>-C</code>	—	Decode mangled/processed symbol names STYLE can be "none", "auto", "gnu-v3", "java", "gnat", "dlang", "rust"	Demangle C++ symbols.
<code>--demangle</code>	STYLE (optional)	Decode mangled/processed symbol names STYLE can be "none", "auto", "gnu-v3", "java", "gnat", "dlang", "rust"	Demangle C++ symbols.
<code>--no-demangle</code>	—	Do not demangle low-level symbol names. (default)	
<code>--recurse-limit</code>	—	Enable a demangling recursion limit. (default)	
<code>--no-recurse-limit</code>	—	Disable a demangling recursion limit	
<code>-n</code>	—	Display the core notes (if present)	Notes, including the build ID that ties a stripped binary to its debug symbols.
<code>--notes</code>	—	Display the core notes (if present)	Notes, including the build ID that ties a stripped binary to its debug symbols.
<code>-r</code>	—	Display the relocations (if present)	Relocations.
<code>--relocs</code>	—	Display the relocations (if present)	Relocations.
<code>-u</code>	—	Display the unwind info (if present)	Unwind information.
<code>--unwind</code>	—	Display the unwind info (if present)	Unwind information.
<code>-d</code>	—	Display the dynamic section (if present)	The dynamic section — needed libraries, RPATH, and the init and fini arrays that run before and after <code>main</code> .

Flag	Argument	What it does	When you would use it
<code>--dynamic</code>	—	Display the dynamic section (if present)	The dynamic section — needed libraries, RPATH, and the init and fini arrays that run before and after <code>main</code> .
<code>-V</code>	—	Display the version sections (if present)	
<code>--version-info</code>	—	Display the version sections (if present)	
<code>-A</code>	—	Display architecture specific information (if any)	
<code>--arch-specific</code>	—	Display architecture specific information (if any)	
<code>-c</code>	—	Display the symbol/file index in an archive	
<code>--archive-index</code>	—	Display the symbol/file index in an archive	
<code>-D</code>	—	Use the dynamic section info when displaying symbols	Use the dynamic symbol table rather than the static one.
<code>--use-dynamic</code>	—	Use the dynamic section info when displaying symbols	Use the dynamic symbol table rather than the static one.
<code>-x</code>	—	Dump the contents of section as bytes	Hex dump of a named section.
<code>--hex-dump</code>	number name	Dump the contents of section as bytes	Hex dump of a named section.
<code>-p</code>	—	Dump the contents of section as strings	String dump of a named section — the targeted alternative to running <code>strings</code> over the whole file.
<code>--string-dump</code>	number name	Dump the contents of section as strings	String dump of a named section — the targeted alternative to running <code>strings</code> over the whole file.
<code>-R</code>	—	Dump the relocated contents of section	
<code>--relocated-dump</code>	number name	Dump the relocated contents of section	
<code>-z</code>	—	Decompress section before dumping it	
<code>--decompress</code>	—	Decompress section before dumping it	
<code>-P</code>	—	Display the contents of non-debug sections in separate debuginfo files. (Implies <code>-wK</code>)	

Flag	Argument	What it does	When you would use it
<code>--process-links</code>	—	Display the contents of non-debug sections in separate debuginfo files. (Implies <code>-wK</code>)	
<code>--dwarf-depth</code>	N	Do not display DIEs at depth N or greater	
<code>--dwarf-start</code>	N	Display DIEs starting at offset N	
<code>--ctf</code>	number name	Display CTF info from section	
<code>--ctf-parent</code>	name	Use CTF archive member as the CTF parent	
<code>--ctf-symbols</code>	number name	Use section as the CTF external symtab	
<code>--ctf-strings</code>	number name	Use section as the CTF external strtab	
<code>--sframe</code>	NAME (optional)	Display SFrame info from section NAME, (default '.sframe')	
<code>-I</code>	—	Display histogram of bucket list lengths	
<code>--histogram</code>	—	Display histogram of bucket list lengths	
<code>-W</code>	—	Allow output width to exceed 80 characters	Wide output that does not truncate.
<code>--wide</code>	—	Allow output width to exceed 80 characters	Wide output that does not truncate.
<code>-H</code>	—	Display this information	
<code>--help</code>	—	Display this information	
<code>-v</code>	—	Display the version number of readelf	
<code>--version</code>	—	Display the version number of readelf	

Gotchas

- The section header table is optional at runtime. Malware strips or corrupts it to break tools, and the binary still runs — when sections look wrong, read `-l` instead, because the loader must be able to.
- `DT_INIT` and `DT_INIT_ARRAY` in `-d` run before `main`. Code placed there executes even if `main` looks harmless.
- An `RPATH` or `RUNPATH` pointing somewhere writable is a hijack waiting to happen, and it shows up here.

See also

`rabin2`, `readelf.py`, `objdump`

readelf.py

Kit	REMnux
Capability	Inspect PE / ELF structure
Version	based on pyelftools 0.33
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://github.com/eliben/pyelftools

← [Capability index](#) · [Kit tool list](#)

Purpose

Python library for parsing and analyzing ELF files and DWARF debugging information.

Synopsis

```
usage: readelf.py [options] <elf-file>
```

Options

All 33 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
<code>-v</code>	—	show program's version number and exit	
<code>--version</code>	—	show program's version number and exit	
<code>-d</code>	—	Display the dynamic section	
<code>--dynamic</code>	—	Display the dynamic section	
<code>-H</code>	—	Display this information	
<code>--help</code>	—	Display this information	
<code>-h</code>	—	Display the ELF file header	
<code>--file-header</code>	—	Display the ELF file header	
<code>-l</code>	—	Display the program headers	
<code>--program-headers</code>	—	Display the program headers	
<code>--segments</code>	—	Display the program headers	
<code>-S</code>	—	Display the sections' headers	

Flag	Argument	What it does	When you would use it
<code>--section-headers</code>	—	Display the sections' headers	
<code>--sections</code>	—	Display the sections' headers	
<code>-e</code>	—	Equivalent to: <code>-h -l -S</code>	
<code>--headers</code>	—	Equivalent to: <code>-h -l -S</code>	
<code>-s</code>	—	Display the symbol table	
<code>--symbols</code>	—	Display the symbol table	
<code>--syms</code>	—	Display the symbol table	
<code>-n</code>	—	Display the core notes (if present)	
<code>--notes</code>	—	Display the core notes (if present)	
<code>-r</code>	—	Display the relocations (if present)	
<code>--relocs</code>	—	Display the relocations (if present)	
<code>-x</code>	number name	Dump the contents of section as bytes	
<code>--hex-dump</code>	number name	Dump the contents of section as bytes	
<code>-p</code>	number name	Dump the contents of section as strings	
<code>--string-dump</code>	number name	Dump the contents of section as strings	
<code>-V</code>	—	Display the version sections (if present)	
<code>--version-info</code>	—	Display the version sections (if present)	
<code>-A</code>	—	Display the architecture-specific information (if present)	
<code>--arch-specific</code>	—	Display the architecture-specific information (if present)	
<code>--debug-dump</code>	what	Display the contents of DWARF debug sections. can one of {info,decodedline,frames,frames-interp,aranges,pubtypes,pubnames,loc,Ranges}	
<code>--traceback</code>	—	Dump the Python traceback on ELFError exceptions from elftools	

See also

`rabin2`, `readelf`, `objdump`

Signature-Explorer (GUI)

Capability	malware triage static
Window title	Signature Explorer II
Captured from	C:\Tools\Explorer Suite\Signature Explorer.exe on 2026-08-02 — control tree in capture/gui/Signature-Explorer/Signature-Explorer.tree.txt

[← Capability index](#) · [Kit tool list](#)

Purpose

Browse, edit and add to the packer signature database that PE Detective and CFF Explorer read. This is how a sample nothing recognises becomes a signature that catches the next one.

When you'd reach for this

Reach for Signature Explorer after a sample comes back unidentified and you have worked out by hand what packed it. It edits the signature database that PE Detective and CFF Explorer both read, so your identification becomes automatic for the next sample in the same family.

That is its entire reason to exist - it analyses nothing. It is the step that turns one analyst's finding into detection the rest of the team gets for free, and the reason a signature database is worth maintaining rather than only consuming.

Sources: <https://www.eyehatemalwares.com/malware-analysis/static-analysis/ma-peid/>

Controls

The parts of this window you will actually touch, read from the application's own accessibility tree rather than from a screenshot. The full node list is in [capture/gui/Signature-Explorer/Signature-Explorer.tree.txt](#).

The window exposes 18 further named controls: **Edit**, **Save Changes**, **Signature**, **Check that the signature doesn't already exist**, **Clear**, **Delete**, **Modify**, **Add**, **Entire Portable Executable:**, **Entry Point:**, **Comments:**, **Name:**, **Signatures**, **Number of signatures: 0**, **Filter:**, **Signature Database**, ..., **Refresh**. The full tree, with every automation id, is in [the capture](#).

Using it

1. Open the signature database from the **Signatures** pane.
2. Use **Check that the signature doesn't already exist** before adding anything — duplicates are the usual way a database rots.
3. Fill **Name:**, **Entry Point:** and, where the pattern is not at the entry point, **Entire Portable Executable:**.
4. Record why the signature exists in **Comments:** — a pattern with no rationale cannot be reviewed later.
5. Press **Add**, then **Save Changes**.

Gotchas

- This edits the database that PE Detective and CFF Explorer read. A bad signature here produces confident wrong answers in both.
- Entry-point signatures miss anything that relocates the entry point, which is exactly what many packers do. Prefer a whole-file pattern when the sample allows it.
- **Save Changes** is a separate action. Adding a signature and closing the window loses it.

unzip

Kit	REMnux
Capability	Detect and reverse packing; Analyse other container formats
Version	UnZip 6.00
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	http://infozip.sourceforge.net

← [Capability index](#) · [Kit tool list](#)

Purpose

Compress and decompress files using the zip algorithm.

Synopsis

```
unzip [-Z] [-opts[modifiers]] file[.zip] [list] [-x xlist] [-d exdir]
Default action is to extract files in list, except those in xlist, to exdir;
file[.zip] may be a wildcard. -Z => ZipInfo mode ("unzip -Z" for usage).
```

Options

All 12 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
<code>-p</code>	—	extract files to pipe, no messages -l list files (short format)	
<code>-f</code>	—	freshen existing files, create none -t test compressed archive data	
<code>-u</code>	—	update files, create if necessary -z display archive comment only	
<code>-v</code>	—	list verbosely/show version info -T timestamp archive to latest	
<code>-x</code>	—	exclude files that follow (in xlist) -d extract files into exdir	
<code>-n</code>	—	never overwrite existing files -q quiet mode (-qq => quieter)	

Flag	Argument	What it does	When you would use it
<code>-o</code>	—	overwrite files WITHOUT prompting -a auto-convert any text files	
<code>-j</code>	—	junk paths (do not make directories) -aa treat ALL files as text	
<code>-U</code>	—	use escapes for all non-ASCII Unicode -UU ignore any Unicode fields	
<code>-C</code>	—	match filenames case-insensitively -L make (some) names lowercase	
<code>-X</code>	—	restore UID/GID info -V retain VMS version numbers	
<code>-K</code>	—	keep setuid/setgid/tacky permissions -M pipe through "more" pager	

See also

`upx`, `die`, `diec`, `binwalk`, `7za`, `msoffcrypto-tool`

upx

Kit	REMnux · Kali Linux · FLARE-VM · SIFT Workstation
Capability	Detect and reverse packing
Version	upx 4.2.4
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://upx.github.io

← [Capability index](#) · [Kit tool list](#)

Purpose

Pack and unpack executables with the UPX compressor.

When you'd reach for this

An analyst reaches for UPX when encountering a sample with .UPX0/.UPX1 sections, running `upx -d` to quickly unpack it before analyzing the decrypted code, as it is straightforward and automated compared to manual unpacking or tools like Unipacker that require emulation for more complex packers.

Sources: <https://inventivehq.com/blog/malware-unpacking-guide>

Synopsis

```
upx [-123456789dlthVL] [-qvfk] [-o file] file..
```

Options

All 37 options parsed from the captured help text; 5 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-1</code>	—	compress faster -9 compress better	
<code>-d</code>	—	decompress -l list compressed file	Decompress — the only flag you normally want in analysis.
<code>-t</code>	—	test compressed file -V display version number	Test the file's integrity without writing anything.
<code>-h</code>	—	give this help -L display software license	
<code>-q</code>	—	be quiet -v be verbose	Quiet output.

Flag	Argument	What it does	When you would use it
<code>-f</code>	—	force compression of suspicious files	Force the operation on a file UPX considers questionable.
<code>--no-color</code>	—	change look	
<code>--mono</code>	—	change look	
<code>--color</code>	—	change look	
<code>--no-progress</code>	—	change look	
<code>--lzma</code>	—	try LZMA [slower but tighter than NRV]	
<code>--brute</code>	—	try all available compression methods & filters [slow]	
<code>--ultra-brute</code>	—	try even more compression variants [very slow]	
<code>-k</code>	—	keep backup files	Keep a backup of the original file.
<code>--backup</code>	—	keep backup files	Keep a backup of the original file.
<code>--no-backup</code>	—	no backup files [default]	
<code>--overlay</code>	copy	copy any extra data attached to the file [default]	
<code>--force-overwrite</code>	—	force overwrite of output files	
<code>--link</code>	—	preserve hard links (Unix only) [USE WITH CARE]	
<code>--no-link</code>	—	do not preserve hard links but rename files [default]	
<code>--no-mode</code>	—	do not preserve file mode (aka permissions)	
<code>--no-owner</code>	—	do not preserve file ownership	
<code>--no-time</code>	—	do not preserve file timestamp	
<code>--coff</code>	—	produce COFF output [default: EXE]	
<code>--8086</code>	—	make compressed com work on any 8086	
<code>--no-reloc</code>	—	put no relocations in to the exe header	
<code>--8-bit</code>	—	uses 8 bit size compression [default: 32 bit]	
<code>--8mib-ram</code>	—	8 megabyte memory limit [default: 2 MiB]	
<code>--boot-only</code>	—	disables client/host transfer compatibility	

Flag	Argument	What it does	When you would use it
<code>--no-align</code>	—	don't align to 2048 bytes [enables: <code>--console-run</code>]	
<code>--le</code>	—	produce LE output [default: EXE]	
<code>--compress-exports</code>	0	do not compress the export section	
<code>--compress-icons</code>	0	do not compress any icons	
<code>--compress-resources</code>	0	do not compress any resources at all	
<code>--keep-resource</code>	list	do not compress resources specified by list	
<code>--strip-relocs</code>	0	do not strip relocations	
<code>--preserve-build-id</code>	—	copy <code>.gnu.note.build-id</code> to compressed output	

Gotchas

- **Work on a copy.** `upx -d` rewrites the file in place, mutating your sample and invalidating its hash.
- Malware routinely uses a modified UPX header so stock `upx -d` refuses. Failure to unpack is itself an indicator, not a dead end.

See also

`die`, `diec`, `binwalk`, `7za`, `unzip`

yara

Kit	REMnux · Kali Linux · FLARE-VM
Capability	Scan with signatures for known-bad
Version	4.2.3
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://virustotal.github.io/yara/

← [Capability index](#) · [Kit tool list](#)

Purpose

Scan files, directories or live processes against YARA rules.

When you'd reach for this

An analyst reaches for YARA when they need to detect specific malware or file patterns using Boolean conditions and tags, running it after collecting files from an incident or system, as it allows efficient filtering and organization of rules through metadata and tags.

Sources: <https://www.eccouncil.org/cybersecurity-exchange/ethical-hacking/mastering-yara-rules-a-complete-guide-with-use-cases-syntax-and-real-world-examples/> · <https://www.picussecurity.com/resource/glossary/what-is-a-yara-rule> · <https://yara.readthedocs.io/en/stable/writingrules.html>

Synopsis

```
yara [OPTION]... [NAMESPACE:]RULES_FILE... FILE | DIR | PID
```

Options

All 55 options parsed from the captured help text; 13 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>--atom-quality-table</code>	FILE	path to a file with the atom quality table	
<code>-C</code>	—	load compiled rules	Load a pre-compiled ruleset (from <code>yarac</code>) instead of source.

Flag	Argument	What it does	When you would use it
<code>--compiled-rules</code>	—	load compiled rules	Load a pre-compiled ruleset (from <code>yarac</code>) instead of source.
<code>-c</code>	—	print only number of matches	Print only the count of matches per file, for a quick sweep.
<code>--count</code>	—	print only number of matches	Print only the count of matches per file, for a quick sweep.
<code>-d</code>	VAR=VALUE	define external variable	Define an external variable a rule expects, e.g. <code>-d filename=x</code> .
<code>--define</code>	VAR=VALUE	define external variable	Define an external variable a rule expects, e.g. <code>-d filename=x</code> .
<code>--fail-on-warnings</code>	—	fail on warnings	
<code>-f</code>	—	fast matching mode	Fast matching mode; faster but can miss some overlapping matches.
<code>--fast-scan</code>	—	fast matching mode	Fast matching mode; faster but can miss some overlapping matches.
<code>-h</code>	—	show this help and exit	
<code>--help</code>	—	show this help and exit	
<code>-i</code>	IDENTIFIER	print only rules named IDENTIFIER	Scan only the rule with the given identifier.
<code>--identifier</code>	IDENTIFIER	print only rules named IDENTIFIER	Scan only the rule with the given identifier.
<code>--max-process-memory-chunk</code>	NUMBER	set maximum chunk size while reading process memory (default=1073741824)	
<code>-l</code>	NUMBER	abort scanning after matching a NUMBER of rules	
<code>--max-rules</code>	NUMBER	abort scanning after matching a NUMBER of rules	
<code>--max-strings-per-rule</code>	NUMBER	set maximum number of strings per rule (default=10000)	
<code>-x</code>	MODULE=FILE	pass FILE's content as extra data to MODULE	
<code>--module-data</code>	MODULE=FILE	pass FILE's content as extra data to MODULE	
<code>-n</code>	—	print only not satisfied rules (negate)	Print files that do NOT match — useful for finding the odd one out.

Flag	Argument	What it does	When you would use it
<code>--negate</code>	—	print only not satisfied rules (negate)	Print files that do NOT match — useful for finding the odd one out.
<code>-N</code>	—	do not follow symlinks when scanning	
<code>--no-follow-symlinks</code>	—	do not follow symlinks when scanning	
<code>-w</code>	—	disable warnings	Suppress warnings, which are noisy on large third-party rulesets.
<code>--no-warnings</code>	—	disable warnings	Suppress warnings, which are noisy on large third-party rulesets.
<code>-m</code>	—	print metadata	Print rule metadata, which usually carries author and reference.
<code>--print-meta</code>	—	print metadata	Print rule metadata, which usually carries author and reference.
<code>-D</code>	—	print module data	
<code>--print-module-data</code>	—	print module data	
<code>-e</code>	—	print rules' namespace	
<code>--print-namespace</code>	—	print rules' namespace	
<code>-S</code>	—	print rules' statistics	
<code>--print-stats</code>	—	print rules' statistics	
<code>-s</code>	—	print matching strings	Print the matching strings and their offsets. Essential for verifying a hit is real rather than trusting the rule name.
<code>--print-strings</code>	—	print matching strings	Print the matching strings and their offsets. Essential for verifying a hit is real rather than trusting the rule name.
<code>-L</code>	—	print length of matched strings	
<code>--print-string-length</code>	—	print length of matched strings	
<code>-g</code>	—	print tags	
<code>--print-tags</code>	—	print tags	
<code>-r</code>	—	recursively search directories	Recurse into directories — the usual mode when triaging a tree.

Flag	Argument	What it does	When you would use it
<code>--recursive</code>	—	recursively search directories	Recurse into directories — the usual mode when triaging a tree.
<code>--scan-list</code>	—	scan files listed in FILE, one per line	
<code>-z</code>	NUMBER	skip files larger than the given size when scanning a directory	
<code>--skip-larger</code>	NUMBER	skip files larger than the given size when scanning a directory	
<code>-k</code>	SLOTS	set maximum stack size (default=16384)	
<code>--stack-size</code>	SLOTS	set maximum stack size (default=16384)	
<code>-t</code>	TAG	print only rules tagged as TAG	Scan only rules carrying a given tag.
<code>--tag</code>	TAG	print only rules tagged as TAG	Scan only rules carrying a given tag.
<code>-p</code>	NUMBER	use the specified NUMBER of threads to scan a directory	Use N threads to speed up a large scan.
<code>--threads</code>	NUMBER	use the specified NUMBER of threads to scan a directory	Use N threads to speed up a large scan.
<code>-a</code>	SECONDS	abort scanning after the given number of SECONDS	
<code>--timeout</code>	SECONDS	abort scanning after the given number of SECONDS	
<code>-v</code>	—	show version information	An analyst would use the -v flag when validating the syntax of a YARA rule to ensure it is correctly formatted before testing it against files.
<code>--version</code>	—	show version information	An analyst would use the -v flag when validating the syntax of a YARA rule to ensure it is correctly formatted before testing it against files.

Gotchas

- A rule that references an external variable fails to compile unless you supply it with `-d`. This is the most common 'the rule is broken' false alarm.
- Scanning a live process needs a PID rather than a path, and root.
- Compile large rulesets once with `yarac` and scan with `-C`; recompiling thousands of rules per scan dominates runtime.

See also

[yarak](#), [clamscan](#), [freshclam](#), [sigtool](#)

yarac

Kit	REMnux · Kali Linux · FLARE-VM
Capability	Scan with signatures for known-bad
Version	4.2.3
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://virustotal.github.io/yara/

← [Capability index](#) · [Kit tool list](#)

Purpose

Identify and classify malware samples using Yara rules.

Synopsis

```
yarac [OPTION]... [NAMESPACE:]SOURCE_FILE... OUTPUT_FILE
```

Options

All 11 options parsed from the captured help text; 2 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>--atom-quality-table</code>	FILE	path to a file with the atom quality table	
<code>-d</code>	VAR=VALUE	define external variable	
<code>--define</code>	VAR=VALUE	define external variable	
<code>--fail-on-warnings</code>	—	fail on warnings	An analyst would use the <code>--fail-on-warnings</code> flag when compiling YARA rules to ensure that no warnings are present, enforcing strict rule validation during the compilation process.
<code>-h</code>	—	show this help and exit	
<code>--help</code>	—	show this help and exit	
<code>--max-strings-per-rule</code>	NUMBER	set maximum number of strings per rule (default=10000)	

Flag	Argument	What it does	When you would use it
<code>-w</code>	—	disable warnings	An analyst would use the <code>--no-warnings</code> flag when compiling YARA rules into a binary to suppress warnings during compilation, allowing the process to proceed without interruptions when external variables or rule syntax may generate non-critical alerts.
<code>--no-warnings</code>	—	disable warnings	An analyst would use the <code>--no-warnings</code> flag when compiling YARA rules into a binary to suppress warnings during compilation, allowing the process to proceed without interruptions when external variables or rule syntax may generate non-critical alerts.
<code>-v</code>	—	show version information	
<code>--version</code>	—	show version information	

See also

`yara`, `clamscan`, `freshclam`, `sigtool`

aeskeyfind

Kit	REMnux · Kali Linux · SIFT Workstation
Capability	Recover encryption keys from memory
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://citp.princeton.edu/our-work/memory/

← [Capability index](#) · [Kit tool list](#)

Purpose

Find 128-bit and 256-bit AES keys in a memory image.

When you'd reach for this

An analyst reaches for aeskeyfind after creating a memory dump using a tool like Volatility to recover AES keys from the dump, as it is specifically designed to locate 128-bit and 256-bit AES keys in memory images; they would run it after the dump is created and before exporting the keys, preferring it over similar tools due to its focus on AES key recovery from memory dumps.

Sources: <https://medium.com/@Frogjump/aeskeyfind-in-kali-linux-72ba6a8ea2fd>

Synopsis

```
aeskeyfind [OPTION]... MEMORY-IMAGE
```

Options

All 3 options parsed from the captured help text; 1 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-v</code>	—	verbose output -- prints the extended keys and the constraints on the rows of the key schedule	An analyst would use the -v flag when examining memory images to obtain detailed verbose output, including extended keys and constraints on the rows of the key schedule, to aid in forensic analysis.
<code>-q</code>	—	don't display a progress bar	
<code>-h</code>	—	displays this help message	

See also

`rsa_keyfind`, `bulk_extractor`

rsakeyfind

Kit	REMnux · Kali Linux · SIFT Workstation
Capability	Recover encryption keys from memory
Captured from	<code>cyberlab-aio</code> via <code>(no args)</code> on 2026-08-11 — raw help output
Documentation	https://citp.princeton.edu/our-work/memory/

← [Capability index](#) · [Kit tool list](#)

Purpose

Find BER-encoded RSA private keys in a memory image.

Synopsis

```
rsakeyfind MEMORY-IMAGE [MODULUS-FILE]
```

Options

No option definitions could be parsed from this tool's help output. It may be subcommand-driven or have no flags; needs manual review.

See also

`aeskeyfind`, `bulk_extractor`

vol

Kit	SIFT Workstation (Volatility 3)
Capability	Analyse a memory image
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Extract and analyse artifacts from a memory image using Volatility 3 plugins.

When you'd reach for this

An analyst reaches for vol when examining memory dumps to detect malicious activity, such as unusual processes or command-line arguments; they may first run plugins like `windows.pslist` or `windows.pstree` to establish context before using `windows.cmdline` or `windows.handles` for deeper analysis, as the tool's use of symbol tables ensures accurate parsing of memory structures over guesswork.

Sources: <https://hivesecurity.gitlab.io/blog/memory-forensics-volatility-attack-detect/> · <https://www.dfirhive.com/post/windows-memory-and-process-analysis-volatility3-walkthrough>

Synopsis

```
vol [-h] [-c CONFIG] [--parallelism [{processes,threads,off}]]
[-e EXTEND] [-p PLUGIN_DIRS] [-s SYMBOL_DIRS] [-v] [-l LOG]
[-o OUTPUT_DIR] [-q] [-f FILE] [--write-config]
[--save-config SAVE_CONFIG] [--clear-cache]
[--cache-path CACHE_PATH] [--offline | -u URL] [--filters FILTERS]
[--hide-columns [HIDE_COLUMNS ...]] [-r RENDERER]
```

Options

All 35 options parsed from the captured help text; 9 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	Show this help message and exit, for specific plugin options use 'vol --help'	

Flag	Argument	What it does	When you would use it
<code>--help</code>	—	Show this help message and exit, for specific plugin options use 'vol --help'	
<code>-c</code>	CONFIG	Load the configuration from a json file	Load saved configuration from a JSON file.
<code>--config</code>	CONFIG	Load the configuration from a json file	Load saved configuration from a JSON file.
<code>--parallelism</code>	{processes,threads,off}	Enables parallelism (defaults to off if no argument given)	
<code>-e</code>	EXTEND	Extend the configuration with a new (or changed) setting	
<code>--extend</code>	EXTEND	Extend the configuration with a new (or changed) setting	
<code>-p</code>	PLUGIN_DIRS	Semi-colon separated list of paths to find plugins	Add a directory of custom plugins.
<code>--plugin-dirs</code>	PLUGIN_DIRS	Semi-colon separated list of paths to find plugins	Add a directory of custom plugins.
<code>-s</code>	SYMBOL_DIRS	Semi-colon separated list of paths to find symbols	Point at a local symbol-table directory — the fix for an air-gapped host that cannot download symbols.
<code>--symbol-dirs</code>	SYMBOL_DIRS	Semi-colon separated list of paths to find symbols	Point at a local symbol-table directory — the fix for an air-gapped host that cannot download symbols.
<code>-v</code>	—	Increase output verbosity	Increase verbosity while diagnosing a symbol-table failure.
<code>--verbosity</code>	—	Increase output verbosity	Increase verbosity while diagnosing a symbol-table failure.
<code>-l</code>	LOG	Log output to a file as well as the console	
<code>--log</code>	LOG	Log output to a file as well as the console	
<code>-o</code>	OUTPUT_DIR	Directory in which to output any generated files	Directory for files the plugin dumps (processes, DLLs, files).
<code>--output-dir</code>	OUTPUT_DIR	Directory in which to output any generated files	Directory for files the plugin dumps (processes, DLLs, files).
<code>-q</code>	—	Remove progress feedback	Quiet — suppress the progress and INFO banner when scripting.

Flag	Argument	What it does	When you would use it
<code>--quiet</code>	—	Remove progress feedback	Quiet — suppress the progress and INFO banner when scripting.
<code>-f</code>	FILE	Shorthand for <code>--single-location=file://</code> if single-location is not defined	The memory image to analyse — required for almost every plugin.
<code>--file</code>	FILE	Shorthand for <code>--single-location=file://</code> if single-location is not defined	The memory image to analyse — required for almost every plugin.
<code>--write-config</code>	—	Write configuration JSON file out to config.json	
<code>--save-config</code>	SAVE_CONFIG	Save configuration JSON file to a file	
<code>--clear-cache</code>	—	Clears out all short-term cached items	
<code>--cache-path</code>	CACHE_PATH	Change the default path (/root/.cache/volatility3) used to store the cache	
<code>--offline</code>	—	Do not search online for additional JSON files	Never attempt a network fetch for symbols. Use this on evidence networks so a scan cannot stall on a download.
<code>-u</code>	URL	Search online for ISF json files	
<code>--remote-isf-url</code>	URL	Search online for ISF json files	
<code>--filters</code>	FILTERS	List of filters to apply to the output (in the form of [+ -]columnname,pattern[!])	
<code>--hide-columns</code>	HIDE_COLUMNS	Case-insensitive space separated list of prefixes to determine which columns to hide in the output if provided	
<code>-r</code>	RENDERER	Determines how to render the output (quick, none, csv, pretty, json, jsonl, arrow, parquet)	Choose the renderer: <code>csv</code> / <code>json</code> when feeding another tool.
<code>--renderer</code>	RENDERER	Determines how to render the output (quick, none, csv, pretty, json, jsonl, arrow, parquet)	Choose the renderer: <code>csv</code> / <code>json</code> when feeding another tool.
<code>--single-location</code>	SINGLE_LOCATION	Specifies a base location on which to stack	
<code>--stackers</code>	STACKERS	List of stackers	
<code>--single-swap-locations</code>	SINGLE_SWAP_LOCATIONS	Specifies a list of swap layer URIs for use with single-location	

Gotchas

- **Timeout-guard** `vol` **in any harness.** On a synthetic or truncated memory image it has pinned a core at 99% indefinitely on this project. Never run it unbounded in an automated pass.
- Volatility 3 takes plugin names, not v2 syntax: `windows.pslist`, not `--profile=... pslist`. Most 'plugin not found' errors are pasted v2 commands.
- The first run against an unfamiliar image downloads symbol tables. On an isolated network that hangs — pre-stage symbols and use `--offline`.

See also

`volatility3`, `volshell`

volatility3

Kit	SIFT Workstation (Volatility 3)
Capability	Analyse a memory image
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Analyse a memory image. These are the framework-wide options; each plugin adds its own, shown by `volatility3 <plugin> --help`.

Synopsis

```
volatility3 [-h] [-c CONFIG] [--parallelism [{processes,threads,off}]]
[-e EXTEND] [-p PLUGIN_DIRS] [-s SYMBOL_DIRS] [-v] [-l LOG]
[-o OUTPUT_DIR] [-q] [-f FILE] [--write-config]
[--save-config SAVE_CONFIG] [--clear-cache]
[--cache-path CACHE_PATH] [--offline | -u URL]
[--filters FILTERS] [--hide-columns [HIDE_COLUMNS ...]]
```

Options

All 35 options parsed from the captured help text; 33 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	Show this help message and exit, for specific plugin options use 'volatility3 --help'	
<code>--help</code>	—	Show this help message and exit, for specific plugin options use 'volatility3 --help'	
<code>-c</code>	CONFIG	Load the configuration from a json file	Load options from a JSON config.
<code>--config</code>	CONFIG	Load the configuration from a json file	Load options from a JSON config.
<code>--parallelism</code>	{processes,threads,off}	Enables parallelism (defaults to off if no argument given)	Enable process or thread parallelism. Off by default, and worth turning on for a long scan.

Flag	Argument	What it does	When you would use it
<code>-e</code>	EXTEND	Extend the configuration with a new (or changed) setting	Override a single configuration setting.
<code>--extend</code>	EXTEND	Extend the configuration with a new (or changed) setting	Override a single configuration setting.
<code>-p</code>	PLUGIN_DIRS	Semi-colon separated list of paths to find plugins	Additional plugin directories, for plugins outside the tree.
<code>--plugin-dirs</code>	PLUGIN_DIRS	Semi-colon separated list of paths to find plugins	Additional plugin directories.
<code>-s</code>	SYMBOL_DIRS	Semi-colon separated list of paths to find symbols	Where to find symbol tables. The usual fix when a Linux or macOS image will not resolve: the ISF for that exact kernel has to be reachable.
<code>--symbol-dirs</code>	SYMBOL_DIRS	Semi-colon separated list of paths to find symbols	Where to find symbol tables — the usual fix when a Linux or macOS image will not resolve.
<code>-v</code>	—	Increase output verbosity	More verbose output; repeat for more.
<code>--verbosity</code>	—	Increase output verbosity	More verbose output; repeat for more.
<code>-l</code>	LOG	Log output to a file as well as the console	Also write output to a log file.
<code>--log</code>	LOG	Log output to a file as well as the console	Also write output to a log file.
<code>-o</code>	OUTPUT_DIR	Directory in which to output any generated files	Directory for files the plugin writes — dumped processes, extracted DLLs. Required before any <code>--dump</code> plugin option does anything useful.
<code>--output-dir</code>	OUTPUT_DIR	Directory in which to output any generated files	Directory for files the plugin writes. Required before any <code>-dump</code> plugin option is useful.
<code>-q</code>	—	Remove progress feedback	Suppress progress feedback, for scripted runs.
<code>--quiet</code>	—	Remove progress feedback	Suppress progress feedback, for scripted runs.
<code>-f</code>	FILE	Shorthand for <code>--single-location=file://</code> if single-location is not defined	The memory image. Almost every invocation starts here.
<code>--file</code>	FILE	Shorthand for <code>--single-location=file://</code> if single-location is not defined	The memory image. Almost every invocation starts here.

Flag	Argument	What it does	When you would use it
<code>--write-config</code>	—	Write configuration JSON file out to config.json	Write the resolved configuration to config.json — useful for making a complex run reproducible.
<code>--save-config</code>	SAVE_CONFIG	Save configuration JSON file to a file	Write the resolved configuration to a named file.
<code>--clear-cache</code>	—	Clears out all short-term cached items	Drop cached items. First thing to try when results look stale or an image was replaced in place.
<code>--cache-path</code>	CACHE_PATH	Change the default path (/root/.cache/volatility3) used to store the cache	Move the cache somewhere with room; it grows.
<code>--offline</code>	—	Do not search online for additional JSON files	Never fetch symbols online. Use it on an analysis host that must not touch the network, and expect failures unless the ISFs are already local.
<code>-u</code>	URL	Search online for ISF json files	Point at an alternative ISF repository.
<code>--remote-isf-url</code>	URL	Search online for ISF json files	Point at an alternative ISF repository.
<code>--filters</code>	FILTERS	List of filters to apply to the output (in the form of [+ -]columnname,pattern[!])	Filter rows as <code>[+ -]column,pattern</code> , so a plugin that returns thousands of rows can be narrowed without a second pass.
<code>--hide-columns</code>	HIDE_COLUMNS	Case-insensitive space separated list of prefixes to determine which columns to hide in the output if provided	Drop columns from the output to keep a wide table readable.
<code>-r</code>	RENDERER	Determines how to render the output (quick, none, csv, pretty, json, jsonl, arrow, parquet)	Output renderer. <code>csv</code> or <code>jsonl</code> when the result feeds another tool; <code>pretty</code> is for reading, not for parsing.
<code>--renderer</code>	RENDERER	Determines how to render the output (quick, none, csv, pretty, json, jsonl, arrow, parquet)	Output renderer. <code>csv</code> or <code>jsonl</code> when the result feeds another tool; <code>pretty</code> is for reading.
<code>--single-location</code>	SINGLE_LOCATION	Specifies a base location on which to stack	The image URI, when it is not a plain local file (<code>-f</code> is shorthand for this).
<code>--stackers</code>	STACKERS	List of stackers	Control the layer stackers used to interpret the image.

Flag	Argument	What it does	When you would use it
<code>--single-swap-locations</code>	SINGLE_SWAP_LOCATIONS	Specifies a list of swap layer URIs for use with single-location	Supply swap files alongside the image, so paged-out memory can be resolved.

Gotchas

- Symbols are the usual failure, not the image. Windows profiles are generated automatically, but Linux and macOS need an ISF matching the exact kernel build — same version is not enough.
- Volatility 3 dropped the Volatility 2 profile system entirely, so `--profile` from older notes and blog posts does not exist here.
- It has pinned CPU indefinitely on a malformed image before. Run long analyses under a timeout rather than assuming progress.

See also

`vol`, `volshell`

volshell

Kit	SIFT Workstation (Volatility 3)
Capability	Analyse a memory image
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

An interactive Python shell over a memory image, with Volatility's object model loaded — for questions no plugin answers.

When you'd reach for this

An analyst reaches for volshell when they need to interactively run plugins or execute custom scripts on a memory image, often after loading the image to extract or analyze specific data. They may use it to generate TreeGrid objects for structured data access or run snippets via rs for quick tasks, preferring it over writing full plugins due to its flexibility and direct framework access.

Sources: <https://github.com/volatilityfoundation/volatility3/blob/develop/doc/source/volshell.rst> · <https://volatility3.readthedocs.io/en/latest/volshell.html>

Synopsis

```
volshell [-h] [-c CONFIG] [-e EXTEND] [-p PLUGIN_DIRS] [-s SYMBOL_DIRS]
[-v] [-o OUTPUT_DIR] [-q] [--log LOG] [-f FILE]
[--write-config] [--save-config SAVE_CONFIG] [--clear-cache]
[--cache-path CACHE_PATH] [--offline | -u URL] [-w | -l | -m]
[--single-location SINGLE_LOCATION]
[--stackers [STACKERS ...]]
```

Options

All 38 options parsed from the captured help text; 21 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	show this help message and exit	
<code>--help</code>	—	show this help message and exit	

Flag	Argument	What it does	When you would use it
<code>-c</code>	CONFIG	Load the configuration from a json file	
<code>--config</code>	CONFIG	Load the configuration from a json file	
<code>-e</code>	EXTEND	Extend the configuration with a new (or changed) setting	
<code>--extend</code>	EXTEND	Extend the configuration with a new (or changed) setting	
<code>-p</code>	PLUGIN_DIRS	Semi-colon separated list of paths to find plugins	
<code>--plugin-dirs</code>	PLUGIN_DIRS	Semi-colon separated list of paths to find plugins	
<code>-s</code>	SYMBOL_DIRS	Semi-colon separated list of paths to find symbols	Where to find symbol tables — the usual fix when a Linux or macOS image will not resolve.
<code>--symbol-dirs</code>	SYMBOL_DIRS	Semi-colon separated list of paths to find symbols	Where to find symbol tables.
<code>-v</code>	—	Increase output verbosity	More verbose output.
<code>--verbosity</code>	—	Increase output verbosity	More verbose output.
<code>-o</code>	OUTPUT_DIR	Directory in which to output any generated files	Directory for files written out of the shell.
<code>--output-dir</code>	OUTPUT_DIR	Directory in which to output any generated files	Directory for files written out of the shell.
<code>-q</code>	—	Remove progress feedback	Quiet.
<code>--quiet</code>	—	Remove progress feedback	Quiet.
<code>--log</code>	LOG	Log output to a file as well as the console	
<code>-f</code>	FILE	Shorthand for <code>--single-location=file://</code> if single-location is not defined	The memory image.
<code>--file</code>	FILE	Shorthand for <code>--single-location=file://</code> if single-location is not defined	The memory image.
<code>--write-config</code>	—	Write configuration JSON file out to config.json	
<code>--save-config</code>	SAVE_CONFIG	Save configuration JSON file to a file	
<code>--clear-cache</code>	—	Clears out all short-term cached items	Drop cached items when results look stale.
<code>--cache-path</code>	CACHE_PATH	Change the default path (<code>/root/.cache/volatility3</code>) used to store the cache	

Flag	Argument	What it does	When you would use it
<code>--offline</code>	—	Do not search online for additional JSON files	Never fetch symbols online.
<code>-u</code>	URL	Search online for ISF json files	
<code>--remote-isf-url</code>	URL	Search online for ISF json files	
<code>-w</code>	—	Run a Windows volshell	Treat the image as Windows.
<code>--windows</code>	—	Run a Windows volshell	Treat the image as Windows.
<code>-l</code>	—	Run a Linux volshell	Treat the image as Linux.
<code>--linux</code>	—	Run a Linux volshell	Treat the image as Linux.
<code>-m</code>	—	Run a Mac volshell	Treat the image as macOS.
<code>--mac</code>	—	Run a Mac volshell	Treat the image as macOS.
<code>--single-location</code>	SINGLE_LOCATION	Specifies a base location on which to stack	
<code>--stackers</code>	STACKERS	List of stackers	
<code>--single-swap-locations</code>	SINGLE_SWAP_LOCATIONS	Specifies a list of swap layer URLs for use with single-location	
<code>--script</code>	SCRIPT	File to load and execute at start	Run a Python script against the image and drop into the shell afterwards — the repeatable form of an interactive session.
<code>--script-only</code>	—	Exit volshell after the script specified in <code>--script</code> completes	Run the script and exit. This is how an exploratory session becomes an automated one.
<code>--pid</code>	PID	Process ID	Enter with a process context already selected, so <code>cc()</code> is not the first thing you type.

Gotchas

- Reach for this when a plugin nearly answers the question but not quite. If a plugin exists, use the plugin — it is tested and this is not.
- Findings from an interactive session are not reproducible by default. Move anything that matters into `--script` so the result can be regenerated and reviewed.
- The same symbol requirement as `volatility3` applies: without an ISF matching the exact kernel build, a Linux or macOS image will not resolve at all.

See also

`vol`, `volatility3`

arp-scan

Kit	SIFT Workstation
Capability	Probe or scan hosts and services
Version	arp-scan 1.10.0
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Target hosts must be specified on the command line unless the `--file` or

When you'd reach for this

An analyst reaches for `arp-scan` when they need to verify the presence of a system with known IP and MAC addresses on a LAN, often running it first with a broadcast to determine the MAC address and then again targeting the specific MAC address for a quieter scan. They may use it after identifying a host via broadcast or before confirming its presence without alerting other network stations, as targeting a specific MAC avoids broadcasting to all devices.

Sources: <https://github.com/royhills/arp-scan/wiki/arp-scan-User-Guide>

Synopsis

```
arp-scan [options] [hosts...]
```

Options

No option definitions could be parsed from this tool's help output. It may be subcommand-driven or have no flags; needs manual review.

See also

`nmap`, `nping`

capinfos

Kit	REMnux · Kali Linux · FLARE-VM · SIFT Workstation
Capability	Read and filter packet captures
Version	Capinfos (Wireshark) 4.0.17.
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.wireshark.org

← [Capability index](#) · [Kit tool list](#)

Purpose

Capture and analyze network traffic with this sniffer.

Synopsis

```
capinfos [options] <infile> ...
```

Common invocations

```
# Generate tab-delimited report with pcap file metadata
capinfos -TtEc *.pcap
# Check duplicate packets in capture file
capinfos -c dupes.pcap
# Generate detailed capture file analysis report
capinfos mycapture.pcap
# Generate tabular report of capture file details
capinfos -T mycapture.pcap
# Generate tab report with pcap file metadata
capinfos -T -t -E -c *.pcap
```

Options

All 4 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	display this help and exit	
<code>--help</code>	—	display this help and exit	

Flag	Argument	What it does	When you would use it
<code>-v</code>	—	display version info and exit	
<code>--version</code>	—	display version info and exit	

See also

`tshark`, `ngrep`, `tcpflow`

editcap

Kit	REMnux · Kali Linux · FLARE-VM · SIFT Workstation
Capability	Split, merge or repair capture files
Version	Editcap (Wireshark) 4.0.17.
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.wireshark.org

← [Capability index](#) · [Kit tool list](#)

Purpose

Cut, split, deduplicate and convert capture files — the tool that makes an unmanageable pcap workable before analysis starts.

When you'd reach for this

An analyst reaches for editcap when they need to remove duplicate packets or split a capture file into smaller segments, often running capinfos first to assess the file's structure, as it directly handles format editing and packet manipulation tasks that other tools like mergcap or tshark do not explicitly address.

Sources: <https://docsislab.wordpress.com/packet-capture/wireshark-command-line/> · <https://wiki.wireshark.org/Tools>

Synopsis

```
editcap [options] ... <infile> <outfile> [ <packet#>[-<packet#>] ... ]
```

Common invocations

```
# Remove duplicate packets from capture file
editcap -d dupes.pcap nodups.pcap
# Split capture into 200-packet segments
editcap -c 200 dbad.pcap dbadsplit.pcap
```

Options

All 25 options parsed from the captured help text; 21 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-r</code>	—	keep the selected packets; default is to delete them.	Invert the selection: keep the specified packets instead of deleting them. Easy to forget, and it reverses the meaning of the whole command.
<code>-A</code>	start time	only read packets whose timestamp is after (or equal to) the given time.	Keep only packets at or after this timestamp.
<code>-B</code>	stop time	only read packets whose timestamp is before the given time. Time format for -A/-B options is YYYY-MM-DDThh:mm:ss[.nnnnnnnnn] [Z][+ -hh:mm] Unix epoch timestamps are also supported.	Keep only packets before this timestamp. With <code>-A</code> , this is how a multi-gigabyte capture becomes the incident window.
<code>--novlan</code>	—	remove vlan info from packets before checking for duplicates.	Ignore VLAN tags when comparing for duplicates, so the same frame seen on two VLANs collapses to one.
<code>-d</code>	—	remove packet if duplicate (window == 5).	Drop duplicate packets using the default 5-packet window. Captures taken from a SPAN port routinely see each packet twice, which distorts every count downstream.
<code>-D</code>	dup window	remove packet if duplicate; configurable . Valid values are 0 to 1000000. NOTE: A of 0 with -V (verbose option) is useful to print MD5 hashes.	Drop duplicates with an explicit window, when <code>-d</code> 's default is too narrow.
<code>-w</code>	dup time window	remove packet if duplicate packet is found EQUAL TO OR LESS THAN prior to current packet. A is specified in relative seconds (e.g. 0.000001). NOTE: The use of the '	Drop duplicates within a time window rather than a packet count.
<code>-s</code>	snaplen	truncate each packet to max. bytes of data.	Truncate each packet to N bytes. Strips payload while keeping headers — the usual way to share a capture without its contents.
<code>-L</code>	—	adjust the frame (i.e. reported) length when chopping and/or snapping.	Adjust the recorded frame length to match after truncating, so the file is not self-inconsistent.

Flag	Argument	What it does	When you would use it
<code>-t</code>	time adjustment	adjust the timestamp of each packet. is in relative seconds (e.g. -0.5).	Shift every timestamp by a relative amount. This is how a capture from a host with a skewed clock is aligned to the rest of the timeline.
<code>-o</code>	change offset	When used in conjunction with <code>-E</code> , skip some bytes from the beginning of the packet. This allows one to preserve some bytes, in order to have some headers untouched.	With <code>-E</code> , skip bytes before introducing errors.
<code>--seed</code>	seed	When used in conjunction with <code>-E</code> , set the seed to use for the pseudo-random number generator. This allows one to repeat a particular sequence of errors.	With <code>-E</code> , fix the random seed so a corrupted-capture test is reproducible.
<code>-I</code>	bytes to ignore	ignore the specified number of bytes at the beginning of the frame during MD5 hash calculation, unless the frame is too short, then the full frame is used. Useful to remove duplicated packets taken on	Ignore N leading bytes when comparing for duplicates.
<code>-c</code>	packets per file	split the packet output to different files based on uniform packet counts with a maximum of each.	Split into files of N packets each. The standard fix for a capture too large for Wireshark to open.
<code>-i</code>	seconds per file	split the packet output to different files based on uniform time intervals with a maximum of each.	Split into files covering N seconds each — the same fix, when time is the natural unit.
<code>-F</code>	capture type	set the output file type; default is pcapng. An empty <code>"-F"</code> option will list the file types.	Output file format; pcapng by default. An empty <code>-F</code> lists the choices.
<code>-T</code>	encap type	set the output file encapsulation type; default is the same as the input file. An empty <code>"-T"</code> option will list the encapsulation types.	Output encapsulation type, when the link type must change.
<code>--discard-all-secrets</code>	—	Discard all decryption secrets from the input file when writing the output file. Does not discard secrets added by <code>"-inject-secrets"</code> in the same command line.	Strip embedded decryption secrets before handing the file to someone else.

Flag	Argument	What it does	When you would use it
<code>--capture-comment</code>	comment	Add a capture file comment, if supported.	Attach a comment to the file — a place to record provenance that travels with the capture.
<code>--discard-capture-comment</code>	—	Discard capture file comments from the input file when writing the output file. Does not discard comments added by " <code>--capture-comment</code> " in the same command line.	Remove existing comments on output.
<code>-h</code>	—	display this help and exit.	
<code>--help</code>	—	display this help and exit.	
<code>-V</code>	—	verbose output. If <code>-V</code> is used with any of the 'Duplicate Packet Removal' options (<code>-d</code> , <code>-D</code> or <code>-w</code>) then Packet lengths and MD5 hashes are printed to standard-error.	Verbose; with the duplicate options it reports what was removed rather than silently dropping packets.
<code>-v</code>	—	print version information and exit.	
<code>--version</code>	—	print version information and exit.	

Gotchas

- `-r` inverts the selection. Without it the named packets are **deleted**, which is the opposite of what most people intend the first time.
- Deduplication is a heuristic over a window, not a proof. A genuine retransmission looks like a duplicate, and dropping it destroys the evidence that a retransmission occurred.
- Splitting rennumbers packets per output file. Frame numbers cited from a split file do not refer to the original capture.

See also

`mergecap`, `reordercap`

inetsim

Kit	REMnux · Kali Linux
Capability	Simulate network services for detonation
Version	INetSim 1.3.2
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.inetsim.org/

← [Capability index](#) · [Kit tool list](#)

Purpose

Emulate common network services and interact with malware.

When you'd reach for this

An analyst reaches for inetsim when setting up a malware analysis lab with VirtualBox and Burp, particularly when encountering installation issues like the "apt-key is deprecated" error, which the documentation addresses with specific keyring setup commands. They may run it after resolving dependencies and configuring the lab environment to simulate network services for analyzing C2 traffic.

Sources: <https://blog.christophetd.fr/malware-analysis-lab-with-virtualbox-inetsim-and-burp/>

Synopsis

```
/usr/bin/inetsim [options]
```

Options

All 14 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
<code>--help</code>	—	Print this help message.	
<code>--version</code>	—	Show version information.	
<code>--config</code>	filename	Configuration file to use.	
<code>--log-dir</code>	directory	Directory logfiles are written to.	
<code>--data-dir</code>	directory	Directory containing service data.	
<code>--report-dir</code>	directory	Directory reports are written to.	

Flag	Argument	What it does	When you would use it
<code>--bind-address</code>	IP address	Default IP address to bind services to. Overrides configuration option 'default_bind_address'.	
<code>--max-chlds</code>	num	Default maximum number of child processes per service. Overrides configuration option 'default_max_chlds'.	
<code>--user</code>	username	Default user to run services. Overrides configuration option 'default_run_as_user'.	
<code>--faketime-init-delta</code>	secs	Initial faketime delta (seconds). Overrides configuration option 'faketime_init_delta'.	
<code>--faketime-auto-delay</code>	secs	Delay for auto incrementing faketime (seconds). Overrides configuration option 'faketime_auto_delay'.	
<code>--faketime-auto-incr</code>	secs	Delta for auto incrementing faketime (seconds). Overrides configuration option 'faketime_auto_increment'.	
<code>--session</code>	id	Session id to use. Defaults to main process id.	
<code>--pidfile</code>	filename	Pid file to use. Defaults to '/var/run/inetsim.pid'.	

mergecap

Kit	REMnux · Kali Linux · FLARE-VM · SIFT Workstation
Capability	Split, merge or repair capture files
Version	Mergecap (Wireshark) 4.0.17.
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.wireshark.org

← [Capability index](#) · [Kit tool list](#)

Purpose

Capture and analyze network traffic with this sniffer.

When you'd reach for this

An analyst reaches for mergecap when merging multiple pcap files captured sequentially into a single file, often running it after capturing or before analysis to consolidate data; they choose it over append mode to maintain correct timestamps and avoid misordering packets, as demonstrated in the documentation.

Sources: <https://osqa-ask.wireshark.org/questions/31113/wireshark-merging-pcap-files/> · <https://osqa-ask.wireshark.org/questions/39951/how-to-simultaneously-filter-and-merge-several-pcap-files/> · <https://wiki.wireshark.org/Tools>

Synopsis

```
mergecap [options] -w <outfile>|- <infile> [<infile> ...]
```

Common invocations

```
# Merge multiple pcap files into a single capture file
mergecap *.pcap -w merged.pcapng
# Merge two network capture files into a single unified pcap file
mergecap -w compare.pcap a.pcap b-shifted.pcap
# Merge capture files into single output file
mergecap -a -w outoforder.pcap download-good.pcap
```

Options

All 7 options parsed from the captured help text; 1 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-a</code>	—	concatenate rather than merge files. default is to merge based on frame timestamps.	An analyst would use the <code>-a</code> flag when they need to concatenate input files in the order they are provided, without reordering packets based on timestamps.
<code>-s</code>	snaplen	truncate packets to bytes of data.	
<code>-h</code>	—	display this help and exit.	
<code>--help</code>	—	display this help and exit.	
<code>-V</code>	—	verbose output.	
<code>-v</code>	—	print version information and exit.	
<code>--version</code>	—	print version information and exit.	

See also

[editcap](#), [reordercap](#)

ngrep

Kit	REMnux · SIFT Workstation
Capability	Read and filter packet captures
Version	V1.47.1
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://github.com/jpr5/ngrep/

← [Capability index](#) · [Kit tool list](#)

Purpose

Grep packet payloads, live or from a capture, with BPF filtering — pattern matching on the wire.

When you'd reach for this

An analyst reaches for ngrep when searching for specific patterns in network traffic, such as detecting "login" in Telnet sessions, using switches like `-w`, `-i`, and `-t` for precise matching and timestamps; they may run it alongside tcpdump to analyze captured packets, preferring it over tcpdump for its grep-style filtering and intuitive regular expression handling.

Sources: <https://www.admin-magazine.com/Articles/Network-Grep>

Synopsis

```
ngrep <-hNXviwqpevxLDtTRM> <-IO pcap_dump> <-n num> <-d dev> <-A num>  
<-s snaplen> <-S limitlen> <-W normal|byline|single|none> <-c cols>  
<-P char> <-F file> <-K count>  
<match expression> <bpf filter>
```

Common invocations

```
# Monitor SMTP traffic across all network interfaces
ngrep -d any port 25
# Monitor HTTP traffic with line-based packet display on port 80
ngrep -W byline port 80
# Search PCAP for specific string occurrences in packets
ngrep -w 'm' -I /tmp/dns.dump
# Search DNS dump for ns3 entries with timestamps and replay packets
ngrep -tD ns3 -I /tmp/dns.dump
# Search network dump for traffic on specific port
ngrep -I /tmp/dns.dump port 80
```

Options

All 29 options parsed from the captured help text; 27 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	is help/usage	
<code>-V</code>	—	is version information	
<code>-q</code>	—	is be quiet (don't print packet reception hash marks)	Suppress the reception hash marks.
<code>-e</code>	—	is show empty packets	Show empty packets, which are otherwise hidden.
<code>-i</code>	—	is ignore case	Case-insensitive match — usually what you want for protocol keywords and hostnames.
<code>-v</code>	—	is invert match	Invert the match, to see everything that is <i>not</i> the known traffic.
<code>-R</code>	—	is don't do privilege revocation logic	Skip privilege revocation.
<code>-x</code>	—	is print in alternate hexdump format	Print payloads as a hexdump — necessary when the protocol is not text.
<code>-X</code>	—	is interpret match expression as hexadecimal	Treat the pattern as hexadecimal, for matching binary signatures rather than text.
<code>-w</code>	—	is word-regex (expression must match as a word)	Match the pattern as a whole word, to stop a short string matching inside longer ones.
<code>-p</code>	—	is don't go into promiscuous mode	Do not enter promiscuous mode — capture only traffic addressed to this host.

Flag	Argument	What it does	When you would use it
<code>-l</code>	—	is make stdout line buffered	Line-buffer stdout, so output appears when piped.
<code>-D</code>	—	is replay pcap_dumps with their recorded time intervals	Replay a pcap at its recorded timing rather than as fast as possible.
<code>-t</code>	—	is print timestamp every time a packet is matched	Print a timestamp on every match.
<code>-T</code>	—	is print delta timestamp every time a packet is matched specify twice for delta from first match	Print the delta since the previous match; twice for delta from the first. Useful for spotting beaconing intervals.
<code>-M</code>	—	is don't do multi-line match (do single-line match instead)	Single-line matching instead of multi-line.
<code>-I</code>	—	is read packet stream from pcap format file pcap_dump	Read from a pcap file instead of an interface. The safe mode: no capture privileges and no risk of touching live traffic.
<code>-O</code>	—	is dump matched packets in pcap format to pcap_dump	Write matched packets to a pcap, turning a search into a smaller capture that Wireshark can open.
<code>-n</code>	—	is look at only num packets	Stop after N packets.
<code>-A</code>	—	is dump num packets after a match	Also print N packets following each match, for the response to the request that matched.
<code>-s</code>	—	is set the bpf caplen	Set the BPF capture length.
<code>-S</code>	—	is set the limitlen on matched packets	Limit how much of a matched packet is shown.
<code>-W</code>	—	is set the dump format (normal, byline, single, none)	Output format: <code>byLine</code> is far more readable for text protocols than the default.
<code>-c</code>	—	is force the column width to the specified size	Force the column width.
<code>-P</code>	—	is set the non-printable display char to what is specified	Set the character shown for non-printable bytes.
<code>-F</code>	—	is read the bpf filter from the specified file	Read the BPF filter from a file, when it is too long to be comfortable on a command line.
<code>-N</code>	—	is show sub protocol number	Show sub-protocol numbers.
<code>-d</code>	—	is use specified device instead of the pcap default	Choose the capture interface rather than the pcap default.

Flag	Argument	What it does	When you would use it
<code>-K</code>	—	is send N packets to kill observed connections	Send packets to kill matched connections. This writes to the network — it is not an analysis option, and it does not belong anywhere near evidence handling.

Gotchas

- It matches within individual packets. A string split across TCP segments will not be found — reassembly is `tshark`'s job, not this one's.
- `-K` is the one flag here that changes the world instead of observing it. Everything else reads; that one transmits.
- Matching payload on a live interface needs capture privileges, and on a busy link `ngrep` drops packets silently. Capture first, search the file afterwards, when the answer has to be complete.

See also

`tshark`, `capinfos`, `tcpflow`

nmap

Kit	Kali Linux · FLARE-VM
Capability	Probe or scan hosts and services
Version	Nmap version 7.93
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Discover hosts, ports and services, and fingerprint what is listening.

Synopsis

```
nmap [Scan Type(s)] [Options] {target specification}
```

Options

All 66 options parsed from the captured help text; 28 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>--exclude</code>	host1[,host2][,host3]	Exclude hosts/networks	Skip these hosts. The safety flag: use it for anything fragile before starting a range scan.
<code>--excludefile</code>	exclude_file	Exclude list from file	Skip the hosts listed in a file.
<code>--dns-servers</code>	serv1[,serv2]	Specify custom DNS servers	
<code>--system-dns</code>	—	Use OS's DNS resolver	
<code>--traceroute</code>	—	Trace hop path to each host	Trace the path to each host.
<code>--scanflags</code>	flags	Customize TCP scan flags	
<code>-b</code>	FTP relay host	FTP bounce scan	
<code>-p</code>	port ranges	Only scan specified ports Ex: -p22; -p1-65535; -p U:53,111,137,T:21-25,80,139,8080,S:9	Which ports. <code>-p-</code> is all 65535 and takes far longer than people expect.
<code>--exclude-ports</code>	port ranges	Exclude the specified ports from scanning	
<code>-F</code>	—	Fast mode - Scan fewer ports than the default scan	Fast scan of the top 100 ports.

Flag	Argument	What it does	When you would use it
<code>-r</code>	—	Scan ports sequentially - don't randomize	
<code>--top-ports</code>	number	Scan most common ports	Scan the N most common ports — the usual time/coverage compromise.
<code>--port-ratio</code>	ratio	Scan ports more common than	
<code>--version-intensity</code>	level	Set from 0 (light) to 9 (try all probes)	How hard <code>-sV</code> tries, 0 to 9.
<code>--version-light</code>	—	Limit to most likely probes (intensity 2)	Intensity 2 — much faster, misses more.
<code>--version-all</code>	—	Try every single probe (intensity 9)	Intensity 9.
<code>--version-trace</code>	—	Show detailed version scan activity (for debugging)	
<code>--script</code>	Lua scripts	is a comma separated list of directories, script-files or script-categories	Run NSE scripts. The category matters — <code>vuln</code> and <code>exploit</code> scripts actively test, and <code>exploit</code> can change the target.
<code>--script-args</code>	n1=v1,[n2=v2,...]	provide arguments to scripts	Arguments for those scripts.
<code>--script-args-file</code>	filename	provide NSE script args in a file	An analyst would use the <code>--script-args-file</code> flag when they need to specify multiple script arguments in a file rather than on the command line, allowing for easier management of complex or repeated argument sets.
<code>--script-trace</code>	—	Show all data sent and received	An analyst would use the <code>--script-trace</code> flag when executing specific scripts to monitor and analyze all incoming and outgoing communication between the scripts and the target system, such as when troubleshooting script behavior or inspecting detailed protocol interactions.
<code>--script-updatedb</code>	—	Update the script database.	An analyst would use the <code>--script-updatedb</code> flag when they have added, removed, or modified the categories of NSE scripts in the default scripts directory, requiring the script database to be updated.

Flag	Argument	What it does	When you would use it
<code>--script-help</code>	Lua scripts	Show help about scripts. is a comma-separated list of script-files or script-categories.	Explain what a script does before running it, which is worth doing for anything outside <code>safe</code> .
<code>-0</code>	—	Enable OS detection	OS fingerprint from the TCP/IP stack. A guess with a confidence, not a fact.
<code>--osscan-limit</code>	—	Limit OS detection to promising targets	
<code>--osscan-guess</code>	—	Guess OS more aggressively	Report near matches rather than staying silent.
<code>-T</code>	0-5	Set timing template (higher is faster)	Timing template 0-5. <code>-T4</code> is the usual choice on a LAN; <code>-T0</code> and <code>-T1</code> exist for evading rate-based detection and take hours.
<code>--min-hostgroup</code>	—	Parallel host scan group sizes	
<code>--min-parallelism</code>	—	Probe parallelization	
<code>--min-rtt-timeout</code>	—	Specifies probe round trip time.	
<code>--max-retries</code>	tries	Caps number of port scan probe retransmissions.	Cap retransmissions on a lossy link.
<code>--host-timeout</code>	time	Give up on target after this long	Give up on a host after this long, so one dead host cannot stall a range.
<code>--min-rate</code>	number	Send packets no slower than per second	
<code>--max-rate</code>	number	Send packets no faster than per second	
<code>-f</code>	—	fragment packets (optionally w/given MTU)	
<code>--mtu</code>	val	fragment packets (optionally w/given MTU)	
<code>-D</code>	decoy1,decoy2[,ME]	Cloak a scan with decoys	Decoy scan.
<code>-S</code>	IP_Address	Spoof source address	Spoof the source address.
<code>-e</code>	iface	Use specified interface	Choose the interface to scan from.
<code>--proxies</code>	url1,[url2]	Relay connections through HTTP/SOCKS4 proxies	
<code>--data</code>	hex string	Append a custom payload to sent packets	

Flag	Argument	What it does	When you would use it
<code>--data-string</code>	string	Append a custom ASCII string to sent packets	
<code>--data-length</code>	num	Append random data to sent packets	
<code>--ip-options</code>	options	Send packets with specified ip options	
<code>--ttl</code>	val	Set IP time-to-live field	
<code>--spoof-mac</code>	mac address	Spoof your MAC address	
<code>--badsum</code>	—	Send packets with a bogus TCP/UDP/SCTP checksum	
<code>-v</code>	—	Increase verbosity level (use -vv or more for greater effect)	Verbose; repeat for more.
<code>-d</code>	—	Increase debugging level (use -dd or more for greater effect)	
<code>--reason</code>	—	Display the reason a port is in a particular state	
<code>--open</code>	—	Only show open (or possibly open) ports	Show only open ports, cutting the closed-port noise.
<code>--packet-trace</code>	—	Show all packets sent and received	
<code>--iflist</code>	—	Print host interfaces and routes (for debugging)	
<code>--append-output</code>	—	Append to rather than clobber specified output files	
<code>--resume</code>	filename	Resume an aborted scan	
<code>--noninteractive</code>	—	Disable runtime interactions via keyboard	
<code>--stylesheet</code>	path	XSL stylesheet to transform XML output to HTML	
<code>--webxml</code>	—	Reference stylesheet from Nmap.Org for more portable XML	
<code>--no-stylesheet</code>	—	Prevent associating of XSL stylesheet w/XML output	
<code>-6</code>	—	Enable IPv6 scanning	Scan IPv6. Hosts frequently expose more on v6 than v4 because the firewall rules were never mirrored.
<code>-A</code>	—	Enable OS detection, version detection, script scanning, and traceroute	Aggressive: version, OS, scripts and traceroute together. Convenient and unmistakably noisy.

Flag	Argument	What it does	When you would use it
<code>--datadir</code>	dirname	Specify custom Nmap data file location	An analyst would use the <code>--datadir</code> flag when needing to specify a custom location for Nmap data files, such as when scripts or configuration files are stored outside the default directories.
<code>--privileged</code>	—	Assume that the user is fully privileged	
<code>--unprivileged</code>	—	Assume the user lacks raw socket privileges	
<code>-V</code>	—	Print version number	
<code>-h</code>	—	Print this help summary page.	

Gotchas

- Scanning is not passive. `-sV` and NSE talk to services properly, `--script exploit` may change the target, and everything here is recorded by anything watching. Have authorisation before running it, and use `--exclude` for hosts that must not be touched.
- `-p-` on a /24 is a very different job from the default scan. Scope the ports before scoping the hosts.
- A closed port and a filtered port are different findings. 'Filtered' means something dropped the probe, which is information about the network rather than about the host.

See also

`nping`, `arp-scan`

nping

Kit	Kali Linux · FLARE-VM
Capability	Probe or scan hosts and services
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Craft and send arbitrary network packets, and report what comes back. Unlike `ping` it will build TCP, UDP, ICMP or raw ARP probes with chosen flags and payloads, which makes it the tool for asking a firewall or an IDS a precise question about what it permits.

When you'd reach for this

An analyst reaches for nping when they need to send custom network packets for testing or forensic analysis, such as probing specific ports or crafting ICMP requests, often after identifying a target range or before verifying network behavior. They may choose it over similar tools due to its detailed target specification options, support for CIDR and octet ranges, and flexibility in packet crafting, as demonstrated in the examples and documentation.

Sources: <https://nmap.org/book/nping-man.html>

Synopsis

```
nping [Probe mode] [Options] {target specification}
```

Common invocations

```
# Test host reachability and open ports
nping scanme.nmap.org
# Test target port responsiveness with two rounds
nping --tcp -c 2 1.1.1.1 -p 100-102
# Testing network connectivity via echo server
nping --echo-server "public" -e wlan0 -vvv
# Test network host with custom TCP packet parameters
nping --tcp -p 80 --flags rst --ttl 2 192.168.1.1
# Send ICMP time-exceeded packets to test network path
nping --icmp --icmp-type time --delay 500ms 192.168.254.254
# Test TCP port responsiveness across multiple hosts with delays
nping --tcp -c 1 --delay 500ms 1.1.1.1 2.2.2.2 3.3.3.3 -p 137-139
```

Options

All 85 options parsed from the captured help text; 3 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>--tcp-connect</code>	—	Unprivileged TCP connect probe mode.	
<code>--tcp</code>	—	TCP probe mode.	An analyst would use the <code>--tcp</code> flag when sending TCP packets to specific ports as part of network testing or scanning, such as in the example where it is used with <code>--flags rst</code> to send a reset packet to port 80.
<code>--udp</code>	—	UDP probe mode.	
<code>--icmp</code>	—	ICMP probe mode.	
<code>--arp</code>	—	ARP/RARP probe mode.	
<code>--tr</code>	—	Traceroute mode (can only be used with TCP/UDP/ICMP modes).	
<code>--traceroute</code>	—	Traceroute mode (can only be used with TCP/UDP/ICMP modes).	
<code>-p</code>	port spec	Set destination port(s).	An analyst would use the <code>-p</code> flag when testing specific TCP ports on a target host, such as checking if a web server is listening on port 80.

Flag	Argument	What it does	When you would use it
<code>--dest-port</code>	port spec	Set destination port(s).	An analyst would use the <code>--dest-port</code> flag when testing connectivity to a specific service on a target host, such as verifying HTTPS availability on port 443.
<code>-g</code>	portnumber	Try to use a custom source port.	
<code>--source-port</code>	portnumber	Try to use a custom source port.	
<code>--seq</code>	seqnumber	Set sequence number.	
<code>--flags</code>	flag list	Set TCP flags (ACK,PSH,RST,SYN,FIN...)	
<code>--ack</code>	acknumber	Set ACK number.	
<code>--win</code>	size	Set window size.	
<code>--badsum</code>	—	Use a random invalid checksum.	
<code>--icmp-type</code>	type	ICMP type.	
<code>--icmp-code</code>	code	ICMP code.	
<code>--icmp-id</code>	id	Set identifier.	
<code>--icmp-seq</code>	n	Set sequence number.	
<code>--icmp-redirect-addr</code>	addr	Set redirect address.	
<code>--icmp-param-pointer</code>	pnt	Set parameter problem pointer.	
<code>--icmp-advert-lifetime</code>	time	Set router advertisement lifetime.	
<code>--icmp-advert-entry</code>	IP,pref	Add router advertisement entry.	
<code>--icmp-orig-time</code>	—	: Set originate timestamp.	
<code>--icmp-recv-time</code>	—	: Set receive timestamp.	
<code>--icmp-trans-time</code>	timestamp	Set transmit timestamp.	
<code>--arp-type</code>	type	Type: ARP, ARP-reply, RARP, RARP-reply.	
<code>--arp-sender-mac</code>	mac	Set sender MAC address.	
<code>--arp-sender-ip</code>	—	: Set sender IP address.	
<code>--arp-target-mac</code>	mac	Set target MAC address.	
<code>--arp-target-ip</code>	—	: Set target IP address.	
<code>-S</code>	—	Set source IP address.	
<code>--source-ip</code>	—	Set source IP address.	

Flag	Argument	What it does	When you would use it
<code>--dest-ip</code>	addr	Set destination IP address (used as an alternative to {target specification}).	
<code>--tos</code>	tos	Set type of service field (8bits).	
<code>--id</code>	—	: Set identification field (16 bits).	
<code>--df</code>	—	Set Don't Fragment flag.	
<code>--mf</code>	—	Set More Fragments flag.	
<code>--evil</code>	—	Set Reserved / Evil flag.	
<code>--ttl</code>	hops	Set time to live [0-255].	
<code>--badsum-ip</code>	—	Use a random invalid checksum.	
<code>--ip-options</code>	S R [route]]L [route]]T U	Set IP options	
<code>--mtu</code>	size	Set MTU. Packets get fragmented if MTU is small enough.	
<code>-6</code>	—	Use IP version 6.	
<code>--IPv6</code>	—	Use IP version 6.	
<code>--hop-limit</code>	—	Set hop limit (same as IPv4 TTL).	
<code>--traffic-class</code>	class	: Set traffic class.	
<code>--flow</code>	label	Set flow label.	
<code>--dest-mac</code>	mac	Set destination mac address. (Disables ARP resolution)	
<code>--source-mac</code>	mac	Set source MAC address.	
<code>--ether-type</code>	type	Set EtherType value.	
<code>--data</code>	hex string	Include a custom payload.	
<code>--data-string</code>	text	Include a custom ASCII text.	
<code>--data-length</code>	len	Include len random bytes as payload.	
<code>--echo-client</code>	passphrase	Run Nping in client mode.	
<code>--echo-server</code>	passphrase	Run Nping in server mode.	
<code>--echo-port</code>	port	Use custom to listen or connect.	
<code>--no-crypto</code>	—	Disable encryption and authentication.	
<code>--once</code>	—	Stop the server after one connection.	

Flag	Argument	What it does	When you would use it
<code>--safe-payloads</code>	—	Erase application data in echoed packets.	
<code>--delay</code>	time	Adjust delay between probes.	
<code>--rate</code>	—	: Send num packets per second.	
<code>-h</code>	—	Display help information.	
<code>--help</code>	—	Display help information.	
<code>-V</code>	—	Display current version number.	
<code>--version</code>	—	Display current version number.	
<code>-c</code>	n	Stop after rounds.	
<code>--count</code>	n	Stop after rounds.	
<code>-e</code>	name	Use supplied network interface.	
<code>--interface</code>	name	Use supplied network interface.	
<code>-H</code>	—	Do not display sent packets.	
<code>--hide-sent</code>	—	Do not display sent packets.	
<code>-N</code>	—	Do not try to capture replies.	
<code>--no-capture</code>	—	Do not try to capture replies.	
<code>--privileged</code>	—	Assume user is fully privileged.	
<code>--unprivileged</code>	—	Assume user lacks raw socket privileges.	
<code>--send-eth</code>	—	Send packets at the raw Ethernet layer.	
<code>--send-ip</code>	—	Send packets using raw IP sockets.	
<code>--bpf-filter</code>	filter spec	Specify custom BPF filter.	
<code>-v</code>	—	Increment verbosity level by one.	
<code>-d</code>	—	Increment debugging level by one.	
<code>-q</code>	—	Decrease verbosity level by one.	
<code>--quiet</code>	—	Set verbosity and debug level to minimum.	
<code>--debug</code>	—	Set verbosity and debug to the max level.	

See also

`nmap`, `arp-scan`

reordercap

Kit	REMnux · Kali Linux · FLARE-VM · SIFT Workstation
Capability	Split, merge or repair capture files
Version	Reordercap (Wireshark) 4.0.17.
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.wireshark.org

← [Capability index](#) · [Kit tool list](#)

Purpose

Capture and analyze network traffic with this sniffer.

When you'd reach for this

An analyst uses reordercap when packets in a capture file are out of chronological order, running it after capturing or extracting the file to reorder packets by timestamp; they avoid using the same input and output file to prevent malformation, and prefer it over manual sorting or other tools because it automatically detects file formats and compression.

Sources: <https://manpages.debian.org/testing/wireshark-common/reordercap.1.en.html> · <https://tshark.dev/edit/reordercap/>

Synopsis

```
reordercap [options] <infile> <outfile>
```

Options

All 3 options parsed from the captured help text; 1 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-n</code>	—	don't write to output file if the input file is ordered.	An analyst would use the <code>-n</code> flag when verifying if a pcap file is already in chronological order to avoid unnecessary processing and output file creation.
<code>-h</code>	—	display this help and exit.	

Flag	Argument	What it does	When you would use it
<code>-v</code>	—	print version information and exit.	

See also

`editcap`, `mergecap`

tcpflow

Kit	REMnux · Kali Linux · SIFT Workstation
Capability	Read and filter packet captures; Extract files and payloads from traffic
Version	TCPFLOW 1.6.1
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://downloads.digitalcorpora.org/downloads/tcpflow/

[← Capability index](#) · [Kit tool list](#)

Purpose

Reassemble TCP streams from a capture and write each conversation to its own file. Where a packet tool shows you frames, this gives you the bytes each side actually sent, in order — which is what you need to read a protocol or recover a transferred file.

When you'd reach for this

An analyst reaches for tcpflow after capturing network traffic with tools like tcpdump, when they need to reconstruct and analyze TCP data streams in files for easier forensic examination; they may run tcpflow on stored pcap files to extract and organize data, preferring it over Wireshark for its ability to save reconstructed flows into conventional files, simplifying further analysis with standard tools.

Sources: <https://forensics.wiki/tcpflow/> · <https://www.systutorials.com/docs/linux/man/1-tcpflow/>

Synopsis

```
tcpflow [-aBCDhIpsvVZ] [-b max_bytes] [-d debug_level]
[-[eE] scanner] [-f max_fds] [-F[ctTXMkmg]] [-h|--help] [-i iface]
[-l files...] [-L semlock] [-m min_bytes] [-o outdir] [-r file] [-R file]
[-S name=value] [-T template] [-U|--relinquish-privileges user] [-v|--verbose]
[-w file] [-x scanner] [-X xmlfile] [-z|--chroot dir] [expression]
```

Common invocations

```
# Extract and decode TCP flows from a pcap capture for analysis
tcpflow -r eth0.pcap
# Reconstruct TCP sessions from multiple pcap files
tcpflow -o out -a -l *.pcap
# Reconstruct network flows from pcap for analysis
tcpflow -a -o outdir -Fk -r packets.pcap
```

Options

All 33 options parsed from the captured help text; 10 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-a</code>	—	do ALL post-processing.	
<code>-b</code>	max_bytes	max number of bytes per flow to save	
<code>-d</code>	debug_level	debug level; default is 1	
<code>-f</code>	—	maximum number of file descriptors to use	
<code>-h</code>	—	print this help message (-hh for more help)	
<code>-H</code>	—	print detailed information about each scanner	
<code>-i</code>	—	network interface on which to listen	An analyst would use the -i flag when they need to capture packets from a specific network interface rather than relying on libpcap's default selection.
<code>-I</code>	—	write for each flow another file *.findx to provide byte-indexed timestamps	
<code>-g</code>	—	output each flow in alternating colors (note change!)	
<code>-l</code>	—	treat non-flag arguments as input files rather than a pcap expression	An analyst would use the -l flag when processing multiple pcap files simultaneously with shell globbing, such as analyzing all capture files in a directory at once.
<code>-L</code>	—	semlock - specifies that writes are locked using a named semaphore	An analyst would use the -L flag when running multiple instances of tcpflow that output to the same standard output to prevent their outputs from overlapping and corrupting each other.

Flag	Argument	What it does	When you would use it
<code>-p</code>	—	don't use promiscuous mode	
<code>-q</code>	—	quiet mode - do not print warnings	
<code>-r</code>	file	read packets from tcpdump pcap file (may be repeated)	An analyst would use the -r flag when processing a pcap file to automatically decode and save TCP flows, such as extracting HTTP responses or reconstructing data from network captures.
<code>-R</code>	file	read packets from tcpdump pcap file TO FINISH CONNECTIONS	An analyst would use the -R flag when processing a pcap file captured with tcpdump -w to rebuild the flows.
<code>-v</code>	—	verbose operation equivalent to -d 10	
<code>-V</code>	—	print version number and exit	
<code>-w</code>	—	file : write packets not processed to file	An analyst would use the -w flag when they need to capture and save UDP packets that were not processed by tcpflow's default handling.
<code>-o</code>	—	outdir : specify output directory (default '.')	An analyst would use the -o flag when they need to specify a particular directory to store the transcript files generated by tcpflow during packet analysis.
<code>-X</code>	—	filename : DFXML output to filename	An analyst would use the -X flag when generating a DFXML report to document every TCP connection and system details for forensic analysis.
<code>-m</code>	—	bytes : specifies skip that starts a new stream (default 16777216).	
<code>-Z</code>	—	do not decompress gzip-compressed HTTP transactions	
<code>-K</code>	—	output keep pcap flow structure.	
<code>-U</code>	user	relinquish privileges and become user (if running as root)	
<code>-z</code>	dir	chroot to dir (requires that -U be used).	

Flag	Argument	What it does	When you would use it
<code>-E</code>	scanner	- turn off all scanners except scanner	
<code>-B</code>	—	binary output, even with -c or -C (normally -c or -C turn it off)	
<code>-c</code>	—	console print only (don't create files)	An analyst would use the -c flag when they need to print the contents of packets to the console in real-time without storing any captured data to files.
<code>-C</code>	—	console print only, but without the display of source/dest header	An analyst would use the -C flag when they need to view flow data in the console without the display of source/destination headers.
<code>-0</code>	—	don't print newlines after packets when printing to console	
<code>-s</code>	—	strip non-printable characters (change to '.')	
<code>-J</code>	—	output json format.	
<code>-D</code>	—	output in hex (useful to combine with -c or -C)	

See also

`tshark`, `capinfos`, `ngrep`, `tcpxtract`, `foremost`

ezhexviewer

Kit	REMnux
Capability	Inspect files by hand
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.decalage.info/python/oletools

[← Capability index](#) · [Kit tool list](#)

Purpose

Microsoft Office OLE2 compound documents.

Options

No option definitions could be parsed from this tool's help output. It may be subcommand-driven or have no flags; needs manual review.

See also

`xxd`, `less`

dnSpy (GUI)

Capability	reverse engineering
Window title	dnSpy v6.5.1 (64-bit, .NET Framework)
Version	v6.5.1
Captured from	C:\Tools\dnSpy\dnSpy.exe on 2026-08-02 — control tree in capture/gui/dnSpy/dnSpy.tree.txt

[← Capability index](#) · [Kit tool list](#)

Purpose

Decompile, browse and debug .NET assemblies. It reconstructs readable C# from IL, so a managed sample is usually faster to understand here than in a disassembler, and it can attach a debugger to the running assembly when static reading stalls.

When you'd reach for this

Reach for dnSpy when the sample is **managed .NET** — a PE carrying a CLR header, which `die` or CFF Explorer reports in seconds. It decompiles IL back to near-original C#, so a .NET dropper often reads almost like the developer's source rather than like disassembly.

What makes it beat a static decompiler on live malware is the built-in debugger. You can attach to a running process, set breakpoints and step through code that only exists after unpacking, and you can edit an assembly and save it back — which is how an anti-debug check or a licence test gets neutered so the next stage will run.

It is the wrong tool for unmanaged native code. No CLR header means nothing here applies, and you want a native disassembler instead.

Sources: <https://dnspy.org/> · <https://www.cybereason.com/blog/research/.net-malware-dropper>

Controls

The parts of this window you will actually touch, read from the application's own accessibility tree rather than from a screenshot. The full node list is in [capture/gui/dnSpy/dnSpy.tree.txt](#).

The window exposes 146 further named controls: **File, Export to Project..., Export to Project..., Save..., _Save..., Save Module..., Save _Module..., Save All..., Save A_ll..., Open..., _Open..., Open from GAC..., Open from _GAC..., Open List..., Open L_ist..., Recent Files, Recent _Files, Reload All Assemblies, _Reload All Assemblies, Close All, Close Old In-Memory Modules, Close All Framework Assemblies, Close All Missing Files, Sort Assemblies, Sor_t Assemblies, Restart as Administrator, Exit, E_xit, Edit, Undo, Redo, Find, _Find, Search Assemblies, _Search Assemblies, Create Assembly..., Create NetModule..., View, Word Wrap, _Word Wrap**, and 106 more. The full tree, with every automation id, is in [the capture](#).

Using it

1. **Open** the .NET assembly.
2. Read the decompiled C# rather than the IL first — .NET decompiles cleanly enough that the source is usually the fastest route.
3. Use **Search Assemblies** for the strings and API names that matter, instead of browsing the namespace tree.
4. Set a breakpoint and start debugging when static reading stalls — this is a debugger as well as a decompiler, which is the reason to choose it.
5. **Export to Project...** when the assembly is worth reading as a whole in an editor.

Gotchas

- Obfuscated assemblies decompile into something that looks like code and is not. Names are meaningless after obfuscation; the control flow may be too. De-obfuscate first with `de4dot`.
- Debugging runs the sample. Do it only on the isolated VM, with no network path out.
- It edits and recompiles assemblies. That is useful and it is also how evidence gets modified by accident — work on a copy.

frida

Kit	REMnux
Capability	Emulate or instrument execution
Version	17.16.3
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://frida.re

← [Capability index](#) · [Kit tool list](#)

Purpose

Trace the execution of a process to analyze its behavior.

When you'd reach for this

An analyst reaches for Frida when dynamically modifying a running mobile application's behavior, such as bypassing SSL pinning or decrypting obfuscated data, often running commands like `frida -U -f` or `frida -U -p` before injecting scripts; they choose it over similar tools because it allows real-time interaction and modification of processes without requiring source code access.

Sources: <https://www.vaadata.com/en/blog/frida-the-tool-dedicated-to-mobile-application-security/>

Synopsis

```
frida [options] target
```

Options

All 66 options parsed from the captured help text; 6 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	show this help message and exit	
<code>--help</code>	—	show this help message and exit	
<code>-D</code>	ID	connect to device with the given ID	
<code>--device</code>	ID	connect to device with the given ID	

Flag	Argument	What it does	When you would use it
<code>-U</code>	—	connect to USB device	When an analyst needs to attach Frida to a target application running on a connected Android device via USB to intercept logs, decrypt data, or hook methods during dynamic instrumentation.
<code>--usb</code>	—	connect to USB device	When an analyst needs to attach Frida to a target application running on a connected Android device via USB to intercept logs, decrypt data, or hook methods during dynamic instrumentation.
<code>-R</code>	—	connect to remote frida-server	
<code>--remote</code>	—	connect to remote frida-server	
<code>-H</code>	HOST	connect to remote frida-server on HOST	
<code>--host</code>	HOST	connect to remote frida-server on HOST	
<code>--certificate</code>	CERTIFICATE	speak TLS with HOST, expecting CERTIFICATE	
<code>--origin</code>	ORIGIN	connect to remote server with "Origin" header set to ORIGIN	
<code>--token</code>	TOKEN	authenticate with HOST using TOKEN	
<code>--keepalive-interval</code>	INTERVAL	set keepalive interval in seconds, or 0 to disable (defaults to -1 to auto-select based on transport)	
<code>--device-option</code>	option	override a backend-specific option, such as "control-endpoint=(string)localabstract:/my-frida-server" (supported types are: string, bool, int)	
<code>--p2p</code>	—	establish a peer-to-peer connection with target	
<code>--stun-server</code>	ADDRESS	set STUN server ADDRESS to use with --p2p	

Flag	Argument	What it does	When you would use it
<code>-f</code>	TARGET	spawn FILE	An analyst would use the <code>-f</code> flag when needing to inject a script at the start of a target process to bypass security mechanisms like SSL pinning or anti-root protection as the application launches.
<code>--file</code>	TARGET	spawn FILE	When an analyst needs to specify the executable file for Frida to monitor and unpack during runtime analysis.
<code>-F</code>	—	attach to frontmost application	
<code>--attach-frontmost</code>	—	attach to frontmost application	
<code>-n</code>	NAME	attach to NAME	
<code>--attach-name</code>	NAME	attach to NAME	
<code>-N</code>	IDENTIFIER	attach to IDENTIFIER	When an analyst needs to inject a script into a specific Android application to modify its behavior, such as bypassing validation or decrypting a flag, they would use the <code>-N</code> flag to target the application by name.
<code>--attach-identifier</code>	IDENTIFIER	attach to IDENTIFIER	When an analyst needs to inject a script into a specific Android application to modify its behavior, such as bypassing validation or decrypting a flag, they would use the <code>-N</code> flag to target the application by name.
<code>-p</code>	PID	attach to PID	When an analyst needs to inject a JS script into an already running target process on a USB-connected device by specifying its process ID (PID) instead of launching the application anew.
<code>--attach-pid</code>	PID	attach to PID	When an analyst needs to inject a JS script into an already running target process on a USB-connected device by specifying its process ID (PID) instead of launching the application anew.
<code>-W</code>	PATTERN	await spawn matching PATTERN	

Flag	Argument	What it does	When you would use it
<code>--await</code>	PATTERN	await spawn matching PATTERN	
<code>--stdio</code>	inherit,pipe	stdio behavior when spawning (defaults to "inherit")	
<code>--aux</code>	option	set aux option when spawning, such as "uid=(int)42" (supported types are: string, bool, int)	
<code>--realm</code>	native,emulated	realm to attach in	
<code>--exceptor</code>	full,handler-only,off	configure the exception handling mode	
<code>--disable-unwind-broker</code>	—	disable the unwind broker	
<code>--disable-exit-monitor</code>	—	disable the exit monitor	
<code>--disable-thread-suspend-monitor</code>	—	disable the thread suspend monitor	
<code>--linker-notifier-offset</code>	OFFSET	add a linker notifier OFFSET (may be specified multiple times)	
<code>--runtime</code>	qjs,v8	script runtime to use	
<code>--debug</code>	—	enable the Node.js compatible script debugger	
<code>--squelch-crash</code>	—	if enabled, will not dump crash report to console	
<code>-O</code>	FILE	text file containing additional command line options	
<code>--options-file</code>	FILE	text file containing additional command line options	
<code>--version</code>	—	show program's version number and exit	
<code>-l</code>	SCRIPT	load SCRIPT	An analyst would use the -l flag when injecting a JavaScript script to modify an application's behavior, such as decrypting obfuscated strings or bypassing security mechanisms like SSL pinning.

Flag	Argument	What it does	When you would use it
<code>--load</code>	SCRIPT	load SCRIPT	An analyst would use the -l flag when injecting a JavaScript script to modify an application's behavior, such as decrypting obfuscated strings or bypassing security mechanisms like SSL pinning.
<code>-P</code>	PARAMETERS_JSON	parameters as JSON, same as Gadget	
<code>--parameters</code>	PARAMETERS_JSON	parameters as JSON, same as Gadget	
<code>-C</code>	USER_CMODULE	load CMODULE	
<code>--cmodule</code>	USER_CMODULE	load CMODULE	
<code>--toolchain</code>	any,internal,external	CModule toolchain to use when compiling from source code	
<code>-c</code>	CODESHARE_URI	load CODESHARE_URI	
<code>--codeshare</code>	CODESHARE_URI	load CODESHARE_URI	
<code>-e</code>	CODE	evaluate CODE	
<code>--eval</code>	CODE	evaluate CODE	
<code>-q</code>	—	quiet mode (no prompt) and quit after -l and -e	
<code>-t</code>	TIMEOUT	seconds to wait before terminating in quiet mode (or 'inf' to run forever)	
<code>--timeout</code>	TIMEOUT	seconds to wait before terminating in quiet mode (or 'inf' to run forever)	
<code>--pause</code>	—	leave main thread paused after spawning program	
<code>-o</code>	LOGFILE	output to log file	
<code>--output</code>	LOGFILE	output to log file	
<code>--eternalize</code>	—	eternalize the script before exit	
<code>--exit-on-error</code>	—	exit with code 1 after encountering any exception in the SCRIPT	
<code>--kill-on-exit</code>	—	kill the spawned program when Frida exits	
<code>--auto-perform</code>	—	wrap entered code with Java.perform	

Flag	Argument	What it does	When you would use it
<code>--auto-reload</code>	—	Enable auto reload of provided scripts and c module (on by default, will be required in the future)	
<code>--no-auto-reload</code>	—	Disable auto reload of provided scripts and c module	

See also

`frida-trace`, `frida-ps`, `frida-discover`, `frida-kill`, `frida-ls-devices`

frida-discover

Kit	REMnux
Capability	Emulate or instrument execution
Version	17.16.3
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://frida.re

← [Capability index](#) · [Kit tool list](#)

Purpose

Trace the execution of a process to analyze its behavior.

Synopsis

```
frida-discover [options] target
```

Options

All 43 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	show this help message and exit	
<code>--help</code>	—	show this help message and exit	
<code>-D</code>	ID	connect to device with the given ID	
<code>--device</code>	ID	connect to device with the given ID	
<code>-U</code>	—	connect to USB device	
<code>--usb</code>	—	connect to USB device	
<code>-R</code>	—	connect to remote frida-server	
<code>--remote</code>	—	connect to remote frida-server	
<code>-H</code>	HOST	connect to remote frida-server on HOST	
<code>--host</code>	HOST	connect to remote frida-server on HOST	

Flag	Argument	What it does	When you would use it
<code>--certificate</code>	CERTIFICATE	speak TLS with HOST, expecting CERTIFICATE	
<code>--origin</code>	ORIGIN	connect to remote server with "Origin" header set to ORIGIN	
<code>--token</code>	TOKEN	authenticate with HOST using TOKEN	
<code>--keepalive-interval</code>	INTERVAL	set keepalive interval in seconds, or 0 to disable (defaults to -1 to auto-select based on transport)	
<code>--device-option</code>	option	override a backend-specific option, such as "control-endpoint=(string)localabstract:/my-frida-server" (supported types are: string, bool, int)	
<code>--p2p</code>	—	establish a peer-to-peer connection with target	
<code>--stun-server</code>	ADDRESS	set STUN server ADDRESS to use with --p2p	
<code>-f</code>	TARGET	spawn FILE	
<code>--file</code>	TARGET	spawn FILE	
<code>-F</code>	—	attach to frontmost application	
<code>--attach-frontmost</code>	—	attach to frontmost application	
<code>-n</code>	NAME	attach to NAME	
<code>--attach-name</code>	NAME	attach to NAME	
<code>-N</code>	IDENTIFIER	attach to IDENTIFIER	
<code>--attach-identifier</code>	IDENTIFIER	attach to IDENTIFIER	
<code>-p</code>	PID	attach to PID	
<code>--attach-pid</code>	PID	attach to PID	
<code>-W</code>	PATTERN	await spawn matching PATTERN	
<code>--await</code>	PATTERN	await spawn matching PATTERN	
<code>--stdio</code>	inherit,pipe	stdio behavior when spawning (defaults to "inherit")	
<code>--aux</code>	option	set aux option when spawning, such as "uid=(int)42" (supported types are: string, bool, int)	
<code>--realm</code>	native,emulated	realm to attach in	

Flag	Argument	What it does	When you would use it
<code>--exceptor</code>	full,handler-only,off	configure the exception handling mode	
<code>--disable-unwind-broker</code>	—	disable the unwind broker	
<code>--disable-exit-monitor</code>	—	disable the exit monitor	
<code>--disable-thread-suspend-monitor</code>	—	disable the thread suspend monitor	
<code>--linker-notifier-offset</code>	OFFSET	add a linker notifier OFFSET (may be specified multiple times)	
<code>--runtime</code>	qjs,v8	script runtime to use	
<code>--debug</code>	—	enable the Node.js compatible script debugger	
<code>--squelch-crash</code>	—	if enabled, will not dump crash report to console	
<code>-O</code>	FILE	text file containing additional command line options	
<code>--options-file</code>	FILE	text file containing additional command line options	
<code>--version</code>	—	show program's version number and exit	

See also

`frida`, `frida-trace`, `frida-ps`, `frida-kill`, `frida-ls-devices`

frida-kill

Kit	REMnux
Capability	Emulate or instrument execution
Version	17.16.3
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://frida.re

← [Capability index](#) · [Kit tool list](#)

Purpose

Trace the execution of a process to analyze its behavior.

Synopsis

```
frida-kill [options] process
```

Options

All 20 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	show this help message and exit	
<code>--help</code>	—	show this help message and exit	
<code>-D</code>	ID	connect to device with the given ID	
<code>--device</code>	ID	connect to device with the given ID	
<code>-U</code>	—	connect to USB device	
<code>--usb</code>	—	connect to USB device	
<code>-R</code>	—	connect to remote frida-server	
<code>--remote</code>	—	connect to remote frida-server	
<code>-H</code>	HOST	connect to remote frida-server on HOST	
<code>--host</code>	HOST	connect to remote frida-server on HOST	

Flag	Argument	What it does	When you would use it
<code>--certificate</code>	CERTIFICATE	speak TLS with HOST, expecting CERTIFICATE	
<code>--origin</code>	ORIGIN	connect to remote server with "Origin" header set to ORIGIN	
<code>--token</code>	TOKEN	authenticate with HOST using TOKEN	
<code>--keepalive-interval</code>	INTERVAL	set keepalive interval in seconds, or 0 to disable (defaults to -1 to auto-select based on transport)	
<code>--device-option</code>	option	override a backend-specific option, such as "control-endpoint=(string)localabstract/my-frida-server" (supported types are: string, bool, int)	
<code>--p2p</code>	—	establish a peer-to-peer connection with target	
<code>--stun-server</code>	ADDRESS	set STUN server ADDRESS to use with --p2p	
<code>-O</code>	FILE	text file containing additional command line options	
<code>--options-file</code>	FILE	text file containing additional command line options	
<code>--version</code>	—	show program's version number and exit	

See also

`frida`, `frida-trace`, `frida-ps`, `frida-discover`, `frida-ls-devices`

frida-ls-devices

Kit	REMnux
Capability	Emulate or instrument execution
Version	17.16.3
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://frida.re

[← Capability index](#) · [Kit tool list](#)

Purpose

Trace the execution of a process to analyze its behavior.

Synopsis

```
frida-ls-devices [options]
```

Options

All 5 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	show this help message and exit	
<code>--help</code>	—	show this help message and exit	
<code>-O</code>	FILE	text file containing additional command line options	
<code>--options-file</code>	FILE	text file containing additional command line options	
<code>--version</code>	—	show program's version number and exit	

See also

`frida`, `frida-trace`, `frida-ps`, `frida-discover`, `frida-kill`

frida-ps

Kit	REMnux
Capability	Emulate or instrument execution
Version	17.16.3
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://frida.re

[← Capability index](#) · [Kit tool list](#)

Purpose

Trace the execution of a process to analyze its behavior.

When you'd reach for this

An analyst reaches for frida-ps when they need to list processes on a remote device, such as after connecting via USB or identifying a specific device ID using frida-ls-devices, to inspect running or installed applications. They may run it before attaching to a target process for further analysis or scripting. They choose it over similar tools because it is explicitly designed for listing processes, a foundational step when interacting with remote systems, as highlighted in the documentation.

Sources: <https://frida.re/docs/frida-ps/> · <https://www.vaadata.com/en/blog/frida-the-tool-dedicated-to-mobile-application-security/>

Synopsis

```
frida-ps [options]
```

Options

All 28 options parsed from the captured help text; 2 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	show this help message and exit	
<code>--help</code>	—	show this help message and exit	

Flag	Argument	What it does	When you would use it
<code>-D</code>	ID	connect to device with the given ID	An analyst would use the <code>-D</code> flag when they need to list processes on a specific device by its ID, such as when targeting a particular connected device during a mobile pentest.
<code>--device</code>	ID	connect to device with the given ID	An analyst would use the <code>-D</code> flag when they need to list processes on a specific device by its ID, such as when targeting a particular connected device during a mobile pentest.
<code>-U</code>	—	connect to USB device	An analyst would use the <code>-U</code> flag when connecting to a device via USB to list its running processes or installed applications during a mobile forensic investigation.
<code>--usb</code>	—	connect to USB device	An analyst would use the <code>-U</code> flag when connecting to a device via USB to list its running processes or installed applications during a mobile forensic investigation.
<code>-R</code>	—	connect to remote frida-server	
<code>--remote</code>	—	connect to remote frida-server	
<code>-H</code>	HOST	connect to remote frida-server on HOST	
<code>--host</code>	HOST	connect to remote frida-server on HOST	
<code>--certificate</code>	CERTIFICATE	speak TLS with HOST, expecting CERTIFICATE	
<code>--origin</code>	ORIGIN	connect to remote server with "Origin" header set to ORIGIN	
<code>--token</code>	TOKEN	authenticate with HOST using TOKEN	
<code>--keepalive-interval</code>	INTERVAL	set keepalive interval in seconds, or 0 to disable (defaults to -1 to auto-select based on transport)	

Flag	Argument	What it does	When you would use it
<code>--device-option</code>	option	override a backend-specific option, such as "control-endpoint=(string)localabstract:/my-frida-server" (supported types are: string, bool, int)	
<code>--p2p</code>	—	establish a peer-to-peer connection with target	
<code>--stun-server</code>	ADDRESS	set STUN server ADDRESS to use with --p2p	
<code>-O</code>	FILE	text file containing additional command line options	
<code>--options-file</code>	FILE	text file containing additional command line options	
<code>--version</code>	—	show program's version number and exit	
<code>-a</code>	—	list only applications	
<code>--applications</code>	—	list only applications	
<code>-i</code>	—	include all installed applications	
<code>--installed</code>	—	include all installed applications	
<code>-j</code>	—	output results as JSON	
<code>--json</code>	—	output results as JSON	
<code>-e</code>	—	exclude icons in output	
<code>--exclude-icons</code>	—	exclude icons in output	

See also

[frida](#), [frida-trace](#), [frida-discover](#), [frida-kill](#), [frida-ls-devices](#)

frida-trace

Kit	REMnux
Capability	Emulate or instrument execution
Version	17.16.3
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://frida.re

← [Capability index](#) · [Kit tool list](#)

Purpose

Trace the execution of a process to analyze its behavior.

When you'd reach for this

An analyst reaches for frida-trace when tracing and modifying application behavior dynamically, such as during reverse engineering or security testing, often after copying frida-server to a remote device and before interacting with the target app's methods. They may choose it over similar tools because it allows real-time modification of method outputs and provides detailed tracing capabilities, as demonstrated by altering return values or inspecting method parameters during execution.

Sources: <https://frida.re/docs/frida-trace/> · <https://www.vaadata.com/en/blog/frida-the-tool-dedicated-to-mobile-application-security/>

Synopsis

```
frida-trace [options] target
```

Options

All 82 options parsed from the captured help text; 10 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	show this help message and exit	
<code>--help</code>	—	show this help message and exit	
<code>-D</code>	ID	connect to device with the given ID	

Flag	Argument	What it does	When you would use it
<code>--device</code>	ID	connect to device with the given ID	
<code>-U</code>	—	connect to USB device	An analyst would use the -U flag when tracing an application running on a remote Android device connected via USB from their host machine.
<code>--usb</code>	—	connect to USB device	An analyst would use the -U flag when tracing an application running on a remote Android device connected via USB from their host machine.
<code>-R</code>	—	connect to remote frida-server	
<code>--remote</code>	—	connect to remote frida-server	
<code>-H</code>	HOST	connect to remote frida-server on HOST	
<code>--host</code>	HOST	connect to remote frida-server on HOST	
<code>--certificate</code>	CERTIFICATE	speak TLS with HOST, expecting CERTIFICATE	
<code>--origin</code>	ORIGIN	connect to remote server with "Origin" header set to ORIGIN	
<code>--token</code>	TOKEN	authenticate with HOST using TOKEN	
<code>--keepalive-interval</code>	INTERVAL	set keepalive interval in seconds, or 0 to disable (defaults to -1 to auto-select based on transport)	
<code>--device-option</code>	option	override a backend-specific option, such as "control-endpoint=(string)localabstract:/my-frida-server" (supported types are: string, bool, int)	
<code>--p2p</code>	—	establish a peer-to-peer connection with target	
<code>--stun-server</code>	ADDRESS	set STUN server ADDRESS to use with --p2p	

Flag	Argument	What it does	When you would use it
<code>-f</code>	TARGET	spawn FILE	An analyst would use the <code>-f</code> flag when launching a specific application on a mobile device to trace its API calls, such as monitoring crypto functions in Snapchat or Java methods in YouTube.
<code>--file</code>	TARGET	spawn FILE	An analyst would use the <code>-f</code> flag when launching a specific application on a mobile device to trace its API calls, such as monitoring crypto functions in Snapchat or Java methods in YouTube.
<code>-F</code>	—	attach to frontmost application	
<code>--attach-frontmost</code>	—	attach to frontmost application	
<code>-n</code>	NAME	attach to NAME	
<code>--attach-name</code>	NAME	attach to NAME	
<code>-N</code>	IDENTIFIER	attach to IDENTIFIER	When the target application is already running and the analyst needs to trace functions using its identifier.
<code>--attach-identifier</code>	IDENTIFIER	attach to IDENTIFIER	When the target application is already running and the analyst needs to trace functions using its identifier.
<code>-p</code>	PID	attach to PID	An analyst would use the <code>-p</code> flag when tracing functions in a specific process by its process ID, such as monitoring a Windows application's memory-related calls in <code>msvcrt.dll</code> .
<code>--attach-pid</code>	PID	attach to PID	An analyst would use the <code>-p</code> flag when tracing functions in a specific process by its process ID, such as monitoring a Windows application's memory-related calls in <code>msvcrt.dll</code> .
<code>-W</code>	PATTERN	await spawn matching PATTERN	
<code>--await</code>	PATTERN	await spawn matching PATTERN	
<code>--stdio</code>	inherit,pipe	stdio behavior when spawning (defaults to "inherit")	

Flag	Argument	What it does	When you would use it
<code>--aux</code>	option	set aux option when spawning, such as "uid=(int)42" (supported types are: string, bool, int)	
<code>--realm</code>	native,emulated	realm to attach in	
<code>--exceptor</code>	full,handler-only,off	configure the exception handling mode	
<code>--disable-unwind-broker</code>	—	disable the unwind broker	
<code>--disable-exit-monitor</code>	—	disable the exit monitor	
<code>--disable-thread-suspend-monitor</code>	—	disable the thread suspend monitor	
<code>--linker-notifier-offset</code>	OFFSET	add a linker notifier OFFSET (may be specified multiple times)	
<code>--runtime</code>	qjs,v8	script runtime to use	
<code>--debug</code>	—	enable the Node.js compatible script debugger	
<code>--squelch-crash</code>	—	if enabled, will not dump crash report to console	
<code>-O</code>	FILE	text file containing additional command line options	An analyst would use the -O flag when dealing with a large number of command line options that exceed the operating system's maximum command line length, allowing them to pass options via text files.
<code>--options-file</code>	FILE	text file containing additional command line options	An analyst would use the -O flag when dealing with a large number of command line options that exceed the operating system's maximum command line length, allowing them to pass options via text files.
<code>--version</code>	—	show program's version number and exit	

Flag	Argument	What it does	When you would use it
<code>-I</code>	MODULE	include MODULE	An analyst would use the -I flag when they need to trace all functions within a specific module, such as to broadly monitor activity in a particular library without specifying individual functions.
<code>--include-module</code>	MODULE	include MODULE	An analyst would use the -I flag when they need to trace all functions within a specific module, such as to broadly monitor activity in a particular library without specifying individual functions.
<code>-X</code>	MODULE	exclude MODULE	
<code>--exclude-module</code>	MODULE	exclude MODULE	
<code>-i</code>	FUNCTION	include [MODULE!]FUNCTION	An analyst would use the -i flag when they need to trace specific functions or modules, such as monitoring particular API calls or methods in a target process.
<code>--include</code>	FUNCTION	include [MODULE!]FUNCTION	An analyst would use the -i flag when they need to trace specific functions or modules, such as monitoring particular API calls or methods in a target process.
<code>-x</code>	FUNCTION	exclude [MODULE!]FUNCTION	An analyst would use the -x flag when they need to exclude specific functions from being traced after including an entire module or a set of functions matching a pattern.
<code>--exclude</code>	FUNCTION	exclude [MODULE!]FUNCTION	An analyst would use the -x flag when they need to exclude specific functions from being traced after including an entire module or a set of functions matching a pattern.
<code>-T</code>	INCLUDE_IMPORTS	include program's imports	
<code>--include-imports</code>	INCLUDE_IMPORTS	include program's imports	
<code>-t</code>	MODULE	include MODULE imports	
<code>--include-module-imports</code>	MODULE	include MODULE imports	
<code>-m</code>	OBJC_METHOD	include OBJC_METHOD	

Flag	Argument	What it does	When you would use it
<code>--include-objc-method</code>	OBJC_METHOD	include OBJC_METHOD	
<code>-M</code>	OBJC_METHOD	exclude OBJC_METHOD	
<code>--exclude-objc-method</code>	OBJC_METHOD	exclude OBJC_METHOD	
<code>-y</code>	SWIFT_FUNC	include SWIFT_FUNC	
<code>--include-swift-func</code>	SWIFT_FUNC	include SWIFT_FUNC	
<code>-Y</code>	SWIFT_FUNC	exclude SWIFT_FUNC	
<code>--exclude-swift-func</code>	SWIFT_FUNC	exclude SWIFT_FUNC	
<code>-j</code>	JAVA_METHOD	include JAVA_METHOD	
<code>--include-java-method</code>	JAVA_METHOD	include JAVA_METHOD	
<code>-J</code>	JAVA_METHOD	exclude JAVA_METHOD	
<code>--exclude-java-method</code>	JAVA_METHOD	exclude JAVA_METHOD	
<code>-s</code>	DEBUG_SYMBOL	include DEBUG_SYMBOL	
<code>--include-debug-symbol</code>	DEBUG_SYMBOL	include DEBUG_SYMBOL	
<code>-q</code>	—	do not format output messages	
<code>--quiet</code>	—	do not format output messages	
<code>-d</code>	—	add module name to generated onEnter log statement	
<code>--decorate</code>	—	add module name to generated onEnter log statement	
<code>-S</code>	PATH	path to JavaScript file used to initialize the session	An analyst would use the -S flag when they need to initialize a frida-trace session by executing custom JavaScript code files to set up the environment, share functions, or add data to the global "state" object before tracing begins.
<code>--init-session</code>	PATH	path to JavaScript file used to initialize the session	An analyst would use the -S flag when they need to initialize a frida-trace session by executing custom JavaScript code files to set up the environment, share functions, or add data to the global "state" object before tracing begins.

Flag	Argument	What it does	When you would use it
<code>-P</code>	PARAMETERS_JSON	parameters as JSON, exposed as a global named 'parameters'	An analyst would use the <code>-P</code> flag when tracing multiple functions and needing to dynamically control handler behavior, such as conditionally printing the process ID based on a JSON parameter passed via the command line.
<code>--parameters</code>	PARAMETERS_JSON	parameters as JSON, exposed as a global named 'parameters'	An analyst would use the <code>-P</code> flag when tracing multiple functions and needing to dynamically control handler behavior, such as conditionally printing the process ID based on a JSON parameter passed via the command line.
<code>-o</code>	OUTPUT	dump messages to file	
<code>--output</code>	OUTPUT	dump messages to file	
<code>--ui-host</code>	UI_HOST	the host to serve the UI on (default localhost)	
<code>--ui-port</code>	UI_PORT	the TCP port to serve the UI on	
<code>--ui-allow-origin</code>	ORIGIN	allow browser requests from ORIGIN; may be specified multiple times	

See also

`frida`, `frida-ps`, `frida-discover`, `frida-kill`, `frida-ls-devices`

idr (GUI)

Capability	reverse engineering
Window title	Interactive Delphi Reconstructor by crypto
Captured from	C:\Tools\idr\idr.exe on 2026-08-02 — control tree in capture/gui/idr/idr.tree.txt

[← Capability index](#) · [Kit tool list](#)

Purpose

Reconstruct Delphi programs: identify the runtime library, recover form definitions and name the event handlers. Delphi binaries are mostly runtime code, so separating the author's few hundred lines from the library's tens of thousands is the difference between a tractable job and an intractable one.

When you'd reach for this

Reach for IDR when the sample is **Delphi**. Delphi binaries are large and mostly runtime library, so a general disassembler buries the author's few thousand lines inside hundreds of thousands of lines of framework code. IDR knows the runtime library and can tell the two apart, which is the difference between a tractable job and an intractable one.

The form viewer is the reason to open it rather than a generic tool. Delphi stores its visual forms inside the binary together with the event handlers wired to each control, so you can go from *the button labelled Install* straight to the routine that runs when it is clicked. On a Delphi dropper that is often the shortest path to the payload.

Sources: <https://gitbook.seguranca-informatica.pt/tools-1/decompilers>

Controls

The parts of this window you will actually touch, read from the application's own accessibility tree rather than from a screenshot. The full node list is in [capture/gui/idr/idr.tree.txt](#).

The window exposes 4 further named controls: **Units (F2)**, **ClassViewer (F7)**, **Branch**, **Tree**. The full tree, with every automation id, is in [the capture](#).

Using it

1. Open the Delphi executable.
2. Let it match the runtime library first; without that, most of the disassembly is Delphi's own code rather than the author's.
3. Read the reconstructed forms and event handlers, which is where Delphi programs keep their logic.

Gotchas

- Delphi binaries are mostly runtime. Library signature matching is what separates the few hundred lines that matter from the tens of thousands that do not.
- The knowledge base is version-specific. A mismatch against the Delphi version used to build the sample leaves the runtime unidentified and the output far less useful.

r2

Kit	REMnux · Kali Linux · SIFT Workstation
Capability	Disassemble and explore a binary
Version	6.1.9-124 r2
Captured from	<code>cyberlab-aio</code> via <code>-h</code> on 2026-08-11 — raw help output
Documentation	https://www.radare.org/n/radare2.html

← [Capability index](#) · [Kit tool list](#)

Purpose

Examine binary files, including disassembling and debugging. Includes r2ai and decai plugins for LLM-powered analysis (API key or local Ollama required), plus the r2ghidra plugin for Ghidra decompilation via the pdg command.

When you'd reach for this

An analyst reaches for r2 when examining stripped or complex binaries to analyze code structure, often running commands like `aa` or `aaa` to identify symbols, entry points, and function trees, as it provides detailed opcode-level insights and customizable analysis loops via its API or scripts, which may offer more precision than default tools.

Sources: https://book.rada.re/analysis/code_analysis.html · <https://r2wiki.readthedocs.io/en/latest/tools/radare2/>

Synopsis

```
r2 [-ACdfjLMnNqStuvwzX] [-P patch] [-p prj] [-a arch] [-b bits] [-c cmd]
[-s addr] [-B baddr] [-m maddr] [-i script] [-e k=v] file|pid|-|--|=
```

Common invocations

```
# Debug the ls program to analyze execution and behavior
r2 -d ls
# Load existing project file
r2 -p test
# Open raw binary for carving analysis
r2 -n dump.bin
# Carve binaries by resizing to original size
r2 -wnqc"r $sz" $a
# Analyze PowerPC 32-bit sub-binary within FAT file
r2 -a ppc -b 32 ls.fat
# Apply script patches to target binary
r2 -i patch.r2 target.bin
```

Options

All 31 options parsed from the captured help text; 3 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
-0	—	print \x00 after init and every command	
-1	—	redirect stderr to stdout	
-2	—	close stderr file descriptor (silent warning messages)	
-a	arch	set asm.arch	An analyst would use the -a flag when specifying the target architecture for analysis, such as when dealing with a fat binary requiring a particular architecture like ppc.
-A	—	run 'aaa' command to analyze all referenced code	
-b	bits	set asm.bits	An analyst would use the -b flag when specifying the bitness (e.g., 32 or 64) of the target binary's architecture during analysis, such as when opening a fat binary with a specific sub-architecture.
-B	baddr	set base address for PIE binaries	
-C	—	file is host:port (alias for -c+=http://%s/cmd/)	

Flag	Argument	What it does	When you would use it
<code>-d</code>	—	debug the executable 'file' or running process 'pid'	An analyst would use the <code>-d</code> flag when debugging a program, attaching to a running process by its PID, or configuring radare2 to handle stdin input during debugging sessions.
<code>-E</code>	—	open cfg.editor with the user rc file	
<code>-f</code>	—	block size = file size	
<code>-i</code>	file	run rlang program, r2script file or load plugin	
<code>-I</code>	file	run script file before the file is opened	
<code>-j</code>	—	use json for -v, -L and maybe others	
<code>-m</code>	addr	map file at given address (loadaddr)	
<code>-M</code>	—	do not demangle symbol names	
<code>-N</code>	—	do not load user settings and scripts	
<code>-q</code>	—	quiet mode (no prompt) and quit after -i	
<code>-Q</code>	—	quiet mode (no prompt) and quit faster (quickLeak=true)	
<code>-p</code>	prj	use project, list if no arg, load if no file	
<code>-P</code>	file	apply rapatch file and quit	
<code>-r</code>	rarun2	specify rarun2 profile to load (same as <code>-e dbg.profile=X</code>)	
<code>-s</code>	addr	initial seek	
<code>-S</code>	—	start r2 in sandbox mode	
<code>-t</code>	—	load rabin2 info in thread	
<code>-u</code>	—	set bin.filter=false to get raw sym/sec/cls names	
<code>-v</code>	—	show radare2 version (-V show lib versions)	
<code>-V</code>	—	show radare2 version (-V show lib versions)	
<code>-w</code>	—	open file in write mode	

Flag	Argument	What it does	When you would use it
<code>-x</code>	—	open without exec-flag (asm.emu will not work), See io.exec	
<code>-X</code>	—	same as -e bin.usextr=false (useful for dyldcache)	

See also

`rabin2`, `rasm2`, `objdump`, `vivbin`, `vdbbin`

rasm2

Kit	REMnux
Capability	Disassemble and explore a binary; Analyse shellcode
Version	rasm2 6.1.9
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://www.radare.org/n/radare2.html

← [Capability index](#) · [Kit tool list](#)

Purpose

Examine binary files, including disassembling and debugging. Includes r2ai and decai plugins for LLM-powered analysis (API key or local Ollama required), plus the r2ghidra plugin for Ghidra decompilation via the pdg command.

Synopsis

```
rasm2 [-ACdDehHLBvw] [-a arch] [-b bits] [-s addr] [-S syntax]
[-f file] [-o file] [-F fil:ter] [-i skip] [-l len] 'code'|hex|0101b|-
```

Common invocations

```
# Assemble x86 instruction into machine code bytes
rasm2 -a x86 -b 32 'mov eax, 33'
# Disassemble hex bytes into assembly instructions
rasm2 -d 90
# List available architecture plugins for disassembly
rasm2 -L
# Convert between assembly instructions and hex representations
rasm2 -a java 'nop'
# Disassemble hex bytes into assembly instructions
rasm2 -a x86 -b 32 -d '90'
```

Options

All 25 options parsed from the captured help text; 5 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
-a	arch	set architecture to assemble/disassemble (see -L)	An analyst would use the -a flag when disassembling code for a specific architecture, such as x86 or Java, to ensure the correct instruction set is used.
-A	—	show Analysis information from given hexpairs	
-b	bits	set cpu register size (8, 16, 32, 64) (RASM2_BITS)	An analyst would use the -b flag when specifying the bitness of the target architecture (e.g., 32 or 64) during disassembly to ensure accurate interpretation of machine code instructions.
-B	—	binary input/output (-I is mandatory for binary input)	
-c	cpu	select specific CPU (depends on arch)	
-C	—	output in C format	
-d	—	disassemble from hexpair bytes (-D show hexpairs)	An analyst would use the -d flag when converting hexadecimal opcodes into human-readable assembly instructions to analyze binary data.
-D	—	disassemble from hexpair bytes (-D show hexpairs)	An analyst would use the -D flag when needing to disassemble hexpair bytes while also viewing the corresponding offset and opcode bytes for detailed analysis.
-e	—	use big endian instead of little endian	
-E	—	display ESIL expression (same input as in -d)	
-f	file	read data from file	
-F	parser	specify which parse filter use (see -LL)	
-i	len	ignore/skip N bytes of the input buffer	An analyst would use the -i flag when they need to skip a specific number of bytes in the input buffer to bypass irrelevant data, such as file headers or padding, while disassembling a binary file.

Flag	Argument	What it does	When you would use it
<code>-j</code>	—	output in json format	
<code>-k</code>	kernel	select operating system (linux, windows, darwin, android, ios, ..)	
<code>-l</code>	len	input/Output length	
<code>-N</code>	—	same as r2 -N (or R2_NOPLUGINS) (not load any plugin)	
<code>-o</code>	file	output file name (rasm2 -Bf a.asm -o a)	
<code>-p</code>	—	run SPP over input for assembly	
<code>-q</code>	—	quiet mode	
<code>-r</code>	—	output in radare commands	
<code>-S</code>	syntax	select syntax (intel, att)	
<code>-v</code>	—	show version information	
<code>-x</code>	—	use hex dwords instead of hex pairs when assembling.	
<code>-w</code>	—	what's this instruction for? describe opcode	

See also

`r2`, `rabin2`, `objdump`, `vivbin`, `vdbbin`, `xortool`

VB- Decompiler (GUI)

Capability	reverse engineering
Window title	VB Decompiler Lite v26.4
Version	v26.4
Captured from	C:\Tools\VB Decompiler\VB Decompiler.exe on 2026-08-02 — control tree in capture/gui/VB- Decompiler/VB-Decompiler.tree.txt

[← Capability index](#) · [Kit tool list](#)

Purpose

Recover source-level structure from Visual Basic executables. P-code binaries decompile close to the original; native-compiled ones yield disassembly with the VB runtime calls identified.

When you'd reach for this

Reach for VB Decompiler when the sample is **Visual Basic 6** or **VB.NET / C#**. VB6 is the case that catches people out, because it compiles two different ways and what comes back depends entirely on which: P-code recovers a large share of the original logic, while native-compiled VB6 leaves you much closer to ordinary disassembly. Establish which you have before judging the tool.

Expect a partial reconstruction rather than compilable source. The vendor quotes roughly 85% logic restoration for P-code and around 95% for .NET. That is enough to read intent, which is usually the question, and not enough to rebuild the program.

For .NET specifically, dnSpy is the better first stop because of its debugger. VB Decompiler earns its place on the VB6 samples dnSpy cannot open at all.

Sources: <https://www.vb-decompiler.org/products.htm> · <https://www.vb-decompiler.org/faq.htm>

Controls

The parts of this window you will actually touch, read from the application's own accessibility tree rather than from a screenshot. The full node list is in [capture/gui/VB-Decompiler/VB-Decompiler.tree.txt](#).

The window exposes 3 further named controls: **Decompile**, **...**, **Decompiler**. The full tree, with every automation id, is in [the capture](#).

Using it

1. Open the Visual Basic executable.
2. Check whether it is p-code or native — the tool tells you, and it decides how much you get back.

3. Read the form definitions: VB malware often carries its logic in event handlers rather than in a main routine.

Gotchas

- P-code decompiles close to source. Native-compiled VB gives you disassembly with VB runtime calls, which is a different and much slower job.
- The Lite edition omits much of the analysis. Check which build you are on before concluding a feature is missing.

vbdec (GUI)

Capability	reverse engineering
Window title	vbdec
Captured from	C:\Tools\vbdec\vbdec.exe on 2026-08-02 — control tree in capture/gui/vbdec/vbdec.tree.txt

[← Capability index](#) · [Kit tool list](#)

Purpose

Decompile VB5 and VB6 binaries, recovering forms, controls and event handlers. Those names usually survive compilation and describe what the program was written to do.

When you'd reach for this

The command-line companion for the same Visual Basic targets as VB Decompiler. Reach for it when you are processing a set of samples rather than opening one, or scripting extraction into a pipeline; use the window when you are reading a single sample and want to navigate it.

The format caveat is identical: VB6 P-code reconstructs well, native-compiled VB6 much less so, and knowing which you have is the first question rather than an afterthought.

Sources: <https://www.vb-decompiler.org/products.htm>

Controls

The parts of this window you will actually touch, read from the application's own accessibility tree rather than from a screenshot. The full node list is in [capture/gui/vbdec/vbdec.tree.txt](#).

The window exposes 1 further named controls: **Search Log**. The full tree, with every automation id, is in [the capture](#).

Using it

1. Open the VB5/VB6 binary.
2. Work from the form and control names, which usually survive and describe the program's intent.

Gotchas

- Scope is VB5 and VB6 only. VB.NET is a .NET assembly — use [dnSpy](#) for those.

vdbbin

Kit	REMnux
Capability	Disassemble and explore a binary
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://github.com/vivisect/vivisect

[← Capability index](#) · [Kit tool list](#)

Purpose

Statically examine and emulate binary files.

Synopsis

```
vdbbin [options] [platformopt=foo, ...]
```

Options

All 27 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	show this help message and exit	
<code>--help</code>	—	show this help message and exit	
<code>-c</code>	COMMAND	Debug a fired command	
<code>--cmd</code>	COMMAND	Debug a fired command	
<code>-p</code>	PROCESS	Attach to process by name or pid	
<code>--process</code>	PROCESS	Attach to process by name or pid	
<code>-Q</code>	—	Run the QT gui	
<code>--qt</code>	—	Run the QT gui	
<code>-R</code>	REMOTEHOST	Attach to remote VDB server	
<code>--remote</code>	REMOTEHOST	Attach to remote VDB server	
<code>-r</code>	—	Do not stop on attach	
<code>--run</code>	—	Do not stop on attach	

Flag	Argument	What it does	When you would use it
<code>-s</code>	SNAPSHOT	Load a vtrace snapshot file	
<code>--snapshot</code>	SNAPSHOT	Load a vtrace snapshot file	
<code>-S</code>	—	—	
<code>--server</code>	—	—	
<code>-v</code>	—	—	
<code>--verbose</code>	—	—	
<code>-t</code>	TARGET	Activate special vdb target (-t ? for list)	
<code>--target</code>	TARGET	Activate special vdb target (-t ? for list)	
<code>--android</code>	—	Debug Android with ADB!	
<code>-e</code>	EVENTID	Used for Windows JIT	
<code>--eventid</code>	EVENTID	Used for Windows JIT	
<code>-w</code>	WAITFOR	Wait for process with name	
<code>--waitfor</code>	WAITFOR	Wait for process with name	
<code>--LI</code>	—	Breakpoint on Library initialization	
<code>--LL</code>	—	Breakpoint on Library load time	

See also

[r2](#), [rabin2](#), [rasm2](#), [objdump](#), [vivbin](#)

vivbin

Kit	REMnux
Capability	Disassemble and explore a binary
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://github.com/vivisect/vivisect

← [Capability index](#) · [Kit tool list](#)

Purpose

Statically examine and emulate binary files.

Synopsis

```
vivbin [options] <workspace|binaries...>
```

Options

All 34 options parsed from the captured help text; 1 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	show this help message and exit	
<code>--help</code>	—	show this help message and exit	
<code>-M</code>	MODNAME	run the file listed as an analysis module in non-gui mode and exit	An analyst would use the -M flag when running a command-line analysis module that modifies the VivWorkspace and requires saving changes to the workspace.
<code>--module</code>	MODNAME	run the file listed as an analysis module in non-gui mode and exit	An analyst would use the -M flag when running a command-line analysis module that modifies the VivWorkspace and requires saving changes to the workspace.
<code>-A</code>	—	Do <i>not</i> do an initial auto-analysis pass	

Flag	Argument	What it does	When you would use it
<code>--skip-analysis</code>	—	Do <i>not</i> do an initial auto-analysis pass	
<code>-B</code>	—	Do <i>not</i> start the gui, just load, analyze and save	
<code>--bulk</code>	—	Do <i>not</i> start the gui, just load, analyze and save	
<code>-C</code>	—	Output vivisect performance profiling (cProfile) info	
<code>--cprofile</code>	—	Output vivisect performance profiling (cProfile) info	
<code>-E</code>	ENTRYPOINTS	Add Entry Point for bulk analysis (can have multiple "-E" args)	
<code>--entrypoint</code>	ENTRYPOINTS	Add Entry Point for bulk analysis (can have multiple "-E" args)	
<code>-O</code>	OPTION	. = (optval must be json syntax)	
<code>--option</code>	OPTION	. = (optval must be json syntax)	
<code>-p</code>	PARSEMOD	Manually specify the parser module (pe/elf/blob/...)	
<code>--parser</code>	PARSEMOD	Manually specify the parser module (pe/elf/blob/...)	
<code>-s</code>	STORAGE_NAME	Specify a storage module by name	
<code>--storage</code>	STORAGE_NAME	Specify a storage module by name	
<code>-S</code>	—	Run Vivisect Server. Last argument should be the VivWorkspaces directory.	
<code>--server</code>	—	Run Vivisect Server. Last argument should be the VivWorkspaces directory.	
<code>-P</code>	SERVER_PORT	Port to run Vivisect Server on (only meaningful with --server)	
<code>--port</code>	SERVER_PORT	Port to run Vivisect Server on (only meaningful with --server)	
<code>-v</code>	—	Enable verbose mode (multiples matter: -vvvv)	
<code>--verbose</code>	—	Enable verbose mode (multiples matter: -vvvv)	
<code>-V</code>	VERSION	Add file version (if available) to save file name	

Flag	Argument	What it does	When you would use it
<code>--version</code>	VERSION	Add file version (if available) to save file name	
<code>-c</code>	CONFIG	Path to a directory to use for config data	
<code>--config</code>	CONFIG	Path to a directory to use for config data	
<code>-a</code>	—	Autosave configuration data	
<code>--autosave</code>	—	Autosave configuration data	
<code>-o</code>	OUTFILE	Name of VivWorkspace file to create (useful for loading multiple binaries into one workspace)	
<code>--outfile</code>	OUTFILE	Name of VivWorkspace file to create (useful for loading multiple binaries into one workspace)	
<code>-m</code>	—	List Architectures and their version/maturity	
<code>--archmaturity</code>	—	List Architectures and their version/maturity	

See also

`r2`, `rabin2`, `rasm2`, `objdump`, `vdbbin`

xortool

Kit	REMnux
Capability	Analyse shellcode; Break simple obfuscation
Version	1.1.0
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output
Documentation	https://github.com/hellman/xortool

← [Capability index](#) · [Kit tool list](#)

Purpose

Recover the key length and key of an XOR-encrypted file by frequency analysis.

When you'd reach for this

An analyst reaches for xortool when dealing with XOR-encrypted data, particularly when the key length is unknown or longer than default limits, and runs it after initial attempts to guess the key fail, using flags like `-m`, `-l`, or `-c` to refine results; they choose it over similar tools because it automates key-length analysis, filters plaintexts by character sets (e.g., Base64), and handles multi-byte keys with adjustable parameters.

Sources: <https://github.com/hellman/xortool> · <https://github.com/hellman/xortool/blob/master/README.md> · <https://www.doyler.net/security-not-included/basic-xortool-usage>

Synopsis

```
xortool [-x] [-m MAX-LEN] [-f] [-t CHARSET] [FILE]
xortool [-x] [-l LEN] [-c CHAR | -b | -o] [-f] [-t CHARSET] [-p PLAIN] [-r PERCENT] [FILE]
xortool [-x] [-m MAX-LEN | -l LEN] [-c CHAR | -b | -o] [-f] [-t CHARSET] [-p PLAIN] [-r PERCENT]
[FILE]
xortool [-h | --help]
xortool --version
```

Common invocations

```
# Analyze XOR-encrypted files to find keys and determine key lengths
xortool file.bin
# Decrypt XOR-encrypted hex file using space character set analysis
xortool -x -c ' ' file.hex
# Analyze XOR-encrypted files to determine key lengths and recover plaintext
xortool -l 11 -c 20 file.bin
# Decrypt XOR-encrypted data with brute force and base64 charset
xortool -b -f -l 23 -t base64 message.enc
# Decrypt XOR-encrypted files using character frequency analysis
xortool tool_xored -c 20
```

Options

All 20 options parsed from the captured help text; 12 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-x</code>	—	input is hex-encoded str	An analyst would use the <code>-x</code> flag when processing a hex-encoded file, such as when decrypting data that has been represented in hexadecimal format.
<code>--hex</code>	—	input is hex-encoded str	An analyst would use the <code>--hex</code> flag when the input data is hex-encoded, as indicated by the tool's option description.
<code>-l</code>	LEN	length of the key	Fix the key length when you already know it.
<code>--key-length</code>	LEN	length of the key	An analyst would use the <code>--key-length</code> flag when they have prior knowledge or suspicion about the specific length of the XOR key used in the encrypted data.
<code>-m</code>	MAX-LEN	maximum key length to probe [default: 65]	Maximum key length to consider.
<code>--max-keylen</code>	MAX-LEN	maximum key length to probe [default: 65]	When analyzing XOR-encrypted data and the key length is unknown but needs to be limited to a specific maximum for efficiency or based on prior knowledge.

Flag	Argument	What it does	When you would use it
<code>-c</code>	CHAR	most frequent char (one char or hex code)	Give the most frequent character of the plaintext — usually <code>20</code> (space) for text, <code>00</code> for binaries. This is the flag that makes or breaks the attack.
<code>--char</code>	CHAR	most frequent char (one char or hex code)	An analyst would use the <code>--char</code> flag when they have prior knowledge or suspicion about the most frequent character in the plaintext, aiding xortool in accurately guessing the XOR key.
<code>-b</code>	—	brute force all possible most frequent chars	Brute-force the most frequent character rather than guessing.
<code>--brute-chars</code>	—	brute force all possible most frequent chars	Brute-force the most frequent character rather than guessing.
<code>-o</code>	—	same as <code>-b</code> but will only check printable chars	An analyst would use the <code>-o</code> flag when brute-forcing possible keys by checking only printable characters to guess the most frequent byte in XOR-encrypted data.
<code>--brute-printable</code>	—	same as <code>-b</code> but will only check printable chars	An analyst would use the <code>-o</code> flag when brute-forcing possible keys by checking only printable characters to guess the most frequent byte in XOR-encrypted data.
<code>-f</code>	—	filter outputs based on the charset	
<code>--filter-output</code>	—	filter outputs based on the charset	
<code>-p</code>	PLAIN	use known plaintext for decoding	An analyst would use the <code>-p</code> flag when they have a known plaintext segment to aid in decrypting XOR-encrypted data, as demonstrated in examples where it's paired with encrypted files and brute-force options.

Flag	Argument	What it does	When you would use it
<code>--known-plaintext</code>	PLAIN	use known plaintext for decoding	An analyst would use the -p flag when they have a known plaintext segment to aid in decrypting XOR-encrypted data, as demonstrated in examples where it's paired with encrypted files and brute-force options.
<code>-r</code>	PERCENT	threshold validity percentage [default: 95]	An analyst would use the -r flag when adjusting the threshold validity percentage for determining the likelihood of correct key guesses during XOR analysis.
<code>--threshold</code>	PERCENT	threshold validity percentage [default: 95]	An analyst would use the -r flag when adjusting the threshold validity percentage for determining the likelihood of correct key guesses during XOR analysis.
<code>-h</code>	—	show this help	
<code>--help</code>	—	show this help	

Gotchas

- Output lands in an `xortool_out/` directory, not stdout. People routinely think it did nothing.
- `-c 00` is right far more often than the default for packed binaries, because null padding dominates.

See also

`rasm2`, `floss`, `xlmdeobfuscator`

AccessEnum (GUI)

Capability	windows artifacts
Window title	AccessEnum License Agreement
Captured from	C:\Tools\sysinternals\AccessEnum.exe on 2026-08-02 — control tree in capture/gui/AccessEnum/AccessEnum.tree.txt

[← Capability index](#) · [Kit tool list](#)

Purpose

Enumerate the effective permissions across a directory tree or registry branch and show them in one list. Sorting by permission surfaces the outlier — the world-writable path that does not belong.

When you'd reach for this

Reach for AccessEnum when the question is *who can write where*. It walks a directory tree or registry branch and lists the effective permissions on each, which is the practical way to find a weak ACL without checking paths one at a time.

Sorting by permission is the point: the outlier surfaces immediately - the world-writable directory under Program Files that lets an unprivileged user replace a binary a service runs as SYSTEM. That is a privilege-escalation path and a persistence mechanism at once, so it matters when hunting and when hardening.

It reports what the permissions *are*. Whether they are wrong is your judgement, not the tool's.

Sources: <https://learn.microsoft.com/en-us/sysinternals/downloads/accessenum>

Controls

The parts of this window you will actually touch, read from the application's own accessibility tree rather than from a screenshot. The full node list is in [capture/gui/AccessEnum/AccessEnum.tree.txt](#).

The window exposes 4 further named controls: **You can also use the /accepteula command-line switch to accept the EULA., Agree, Decline, Print.** The full tree, with every automation id, is in [the capture](#).

Using it

1. Accept the licence on first run; until that is done the tool never reaches its main window.
2. Choose the directory or registry key to enumerate.
3. Run the scan, then sort by the permissions column rather than by path — the outliers are the finding, and they do not cluster by location.
4. Save the results as the before-state when you are about to change anything.

Gotchas

- It reports what the ACLs say, not what is reachable. Group membership, inheritance and share permissions all sit on top of this.
- The capture behind this page is the licence dialog, not the tool: a first run shows the EULA and nothing else. Sysinternals binaries need `-accepteula` before they can be driven automatically, so the control table here documents that dialog rather than the main window.

AmcacheParser

Kit	FLARE-VM / SIFT (Eric Zimmerman tools)
Capability	Parse ESE / SRUM / Amcache databases; Parse execution and persistence artifacts
Version	2026.5.0
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Parse Amcache.hve — the record of programs present on a host, with SHA-1 hashes, including binaries that have since been deleted.

When you'd reach for this

An analyst reaches for AmcacheParser after manually examining the AmCache hive with Registry Explorer or when needing structured CSV output for timeline analysis, as it automates extraction of AmCache data into a CSV file, which is more efficient than manual methods or RegRipper's plugin-based reports. They may run it following the extraction of the Amcache.hve file and before analyzing results in Timeline Explorer, prioritizing its automation and compatibility with further analysis tools.

Sources: <https://www.menovanveenendaal.com/posts/The-Windows-AmCache-and-ShimCache-Artifacts/>

Synopsis

```
AmcacheParser [options]
```

Common invocations

```
# Extract execution data and SHA1 hashes from Amcache.hve for analysis
AmcacheParser.exe -f "<EV>\Windows\AppCompat\Programs\Amcache.hve" -i --csv "<OUT>\Amcache" --csvf
amcache.csv

# Extract application data from Amcache hive with whitelist filtering
AmcacheParser.exe -f "C:\Temp\amcache\AmcacheWin10.hve" -w "c:\temp\whitelist.txt" --csv C:\temp
```

Options

All 15 options parsed from the captured help text; 9 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-f</code>	f	Amcache.hve file to parse	The Amcache.hve to parse.
<code>-i</code>	—	Include file entries for Programs entries	Include file entries associated with Programs entries. More complete, and noisier.
<code>-w</code>	w	Path to file containing SHA-1 hashes to <i>exclude</i> from the results. Blacklisting overrides whitelisting	Blacklist of SHA-1 hashes to exclude. Blacklisting overrides whitelisting, so a hash in both is dropped — the safe default when suppressing known-good noise.
<code>-b</code>	b	Path to file containing SHA-1 hashes to <i>include</i> from the results. Blacklisting overrides whitelisting	Whitelist of SHA-1 hashes to include.
<code>--csv</code>	csv	Directory to save CSV formatted results to. Be sure to include the full path in double quotes	Write CSV to a directory. The usual output.
<code>--csvf</code>	csvf	File name to save CSV formatted results to. When present, overrides default name	Override the generated CSV filename.
<code>--dt</code>	dt	The custom date/time format to use when displaying time stamps. See https://goo.gl/CNVq0k for options. Default is: yyyy-MM-dd HH:mm:ss [default: yyyy-MM-dd HH:mm:ss]	Custom timestamp format for the output.
<code>--mp</code>	—	When true, display higher precision for timestamps	Higher-precision timestamps.
<code>--nl</code>	—	When true, ignore transaction log files for dirty hives. Default is FALSE	Ignore transaction logs for a dirty hive. Leave this off unless you know why you want it: skipping the logs means parsing a hive that is missing its most recent changes.
<code>--debug</code>	—	Show debug information during processing	
<code>--trace</code>	—	Show trace information during processing	
<code>-?</code>	—	Show help and usage information	

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	Show help and usage information	
<code>--help</code>	—	Show help and usage information	
<code>--version</code>	—	Show version information	

Gotchas

- Amcache records that a binary was **present**, not that it ran. It is evidence of existence; Prefetch and event logs are evidence of execution. Conflating the two is the standard error with this artifact.
- It carries SHA-1 for entries, which makes it the fastest way to tie a deleted binary to threat intelligence long after the file is gone.
- The hive is usually dirty when collected from a live host. Let the transaction logs replay — the entries only in the logs are the most recent, which is normally the part you care about.

See also

[esedbexport](#), [esedbinfo](#), [SrumECmd](#), [PECmd](#), [AppCompatCacheParser](#), [MFTECmd](#)

AppCompatCacheParser

Kit	FLARE-VM / SIFT (Eric Zimmerman tools)
Capability	Parse execution and persistence artifacts
Version	2026.5.0
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

← [Capability index](#) · [Kit tool list](#)

Purpose

Parse the Application Compatibility Cache (Shimcache) out of the SYSTEM registry hive. Windows records an executable here when the shim engine examines it, which happens for programs that were run and for some that were merely present — so it is evidence of existence and interest, not proof of execution.

When you'd reach for this

An analyst reaches for AppCompatCacheParser when examining ShimCache for historical execution evidence, often after checking UserAssist or before parsing AmCache, as it converts the registry's AppCompatCache into a readable CSV, providing file names, sizes, and timestamps that manual analysis cannot easily extract. They may prefer it over AmCacheParser when focusing on ShimCache-specific data rather than AmCache's more detailed but differently structured entries.

Sources: <https://hackers-arise.com/digital-forensics-registry-analysis-for-beginners-part-3-evidence-of-execution/> · <https://hivesecurity.gitlab.io/blog/dfir-incident-response-complete-guide-2026/> · <https://nullsec.us/windows-10-11-appcompatcache-deep-dive/>

Synopsis

```
AppCompatCacheParser [options]
```

Common invocations

```
# Extract ShimCache entries from SYSTEM hive for analysis
AppCompatCacheParser.exe -f "<EV>\Windows\System32\config\SYSTEM" --csv "<OUT>\ShimCache" --csvf
shimcache.csv
```

Options

All 13 options parsed from the captured help text; 2 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-f</code>	f	Full path to SYSTEM hive to process. If this option is not specified, the live Registry will be used	
<code>--csv</code>	csv	Directory to save CSV formatted results to. Be sure to include the full path in double quotes	An analyst would use the <code>--csv</code> flag when parsing the ShimCache from the SYSTEM hive to generate CSV output for analyzing execution evidence, such as identifying unusual or first-time executions.
<code>--csvf</code>	csvf	File name to save CSV formatted results to. When present, overrides default name	An analyst would use the <code>--csvf</code> flag when parsing the ShimCache from the SYSTEM hive to generate a CSV-formatted output file for further analysis.
<code>-c</code>	c	The ControlSet to parse. Default is to extract all control sets [default: -1]	
<code>-t</code>	—	Sorts last modified timestamps in descending order	
<code>--dt</code>	dt	The custom date/time format to use when displaying time stamps. See https://goo.gl/CNVq0k for options [default: yyyy-MM-dd HH:mm:ss]	
<code>--nl</code>	—	When true, ignore transaction log files for dirty hives	
<code>--debug</code>	—	Show debug information during processing	
<code>--trace</code>	—	Show trace information during processing	
<code>-?</code>	—	Show help and usage information	
<code>-h</code>	—	Show help and usage information	
<code>--help</code>	—	Show help and usage information	
<code>--version</code>	—	Show version information	

See also

[PECmd](#), [MFTECmd](#), [AmcacheParser](#)

chainsaw

Kit	SIFT / Security Onion (Sigma-based log hunting)
Capability	Parse Windows event logs
Version	chainsaw 2.16.0
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Hunt through Windows event logs with Sigma rules and built-in detection logic, at speed.

Synopsis

```
chainsaw [OPTIONS] <COMMAND>
```

Common invocations

```
# Detect missing log entries and time gaps
./chainsaw analyse gaps ./Logs/
# Search event logs for mimikatz indicators in attack samples
./chainsaw search mimikatz -i evtx_attack_samples/
# Searching EVTX logs for specific event IDs
./chainsaw search -t 'Event.System.EventID: =4104' evtx_attack_samples/
# Detect threats in event logs using Sigma rules
./chainsaw hunt EVTX-ATTACK-SAMPLES/ -s sigma/ --mapping mappings/sigma-event-logs-all.yml
```

Options

All 6 options parsed from the captured help text; 2 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>--no-banner</code>	—	Hide Chainsaw's banner	Suppress the banner, for clean output in a report or a pipeline.
<code>--num-threads</code>	NUM_THREADS	Limit the thread number (default: num of CPUs)	Cap the thread count. Defaults to every core, which is usually right on a dedicated analysis box and rude on a shared one.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	Print help	
<code>--help</code>	—	Print help	
<code>-V</code>	—	Print version	
<code>--version</code>	—	Print version	

Gotchas

- The interesting options live on the subcommands — `hunt`, `search`, `dump` — not at the top level captured here. Run `chainsaw hunt --help` for the ones that matter.
- Rules are not bundled with the binary. Without a Sigma rule set and the mapping file, `hunt` runs and finds nothing, which looks identical to a clean host.

See also

`evtxexport`, `evtxinfo`, `EvtxECmd`, `hayabusa`

esedbexport

Kit	SIFT Workstation (libyal)
Capability	Parse ESE / SRUM / Amcache databases
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Use esedbexport to export items stored in an Extensible Storage Engine (ESE)

When you'd reach for this

When analyzing EDB files from applications like Active Directory, an analyst uses esedbexport after mounting the file via Docker, as shown in the example command, to extract structured data from the database. They might run it after obtaining the EDB file through imaging or extraction tools, and choose it because it is specifically designed for ESE databases, as indicated by the documentation's mention of its use in Windows Mail, Exchange, and Active Directory.

Sources: <https://github.com/4k4xs4pH1r3/libesedb-utils/blob/master/libesedb.md> · <https://github.com/security-dockerfiles/esedbexport>

Synopsis

```
esedbexport [ -c codepage ] [ -l logfile ] [ -m mode ] [ -t target ]
[ -T table_name ] [ -hvV ] source
```

Options

All 8 options parsed from the captured help text; 1 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-c</code>	—	codepage of ASCII strings, options: ascii, windows-874, windows-932, windows-936, windows-1250, windows-1251, windows-1252 (default), windows-1253, windows-1254 windows-1255, windows-1256, windows-125	
<code>-h</code>	—	shows this help	

Flag	Argument	What it does	When you would use it
<code>-l</code>	—	logs information about the exported items	
<code>-m</code>	—	export mode, option: all, tables (default) 'all' exports all the tables or a single specified table with indexes, 'tables' exports all the tables or a single specified table	
<code>-t</code>	—	specify the basename of the target directory to export to (default is the source filename) esedbexport will add the suffix .export to the basename	An analyst would use the -t flag when extracting data from the NTDS (Active Directory) database file (ntds.dit) using the esedbexport tool in a Docker container.
<code>-T</code>	—	exports only a specific table	
<code>-v</code>	—	verbose output to stderr	
<code>-V</code>	—	print version	

See also

[esedbinfo](#) , [SrumECmd](#) , [AmcacheParser](#)

esedbinfo

Kit	SIFT Workstation (libyal)
Capability	Parse ESE / SRUM / Amcache databases
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Use esedbinfo to determine information about an Extensible Storage Engine (ESE)

When you'd reach for this

An analyst reaches for esedbinfo when examining Extensible Storage Engine (ESE) Database Files (EDB) to retrieve metadata such as file format, page size, tables, columns, and indexes, as demonstrated by the example `esedbinfo Windows.edb`. They may run it after obtaining an EDB file from a system, such as one used by Exchange or Active Directory, to understand its structure before deeper analysis. The tool is chosen for its specific focus on ESE databases and its ability to provide detailed catalog information, as described in the documentation.

Sources: <https://github.com/4k4xs4pH1r3/libesedb-utils/blob/master/libesedb.md> · <https://manpages.debian.org/unstable/libesedb-utils/esedbinfo.1.en.html>

Synopsis

```
esedbinfo [ -hvV ] source
```

Options

All 3 options parsed from the captured help text; 1 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-h</code>	—	shows this help	
<code>-v</code>	—	verbose output to stderr	An analyst would use the -v flag when needing detailed verbose output about an ESE Database File's structure and contents, such as page size, table counts, and column details.
<code>-V</code>	—	print version	

See also

[esedbexport](#), [SrumECmd](#), [AmcacheParser](#)

EvtxECmd

Kit	FLARE-VM / SIFT (Eric Zimmerman tools)
Capability	Parse Windows event logs
Version	2026.5.0
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Parse Windows event logs into a normalised, filterable CSV, mapping the useful fields out of the XML payload.

When you'd reach for this

An analyst reaches for EvtxECmd when processing Windows Event Log (EVTX) files as part of a KAPE workflow, often after collecting event data or before generating actionable output through KAPE modules; they choose it because it is specifically integrated with KAPE and includes maps for structured EVTX analysis, which may offer more streamlined processing compared to standalone tools.

Sources: <https://ericzimmerman.github.io/> · <https://github.com/AndrewRathbun/Awesome-KAPE>

Synopsis

```
EvtxECmd [options]
```

Common invocations

```
# Normalize event logs into structured CSV format
EvtxECmd.exe --sync

# Convert event logs to normalized CSV using community maps
EvtxECmd.exe -f "<EV>\...\Security.evtx" --csv "<OUT>\EVTX" --csvf security.csv
```

Options

All 26 options parsed from the captured help text; 20 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-f</code>	f	File to process. Either this or -d is required	A single log, when you already know which one matters.
<code>-d</code>	d	Directory to recursively process. Either this or -f is required	Recurse a directory of .evtx files — the usual mode when working from a collected <code>winevt\Logs</code> .
<code>--csv</code>	csv	Directory to save CSV formatted results to. Be sure to include the full path in double quotes	Write CSV to a directory. The normal output, and the form the rest of a timeline workflow expects.
<code>--csvf</code>	csvf	File name to save CSV formatted results to. When present, overrides default name	Override the generated CSV filename.
<code>--json</code>	json	Directory to save JSON formatted results to. Be sure to include the full path in double quotes	JSON output, for feeding another tool.
<code>--jsonf</code>	jsonf	File name to save JSON formatted results to. When present, overrides default name	Override the generated JSON filename.
<code>--xml</code>	xml	Directory to save XML formatted results to	XML output.
<code>--xmlf</code>	xmlf	File name to save XML formatted results to. When present, overrides default name	Override the generated XML filename.
<code>--dt</code>	dt	The custom date/time format to use when displaying time stamps. See https://goo.gl/CNVq0k for options [default: yyyy-MM-dd HH:mm:ss.ffffff]	Custom timestamp format for the output.
<code>--inc</code>	inc	List of Event IDs to process. All others are ignored. Overrides -exc Format is 4624,4625,5410,5500-5600	Process only these Event IDs. The fastest way to cut a multi-gigabyte log down to the question being asked — ranges are allowed (<code>4624,4625,5410-5500</code>).
<code>--exc</code>	exc	List of Event IDs to IGNORE. All others are included. Format is 4624,4625,5410,5500-5600	Process everything except these Event IDs. <code>--inc</code> wins if both are given.

Flag	Argument	What it does	When you would use it
<code>--sd</code>	sd	Start date for including events (UTC). Anything OLDER than this is dropped. Format should match --dt	Drop events older than this date (UTC).
<code>--ed</code>	ed	End date for including events (UTC). Anything NEWER than this is dropped. Format should match --dt	Drop events newer than this date (UTC). With <code>--sd</code> , scopes the parse to the incident window instead of all history.
<code>--fj</code>	—	When true, export all available data when using --json	Export all available data in JSON rather than the mapped subset.
<code>--tdt</code>	tdt	The number of seconds to use for time discrepancy detection. Default is 1 [default: 1]	Seconds of tolerance for time-discrepancy detection — flags records whose timestamps disagree, a clock-tampering signal.
<code>--met</code>	—	When true, show metrics about processed event log. Default is true	Show per-log metrics about what was processed.
<code>--maps</code>	maps	The path where event maps are located. Defaults to 'Maps' folder where program was executed [default: /opt/ezttools/EvtxECmd/Maps]	Where the event maps live. The maps are what turn raw XML into named columns; without the right ones, useful fields stay buried in the payload.
<code>--vss</code>	—	Process all Volume Shadow Copies that exist on drive specified by -f or -d	Also parse every Volume Shadow Copy on the drive, which is where cleared or rotated logs may survive.
<code>--dedupe</code>	—	Deduplicate -f or -d & VSCs based on SHA-1. First file found wins	Drop duplicates by SHA-1 across the source and shadow copies. Use it whenever <code>--vss</code> is on.
<code>--sync</code>	—	If true, the latest maps from https://github.com/EricZimmerman/evtx/tree/master/evtx/Maps are downloaded and local maps updated	Pull the latest maps from upstream. Worth doing before a big parse — map coverage improves continuously.
<code>--debug</code>	—	Show debug information during processing	
<code>--trace</code>	—	Show trace information during processing	
<code>-?</code>	—	Show help and usage information	
<code>-h</code>	—	Show help and usage information	

Flag	Argument	What it does	When you would use it
<code>--help</code>	—	Show help and usage information	
<code>--version</code>	—	Show version information	

Gotchas

- Without a map for an Event ID, the interesting values stay inside the XML payload rather than becoming columns. If an expected field is missing, the map is usually the reason, not the log.
- A cleared log is itself the finding: Security 1102 and System 104 record the clearing. Include them explicitly when hunting anti-forensics.
- Event log timestamps are recorded in UTC but `--sd` / `--ed` are only as good as your assumption about the host's clock. Corroborate before building a timeline on them.

See also

`evtxexport`, `evtxinfo`, `chainsaw`, `hayabusa`

evtxexport

Kit	SIFT Workstation (libyal)
Capability	Parse Windows event logs
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Use evtxexport to export items stored in a Windows XML Event Viewer

When you'd reach for this

An analyst reaches for evtxexport when exporting event records from an XML Event Log (.evtx) file, often after mounting a volume or image to access logs, as it supports exporting full event messages requiring SYSTEM and SOFTWARE registry files; they may use it after mounting a QEMU VM image and before analyzing event data in text or XML format, preferring it over similar tools for its ability to handle multi-language resources and full message exports.

Sources: <https://github.com/libyal/libevtx/wiki/Tools>

Synopsis

```
evtxexport [ -c codepage ] [ -f format ] [ -l log_file ]  
[ -m mode ] [ -p resource_files_path ]  
[ -r registry_files_path ] [ -s system_file ]  
[ -S software_file ] [ -t event_log_type ]  
[ -hTvV ] source
```

Common invocations

```
# Export Windows event log entries for analysis  
evtxexport -p c/ -r c/Windows/System32/config/ c/Windows/System32/winevt/Logs/Application.Evtx  
# Export event logs to XML format from file  
evtxexport -f xml p1/Windows/System32/winevt/Logs/Application.evtx
```

Options

All 13 options parsed from the captured help text; 7 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-c</code>	—	codepage of ASCII strings, options: ascii, windows-874, windows-932, windows-936, windows-949, windows-950, windows-1250, windows-1251, windows-1252 (default), windows-1253, windows-1254, windows-1255	
<code>-f</code>	—	output format, options: xml, text (default)	An analyst would use the -f flag when exporting event records from an EVTX file in a specific format, such as XML, to ensure the data is structured for analysis or integration with other tools.
<code>-h</code>	—	shows this help	
<code>-l</code>	—	logs information about the exported items	An analyst would use the -l flag when specifying the path to a particular EVTX log file to be processed by evtxexport.
<code>-m</code>	—	export mode, option: all, items (default), recovered 'all' exports the (allocated) items and recovered items, 'items' exports the (allocated) items and 'recovered' exports the recovered items	
<code>-p</code>	—	search PATH for the resource files	An analyst would use the -p flag when specifying the path to a mounted file system or volume containing Windows event logs and registry files for extraction.
<code>-r</code>	—	name of the directory containing the SOFTWARE and SYSTEM (Windows) Registry file	An analyst would use the -r flag when specifying the directory containing the SYSTEM and SOFTWARE registry files to properly parse event log data from a mounted Windows volume.
<code>-s</code>	—	filename of the SYSTEM (Windows) Registry file. This option overrides the path provided by -r	An analyst would use the -s flag when specifying the path to the SYSTEM registry file to export event log data that requires registry information for proper interpretation.

Flag	Argument	What it does	When you would use it
<code>-S</code>	—	filename of the SOFTWARE (Windows) Registry file. This option overrides the path provided by -r	An analyst would use the -S flag when exporting event logs from a mounted volume and needing to include the SOFTWARE registry file to resolve software-specific information referenced in the event data.
<code>-t</code>	—	event log type, options: application, security, system if not specified the event log type is determined based on the filename.	
<code>-T</code>	—	use event template definitions to parse the event record data	
<code>-v</code>	—	verbose output to stderr	An analyst would use the -v flag when they need detailed error, verbose, or debug output printed to stderr during the processing of EVTX files to troubleshoot issues or understand the tool's operation.
<code>-V</code>	—	print version	

See also

[evtxinfo](#), [EvtxECmd](#), [chainsaw](#), [hayabusa](#)

evtxinfo

Kit	SIFT Workstation (libyal)
Capability	Parse Windows event logs
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

← [Capability index](#) · [Kit tool list](#)

Purpose

Use evtxinfo to determine information about a Windows XML Event Viewer

When you'd reach for this

An analyst reaches for evtxinfo after extracting EVTX files from memory dumps using tools like volatility's dumpfiles, running it to inspect file headers and chunk metadata before using evtxdump to parse event data, as it provides structural insights without full log parsing.

Sources: <https://manpages.debian.org/unstable/libevtx-utils/evtxexport.1.en.html> · <https://www.rocheston.com/fire/> · https://www.tophertimzen.com/resources/cs407/slides/week04_02-EventLogs.html

Synopsis

```
evtxinfo [ -c codepage ] [ -hvV ] source
```

Options

All 4 options parsed from the captured help text; 1 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-c</code>	—	codepage of ASCII strings, options: ascii, windows-874, windows-932, windows-936, windows-949, windows-950, windows-1250, windows-1251, windows-1252 (default), windows-1253, windows-1254, windows-1255	An analyst would use the -c flag when the ASCII strings in the EVTX file are encoded using a codepage different from the default (windows-1252).
<code>-h</code>	—	shows this help	
<code>-v</code>	—	verbose output to stderr	
<code>-V</code>	—	print version	

See also

[evtxexport](#), [EvtxECmd](#), [chainsaw](#), [hayabusa](#)

hayabusa

Kit	SIFT / Security Onion (Sigma-based log hunting)
Capability	Parse Windows event logs
Version	error: unexpected argument '--version' found
Captured from	<code>cyberlab-aio</code> via <code>help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Scan Windows event logs against a bundled Sigma rule set and produce a ranked timeline of what looks like attacker activity. It is built for speed over a whole log directory, so it is the first pass that tells you which hosts and which hours deserve a closer look.

When you'd reach for this

An analyst reaches for Hayabusa when generating fast forensics timelines from Windows event logs, either after collecting logs for offline analysis or during live-response investigations; they may run it before importing data into tools like Elastic Stack or Timesketch, as it consolidates events into structured formats and leverages Sigma rules for detection, offering speed and compatibility with enterprise-scale analysis platforms.

Sources: <https://github.com/Yamato-Security/hayabusa>

Synopsis

```
hayabusa.exe <COMMAND> [OPTIONS]
hayabusa.exe help <COMMAND> or hayabusa.exe <COMMAND> -h
```

Options

No option definitions could be parsed from this tool's help output. It may be subcommand-driven or have no flags; needs manual review.

See also

[evtxexport](#) , [evtxinfo](#) , [EvtxECmd](#) , [chainsaw](#)

hivexsh

Kit	Kali Linux
Capability	Parse registry hives
Captured from	<code>cyberlab-aio</code> via <code>help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

If you think this file is a valid Windows binary hive file (*not*

When you'd reach for this

When an analyst needs to examine Windows Registry hive files, they use hivexsh after obtaining the hive file (e.g., via virt-cat or guestfish) to navigate and inspect its keys and subkeys, as it is specifically designed for this task and provides interactive shell commands for structured exploration.

Sources: <https://libguestfs.org/hivexsh.1.html> · <https://manpages.ubuntu.com/manpages/xenial/man1/hivexsh.1.html>

Options

No option definitions could be parsed from this tool's help output. It may be subcommand-driven or have no flags; needs manual review.

See also

`rip.pl`, `regripper`, `regfexport`, `regfinfo`, `regfmount`, `regipy-dump`, `regipy-parse-header`, `regipy-plugins-run`

MFTECmd

Kit	FLARE-VM / SIFT (Eric Zimmerman tools)
Capability	Parse execution and persistence artifacts
Version	2026.5.0
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Parse NTFS metadata files — \$MFT, \$J, \$Boot, \$SDS, \$I30 — into CSV or bodyfile, including entries for deleted files.

Synopsis

```
MFTECmd [options]
```

Options

All 32 options parsed from the captured help text; 26 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-f</code>	f	File to process (\$MFT \$J \$Boot \$SDS \$I30). Required	The metadata file to parse. Required, and the file type is detected from its contents rather than its name.
<code>-m</code>	m	\$MFT file to use when -f points to a \$J file (Use this to resolve parent path in \$J CSV output)	Supply the \$MFT alongside a \$J. Without it the journal shows file names with no path, because the parent directory only exists in the \$MFT.
<code>--csv</code>	csv	Directory to save CSV formatted results to. Be sure to include the full path in double quotes. This or --json required unless --de or --body is specified	Write CSV to a directory. The normal output.
<code>--csvf</code>	csvf	File name to save CSV formatted results to. When present, overrides default name	Override the generated CSV filename.

Flag	Argument	What it does	When you would use it
<code>--json</code>	json	Directory to save JSON formatted results to. Be sure to include the full path in double quotes. This or --csv required unless --de or --body is specified	JSON output, for a pipeline.
<code>--jsonf</code>	jsonf	File name to save JSON formatted results to. When present, overrides default name	Override the generated JSON filename.
<code>--body</code>	body	Directory to save bodyfile formatted results to. --bdl is also required when using this option	Bodyfile output, which is what <code>mactime</code> consumes — the bridge from NTFS metadata into a classic timeline.
<code>--bodyf</code>	bodyf	File name to save body formatted results to. When present, overrides default name	Override the generated bodyfile name.
<code>--bdl</code>	bdl	Drive letter (C, D, etc.) to use with bodyfile. Only the drive letter itself should be provided	Drive letter to record in the bodyfile. Required with <code>--body</code> , because a bodyfile path is meaningless without the volume it came from.
<code>--blf</code>	—	When true, use LF vs CRLF for newlines	Use LF rather than CRLF, when the output is going to a Unix toolchain.
<code>--dd</code>	dd	Directory to save exported \$MFT FILE record. --do is also required when using this option	Directory to write an exported FILE record to.
<code>--do</code>	do	Offset of the \$MFT FILE record to dump as decimal or hex. Ex: 5120 or 0x1400 Use --de or --debug to see offsets	Offset of the FILE record to dump, decimal or hex.
<code>--de</code>	de	Dump full details for \$MFT entry/sequence #. Format is 'Entry' or 'Entry-Seq' as decimal or hex. Example: 5, 624-5 or 0x270-0x5.	Dump full detail for one entry, as <code>Entry</code> or <code>Entry-Seq</code> . The flag for interrogating a single suspicious file.
<code>--dr</code>	—	When true, dump \$MFT resident files to dir specified by --csv or --json, in 'Resident' subdirectory. Files will be named '--_bin'	Dump resident files out of the \$MFT. Small files live entirely inside their MFT record, so this recovers content with no data runs to follow.

Flag	Argument	What it does	When you would use it
<code>--fls</code>	—	When true, displays contents of directory from \$MFT specified by --de. Ignored when --de points to a file	List a directory's contents from the \$MFT, for the entry given by <code>--de</code> .
<code>--ds</code>	ds	Dump full details for Security Id from \$SDS as decimal or hex. Example: 624 or 0x270	Dump a security descriptor from \$SDS by Id — how you get from a file to the ACL that was on it.
<code>--dt</code>	dt	The custom date/time format to use when displaying time stamps. See https://goo.gl/CNVq0k for options [default: yyyy-MM-dd HH:mm:ss.ffffff]	Custom timestamp format for the output.
<code>--sn</code>	—	Include DOS file name types in \$MFT output	Include DOS 8.3 short names.
<code>--fl</code>	—	Generate condensed file listing of parsed \$MFT contents. Requires --csv	Condensed file listing instead of the full attribute dump.
<code>--at</code>	—	When true, include all timestamps from 0x30 attribute vs only when they differ from 0x10 in the \$MFT	Include all \$STANDARD_INFORMATION timestamps rather than only those that differ from \$FILE_NAME. Differences between the two attribute sets are the classic timestamping signal, so include them when that is the question.
<code>--rs</code>	—	When true, recover slack space from FILE records when processing \$MFT files. This option has no effect for \$I30 files	Recover slack space from FILE records. Old entries survive in the unused tail of a record, so this reaches deleted metadata that a straight parse skips.
<code>--vss</code>	—	Process all Volume Shadow Copies that exist on drive specified by -f	Also parse every Volume Shadow Copy, which is where an older \$MFT still holds entries the live one has reused.
<code>--dedupe</code>	—	Deduplicate -f & VSCs based on SHA-1. First file found wins	Drop duplicates by SHA-1 across the source and shadow copies. Use it whenever <code>--vss</code> is on.
<code>--ir</code>	—	Include resident data in JSON/CSV output	Include resident data inline in the output rather than as separate files.

Flag	Argument	What it does	When you would use it
<code>--re</code>	re	Comma-separated list of extensions to include for resident data (e.g., '.txt,.ps1,.bat'). If omitted, includes all	Restrict resident extraction to these extensions.
<code>--rm</code>	rm	Maximum size in bytes for resident data to include (max: 1024000) [default: 1024]	Cap the size of resident data included.
<code>--debug</code>	—	Show debug information during processing	
<code>--trace</code>	—	Show trace information during processing	
<code>-?</code>	—	Show help and usage information	
<code>-h</code>	—	Show help and usage information	
<code>--help</code>	—	Show help and usage information	
<code>--version</code>	—	Show version information	

Gotchas

- \$MFT entries are reused. A deleted file's record is overwritten by the next file that needs it, so an entry describing a deleted file may already belong to something else — check the sequence number before asserting the two are the same file.
- \$STANDARD_INFORMATION timestamps are trivially forged; \$FILE_NAME timestamps are not, because they update only through the kernel. `--at` is what lets you compare them, and a mismatch is the strongest cheap indicator of timestomping.
- Parsing a live volume's \$MFT copied with a normal file copy will fail — it is locked. Extract it with a forensic tool or from a shadow copy.

See also

[PECmd](#), [AppCompatCacheParser](#), [AmcacheParser](#)

PECmd

Kit	FLARE-VM / SIFT (Eric Zimmerman tools)
Capability	Parse execution and persistence artifacts
Version	2026.5.0
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Parse Windows Prefetch files into evidence of what executed, when, how often, and which files each run touched.

When you'd reach for this

An analyst reaches for PECmd when processing Windows Prefetch files to extract execution details, often running it after collecting .pf files from a system or before exporting data for further analysis; they may use it over similar tools due to its structured output options like JSON or CSV, and its specific focus on Windows Prefetch parsing.

Sources: <https://github.com/EricZimmerman/PECmd>

Synopsis

```
PECmd [options]
```

Options

All 20 options parsed from the captured help text; 14 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-f</code>	f	File to process. Either this or -d is required	A single .pf file, when chasing one binary.
<code>-d</code>	d	Directory to recursively process. Either this or -f is required	Recurse a directory — the normal mode, since Prefetch is only meaningful as a set. Point it at <code>C:\Windows\Prefetch</code> .

Flag	Argument	What it does	When you would use it
<code>-k</code>	k	Comma separated list of keywords to highlight in output. By default, 'temp' and 'tmp' are highlighted. Any additional keywords will be added to these	Highlight extra keywords in the output. <code>temp</code> and <code>tmp</code> are highlighted by default; add the names you are hunting.
<code>-o</code>	o	When specified, save prefetch file bytes to the given path. Useful to look at decompressed Win10 files	Save the decompressed Prefetch bytes. Win10+ Prefetch is MAM-compressed, so this is how you get something another tool or a hex editor can read.
<code>-q</code>	—	Do not dump full details about each file processed. Speeds up processing when using <code>--json</code> or <code>--csv</code>	Suppress the per-file detail. Worth it on a large directory when the CSV is the real output.
<code>--csv</code>	csv	Directory to save CSV formatted results to. Be sure to include the full path in double quotes	Write CSV to a directory. This is the output that matters: Prefetch is a timeline source, and Timeline Explorer or a spreadsheet is where the pattern shows up.
<code>--csvf</code>	csvf	File name to save CSV formatted results to. When present, overrides default name	Override the generated CSV filename.
<code>--json</code>	json	Directory to save JSON formatted results to. Be sure to include the full path in double quotes	JSON output, when the results feed another tool.
<code>--jsonf</code>	jsonf	File name to save JSON formatted results to. When present, overrides default name	Override the generated JSON filename.
<code>--html</code>	html	Directory to save xhtml formatted results to. Be sure to include the full path in double quotes	XHTML report, for handing to someone who will not open a CSV.
<code>--dt</code>	dt	The custom date/time format to use when displaying time stamps. See https://goo.gl/CNVq0k for options [default: yyyy-MM-dd HH:mm:ss]	Custom timestamp format for the output.
<code>--mp</code>	—	When true, display higher precision for timestamps	Higher-precision timestamps, when ordering events within the same second matters.

Flag	Argument	What it does	When you would use it
<code>--vss</code>	—	Process all Volume Shadow Copies that exist on drive specified by -f or -d	Also parse every Volume Shadow Copy on the drive. Prefetch rolls over at 1024 entries on Win10+, so shadow copies are often the only place an older execution still exists.
<code>--dedupe</code>	—	Deduplicate -f or -d & VSCs based on SHA-1. First file found wins	Drop duplicates by SHA-1 across the source and the shadow copies. Effectively mandatory with <code>--vss</code> , which otherwise returns the same file many times over.
<code>--debug</code>	—	Show debug information during processing	
<code>--trace</code>	—	Show trace information during processing	
<code>-?</code>	—	Show help and usage information	
<code>-h</code>	—	Show help and usage information	
<code>--help</code>	—	Show help and usage information	
<code>--version</code>	—	Show version information	

Gotchas

- Prefetch proves a program **ran**; it does not prove who ran it or what it did. Pair it with event logs or `AmcacheParser` before attributing anything.
- Absence is not evidence of absence. Prefetch can be disabled, is commonly off on SSD-era server builds, and rolls over — a missing entry means nothing on its own.
- The last-run timestamps are the eight most recent executions only. Older runs are gone from the file even though the run count keeps counting them.

See also

`AppCompatCacheParser`, `MFTECmd`, `AmcacheParser`

RECcmd

Kit	FLARE-VM / SIFT (Eric Zimmerman tools)
Capability	Parse registry hives
Version	2026.5.0
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Query and export Windows registry hives from the command line, using batch files of plugin definitions to pull known-interesting keys in one pass. The batch approach is the point: it turns 'check the usual persistence locations' into one reproducible command.

When you'd reach for this

An analyst reaches for RECcmd during incident triage after checking event logs, file system changes, and amcache data to investigate persistence mechanisms like Run keys, services, or tasks; they use it alongside tools like EvtxECmd and MFTECmd to build a timeline of suspicious activity, as RECcmd specifically targets registry artifacts for persistence analysis.

Sources: <https://ridgelinecyber.com/resources/kape-ez-tools/> · <https://ridgelinecyber.com/training/modules/free/ir01-toolkit-setup/03-eztools/>

Synopsis

```
RECcmd [options]
```

Common invocations

```
# Extract user activity data from registry hive
RECcmd.exe -f "<EV>\Users\{user}\NTUSER.DAT" --bn BatchExamples\UserActivity.reb --csv "
<OUT>\Registry"
```

Options

All 33 options parsed from the captured help text; 4 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-f</code>	f	Hive to search. -f or -d is required	An analyst would use the -f flag when processing a single registry hive file against a specific rule file to extract forensic artifacts.
<code>-d</code>	d	Directory to look for hives (recursively). -f or -d is required	An analyst would use the -d flag with RECcmd when batch-processing hives against a community ruleset to extract forensic values like RunOnce persistence or user activity from the registry.
<code>--kn</code>	kn	Display details for key name. Includes subkeys and values	
<code>--vn</code>	vn	Value name. Only this value will be dumped	
<code>--bn</code>	bn	Use settings from supplied file to find keys/values. See included sample file for examples	
<code>--csv</code>	csv	Directory to save CSV formatted results to. Be sure to include the full path in double quotes	An analyst would use the --csv flag when batch-processing registry hives against the community ruleset to generate structured CSV output for forensic analysis of persistence mechanisms, user activity, and system configuration details.
<code>--csvf</code>	csvf	File name to save CSV formatted results to. When present, overrides default name	An analyst would use the --csvf flag when processing registry hives with RECcmd to generate a CSV output file for further analysis or documentation during a forensic investigation.
<code>--saveTo</code>	saveTo	Saves --vn value data in binary form to file. Expects path to a FILE	
<code>--json</code>	json	Directory to save JSON formatted results to. Be sure to include the full path in double quotes	
<code>--jsonf</code>	jsonf	File name to save JSON formatted results to. When present, overrides default name	
<code>--details</code>	—	Show more details when displaying results	

Flag	Argument	What it does	When you would use it
<code>--base64</code>	base64	Find Base64 encoded values with size >= Base64 (specified in bytes)	
<code>--minSize</code>	minSize	Find values with data size >= MinSize (specified in bytes)	
<code>--sa</code>	sa	Search for in keys, values, data, and slack	
<code>--sk</code>	sk	Search for in value record's key names	
<code>--sv</code>	sv	Search for in value record's value names	
<code>--sd</code>	sd	Search for in value record's value data	
<code>--ss</code>	ss	Search for in value record's value slack	
<code>--literal</code>	—	If true, --sd and --ss search value will not be interpreted as ASCII or Unicode byte strings	
<code>--nd</code>	—	If true, do not show data when using --sd or --ss	
<code>--regex</code>	—	If present, treat in --sk, --sv, --sd, and --ss as a regular expression	
<code>--dt</code>	dt	The custom date/time format to use when displaying time stamps. See https://goo.gl/CNVq0k for options [default: yyyy-MM-dd HH:mm:ss.ffffff]	
<code>--nl</code>	—	When true, allow transaction log files to not exist for dirty hives	
<code>--recover</code>	—	If true, recover deleted keys/values. Default is true	
<code>--vss</code>	—	Process all Volume Shadow Copies that exist on drive specified by -f or -d	
<code>--dedupe</code>	—	Deduplicate -f or -d & VSCs based on SHA-1. First file found wins	

Flag	Argument	What it does	When you would use it
<code>--sync</code>	—	If true, the latest batch files from https://github.com/EricZimmerman/RECcmd/tree/master/Batch Examples are downloaded and local files updated	
<code>--debug</code>	—	Show debug information during processing	
<code>--trace</code>	—	Show trace information during processing	
<code>-?</code>	—	Show help and usage information	
<code>-h</code>	—	Show help and usage information	
<code>--help</code>	—	Show help and usage information	
<code>--version</code>	—	Show version information	

See also

[rip.pl](#), [regripper](#), [hivexsh](#), [regfexport](#), [regfinfo](#), [regfmount](#), [regipy-dump](#), [regipy-parse-header](#)

regfexport

Kit	SIFT Workstation (libyal)
Capability	Parse registry hives
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Use regfexport to export information from a Windows NT

Synopsis

```
regfexport [ -c codepage ] [ -K key_path ] [ -l logfile ]  
[ -hvV ] source
```

Options

All 6 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
<code>-c</code>	—	codepage of ASCII strings, options: ascii, windows-874, windows-932, windows-936, windows-949, windows-950, windows-1250, windows-1251, windows-1252 (default), windows-1253, windows-1254, windows-1255	
<code>-h</code>	—	shows this help	
<code>-K</code>	—	show information about a specific key path.	
<code>-l</code>	—	logs information about the exported items	
<code>-v</code>	—	verbose output to stderr	
<code>-V</code>	—	print version	

See also

rip.pl , regripper , hivexsh , regfinfo , regfmount , regipy-dump , regipy-parse-header , regipy-plugins-run

reginfo

Kit	SIFT Workstation (libyal)
Capability	Parse registry hives
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Use reginfo to determine information about a Windows NT

When you'd reach for this

An analyst reaches for reginfo when examining Windows NT Registry Files (REGF), such as NTUSER.DAT, to retrieve information about the registry's structure and contents. They might run it after extracting the registry file from a disk image or before using other tools that require the key and value hierarchy, as it provides structured output options like bodyfile and verbose diagnostics.

Sources: <https://manpages.debian.org/unstable/libregf-utils/reginfo.1.en.html>

Synopsis

```
reginfo [ -B bodyfile ] [ -c codepage ] [ -hHvV ] source
```

Options

All 6 options parsed from the captured help text; 4 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-B</code>	—	output key and value hierarchy as a bodyfile	An analyst would use the -B flag when they need to output the key and value hierarchy of a REGF file as a bodyfile for further processing or analysis.

Flag	Argument	What it does	When you would use it
<code>-c</code>	—	codepage of ASCII strings, options: ascii, windows-874, windows-932, windows-936, windows-949, windows-950, windows-1250, windows-1251, windows-1252 (default), windows-1253, windows-1254, windows-1255	An analyst would use the -c flag when the ASCII strings in the REGF file are encoded using a specific codepage other than the default (windows-1252).
<code>-h</code>	—	shows this help	
<code>-H</code>	—	shows the key and value hierarchy	An analyst would use the -H flag when examining a Windows NT Registry File to display its key and value hierarchy for forensic analysis.
<code>-v</code>	—	verbose output to stderr	An analyst would use the -v flag when they need detailed error or debug information printed to stderr during the analysis of a Windows NT Registry File.
<code>-V</code>	—	print version	

See also

[rip.pl](#), [regripper](#), [hivexsh](#), [regfexport](#), [regfmount](#), [regipy-dump](#), [regipy-parse-header](#), [regipy-plugins-run](#)

regfmount

Kit	SIFT Workstation (libyal)
Capability	Parse registry hives
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Use regfmount to mount a Windows NT Registry File (REGF)

When you'd reach for this

An analyst reaches for regfmount when examining Windows registry hive files to explore their structure and contents, often after extracting the hive from a disk image or virtual machine; they may run commands like `ls` and `cat` on the mounted directory to inspect keys and values, preferring it over similar tools for its ability to present registry data as a navigable file system with editable text files.

Sources: <https://miloserdov.org/?p=5448>

Synopsis

```
regfmount [ -c codepage ] [ -X extended_options ] [ -hvV ] file
mount_point
```

Options

All 5 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
<code>-c</code>	—	codepage of ASCII strings, options: ascii, windows-874, windows-932, windows-936, windows-949, windows-950, windows-1250, windows-1251, windows-1252 (default), windows-1253, windows-1254, windows-1255	
<code>-h</code>	—	shows this help	

Flag	Argument	What it does	When you would use it
<code>-v</code>	—	verbose output to stderr, while regfmount will remain running in the foreground	
<code>-V</code>	—	print version	
<code>-X</code>	—	extended options to pass to sub system	

See also

`rip.pl`, `regripper`, `hivexsh`, `regfexport`, `regfinfo`, `regipy-dump`, `regipy-parse-header`, `regipy-plugins-run`

regipy-diff

Kit	SIFT Workstation (regipy)
Capability	Parse registry hives
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Diff two registry hives and report what changed between them. The artifact-level version of a before-and-after detonation: snapshot, run the sample, snapshot again, and read the delta.

Synopsis

```
regipy-diff [OPTIONS] FIRST_HIVE_PATH SECOND_HIVE_PATH
```

Options

All 4 options parsed from the captured help text; 1 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-o</code>	FILE	—	An analyst would use the <code>-o</code> flag when comparing registry hives to save the resulting differences to a CSV file for further analysis or documentation.
<code>-v</code>	—	Verbosity	
<code>--verbose</code>	—	Verbosity	
<code>--help</code>	—	Show this message and exit.	

See also

[rip.pl](#), [regripper](#), [hivexsh](#), [regfexport](#), [regfinfo](#), [regfmount](#), [regipy-dump](#), [regipy-parse-header](#)

regipy-dump

Kit	SIFT Workstation (regipy)
Capability	Parse registry hives
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Dump a registry hive to JSON with regipy, so the contents can be searched, diffed or fed into other tooling rather than read key by key.

When you'd reach for this

An analyst reaches for regipy-dump when they need to extract and analyze the contents of a registry hive, often after ensuring the hive is clean or applying transaction logs, and before running plugins or comparisons; they may use it to generate a JSON or timeline output for further examination, as it handles checksum validation and transaction logs during parsing.

Sources: <https://github.com/mkorman90/regipy>

Synopsis

```
regipy-dump [OPTIONS] HIVE_PATH
```

Options

All 18 options parsed from the captured help text; 1 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-o</code>	FILE	—	
<code>-p</code>	TEXT	A registry path to start iterating from	
<code>--registry-path</code>	TEXT	A registry path to start iterating from	
<code>-t</code>	—	Create a CSV timeline instead	An analyst would use the -t flag when they need to output a timeline of the registry hive data instead of a JSON file.

Flag	Argument	What it does	When you would use it
<code>--timeline</code>	—	Create a CSV timeline instead	An analyst would use the <code>-t</code> flag when they need to output a timeline of the registry hive data instead of a JSON file.
<code>-l</code>	TEXT	Specify a hive type, if it could not be identified for some reason	
<code>--hive-type</code>	TEXT	Specify a hive type, if it could not be identified for some reason	
<code>-r</code>	TEXT	The path from which the partial hive actually starts, for example: <code>-t ntuser -r "/Software"</code> would mean this is actually a HKCU hive, starting from HKCU/Software	
<code>--partial_hive_path</code>	TEXT	The path from which the partial hive actually starts, for example: <code>-t ntuser -r "/Software"</code> would mean this is actually a HKCU hive, starting from HKCU/Software	
<code>-v</code>	—	Verbosity	
<code>--verbose</code>	—	Verbosity	
<code>-d</code>	—	Not fetching the values for each subkey makes the iteration way faster. Values count will still be returned	
<code>--do-not-fetch-values</code>	—	Not fetching the values for each subkey makes the iteration way faster. Values count will still be returned	
<code>-s</code>	TEXT	If <code>"-s"</code> was specified, fetch only values for subkeys starting this timestamp in isoformat	
<code>--start-date</code>	TEXT	If <code>"-s"</code> was specified, fetch only values for subkeys starting this timestamp in isoformat	
<code>-e</code>	TEXT	If <code>"-e"</code> was specified, fetch only values for subkeys until this timestamp in isoformat	
<code>--end-date</code>	TEXT	If <code>"-e"</code> was specified, fetch only values for subkeys until this timestamp in isoformat	
<code>--help</code>	—	Show this message and exit.	

See also

[rip.pl](#), [regripper](#), [hivexsh](#), [regfexport](#), [regfinfo](#), [regfmount](#), [regipy-parse-header](#), [regipy-plugins-run](#)

regipy-parse-header

Kit	SIFT Workstation (regipy)
Capability	Parse registry hives
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Print a hive's header — sequence numbers, timestamp and whether it was cleanly unmounted. A dirty hive means transaction logs still hold recent changes, so this is the check that tells you whether you are reading the whole story.

When you'd reach for this

An analyst reaches for regipy-parse-header when examining the header of a registry hive file to quickly retrieve metadata such as sequence numbers and modification times, often running it before deeper analysis of the hive's contents. They may choose it because the Rust backend significantly reduces parsing time compared to the default Python parser, though the Python version remains the default if the Rust backend is not installed.

Sources: <https://github.com/mkorman90/regipy>

Synopsis

```
regipy-parse-header [OPTIONS] HIVE_PATH
```

Options

All 3 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
<code>-v</code>	—	Verbosity	
<code>--verbose</code>	—	Verbosity	
<code>--help</code>	—	Show this message and exit.	

See also

[rip.pl](#), [regripper](#), [hivexsh](#), [regfexport](#), [regfinfo](#), [regfmount](#), [regipy-dump](#), [regipy-plugins-run](#)

regipy-plugins-run

Kit	SIFT Workstation (regipy)
Capability	Parse registry hives
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Run regipy's plugins over a hive and emit structured results. The Python counterpart to RegRipper, and the easier one to embed in a pipeline because the output is JSON rather than formatted text.

When you'd reach for this

An analyst reaches for regipy-plugins-run after dumping a registry hive to disk, as it automatically detects the hive type and executes relevant plugins for analysis, offering efficiency over manual plugin selection or alternative tools that lack automatic hive-type detection.

Sources: <https://github.com/mkorman90/regipy>

Synopsis

```
regipy-plugins-run [OPTIONS] HIVE_PATH
```

Options

All 11 options parsed from the captured help text. The final column is filled in by review.

Flag	Argument	What it does	When you would use it
<code>-o</code>	FILE	Output path for plugins result [required]	
<code>-p</code>	TEXT	A plugin or list of plugins to execute command separated	
<code>--plugins</code>	TEXT	A plugin or list of plugins to execute command separated	
<code>-t</code>	TEXT	Specify a hive type, if it could not be identified for some reason	

Flag	Argument	What it does	When you would use it
<code>--hive-type</code>	TEXT	Specify a hive type, if it could not be identified for some reason	
<code>-r</code>	TEXT	The path from which the partial hive actually starts, for example: <code>-t ntuser -r "/Software"</code> would mean this is actually a HKCU hive, starting from HKCU/Software	
<code>--partial_hive_path</code>	TEXT	The path from which the partial hive actually starts, for example: <code>-t ntuser -r "/Software"</code> would mean this is actually a HKCU hive, starting from HKCU/Software	
<code>-v</code>	—	Verbosity	
<code>--verbose</code>	—	Verbosity	
<code>--include-unvalidated</code>	—	Include plugins that don't have validation test cases. These plugins may return incomplete or inaccurate data. Use at your own risk.	
<code>--help</code>	—	Show this message and exit.	

See also

[rip.pl](#), [regripper](#), [hivexsh](#), [regfexport](#), [regfinfo](#), [regfmount](#), [regipy-dump](#), [regipy-parse-header](#)

regripper

Kit	Kali Linux
Capability	Parse registry hives
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

Run plugins against a Windows registry hive and print what each finds. The plugins encode where the interesting keys live and how to interpret them, so it answers 'what is in this hive that matters?' without you memorising key paths.

When you'd reach for this

An analyst reaches for RegRipper when examining registry hive files to quickly extract and decode data using pre-canned plugins, often after extracting the hive from a forensic image or system; they may run it alongside manual registry analysis to validate findings, as it automates complex data extraction tasks like decoding ROT-13 or translating binary values, which would be time-consuming manually.

Sources: <https://www.sans.org/blog/regripper-ripping-registries-with-ease>

Options

No option definitions could be parsed from this tool's help output. It may be subcommand-driven or have no flags; needs manual review.

See also

[rip.pl](#), [hivexsh](#), [regfexport](#), [regfinfo](#), [regfmount](#), [regipy-dump](#), [regipy-parse-header](#), [regipy-plugins-run](#)

rip.pl

Kit	Kali Linux
Capability	Parse registry hives
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

[← Capability index](#) · [Kit tool list](#)

Purpose

The command-line entry point to RegRipper — run one plugin or a whole profile against a registry hive. Scriptable in a way the GUI is not, which is what makes it the form used in a pipeline.

When you'd reach for this

An analyst reaches for rip.pl when parsing Windows registry hives to extract forensic artifacts, often after extracting the hive files from a disk image or memory dump, and may list plugins first with `perl rip.pl -l` to determine which analysis to perform; they choose it because it is pre-installed on the SIFT workstation and supports a large number of plugins for detailed registry analysis.

Sources: <https://fwhibbit.es/en/windows-registry-prepare-the-coffeemaker> · <https://linuxconfig.org/how-to-install-regripper-registry-data-extraction-tool-on-linux> · <https://www.sans.org/blog/regripper-ripping-registries-with-ease>

Options

No option definitions could be parsed from this tool's help output. It may be subcommand-driven or have no flags; needs manual review.

See also

`regripper`, `hivexsh`, `regfexport`, `regfinfo`, `regfmount`, `regipy-dump`, `regipy-parse-header`, `regipy-plugins-run`

SrumECmd

Kit	FLARE-VM / SIFT (Eric Zimmerman tools)
Capability	Parse ESE / SRUM / Amcache databases
Version	2026.5.0
Captured from	<code>cyberlab-aio</code> via <code>--help</code> on 2026-08-11 — raw help output

← [Capability index](#) · [Kit tool list](#)

Purpose

Parse the System Resource Usage Monitor database, which Windows keeps for roughly 30 days. It records bytes sent and received per application per user — the artifact that answers 'how much data left this host, and which process sent it?' long after the network logs have rolled.

When you'd reach for this

An analyst reaches for SrumECmd during incident response after checking logs, processes, file changes, and persistence mechanisms to analyze data egress volume per application by processing SRUDB.dat and SOFTWARE hive data, as it specifically addresses network and application-based data movement insights not covered by other tools in the triage sequence.

Sources: <https://ericzimmerman.github.io/> · <https://ridgelinecyber.com/resources/kape-ez-tools/> · <https://ridgelinecyber.com/training/modules/free/ir01-toolkit-setup/03-eztools/>

Synopsis

```
SrumECmd [options]
```

Common invocations

```
# Extract app network usage from SRUM database
SrumECmd.exe -f "<EV>\Windows\System32\sru\SRUDB.dat" -r "<EV>\Windows\System32\config\SOFTWARE" --
csv "<OUT>\SRUM"
```

Options

All 11 options parsed from the captured help text; 1 reviewed with usage guidance.

Flag	Argument	What it does	When you would use it
<code>-f</code>	f	SRUDB.dat file to parse	An analyst would use the -f flag with SrumECmd when extracting per-app network usage data from the SRUDB.dat file to investigate network activity over the last 30-60 days.
<code>-r</code>	r	SOFTWARE hive to process. This is optional, but recommended	
<code>-d</code>	d	Directory to recursively process, looking for SRUDB.dat and SOFTWARE hive. This mode is primarily used with KAPE so both SRUDB.dat and SOFTWARE hive can be located	
<code>--csv</code>	csv	Directory to save CSV formatted results to. Be sure to include the full path in double quotes	
<code>--dt</code>	dt	The custom date/time format to use when displaying time stamps. See https://goo.gl/CNVq0k for options [default: yyyy-MM-dd HH:mm:ss]	
<code>--debug</code>	—	Show debug information during processing	
<code>--trace</code>	—	Show trace information during processing	
<code>-?</code>	—	Show help and usage information	
<code>-h</code>	—	Show help and usage information	
<code>--help</code>	—	Show help and usage information	
<code>--version</code>	—	Show version information	

See also

`esedbexport`, `esedbinfo`, `AmcacheParser`

Alphabetical Tool Index

Every tool in the guide, A–Z, with the page its entry starts on.

#

7za Malware triage — static 291

A

AccessEnum-gui	Windows artifacts	455	affconvert	Acquire & preserve	65	AppCompatCacheParser	Windows artifacts	460
aeskeyfind	Memory forensics	363	affinfo	Acquire & preserve	67	arp-scan	Network analysis	377
affcat	Acquire & preserve	63	AmcacheParser	Windows artifacts	457			

B

base64dump.py	Malware triage — static	293	binwalk	Examine the filesystem	173	bulk_extractor	Examine the filesystem	183
bdeinfo	Acquire & preserve	68	blkls	Examine the filesystem	180			

C

capa	Malware triage — static	297	chainsaw	Windows artifacts	463	cyberchef	Decode & deobfuscate	151
capinfos	Network analysis	378	clamscan	Malware triage — static	302			
CFF-Explorer-gui	Malware triage — static	300	CryptoTester-gui	Decode & deobfuscate	149			

D

dc3dd	Acquire & preserve	70	die	Malware triage — static	307	dnSpy-gui	Reverse engineering	410
dcfldd	Acquire & preserve	72	die-gui	Malware triage — static	308	dumpcap	Acquire & preserve	76
dd	Acquire & preserve	74	diec	Malware triage — static	312			

E

editcap	Network analysis	380	evtxexport	Windows artifacts	473	ewfinfo	Acquire & preserve	89
esedbexport	Windows artifacts	465	evtxinfo	Windows artifacts	476	ewfmount	Acquire & preserve	91
esedbinfo	Windows artifacts	467	ewfacquire	Acquire & preserve	81	ewfverify	Acquire & preserve	93
EvtxECmd	Windows artifacts	469	ewfexport	Acquire & preserve	86	ezhexviewer	Report & support	409

F

ffind	Examine the filesystem	189	freshclam	Malware triage — static	318	frida-ls-devices	Reverse engineering	423
file	Examine the filesystem	191	frida	Reverse engineering	412	frida-ps	Reverse engineering	424
floss	Malware triage — static	316	frida-discover	Reverse engineering	418	frida-trace	Reverse engineering	427
fls	Examine the filesystem	197	frida-kill	Reverse engineering	421	fsstat	Examine the filesystem	203
foremost	Examine the filesystem	201						

H

hashcat	Decode & deobfuscate	152	hayabusa	Windows artifacts	478
HashMyFiles-gui	Acquire & preserve	95	hivexsh	Windows artifacts	479
			hydra	Decode & deobfuscate	164

I

icat	Examine the filesystem	205	ils	Examine the filesystem	207	inetsim	Network analysis	384
idr-gui	Reverse engineering	434	img_stat	Acquire & preserve	97	istat	Examine the filesystem	209

J

john Decode & deobfuscate 165

L

<code>log2timeline.py</code>	Build the timeline	121
------------------------------	--------------------	-----

M

<code>mactime</code>	Build the timeline	134	<code>mmls</code>	Examine the filesystem	212	<code>msodd</code>	Malware triage — documents	234
<code>md5sum</code>	Acquire & preserve	98	<code>mraptor</code>	Malware triage — documents	232	<code>msoffcrypto-tool</code>	Malware triage — documents	237
<code>mergecap</code>	Network analysis	386						
<code>MFTECmd</code>	Windows artifacts	480						

N

<code>ngrep</code>	Network analysis	388	<code>nping</code>	Network analysis	397	<code>numbers-to-string.py</code>	Malware triage — static	321
<code>nmap</code>	Network analysis	392	<code>ntfs-3g</code>	Acquire & preserve	101			

O

<code>objdump</code>	Malware triage — static	324	<code>olefil</code>	Malware triage — documents	250	<code>oleobj</code>	Malware triage — documents	256
<code>OffVis-gui</code>	Malware triage — documents	239	<code>oleid</code>	Malware triage — documents	252	<code>oletime</code>	Malware triage — documents	258
<code>olebrows</code>	Malware triage — documents	241	<code>olema</code>	Malware triage — documents	253	<code>olevb</code>	Malware triage — documents	260
<code>oledi</code>	Malware triage — documents	242	<code>olemet</code>	Malware triage — documents	254	<code>openssl</code>	Decode & deobfuscate	169
<code>oledump.py</code>	Malware triage — documents	244						

P

<code>pcodedm</code>	Malware triage — documents	264	<code>pdfid.py</code>	Malware triage — documents	277	<code>pinfo.py</code>	Build the timeline	136
<code>pdf-</code>	Malware triage — documents	266	<code>PDFStreamDump</code>	Malware triage — documents	280	<code>psort.py</code>	Build the timeline	139
<code>parser</code>	Malware triage — documents	270	<code>per-gui</code>	Malware triage — static	330	<code>pyxswf</code>	Malware triage — documents	282
<code>parser.py</code>	Malware triage — documents	274	<code>PE-Detective-gui</code>	Windows artifacts	484			
<code>pdfi</code>	Malware triage — documents	274	<code>PECmd</code>	Examine the filesystem	214			
<code>d</code>			<code>photorec</code>					

R

<code>r2</code>	Reverse engineering	436	<code>RECmd</code>	Windows artifacts	487	<code>regripper</code>	Windows artifacts	504
<code>rabin2</code>	Malware triage — static	332	<code>regfexport</code>	Windows artifacts	491	<code>reordercap</code>	Network analysis	403
<code>radiff2</code>	Malware triage — static	336	<code>regfinfo</code>	Windows artifacts	493	<code>rip.pl</code>	Windows artifacts	505
<code>rafind2</code>	Examine the filesystem	216	<code>regfmount</code>	Windows artifacts	495	<code>rsakeyfind</code>	Memory forensics	365
<code>rahash2</code>	Acquire & preserve	102	<code>regipy-diff</code>	Windows artifacts	497	<code>rtfobj</code>	Malware triage — documents	285
<code>rasm2</code>	Reverse engineering	440	<code>regipy-dump</code>	Windows artifacts	498			
<code>rax2</code>	Decode & deobfuscate	170	<code>regipy-parse-header</code>	Windows artifacts	501			
<code>readelf</code>	Malware triage — static	340	<code>regipy-plugins-run</code>	Windows artifacts	502			
<code>readelf.py</code>	Malware triage — static	346						

S

<code>scalpel</code>	Examine the filesystem	217	<code>Signature-Explorer-gui</code>	Malware triage — static	349	<code>SrumECmd</code>	Windows artifacts	506
<code>sha256sum</code>	Acquire & preserve	105	<code>sigtool</code>	Acquire & preserve	107	<code>ssdeep</code>	Acquire & preserve	110
						<code>strings</code>	Examine the filesystem	220

T

<code>tcpflow</code>	Network analysis	405	<code>tshark</code>	Acquire & preserve	112	<code>tsk_recover</code>	Examine the filesystem	227
<code>tcpextract</code>	Examine the filesystem	223	<code>tsk_gettimes</code>	Build the timeline	147			
<code>testdisk</code>	Examine the filesystem	225						

U

unzip	Malware triage — static	351	upx	Malware triage — static	353
--------------	-------------------------	-----	------------	-------------------------	-----

V

VB-Decompiler-	Reverse	443	vdbbin	Reverse engineering	446	volatility3	Memory forensics	370
gui	engineering		vivbin	Reverse engineering	448	volshell	Memory forensics	374
vbdec-gui	Reverse engineering	445	vol	Memory forensics	366	vshadowinfo	Acquire & preserve	119

X

xlmdeobfusc	Malware triage —	287	xortool	Reverse engineering	451
ator	documents		xxd	Examine the filesystem	229

Y

yara	Malware triage — static	356	yarac	Malware triage — static	361
-------------	-------------------------	-----	--------------	-------------------------	-----